
Python Extension Patterns

Release 0.3.1rc1

Paul Ross <apaulross@gmail.com>

Apr 01, 2025

CONTENTS

1	Introduction	1
1.1	A Personal Note	1
1.2	Firstly Why?	1
1.2.1	Performance	2
1.2.2	Interface with C/C++ libraries	2
1.2.3	Less memory	2
1.2.4	The GIL	2
1.3	Why Not?	2
1.4	Alternatives to C Extensions	2
1.4.1	ctypes	2
1.4.2	Code Generators	2
1.5	A Faustian Bargain	3
1.6	Summary Advice	3
1.7	Python Documentation	3
1.8	Project Links	3
2	A Simple Example	5
2.1	Faster Please	5
2.1.1	The C Equivalent Function	5
2.1.2	The Python Interface to C	6
2.1.3	The Python Module	6
2.1.4	The setup.py File	6
2.1.5	Trying it Out	7
2.2	How Did We Do?	7
2.3	It's Not Over Yet	8
2.4	C Fights Back	8
2.5	Summary	9
3	PyObjects and Reference Counting	11
3.1	Access After Free	13
3.2	Memory Leaks	13
3.3	Warning About Py_SETREF() and Py_XSETREF()	14
3.4	Warning on Relying on Reference Counts of Unity or Less	15
3.5	Python's Garbage Collector	15
3.6	Python Terminology	16
3.6.1	"New" References	16
3.6.2	"Stolen" References	17
3.6.3	Warning on "Stolen" References With Containers	18
3.6.4	"Borrowed" References	19
3.7	Strong and Weak References	21

3.7.1	Examples	22
3.8	A Possible Precautionary Principle	23
3.9	Summary	24
4	Containers and Reference Counts	25
4.1	Some Additional Terminology	25
4.1.1	Discarded References	25
4.1.2	Abandoned References	26
4.2	Exploring the CPython C API	26
4.3	Tuples	27
4.3.1	PyTuple_SetItem()	27
4.3.2	PyTuple_SET_ITEM()	30
4.3.3	Setting and Replacing NULL	32
4.3.4	PyTuple_Pack()	33
4.3.5	Py_BuildValue()	34
4.3.6	Tuple Getters	35
4.3.7	Summary	35
4.4	Lists	35
4.4.1	PyList_SetItem() and PyList_SET_ITEM()	35
4.4.2	PyList_Append()	35
4.4.3	PyList_Insert()	36
4.4.4	Py_BuildValue()	37
4.4.5	List Getters	37
4.4.6	Summary	37
4.5	Dictionaries	38
4.5.1	PyDict_SetItem()	38
4.5.2	PyDict_SetDefault()	39
4.5.3	PyDict_SetDefaultRef() [Python 3.13+]	40
4.5.4	PyDict_GetItem()	42
4.5.5	PyDict_GetItemRef() [Python 3.13+]	43
4.5.6	PyDict_Pop() [Python 3.13+]	43
4.5.7	Other APIs	45
4.6	Sets	47
4.6.1	PySet_Add()	47
4.6.2	PySet_Discard()	48
4.6.3	PySet_Pop()	49
4.7	Summary	50
4.7.1	Tuples	50
4.7.2	Lists	50
4.7.3	Dictionaries	51
4.7.4	Sets	51
5	Struct Sequence (a namedtuple in C)	53
5.1	Differences Between a C Struct Sequence and a Python namedtuple	53
5.2	A Basic C Struct Sequence	54
5.2.1	Documentation String	54
5.2.2	Field Specifications	54
5.2.3	Struct Sequence Type Specification	54
5.2.4	Creating an Instance	54
5.2.5	Using an Instance	56
5.3	Whether to Provide Access to the Type from Python	56
5.3.1	Exposing the Type from the CPython Module	57
5.3.2	Hiding the Type in the Module	58
5.4	Converting a C struct to a namedtuple	60

5.5	Controlling Member Access	62
5.5.1	The Importance of the <code>n_in_sequence</code> Field	62
5.6	Unnamed Fields	62
5.6.1	Field Declaration	63
5.6.2	Type Declaration	64
5.6.3	Struct Sequence Creation	65
5.6.4	Access from Python	66
6	Exception Raising	67
6.1	Common Exception Patterns	68
6.1.1	Type Checking	68
6.2	Creating Specialised Exceptions	69
6.2.1	Declaring Specialised Exceptions	69
6.2.2	Example Module	69
6.2.3	Initialising Specialised Exceptions	70
6.2.4	Raising Specialise Exceptions	71
7	A Pythonic Coding Pattern for C Functions	73
7.1	Principle	73
7.2	Coding the Function	73
7.3	The Function Code as One	76
8	Parsing Python Arguments	77
8.1	Specifying the Function Arguments	77
8.1.1	C Function Declaration	77
8.1.2	Setting the <code>ml_flags</code> Field	77
8.2	Parsing the Arguments	79
8.3	Examples	80
8.3.1	No Arguments	80
8.3.2	One Argument	80
8.3.3	Variable Number of Arguments and Keyword Arguments	83
8.3.4	Default String and Bytes Arguments	84
8.3.5	Positional Only and Keyword Only Arguments	85
8.3.6	Parsing Arguments With Functional Conversion to C	86
8.4	Being Pythonic with Default Mutable Arguments	87
8.5	Helper Macros	91
8.5.1	Immutable Arguments	92
8.5.2	Mutable Arguments	93
9	Creating New Types	97
9.1	Properties	97
9.1.1	Referencing Existing Properties	97
9.1.2	Created Properties	98
9.2	Subclassing	98
9.3	Examples	98
9.3.1	Setting, Getting and Deleting Attributes Dynamically	98
9.4	Emulating Sequence Types	103
9.4.1	The Sequence Function Table	105
9.4.2	<code>sq_length</code>	105
9.4.3	<code>sq_concat</code>	106
9.4.4	<code>sq_repeat</code>	107
9.4.5	<code>sq_item</code>	109
9.4.6	<code>sq_ass_item</code>	110
9.4.7	<code>sq_ass_contains</code>	114
9.4.8	<code>sq_inplace_concat</code>	115

9.4.9	sq_inplace_repeat	115
9.4.10	The Function Table	116
9.4.11	Module Initialisation	117
9.5	TODOs:	117
10	Setting and Getting Module Globals	119
10.1	Initialising Module Globals	119
10.2	Getting and Setting	121
10.2.1	From Python	121
10.2.2	Getting Module Globals From C	122
10.2.3	Setting Module Globals From C	123
11	Logging and Frames	125
11.1	Logging From C	125
11.1.1	Logging C Declarations	125
11.1.2	Logging C Globals	125
11.1.3	Logging C Functions	125
11.1.4	cLogging Module	127
11.1.5	Using and Testing	129
11.2	Frames	130
11.2.1	The Python Frame	130
11.2.2	The C “Frame”	132
12	File Paths and Files	133
12.1	File Paths	133
12.1.1	Parsing File Paths as Arguments	133
12.2	Files	135
12.2.1	Reading a Python File	135
12.2.2	Writing to a Python File	136
13	Subclassing and Using super()	141
13.1	Basic Subclassing	141
13.1.1	Writing the C Extension	141
13.1.2	Setup and Build	143
13.1.3	Test	143
13.2	Calling super() from C	143
13.2.1	The Obvious Way is Wrong	144
13.2.2	Doing it Right	145
13.3	Subclassing with Overloading	150
13.3.1	Writing the C Extension	150
13.3.2	Setup and Build	152
13.3.3	Test	152
13.3.4	Another Example of Using this API	153
14	Capsules	155
14.1	A Simple Example	155
14.1.1	Basic Extension	155
14.1.2	An Exportable Extension	156
14.1.3	A Client Extension	159
14.2	Using an Existing Capsule	161
14.2.1	Writing the Code for the Object	161
14.2.2	Building	163
14.2.3	Testing	163
15	Iterators and Generators	169

15.1	Iterators	169
15.1.1	Example of a Sequence	170
15.1.2	Adding an Iterator	172
15.1.3	A Note on Module Initialisation	175
15.1.4	Iterating a Python Object in C	176
15.1.5	Examples	178
15.1.6	Reverse Iterators	179
15.2	Generators	180
15.2.1	Our Iterator as a Generator	181
16	Context Managers	183
16.1	C Functions	183
16.1.1	<code>__enter__</code>	183
16.1.2	<code>__exit__</code>	183
16.1.3	Method declarations	184
16.2	Understanding the Context Manager	184
16.2.1	A Context Manager Without a Target	184
16.2.2	A Context Manager With a Target	184
16.3	Minimal Context Manager in C	185
16.3.1	<code>__enter__</code> and <code>__exit__</code> Methods	186
16.3.2	Type Declaration	186
16.3.3	Module	186
16.3.4	Setup	187
16.3.5	Testing	187
17	Pickling C Extension Types	191
17.1	Pickle Version Control	191
17.2	Implementing <code>__getstate__</code>	192
17.3	Implementing <code>__setstate__</code>	192
17.3.1	Error Checking	192
17.3.2	Set the first Member	193
17.3.3	Set the last Member	193
17.3.4	Set the number Member	194
17.3.5	<code>__setstate__</code> in Full	194
17.4	Add the Special Methods	196
17.5	Pickling a <code>custom2.Custom</code> Object	197
17.6	The Pickled Object in Detail	198
17.7	Pickling Objects with External State	200
17.8	References	200
18	Watchers [Python 3.12+]	201
18.1	Dictionary Watchers	201
18.1.1	Low Level C Implementation	202
18.1.2	Exposing This to CPython	206
18.1.3	Creating the Context Manager	207
18.1.4	Module, Setup and Test	209
18.1.5	Without the Context Manager	210
18.2	Type Watchers	210
18.3	Function Watchers	210
18.4	Code Watchers	210
19	Setting Compiler Flags	211
19.1	From the Command Line	211
19.2	Programmatically from Within a Python Process	211
19.3	From the Command Line using <code>sysconfig</code>	212

19.4	Setting Flags Automatically in <code>setup.py</code>	212
19.5	Getting C/C++ Flags from a CMake Build	213
19.6	Other Resources	215
20	Debugging	217
20.1	Debugging Tools	217
20.1.1	Build a Debug Version of Python	217
20.1.2	Valgrind	217
20.1.3	A Simple Memory Monitor	217
20.2	Building and Using a Debug Version of Python	219
20.2.1	Building a Standard Debug Version of Python	219
20.2.2	Specifying Macros	220
20.2.3	The Debug Builds	220
20.2.4	Python's Memory Allocator	221
20.2.5	Identifying the Python Build Configuration from the Runtime	224
20.3	Valgrind	225
20.3.1	Building Valgrind	225
20.3.2	Building Python for Valgrind	225
20.3.3	Using Valgrind	226
20.4	Leaked New References	226
20.4.1	A Leak Example	226
20.4.2	Recognising Leaked New References	228
20.4.3	Finding Where the Leak is With Valgrind	229
20.5	Debugging Tactics	230
20.5.1	Access After Free	231
20.6	Using gcov for C/C++ Code Coverage	231
20.6.1	gcov under Mac OS X	231
20.6.2	Using gcov on CPython Extensions	232
20.7	Debugging Python C Extensions in an IDE	234
20.7.1	Creating a Python Unit Test to Execute	234
20.7.2	Writing a C Function to call any Python Unit Test	234
20.7.3	Debugging Python C Extensions in Xcode	240
20.7.4	Using a Debug Version of Python C with Xcode	240
20.7.5	Debugging Python C Extensions in Eclipse	241
20.8	Instrumenting the Python Process for Your Structures	241
20.8.1	An Implementation of a Counter	241
20.8.2	Instrumenting the Counter	244
21	Memory Leaks	247
21.1	Introduction	247
21.1.1	Is it a Leak?	247
21.1.2	Sources of Leaks	247
21.1.3	A Bit About (C)Python Memory Management	247
21.2	Tools for Detecting Memory Leaks	251
21.2.1	Platform Tools	251
21.2.2	Python Tools	253
21.2.3	Debugging Tools	255
21.2.4	Building a Debug Version of Python	255
21.3	Techniques	255
21.3.1	High Level	255
21.4	pymemtrace	256
21.4.1	pymemtrace Tools	256
21.4.2	Tool Characteristics	256
21.4.3	pymemtrace Process	257

21.4.4	pymemtrace cPyMemTrace	259
21.4.5	pymemtrace and DTrace	262
21.4.6	pymemtrace trace_malloc	262
21.4.7	pymemtrace Debug Malloc Stats	263
22	Thread Safety	267
22.1	When You Need a Lock	267
22.2	Coding up the Lock	268
22.2.1	Adding a PyThread_type_lock to our object	268
22.2.2	Initialising and Deallocating the Lock	268
22.3	Using the Lock	269
22.3.1	From C Code	269
22.3.2	From C++ Code	270
22.4	Example Code and Tests	271
22.4.1	Example Code	271
22.4.2	Example Tests	272
23	Source Code Layout	275
23.1	Testing CPython Utility Code	275
24	Using C++ With CPython Code	277
24.1	C++ RAII Wrappers Around PyObject*	277
24.1.1	C++ RAII Wrapper for a Borrowed PyObject*	277
24.1.2	C++ RAII Wrapper for a New PyObject*	278
24.2	Handling Default Arguments	278
24.2.1	Immutable Default Arguments	279
24.2.2	Mutable Default Arguments	280
24.3	Homogeneous Python Containers and C++	283
24.3.1	A Problematic Example	284
24.4	A CPython Extension containing C++ Objects	297
24.4.1	Allocation of C++ Objects and Placement new	297
24.5	Python Unicode Strings and C++	299
24.5.1	Basic Handling of Unicode	299
24.5.2	Working with bytes, bytearray and UTF-8 Unicode Arguments	302
24.6	C++ and the Numpy C API	303
24.6.1	Initialising Numpy	303
24.6.2	Verifying Numpy is Initialised	303
24.6.3	Numpy Initialisation Techniques	304
24.7	C++ and the Python Buffer Protocol	308
24.7.1	References:	308
25	Miscellaneous	309
25.1	No PyInit_... Function Found	309
25.2	Migrating from Python to a C Extension	309
26	Installation	311
26.1	Project Links	311
26.2	Setup	311
26.3	Stable release	311
26.4	From sources	311
26.5	Install Requirements	312
26.6	What is in the Package	312
26.7	Running the Tests	313
26.8	Building the Documentation	313
26.9	Building Everything	314

27 Further Reading	315
27.1 C Extensions - General	315
27.2 Books	315
27.3 Projects	315
28 TODO	317
28.1 Existing TODOs	317
28.2 Work Estimate for v0.3.0 Release	319
28.2.1 New Chapter “Debugging Python with CLion”	319
28.2.2 TOTAL	319
29 History	321
29.1 0.3.1rc0 (2025-04-01)	321
29.2 0.3.0 (2025-03-20)	321
29.2.1 Added Chapters	321
29.2.2 Changed Chapters	321
29.2.3 Other	322
29.2.4 TODO	322
29.3 0.2.2 (2024-10-21)	322
29.4 0.2.1 (2024-07-29)	322
29.4.1 Contributors	322
29.5 0.1.0 (2014-09-09)	323
30 Search	325
Index	327

INTRODUCTION

This project explores reliable patterns of coding Python Extensions in C. It covers the essentials of reference counts, exceptions and creating objects and functions that are safe and efficient.

Writing Python C Extensions can be daunting; you have to cast aside the security and fluidity of Python and embrace C, not just C but Python's C API, which is huge¹ and changes between versions². Not only do you have to worry about just your standard `malloc()` and `free()` cases but now you have to contend with how CPython's does its memory management which is by *reference counting*.

I describe some of the pitfalls you (I am thinking of you as a savvy C coder) can encounter and some of the coding patterns that you can use to avoid them.

This also might help if you are considering code to submit to the Python standard library which depends extensively on C extensions.

This is primarily a documentation project, with code. If you want that code see [Installation](#), otherwise read on.

1.1 A Personal Note

This project has its roots when, long ago, I joined a tech company that had created many production critical Python extensions in C. These were all written by a single engineer who left shortly after I joined. The CTO appointed me as the replacement on the dubious basis that I knew Python and C although I had never written a Python extension in C.

I really struggled to bring my knowledge of both languages to their very complicated, and crucial, codebase. To be honest I don't think I did a great job, but as I was the 'owner' I somehow got away with it.

After some time it occurred to me that, rather learning from their scrappy CPython C code, I asked myself "how would you write a Python C Extension from scratch?". So on the commute and at weekends I did just that and slowly things became clearer. This eventually led me to being invited to PyConUS to give a talk about the subject. This document is a synthesis of the latter journey which ended up giving me far more confidence about the subject than during my earlier difficulties.

My fond hope is that you will find that this document makes it much easier to work in this field than I found initially. Another way of saying that is that I dedicate this document to you, and your work.

So why write Python C Extensions?

1.2 Firstly Why?

There are several reasons why you might want to write a C extension:

¹ Huge, but pretty consistent once mastered.

² Version 0.3 of this project supports Python versions: 3.9, 3.10, 3.11, 3.12, 3.13.

1.2.1 Performance

This is the most compelling reason, a 50x or 100x improvement over pure Python is not unusual.

1.2.2 Interface with C/C++ libraries

If you have a library in C or C++ you will have to write a C extension to give a Python interface to that library.

1.2.3 Less memory

Python is pretty memory hungry, for example a float in Python is 24 bytes, the corresponding double in C is 8 bytes. C and C++ have more specific deallocation policies than with a garbage collected language.

1.2.4 The GIL

C Extensions do not need the Global Interpreter Lock (GIL) when working with C/C++ code which do *not* make calls to the CPython API.

1.3 Why Not?

Like everything, there are disadvantages, here are some:

- C Extensions are something of a niche skill. Achieving and maintaining that skill has costs. Hopefully this project reduces those.
- Writing C Extensions is more time consuming than pure Python. There is also the intellectual problem that you are dealing with Pure C/CPython/Python code which expects a lot of context switching. This project proposes patterns of code that should reduce the cognitive overhead of all of that.
- Testing C Extensions, whilst excellent at a high level, can be really tricky at a line-of-code level.

1.4 Alternatives to C Extensions

There are several alternatives to writing an extension directly in C, here are some:

1.4.1 ctypes

`ctypes` is a well documented foreign function library for Python and is part of Python's standard library. The module allows direct access to C/C++ libraries (such as `libc`). If you need this functionality, for example you need to access a binary library where you do not have the original source code so you can not build the library into your own code.

1.4.2 Code Generators

There are a number of projects out there that take high level code and generate C/C++ code that can then be built into a Python module. Some examples are:

- [SWIG](#) is a very well established project. An advantage that distinguishes it from other projects is its multi-language support.
- [Cython](#) is another well established project. You write in Python-like pseudo code that is translated into C which is then compiled into a Python module. A notable feature is its excellent support for working with `numpy`. If you are using Cython you might find another project of mine, [Notes on Cython](#), useful.
- [PyBind11](#) is an excellent and ingenious project that uses C++ template to do the bulk of the work in generating code for a Python module.

There are common drawbacks of code generators:

- The testing and debug story is generally poor.
- There is occasional pathological behaviour where a small change or version upgrade can introduce a large performance degradation.
- If you are crossing the boundary between the Python interpreter and compiled C/C++ at a high frequency, perhaps with many small objects, code generators can create a performance overhead compared to C extensions. An example is shown here with my project on [XML creation](#).

There are many other alternatives such as `pypy`, `numba` that are worth knowing about.

1.5 A Faustian Bargain

The ability to write C code and link it to the Python runtime has played a huge part in Python's success story. Much of the `stdlib`, and third party packages like `numpy` gives C like performance with Python's simple interface. This gives the reference implementation, CPython, huge power and ease of use.

However the downside is that it becomes very difficult to create *alternative* implementations of the Python language and, who knows, these might be faster, use less memory or have better parallelism. This is because any alternate implementation must work the thousands of CPython C extensions out there, with all their quirks, to have any chance of being compatible with existing code.

Python C extensions will be around for a long time. It is a skill worth learning.

1.6 Summary Advice

My advice if you are thinking about extensions:

- They can be really powerful, 100x powerful
- They can be expensive to write and maintain
- It helps to follow established patterns
- Write everything in Python, benchmark/profile before deciding what to put into CPython
- Use them for low level, stable, library code
- Keep the CPython layer as thin as possible
- Testing, testing testing!

1.7 Python Documentation

Some of the official Python documentation is wrong, misleading or missing and this project goes some way to correcting that. For example see the chapters [Containers and Reference Counts](#), [Struct Sequence \(a namedtuple in C\)](#) and [Creating New Types](#). There is an index entry "Documentation Lacunae" that identifies sections that improve or correct the official Python documentation.

1.8 Project Links

- Source is on [GitHub](#).
- Documentation [Read the Docs](#).
- Project is on [PyPi](#).

Next up: a simple example showing the effect on code performance.

A SIMPLE EXAMPLE

This very artificial example illustrates some of the benefits and drawbacks of Python C Extensions.

Suppose you have some Python code such as this that is performing slowly:

```
def fibonacci(index: int) -> int:
    if index < 2:
        return index
    return fibonacci(index - 2) + fibonacci(index - 1)
```

And that code is in pFibA.py. In the repl we can measure its performance with `timeit`:

```
>>> import pFibA
>>> pFibA.fibonacci(30)
832040
>>> import timeit
>>> ti_py = timeit.timeit(f'pFibA.fibonacci(30)', setup='import pFibA', number=10)
>>> print(f'Python timeit: {ti_py:8.6f}')
Python timeit: 1.459842
>>>
```

2.1 Faster Please

Now we want something faster so we turn to creating a C extension.

2.1.1 The C Equivalent Function

Firstly we can write the C equivalent to `fibonacci()` in the file `cFibA.c`, note the inclusion of `"Python.h"` which will give us access to the whole Python C API (we will use that later on):

```
#define PPY_SSIZE_T_CLEAN
#include "Python.h"

long fibonacci(long index) {
    if (index < 2) {
        return index;
    }
    return fibonacci(index - 2) + fibonacci(index - 1);
}
```

2.1.2 The Python Interface to C

So far we have a pure C function, we now write a C function that takes Python objects as arguments, converts them to C objects (so-called ‘un-boxing’), calls `fibonacci()` then converts the C result to a Python object (so-called ‘boxing’).

```
1 static PyObject *
2 py_fibonacci(PyObject *Py_UNUSED(module), PyObject *args) {
3     long index;
4
5     if (!PyArg_ParseTuple(args, "l", &index)) {
6         return NULL;
7     }
8     long result = fibonacci(index);
9     return Py_BuildValue("l", result);
10 }
```

2.1.3 The Python Module

Then we need to write some C code that defines the Python module that contains this function. The first is a data structure to define the Python functions in the module:

```
static PyMethodDef module_methods[] = {
    {
        "fibonacci",
        (PyCFunction) py_fibonacci,
        METH_VARARGS,
        "Returns the Fibonacci value."
    },
    {NULL, NULL, 0, NULL} /* Sentinel */
};
```

Then we have a structure that defines the module itself, its name and so on. Note that this references the `module_methods` structure above:

```
static PyModuleDef cFibA = {
    PyModuleDef_HEAD_INIT,
    .m_name = "cFibA",
    .m_doc = "Fibonacci in C.",
    .m_size = -1,
    .m_methods = module_methods,
};
```

Lastly a function to initialise the module:

```
PyMODINIT_FUNC PyInit_cFibA(void) {
    PyObject *m = PyModule_Create(&cFibA);
    return m;
}
```

2.1.4 The setup.py File

Finally we add a `setup.py` that specifies how to compile this code:


```

1 from setuptools import setup, Extension
2
3 setup(
4     name='cPyExtPatt',
5     version='0.1.0',
6     author='AUTHOR',
7     description='Python Extension example.',
8     ext_modules=[
9         Extension(
10             "cPyExtPatt.SimpleExample.cFibA",
11             sources=['src/cpy/SimpleExample/cFibA.c', ],
12             include_dirs=[],
13             library_dirs=[],
14             libraries=[],
15             extra_compile_args=[
16                 '-Wall', '-Wextra', '-Werror', '-Wfatal-errors', '-Wpedantic',
17                 '-Wno-unused-function', '-Wno-unused-parameter',
18                 '-Qunused-arguments', '-std=c99',
19                 '-UDEBUG', '-DNDEBUG', '-Ofast', '-g',
20             ],
21         )
22     ]
23 )

```

Running `python setup.py develop` will compile and build the module which can be used thus:

2.1.5 Trying it Out

```

>>> from cPyExtPatt.SimpleExample import cFibA
>>> cFibA.fibonacci(8)
21

```

Great, it works!

2.2 How Did We Do?

There is a test file that uses `timeit` to check the performance of the Python and C code at `src/cpy/SimpleExample/timeit_test.py`.

```

(PythonExtPatt3.11_A) SimpleExample git:(develop) $ python timeit_test.py
Index: 32 number of times: 20
Version A, no cacheing:
Python timeit: 7.321638
      C timeit: 0.131115
C is 55.8 times FASTER.

```

So with a small bit of work we have got a performance improvement of 55x.

2.3 It's Not Over Yet

Suppose we change the Python code by adding a couple of lines thus that uses a local cache for the results. We put this in the file pFibB.py:

```
import functools

@functools.cache
def fibonacci(index: int) -> int:
    if index < 2:
        return index
    return fibonacci(index - 2) + fibonacci(index - 1)
```

Now what does our timing code say?

```
Version A with Python cache, no C cache:
Python timeit: 0.000012
    C timeit: 0.130394
C is 11058.7 times SLOWER.
```

So our Python code is now vastly faster than our C code. This emphasises that performance can also come from a good choice of libraries, data structures, algorithms, cacheing and other techniques as well as the choice of the language of the implementation.

2.4 C Fights Back

Whatever we can do in Python we can do in C so what if we write cFibB.c to change the `fibonacci()` function to have a cache as well?

```
1 long fibonacci(long index) {
2     static long *cache = NULL;
3     if (!cache) {
4         /* FIXME */
5         cache = calloc(1000, sizeof(long));
6     }
7     if (index < 2) {
8         return index;
9     }
10    if (!cache[index]) {
11        cache[index] = fibonacci(index - 2) + fibonacci(index - 1);
12    }
13    return cache[index];
14 }
```

Now what does our timing code say?

```
Version B, both are cached:
Python timeit: 0.000004
    C timeit: 0.000007
C is 1.9 times SLOWER.
```

So our C code is back in the game but still slower. What is more the C code has added significant complexity to our codebase. And this codebase has to be maintained, and at what cost? The C code has also added significant risk as well as identified by the `/* FIXME */` comment above.

So while the options are available there are tradeoffs to be made.

2.5 Summary

- C Extensions can give vastly improved performance.
- A good choice of Python libraries, algorithms, code architecture and design can improve performance less expensively than going to C.
- All of this exposes the possible tradeoffs between the techniques.
- It is very useful in software engineering to have tradeoffs such as these that are explicit and visible.

Next up: understanding reference counts and Python's terminology.

PYOBJECTS AND REFERENCE COUNTING

A `PyObject` can represent any Python object. It is a fairly minimal C struct consisting of a reference count and a pointer to the object type¹:

```
typedef struct _object {  
    Py_ssize_t ob_refcnt;  
    struct _typeobject *ob_type;  
} PyObject;
```

Note

The struct `_typeobject` is crucial for defining new types. This structure changes over various Python versions. For convenience I have included the structure definitions for Python types in `type_objects/` for Python versions 3.6 to 3.13 which allows easy comparison between versions.

In Python C extensions you always create and deallocate these `PyObject`s *indirectly*. Creation is via Python's C API and destruction is done by decrementing the reference count. If this count hits zero then CPython will free all the resources used by the object.

The macros to manipulate reference counts are:

¹ This varies slightly between different Python versions.

Table 1: Reference Count Macros

Macro	Description
<code>Py_REFCNT()</code>	Get the reference count of an object. This expands to <code>((PyObject*)(o))->ob_refcnt</code> . This will segfault if passed <code>NULL</code> .
<code>Py_INCREF()</code>	Increments the reference count of the given <code>PyObject *</code> . This will segfault if passed <code>NULL</code> .
<code>Py_XINCREF()</code>	As <code>Py_INCREF()</code> but does nothing if passed <code>NULL</code> .
<code>Py_DECREF()</code>	Decrements the reference count of the given <code>PyObject *</code> . This will segfault if passed <code>NULL</code> . If the reference count reaches zero then the object will be deallocated and the memory <i>may</i> be reused. Warning: De-allocation might cause the execution of arbitrary Python code has free access to all Python global variables. Warning: Calling this when the reference count is 1 might reuse the memory so on exit the object reference count could be anything.
<code>Py_XDECREF()</code>	As <code>Py_DECREF()</code> but does nothing if passed <code>NULL</code> .
<code>Py_SETREF()</code>	This decrements the first arguments reference count then replaces it with the second argument. Thus it gracefully replaces one <code>PyObject *</code> with another.  Warning See <i>Warning About Py_SETREF() and Py_XSETREF()</i>.
<code>Py_XSETREF()</code>	As <code>Py_SETREF()</code> but if passed <code>NULL</code> as the first argument replaces the first argument with the second.  Warning See <i>Warning About Py_SETREF() and Py_XSETREF()</i>.

Here is an example of a normal `PyObject` creation, print and de-allocation:

```

1  #include "Python.h"
2
3  void print_hello_world(void) {
4      PyObject *pObj = NULL;
5
6      pObj = PyBytes_FromString("Hello world\n"); /* Object creation, ref count = 1. */
7      PyObject_Print(pObj, stdout, 0);
8      Py_DECREF(pObj); /* ref count becomes 0, object deallocated.
9                      * Miss this step and you have a memory leak. */
10 }

```

The twin challenges in Python extensions are:

- Avoiding undefined behaviour such as object access after an object's reference count is zero. This is analogous in C to access after `free()` or a using *dangling pointer*.
- Avoiding memory leaks where an object's reference count never reaches zero and there are no references to the object. This is analogous in C to a `malloc()` with no corresponding `free()`.

Here are some examples of where things can go wrong:

3.1 Access After Free

Taking the above example of a normal PyObject creation and deallocation then in the grand tradition of C memory management after the Py_DECREF the pObj is now referencing free'd memory:

```

1  #include "Python.h"
2
3  void print_hello_world(void) {
4      PyObject *pObj = NULL;
5
6      pObj = PyBytes_FromString("Hello world\n");    /* Object creation, ref count = 1. */
7      PyObject_Print(pObj, stdout, 0);
8      Py_DECREF(pObj);                                /* ref count = 0 so object deallocated.
9      /*
10     /* Accidentally use pObj... */
11     PyObject_Print(pObj, stdout, 0);

```

Accessing pObj may or may not give you something that looks like the original object.

The corresponding issue is if you decrement the reference count without previously incrementing it then the caller might find *their* reference invalid:

```

1  static PyObject *bad_incref(PyObject *pObj) {
2
3      /* Use pObj... */
4
5      Py_DECREF(pObj); /* Might make reference count zero. */
6      Py_RETURN_NONE; /* On return caller might find their object free'd. */
7  }

```

After the function returns the caller *might* find the object they naively trusted you with but probably not. A classic access-after-free error.

3.2 Memory Leaks

Memory leaks occur with a PyObject if the reference count never reaches zero and there is no Python reference or C pointer to the object in scope. Here is where it can go wrong: in the middle of a great long function there is an early return on error. On that path this code has a memory leak:

```

1  static PyObject *bad_incref(PyObject *pObj) {
2      Py_INCREF(pObj);
3      /* ... a metric ton of code here ... */
4      if (error) {
5          /* No matching Py_DECREF, pObj is leaked. */
6          return NULL;
7      }
8      /* ... more code here ... */
9      Py_DECREF(pObj);
10     Py_RETURN_NONE;
11 }

```

The problem is that the reference count was not decremented before the early return, if pObj was a 100 Mb string then that memory is lost. Here is some C code that demonstrates this:

```
static PyObject *bad_incref(PyObject *pModule, PyObject *pObj) {
    Py_INCREF(pObj);
    Py_RETURN_NONE;
}
```

And here is what happens to the memory if we use this function from Python (`cPyRefs.incref(...)` in Python calls `bad_incref()` in C):

```
1 >>> from cPyExtPatt import cPyRefs          # Process uses about 1Mb
2 >>> s = ' ' * 100 * 1024**2 # Process uses about 101Mb
3 >>> del s                                     # Process uses about 1Mb
4 >>> s = ' ' * 100 * 1024**2 # Process uses about 101Mb
5 >>> cPyRefs.bad_incref(s)                   # Now do an increment without decrement
6 >>> del s                                     # Process still uses about 101Mb - leaked
7 >>> s                                         # Officially 's' does not exist
8 Traceback (most recent call last):
9   File "<stdin>", line 1, in <module>
10 NameError: name 's' is not defined
11 >>>                                         # But process still uses about 101Mb - 's' is leaked
```

3.3 Warning About Py_SETREF() and Py_XSETREF()

Warning

`Py_SETREF()` and `Py_XSETREF()` contain a flaw. The idea behind them is to ‘swap’ the arguments, decrementing the first argument’s reference count.

The implementation (for `Py_SETREF()`) looks like this:

```
#define Py_SETREF(op, op2) \
    do { \
        PyObject *_py_tmp = _PyObject_CAST(op); \
        (op) = (op2); \
        Py_DECREF(_py_tmp); \
    } while (0)
```

However if called with both arguments pointing to the same `PyObject *` and that has a reference count of unity this will deallocate the object whilst maintaining a reference to it. Then a segfault is certain to happen later on as soon the de-referenced `PyObject *` is accessed.

The greater the first argument’s reference count is, then the further away from the call to `Py_SETREF()` the segfault will happen.

For example, in the simple case:

```
PyObject *op = PyUnicode_FromString("My test string.");
assert(Py_REFCNT(op) == 1);
Py_SETREF(op, op);
/* This is now undefined as it is access after free
 * and will most likely a segfault. */
Py_DECREF(op);
```

This issue is at the heart of the problem described in *PyTuple_SetItem() Replacement*.

A simple change to these macros would eliminate this problem:


```

#define Py_SETREF(op, op2) \
    do { \
        if (op != op2) { \
            PyObject *_py_tmp = _PyObject_CAST(op); \
            (op) = (op2); \
            Py_DECREF(_py_tmp); \
        } \
    } while (0)

```

3.4 Warning on Relying on Reference Counts of Unity or Less

Warning

Do not be tempted to read the reference count itself to determine if the object is alive. The reason is that if `Py_DECREF` sees a refcount of one it can free and then reuse the address of the refcount field for a completely different object which makes it highly unlikely that that field will have a zero in it. There are some examples of this later on.

Here is a simple example:

```

PyObject *op = PyUnicode_FromString("My test string.");
assert(Py_REFCNT(op) == 1);
Py_DECREF(op);
/* Py_REFCNT(op) can be anything here. */

```

For example this code is asking for trouble:

```

PyObject *op;
/* Do something ... */
while (op->ob_refcnt) {
    Py_DECREF(op);
}

```

This will either loop forever or segfault.

3.5 Python's Garbage Collector

CPython is a garbage collected language however the primary means of garbage collection is by reference counting. The 'garbage collector' is a secondary device that attempts to resolve circular references. If the reference counts are wrong the 'garbage collector' can do nothing to recover memory².

For example this will leak whatever efforts CPython's garbage collector makes:

```

void leak(void) {
    PyObject *value = PyLong_FromLong(123456L);
    /* value has been allocated on the heap with a reference count of 1. */
    /* On return, value will be leaked. */
}

```

² If you are interested in some of the technical details of memory management terms and techniques in a wide variety of languages then a great place to start is <https://www.memorymanagement.org>. For example here is their entry about [reference counting](#).

3.6 Python Terminology

The Python documentation uses the terminology “New”, “Stolen” and “Borrowed” references throughout. These terms identify who is the *real owner* of the reference and whose job it is to clean it up when it is no longer needed:

- **New** references occur when a `PyObject` is constructed, for example when creating a new list.
- **Stolen** references occur when composing a `PyObject`, for example appending a value to a list. “Setters” in other words.
- **Borrowed** references occur when inspecting a `PyObject`, for example accessing a member of a list. “Getters” in other words. *Borrowed* does not mean that you have to return it, it just means you that don’t own it. If *shared* references or *pointer aliases* mean more to you than *borrowed* references that is fine because that is exactly what they are.

This is about programming by contract and the following sections describe the contracts for each reference type.

First up **New** references.

3.6.1 “New” References

When you create a “New” `PyObject` from a Python C API then you own it and it is your job to either:

- Dispose of the object when it is no longer needed with `Py_DECREF`³.
- Give it to someone else who will do that for you.

If neither of these things is done you have a memory leak just like a `malloc()` without a corresponding `free()`.

Here is an example of a well behaved C function that take two C longs, converts them to Python integers and, subtracts one from the other and returns the Python result:

```

1 static PyObject *subtract_long(long a, long b) {
2     PyObject *pA, *pB, *r;
3
4     pA = PyLong_FromLong(a);           /* pA: New reference. */
5     pB = PyLong_FromLong(b);           /* pB: New reference. */
6     r = PyNumber_Subtract(pA, pB);     /* r: New reference. */
7     Py_DECREF(pA);                     /* My responsibility to decref. */
8     Py_DECREF(pB);                     /* My responsibility to decref. */
9     return r;                           /* Callers responsibility to decref. */
10 }
```

`PyLong_FromLong()` returns a *new* reference which means we have to clean up ourselves by using `Py_DECREF`.

`PyNumber_Subtract()` also returns a *new* reference but we expect the caller to clean that up. If the caller doesn’t then there is a memory leak.

So far, so good but what would be bad is this:

```
r = PyNumber_Subtract(PyLong_FromLong(a), PyLong_FromLong(b));
```

You have passed in two *new* references to `PyNumber_Subtract()` and that function has no idea that they have to be decref’d once used so the two `PyLong` objects are leaked.

The contract with *new* references is: either you decref it or give it to someone who will. If neither happens then you have a memory leak.

³ To be picky we just need to decrement the use of *our* reference to it. Other code that has incremented the same reference is responsible for decrementing their use of the reference.

An Example Leak Problem

Here is an example that exhibits a leak. The object is to add the integers 400 to 404 to the end of a list. You might want to study it to see if you can spot the problem:

```
static PyObject *
list_append_one_to_four(PyObject *list) {
    for (int i = 400; i < 405; ++i) {
        PyList_Append(list, PyLong_FromLong(i));
    }
    Py_RETURN_NONE;
}
```

The problem is that `PyLong_FromLong` creates `PyObject` (an int) with a reference count of 1 **but** `PyList_Append` increments the reference count of the object passed to it by 1 to 2. This means when the list is destroyed the list element reference counts drop by one (to 1) but *no lower* as nothing else references them. Therefore they never get deallocated so there is a memory leak.

The append operation *must* behave this way, consider this Python code

```
l = []
a = 400
# The integer object '400' has a reference count of 1 as only
# one symbol references it: a.
l.append(a)
# The integer object '400' must now have a reference count of
# 2 as two symbols reference it: a and l, specifically l[-1].
```

The fix is to create a temporary item and then *decref* *that* once appended (error checking omitted):

```
1 static PyObject *
2 list_append_one_to_four(PyObject *list) {
3     PyObject *temporary_item = NULL;
4
5     for (int i = 400; i < 405; ++i) {
6         /* Create the object to append to the list. */
7         temporary_item = PyLong_FromLong(i);
8         /* temporary_item->ob_refcnt == 1 now */
9         /* Append it. This will increment the reference count to 2. */
10        PyList_Append(list, temporary_item);
11        /* Decrement our reference to it leaving the list having the only reference. */
12        Py_DECREF(temporary_item);
13        /* temporary_item->ob_refcnt == 1 now */
14        temporary_item = NULL; /* Good practice... */
15    }
16    Py_RETURN_NONE;
17 }
```

3.6.2 “Stolen” References

This is also to do with object creation but where another object takes responsibility for decref’ing (possibly freeing) the object. Typical examples are when you create a `PyObject` that is then inserted into an existing container such as a tuple, list, dict etc.

The analogy with C code is `malloc`’ing some memory, populating it and then passing that pointer to a linked list which then takes on the responsibility to free the memory if that item in the list is removed. If you were to free the memory

you had malloc'd then you will get a double free when the linked list (eventually) frees its members.

Here is an example of creating a 3-tuple, the comments describe what is happening contractually:

```

1 static PyObject *make_tuple(void) {
2     PyObject *r;
3     PyObject *v;
4
5     r = PyTuple_New(3);           /* New reference. */
6     v = PyLong_FromLong(1L);     /* New reference. */
7     /* PyTuple_SetItem "steals" the new reference v. */
8     PyTuple_SetItem(r, 0, v);
9     /* This is fine. */
10    v = PyLong_FromLong(2L);
11    PyTuple_SetItem(r, 1, v);
12    /* More common pattern. */
13    PyTuple_SetItem(r, 2, PyUnicode_FromString("three"));
14    return r; /* Callers responsibility to decref. */
15 }

```

Note line 10 where we are overwriting an existing pointer with a new value, this is fine as `r` has taken responsibility for the original pointer value. This pattern is somewhat alarming to dedicated C programmers so the more common pattern, without the assignment to `v` is shown in line 13.

What would be bad is this:

```

v = PyLong_FromLong(1L); /* New reference. */
PyTuple_SetItem(r, 0, v); /* r takes ownership of the reference. */
Py_DECREF(v);           /* Now we are interfering with r's internals. */

```

Once `v` has been passed to `PyTuple_SetItem` then your `v` becomes a *borrowed* reference with all of their problems which is the subject of the next section.

The contract with *stolen* references is: the thief will take care of things so you don't have to. If you try to the results are undefined.

3.6.3 Warning on “Stolen” References With Containers

Warning

The example above describes tuples that *do* “steal” references. Other containers, such as a `dict` and `set` do *not*.

A consequence is that `PyTuple_SetItem(pTuple, 0, PyLong_FromLong(12345L))` does *not* leak with *new* references but `PyDict_SetItem(pDict, PyLong_FromLong(12345L), PyLong_FromLong(123456L))` *does* leak with *new* references. To avoid that particular leak then create temporaries, then call `PyDict_SetItem` with them and then decref your temporaries. If the key/value objects are *borrowed* references then there is nothing to do, `PyDict_SetItem` will increment them correctly.

Unfortunately this was only made clear in the Python documentation for `PyDict_SetItem` in Python version 3.8+: <https://docs.python.org/3/c-api/dict.html>

This also happens with `Py_BuildValue` when building with newly created Python objects (using `Py_BuildValue("{00}", ...)`).

This warning also applies to `PySet_Add()` which also increments the reference rather than stealing it. The documentation does not mention this at all (as of Python 3.13).

See `src/cpy/RefCount/cRefCount.c` and `tests/unit/test_c_ref_count.py` for verification of this.

The next chapter *Containers and Reference Counts* goes into more detail about this, how it can bite you and how you can defend yourself from these issues.

3.6.4 “Borrowed” References

When you obtain a reference to an existing `PyObject` in a container using a ‘getter’ you are given a *borrowed* reference and this is where things can get tricky. The most subtle bugs in Python C Extensions are usually because of the misuse of borrowed references.

If you find the term “borrowed references” mystifying then perhaps “shared references” might be clearer, because that is exactly what they are.

The analogy in C is having two pointers to the same memory location: so who is responsible for freeing the memory and what happens if the other pointer tries to access that free’d memory?

Here is an example where we are accessing the last member of a list with a “borrowed” reference. This is the sequence of operations:

- Get a *borrowed* reference to a member of the list.
- Do some operation on that list, in this case call `do_something()`.
- Access the *borrowed* reference to the member of the original list, in this case just print it out.

Here is a C function that *borrow*s a reference to the last object in a list, prints out the object’s reference count, calls another C function `do_something()` with that list, prints out the reference count of the object again and finally prints out the Python representation of the object:

```

1 static PyObject *pop_and_print_BAD(PyObject *pList) {
2     PyObject *pLast;
3
4     pLast = PyList_GetItem(pList, PyList_Size(pList) - 1);
5     fprintf(stdout, "Ref count was: %zd\n", pLast->ob_refcnt);
6     do_something(pList);
7     fprintf(stdout, "Ref count now: %zd\n", pLast->ob_refcnt);
8     PyObject_Print(pLast, stdout, 0);
9     fprintf(stdout, "\n");
10    Py_RETURN_NONE;
11 }
```

The problem is that if `do_something()` mutates the list it might invalidate the item that we have a pointer to.

Suppose `do_something()` ‘removes’ every item in the list⁴. Such as:

```

void do_something(PyObject *pList) {
    while (PyList_Size(pList) > 0) {
        PySequence_DelItem(pList, 0);
    }
}
```

Then whether reference `pLast` is still “valid” depends on what other references to it exist and you have no control over that.

Here are some examples of what might go wrong in that case.

⁴ Of course we never *remove* items in a list we merely decrement their reference count, and if that hits zero then they are deleted.

```
>>> l = ["Hello", "World"]
>>> cPyRefs.pop_and_print_BAD(l)      # l will become empty
Ref count was: 1
Ref count now: 4302027608
'World'
```

The reference count is bogus, however the memory has not been *completely* overwritten so the object (the string “World”) *appears* to be OK.

If we try a different string:

```
>>> l = ['abc' * 200]
>>> cPyRefs.pop_and_print_BAD(l)
Ref count was: 1
Ref count now: 2305843009213693952
Segmentation fault: 11
```

At least this will get your attention!

Note

Incidentally from Python 3.3 onwards there is a module `faulthandler` that can give useful debugging information (file `FaultHandlerExample.py`):

```
1 import faulthandler
2 faulthandler.enable()
3 from cPyExtPatt import cPyRefs
4 l = ['abc' * 200]
5 cPyRefs.pop_and_print_BAD(l)
```

And this is what you get:

```
$ python3 FaultHandlerExample.py
Ref count was: 1
Ref count now: 2305843009213693952
Fatal Python error: Segmentation fault

Current thread 0x000007fff73c88310:
  File "FaultHandlerExample.py", line 7 in <module>
Segmentation fault: 11
```

“Borrowed” References Go Bad

There is a more subtle issue; suppose that in your Python code there is a reference to the last item in the list, then the problem suddenly “goes away”:

```
>>> l = ["Hello", "World"]
>>> a = l[-1]
>>> cPyRefs.pop_and_print_BAD(l)
Ref count was: 2
Ref count now: 1
'World'
```

The reference count does not go to zero so the object is preserved. The problem is that the correct behaviour of your C function depends entirely on that caller code having an extra reference.

This can happen implicitly as well:

```
>>> l = list(range(8))
>>> cPyRefs.pop_and_print_BAD(1)
Ref count was: 20
Ref count now: 19
7
```

The reason for this is that (for efficiency) CPython maintains the integers -5 to 255 permanently so they never go out of scope. If you use different integers we are back to the same access-after-free problem:

```
>>> l = list(range(800, 808))
>>> cPyRefs.pop_and_print_BAD(1)
Ref count was: 1
Ref count now: 4302021872
807
```

The problem with detecting these errors is that the bug is data dependent so your code might run fine for a while but some change in user data could cause it to fail. And it will fail in a manner that is not easily detectable.

“Borrowed” References Solution

Fortunately the solution is easy: with borrowed references you should increment the reference count whilst you have an interest in the object, then decrement it when you no longer want to do anything with it:

```
1 static PyObject *pop_and_print_GOOD(PyObject *pList) {
2     PyObject *pLast;
3
4     pLast = PyList_GetItem(pList, PyList_Size(pList) - 1);
5     Py_INCREF(pLast);      /* Prevent pLast being deallocated. */
6     /* ... */
7     do_something(pList);
8     /* ... */
9     Py_DECREF(pLast);      /* No longer interested in pLast, it might */
10    pLast = NULL;          /* get deallocated here but we shouldn't care. */
11    /* ... */
12    Py_RETURN_NONE;
13 }
```

The `pLast = NULL;` line is not necessary but is good coding style as it will cause any subsequent accesses to `pLast` to fail.

An important takeaway here is that incrementing and decrementing reference counts is a cheap operation but the consequences of getting it wrong can be expensive. A precautionary approach in your code might be to *always* increment borrowed references when they are instantiated and then *always* decrement them before they go out of scope. That way you incur two cheap operations but eliminate a vastly more expensive one.

3.7 Strong and Weak References

Another mental model to look at this is the concept of *strong* and *weak* references to an object. This model is commonly used in other software domains⁵. If this model suits you then use it!

Here are the essential details between this model and the Python one. The mapping between the Python new/stolen/borrowed terminology and strong/weak terminology is:

⁵ Here is <https://www.memorymanagement.org/>'s definition of a strong reference and a weak reference.

- A “new” reference is a single *strong* reference.
- A “stolen” reference is handing over responsibility for a *strong* reference, any previous reference becomes a *weak* reference.
- A “borrowed” reference is a new *weak* reference.

And the Python implementation gives:

- `Py_REFCNT()` returns the number of *strong* references.
- `Py_INCREF()` creates a new *strong* reference.
- `Py_DECREF()` releases a *strong* reference.

The two types of errors are:

- Any unreachable *strong* reference is a memory leak (an unreachable object with a reference count > 0).
- Accessing an object through a *weak* reference where no *strong* references exist leads to undefined behaviour. Note the warning above: *Warning on Relying on Reference Counts of Unity or Less*.

3.7.1 Examples

Unreachable Strong Reference

Here we create a new object with a *strong* reference count of unity and then abandon it:

```
void unreachable(void) {
    /* Create a new object, a strong reference (a "new" reference). */
    PyObject *value = PyUnicode_FromString("My test string.");
    /* Check the strong reference count. */
    assert(Py_REFCNT(value) == 1);
    /* The object is now unreachable as 'value' goes out of scope.
       * The strong reference count is still unity so the object is leaked. */
    return;
}
```

“New” References

Here is a walked through example of the lifetime of creating a tuple containing a single string. First create the string:

```
Py_ssize_t strong_ref_count;

PyObject *value = PyUnicode_FromString("My test string.");
strong_ref_count = Py_REFCNT(value);
assert(strong_ref_count == 1);
```

There is one strong reference to the string and it is ‘owned’ by value. Now create a tuple:

```
PyObject *container = PyTuple_New(1);
strong_ref_count = Py_REFCNT(container);
assert(strong_ref_count == 1);
```

There is one strong reference to the container and it is ‘owned’ by container. Now insert the value into the tuple (error checking omitted):


```
PyTuple_SetItem(container, 0, value);
strong_ref_count = Py_REFCNT(value);
assert(strong_ref_count == 1);
```

At this point, in the strong/weak model, we have two references to the original string. One is *strong* (since `Py_REFCNT(value)` is 1), the other must then be a *weak* reference. But which is which? In other words is `value` or `container[0]` *strong* or *weak*?

This model determines that `container` holds the *strong* reference since on destruction of that container `Py_DECREF()` will be called on that reference reducing the strong reference count to zero. Therefore `value`, once a *strong* reference is now a *weak* reference. This expresses the concept of *stealing* a reference. So we end up with:

- A *strong* reference which is held by `container`, specifically `container[0]`.
- A *weak* reference which is held by `value`.

Now lets destroy the container.

```
Py_DECREF(container);
```

This will destroy the contents, by remove one *strong* reference for each value of the container then removing the *strong* reference to the container. This now makes `container` a weak reference to an object that has no *strong* references:

So we are left with no strong references but still have two weak references, held by `value` and `container`. Now accessing an object that has no strong references through a weak reference is undefined behaviour such as this:

```
PyObject_Print(container, stdout, Py_PRINT_RAW);
PyObject_Print(value, stdout, Py_PRINT_RAW);
```

Python's documentation on [strong references](#) and [borrowed \(weak\) references](#).

3.8 A Possible Precautionary Principle

Getting the reference counts wrong at any stage of the program risks:

- If the reference count is unnecessarily *high* you will have a memory leak.
- If the reference count is unnecessarily *low* you might have a segmentation fault or undefined behaviour.

Note that memory leaks are usually harder, sometimes *much* harder, to fix than segmentation faults.

To avoid both possibilities you need to get the reference counts *exactly* right and requires a great attention to detail which is expensive to do. This problem is exacerbated if the code base is large and constantly changing.

So are there some other trade-offs that could be made? Well if your process is short running you might not care about memory leaks at all as the OS will reclaim all the memory at the process end. In this case you could make the choice to ignore decrementing reference counts (or gratuitously increase the reference count) so that objects are never free'd thus a segmentation fault is impossible. This is similar to the way some compilers use `malloc()` but never bother with `free()`. The rationale is that there may be any number of references to an internal data structure and it is dangerous to invalidate any of them whereas if the compiler is a short running process then leaks are unimportant (you hope). This is also referred to as the [Null Garbage Collector](#) [[devblogs.microsoft.com](#)].

However if you want to create a Python Extension for a long running process (say a server) and you can't put up with memory leaks then you have no choice but to control the reference counts carefully.

Budget accordingly.

3.9 Summary

The contracts you enter into with these three reference types are:

Table 2: Python References.

Type	Contract
New	Either you decref it or give it to someone who will, otherwise you have a memory leak.
Stolen	The thief will take of things so you don't have to. If you try to the results are undefined.
Borrowed	The lender can invalidate the reference at any time without telling you. Bad news. So increment a borrowed reference whilst you need it and decrement it when you are finished.

The strong reference/weak reference model maps well to the Python model.

In the next chapter *Containers and Reference Counts* we look in more detail about the interplay of reference counts with Python objects and Python containers such as `tuple`, `list`, `set` and `dict`.

CONTAINERS AND REFERENCE COUNTS

This chapter looks in more detail of how the Python C API works with different containers, such as `tuple`, `list`, `set` and `dict`¹. It also clarifies and correct the Python documentation where that is inaccurate, incomplete or misleading. This also shows where the Python C API has some undocumented failure modes, some of which can lead to undefined behaviour.

This chapter includes examples and tests that you can step through to better understand the interplay between the container and the objects in that container.

Of particular interest are *Setters*, *Getters* and the behaviour of `Py_BuildValue` for each of those containers².

Note

The Python documentation for the [Concrete Objects Layer](#) has a general warning that passing NULL values into functions gives rise to undefined behaviour. This is not always the case and this chapter explores this in more detail than the official Python documentation.

4.1 Some Additional Terminology

As well as “*New*” *References*, “*Stolen*” *References* and “*Borrowed*” *References* described in the previous chapter some other *behaviors* of containers are worth defining when they interact with objects.

4.1.1 Discarded References

This is when a container has a reference to an object but is required to replace it with another object. In this case the container decrements the reference count of the original object before replacing it with the other object. This is to prevent a memory leak of the previous object.

Warning

If the the replacement object is the **same** as the existing object then very bad things *might* happen. For example see the warning in [PyTuple_SetItem\(\)](#) *PyTuple_SetItem() Replacement*.

¹ The official [Python documentation](#) categorises tuples and lists as *sequence objects* because they support the [Sequence Protocol](#) and dictionaries and sets as *container objects* because they support the [Mapping Protocol](#). In this chapter I use looser language by describing all four as *containers*.

² `Py_BuildValue` can not create a set, only a tuple, list or dictionary.

4.1.2 Abandoned References

This is when a container has a reference to an object but is required to replace it with another object. In this case the container *does not* decrement the reference count of the original object before replacing it with the other object. This *will* lead to a memory leak *unless* the replacement object is the same as the existing object. Of course if the original reference is NULL there is no leak.

An example of this is `PyTuple_SET_ITEM()` *PyTuple_SET_ITEM() Replacement*.

4.2 Exploring the CPython C API

The code in this chapter explores the CPython C API in several ways:

- C code that can be stepped through in the debugger. This code is in `src/cpy/Containers/DebugContainers.c`. This uses `asserts` to check the results, particularly reference counts, so should always be compiled with `-DDEBUG`. The tests are exercised by `src/main.c`.
- Test code is in `src/cpy/RefCount/cRefCount.c` which is built into the Python module `cPyExtPatt.cRefCount`. This can be run under `pytest` for multiple Python versions by `build_all.sh`.
- A study of the Python source code.
- A review of the Python C API documentation.

Note

The examples below use code that calls a function `new_unique_string()`. This function is designed to create a new, unique, `PyObject` (a string) that is never cached so always starts with a reference count of unity. The implementation is in `src/cpy/Containers/DebugContainers.c` and looks something like this:

```
static long debug_test_count = 0L;

PyObject *
new_unique_string(const char *function, const char *suffix) {
    if (suffix){
        return PyUnicode_FromFormat(
            "%s-%s-%ld", function, suffix, debug_test_count++
        );
    }
    return PyUnicode_FromFormat("%s-%ld", function, debug_test_count++);
}
```

Here is an example test function that checks that `PyTuple_SetItem()` *steals* a reference:

```
1  /**
2   * A function that checks whether a tuple steals a reference when using PyTuple_SetItem.
3   * This can be stepped through in the debugger.
4   * asserts are use for the test so this is expected to be run in DEBUG mode.
5   */
6  void dbg_PyTuple_SetItem_steals(void) {
7      printf("%s():\n", __FUNCTION__);
8      if (PyErr_Occurred()) {
9          fprintf(stderr, "%s(): On entry PyErr_Print() %s#%d:\n", __FUNCTION__, __FILE__
10         ↪ NAME__, __LINE__);
11         PyErr_Print();
12     }
```

(continues on next page)

(continued from previous page)

```

11     return;
12 }
13 assert(!PyErr_Occurred());
14 Py_ssize_t ref_count;
15
16 PyObject *container = PyTuple_New(1);
17 assert(container);
18 ref_count = Py_REFCNT(container);
19 assert(ref_count == 1);
20
21 PyObject *value = new_unique_string(__FUNCTION__, NULL);
22 ref_count = Py_REFCNT(value);
23 assert(ref_count == 1);
24
25 if (PyTuple_SetItem(container, 0, value)) {
26     assert(0);
27 }
28 ref_count = Py_REFCNT(value);
29 assert(ref_count == 1);
30
31 PyObject *get_item = PyTuple_GET_ITEM(container, 0);
32 ref_count = Py_REFCNT(get_item);
33 assert(ref_count == 1);
34
35 Py_DECREF(container);
36 /* NO as container deals with this. */
37 /* Py_DECREF(value); */
38 assert(!PyErr_Occurred());
39 }

```

Firstly Tuples, I'll go into quite a lot of detail here because it is very similar to the C API for lists which I'll cover with more brevity in a later section.

4.3 Tuples

The Python documentation for the [Tuple C API](#) is here.

Firstly setters, there are two APIs for setting an item in a tuple; `PyTuple_SetItem()` and `PyTuple_SET_ITEM()`.

4.3.1 PyTuple_SetItem()

`PyTuple_SetItem()` (a C function) inserts an object into a tuple with error checking. This function returns non-zero on error, these are described below in [PyTuple_SetItem\(\) Failures](#). The failure of `PyTuple_SetItem()` has serious consequences for the value that is intended to be inserted.

PyTuple_SetItem() Basic Usage

`PyTuple_SetItem()` *steals* a reference, that is, the container assumes responsibility for freeing that value when the container is free'd ('freeing' meaning decrementing the reference count). For example:

```

PyObject *container = PyTuple_New(1); /* container ref count will be 1. */
PyObject *value = new_unique_string(__FUNCTION__, NULL); /* value ref count will be 1. */

```

(continues on next page)

(continued from previous page)

```
PyTuple_SetItem(container, 0, value); /* value ref count will remain at 1. */
/* get_item == value and value ref count will be 1. */
PyObject *get_item = PyTuple_GET_ITEM(container, 0);
/* The contents of the container and value, will be decref'd
 * In this particular case both will go to zero and free'd. */
Py_DECREF(container);
/* Do not do this as the container has dealt with this. */
/* Py_DECREF(value); */
```

For code and tests see:

- **C:** in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyTuple_SetItem_steals`
- **CPython:** in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyTuple_SetItem_steals()`
- **Python:** in `tests/unit/test_c_ref_count.py`:
 - `test_PyTuple_SetItem_steals()`.

Whilst we are here this is an example of testing the behaviour by manipulating reference counts which we then check with `assert()`. The rationale is that you can't check reference counts after an object is destroyed. For example:

```
PyObject *value = new_unique_string(__FUNCTION__, NULL);
assert(Py_REFCNT(value) == 1);
Py_DECREF(value);
/* This will fail, the reference count will have an arbitrary value. */
assert(Py_REFCNT(value) == 0);
```

Once an object has been free'd you can not rely on the reference count field. Instead, deliberately increment the reference count before the critical section and check it afterwards. For example:

```
PyObject *container = PyTuple_New(1);
PyObject *value = new_unique_string(__FUNCTION__, NULL);
assert(Py_REFCNT(value) == 1);
PyTuple_SetItem(container, 0, value);
assert(Py_REFCNT(value) == 1);
/* Increment the reference count so we can see destruction. */
Py_INCREF(value);
assert(Py_REFCNT(value) == 2);
/* Check destruction. */
Py_DECREF(container);
assert(Py_REFCNT(value) == 1);
/* Clean up. */
Py_DECREF(value);
```

PyTuple_SetItem() Replacement

What happens when you use `PyTuple_SetItem()` to replace an existing element in a tuple? `PyTuple_SetItem()` still *steals* a reference, but what happens to the original reference? Lets see:

```
PyObject *container = PyTuple_New(1); /* container ref count will be 1. */
PyObject *value_a = new_unique_string(__FUNCTION__, NULL);
```

(continues on next page)

(continued from previous page)

```

/* value_a ref count will be 1. */
PyTuple_SetItem(container, 0, value_a); /* value_a ref count will remain 1. */
PyObject *value_b = new_unique_string(__FUNCTION__, NULL);
/* value_b ref count will be 1. */
PyTuple_SetItem(container, 0, value_b);
/* Now value_b ref count will remain 1 and value_a ref count will have been decremented
 * In this case value_a will have been free'd. */

```

So `PyTuple_SetItem()` discards (see *Discarded References*) the original reference.

Warning

What happens if you use `PyTuple_SetItem()` to replace a value with the *same* value? For example:

```

PyObject *container = PyTuple_New(1);
/* container ref count will be 1. */
PyObject *value = new_unique_string(__FUNCTION__, NULL);
/* value ref count will be 1. */
PyTuple_SetItem(container, 0, value);
/* value ref count is still 1 as it has been *stolen*. */

```

Now repeat the replacement:

```

/* Repeating the same call will only lead to trouble later on (maybe). */
PyTuple_SetItem(container, 0, value);
/* And this will segfault as, during execution, it will
 * try to decrement a value that does not exist. */
Py_DECREF(container);
/* So what is going on? */

```

What is happening is that the second time `PyTuple_SetItem()` is called it decrements the reference count of the existing member that happens to be `value`. In this case this brings `value`'s reference count from one down to zero. At that point `value` is free'd. Then `PyTuple_SetItem()` blithely sets `value` which is now, likely, garbage.

This is not described in the Python documentation.

A simple change to `PyTuple_SetItem()` would prevent this from producing undefined behaviour by checking if the replacement is the same as the existing value.

`PyTuple_SET_ITEM()` does not exhibit this problem as it *abandons* (see *Abandoned References*) values rather than *discarding* (see *Discarded References*) them.

For code and tests see:

- **C:** in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyTuple_SetItem_steals_replace`
 - `dbg_PyTuple_SetItem_replace_with_same`
- **CPython:** in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyTuple_SetItem_steals_replace`
 - `test_PyTuple_SetItem_replace_same`
- **Python:** in `tests.unit.test_c_ref_count`
 - `test_PyTuple_SetItem_steals_replace()`

– test_PyTuple_SetItem_replace_same()

PyTuple_SetItem() Failures

PyTuple_SetItem() can return a non-zero error code for these reasons:

- The given container is not a tuple.
- The index is out of range; index < 0 or index >= tuple length (negative indexes are not allowed).

Warning

A consequence of failure is that the value being inserted will be decref'd. For example this code will segfault:

```
PyObject *container = PyTuple_New(1); /* Reference count will be 1. */
PyObject *value = new_unique_string(__FUNCTION__, NULL); /* Ref count will be 1. */
PyTuple_SetItem(container, 1, value); /* Index out of range. */
Py_DECREF(value); /* value has already been decrefd and free'd so this will SIGSEGV */
```

This is not described in the Python documentation.

For code tests see, when the container is not a tuple:

- C: dbg_PyTuple_SetItem_fails_not_a_tuple in src/cpy/Containers/DebugContainers.c.
- CPython: test_PyTuple_SetItem_fails_not_a_tuple in src/cpy/RefCount/cRefCount.c.
- Python: tests.unit.test_c_ref_count.test_PyTuple_SetItem_fails_not_a_tuple.

And, when the index out of range:

- C: dbg_PyTuple_SetItem_fails_out_of_range in src/cpy/Containers/DebugContainers.c.
- CPython: test_PyTuple_SetItem_fails_out_of_range in src/cpy/RefCount/cRefCount.c.
- Python: tests.unit.test_c_ref_count.test_PyTuple_SetItem_fails_out_of_range.

Note

I'm not really sure why the PyTuple_SetItem() API exists. Tuples are meant to be immutable but this API treats existing tuples as mutable. It would seem like PyTuple_SET_ITEM() would be enough.

4.3.2 PyTuple_SET_ITEM()

PyTuple_SET_ITEM() is a function like macro that inserts an object into a tuple without any error checking (see *PyTuple_SET_ITEM() Failures* below). However type checking is performed as an assertion if Python is built in debug mode or with assertions. Because of the absence of checks, it is slightly faster than PyTuple_SetItem(). This is usually used on newly created tuples.

Importantly PyTuple_SET_ITEM() behaves **differently** to PyTuple_SetItem() when replacing another object.

PyTuple_SET_ITEM() Basic Usage

PyTuple_SET_ITEM() *steals* a reference just like PyTuple_SetItem().

```
PyObject *container = PyTuple_New(1); /* Reference count will be 1. */
PyObject *value = new_unique_string(__FUNCTION__, NULL); /* Ref count will be 1. */
```

(continues on next page)

(continued from previous page)

```
PyTuple_SET_ITEM(container, 0, value); /* Ref count of value will be 1. */
/* get_item == value and Ref count will be 1. */
PyObject *get_item = PyTuple_GET_ITEM(container, 0);
assert(get_item == value && Py_REFCNT(value) == 1);
Py_DECREF(container); /* The contents of the container, value, will be decrefd */
/* Do not do this as the container deals with this. */
/* Py_DECREF(value); */
```

For code and tests see:

- **C:** in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyTuple_PyTuple_SET_ITEM_steals()`
- **CPython:** in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyTuple_PyTuple_SET_ITEM_steals()`
- **Python:** in `tests/unit/test_c_ref_count`
 - `test_PyTuple_PyTuple_SET_ITEM_steals()`

PyTuple_SET_ITEM() Replacement

`PyTuple_SET_ITEM()` differs from `PyTuple_SetItem()` when replacing an existing element in a tuple as the original reference will be leaked. This is because `PyTuple_SET_ITEM()` *abandons* the previous reference (see [Abandoned References](#)):

```
PyObject *container = PyTuple_New(1); /* Reference count will be 1. */
PyObject *value_a = new_unique_string(__FUNCTION__, NULL); /* Ref count will be 1. */
PyTuple_SET_ITEM(container, 0, value_a); /* Ref count of value_a will be 1. */
PyObject *value_b = new_unique_string(__FUNCTION__, NULL); /* Ref count will be 1. */
PyTuple_SET_ITEM(container, 0, value_b);
assert(Py_REFCNT(value_a) == 1);
/* Ref count of value_b will be 1,
 * value_a ref count will still be at 1 and value_a will be leaked unless decrefd. */
```

Note

Because `PyTuple_SET_ITEM()` *abandons* the previous reference it does not have the problem with undefined behaviour that `PyTuple_SetItem()` has. For that see the warning about undefined behaviour in `PyTuple_SetItem()` [PyTuple_SetItem\(\) Replacement](#).

For code and tests see:

- **C:** in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyTuple_SET_ITEM_steals_replace()`
 - `dbg_PyTuple_SET_ITEM_replace_with_same()`
- **CPython:** in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyTuple_SetItem_steals_replace()`
 - `test_PyTuple_SET_ITEM_replace_same()`
- **Python:** in `tests/unit/test_c_ref_count.py`:

- test_PyTuple_SetItem_steals_replace
- test_PyTuple_SET_ITEM_replace_same.

PyTuple_SET_ITEM() Failures

PyTuple_SET_ITEM() minimises the error checking that PyTuple_SetItem() does, so:

- It does not check if the given container is a Tuple.
- It does not check if the index is in range.

If either of those is wrong then PyTuple_SET_ITEM() is capable of writing to arbitrary memory locations, and the result is likely to be tragic, mostly undefined behaviour and/or memory corruption.

4.3.3 Setting and Replacing NULL

This looks at what happens when setting or replacing a NULL pointer in a tuple. Both PyTuple_SetItem() and PyTuple_SET_ITEM() behave the same way.

Setting NULL

Setting a NULL will not cause an error:

```
PyObject *container = PyTuple_New(1);
assert(!PyErr_Occurred());
PyTuple_SetItem(container, 0, NULL);
assert(!PyErr_Occurred());
```

For code and tests see:

- **C:** in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyTuple_SetItem_NULL()`
- **CPython:** in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyTuple_SetItem_NULL()`
- **Python:** in `tests/unit/test_c_ref_count.py`
 - `test_PyTuple_SetItem_NULL()`.

And:

```
PyObject *container = PyTuple_New(1);
assert(!PyErr_Occurred());
PyTuple_SET_ITEM(container, 0, NULL);
assert(!PyErr_Occurred());
```

For code and tests see:

- **C:** in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyTuple_SET_ITEM_NULL()`
- **CPython:** in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyTuple_SET_ITEM_NULL()`
- **Python:** in `tests/unit/test_c_ref_count.py`
 - `test_PyTuple_SET_ITEM_NULL()`.

Replacing NULL

Replacing a NULL will not cause an error, the original value is *abandoned* (see *Abandoned References*) and the replaced value reference is *stolen* (see “*Stolen*” *References*):

```
PyObject *container = PyTuple_New(1);
PyTuple_SetItem(container, 0, NULL);
PyObject *value = new_unique_string(__FUNCTION__, NULL); /* Ref Count of value is 1. */
PyTuple_SetItem(container, 0, value); /* Ref Count of value is still 1. */
```

For code and tests see:

- **C: in src/cpy/Containers/DebugContainers.c:**
 - dbg_PyTuple_SetItem_NULL_SetItem
- **CPython: in src/cpy/RefCount/cRefCount.c:**
 - test_PyTuple_SetItem_NULL_SetItem
- **Python: in tests/unit/test_c_ref_count.py:**
 - test_PyTuple_SetItem_NULL_SetItem().

And:

```
PyObject *container = PyTuple_New(1);
PyTuple_SET_ITEM(container, 0, NULL);
PyObject *value = new_unique_string(__FUNCTION__, NULL); /* Ref Count of value is 1. */
PyTuple_SET_ITEM(container, 0, value); /* Ref Count of value is still 1. */
```

For code and tests see:

- **C: in src/cpy/Containers/DebugContainers.c:**
 - dbg_PyTuple_SET_ITEM_NULL_SET_ITEM()
- **CPython: in src/cpy/RefCount/cRefCount.c:**
 - test_PyTuple_SET_ITEM_NULL_SET_ITEM()
- **Python: in tests/unit/test_c_ref_count.py:**
 - test_PyTuple_SET_ITEM_NULL_SET_ITEM().

4.3.4 PyTuple_Pack()

`PyTuple_Pack()` takes a length and a variable argument list of PyObjects. Each of those PyObjects reference counts will be incremented. In that sense it behaves as `Py_BuildValue()`.

Note

`PyTuple_Pack()` is implemented as a separate low level routine, it does not invoke `PyTuple_SetItem()` or `PyTuple_SET_ITEM()`.

For example:

```
PyObject *value_a = new_unique_string(__FUNCTION__, NULL);
PyObject *value_b = new_unique_string(__FUNCTION__, NULL);
PyObject *container = PyTuple_Pack(2, value_a, value_b);
```

(continues on next page)

(continued from previous page)

```

assert(Py_REFCNT(value_a) == 2);
assert(Py_REFCNT(value_b) == 2);

Py_DECREF(container);

/* Leaks: */
assert(Py_REFCNT(value_a) == 1);
assert(Py_REFCNT(value_b) == 1);
/* Fix leaks: */
Py_DECREF(value_a);
Py_DECREF(value_b);

```

For code and tests see:

- **C:** in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyTuple_PyTuple_Pack`
- **CPython:** in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyTuple_Py_PyTuple_Pack`
- **Python:** in `tests/unit/test_c_ref_count.py`:
 - `test_PyTuple_Py_PyTuple_Pack()`.

4.3.5 Py_BuildValue()

`Py_BuildValue()` is a very convenient way to create tuples, lists and dictionaries. `Py_BuildValue("(O)", value);` will increment the refcount of `value` and this can, potentially, leak:

```

PyObject *value = new_unique_string(__FUNCTION__, NULL);
/* value reference count is 1. */
PyObject *container = Py_BuildValue("(O)", value);
/* value reference count is incremented to 2. */
assert(Py_REFCNT(value) == 2);

Py_DECREF(container);
/* value reference count is decremented to 1. */
assert(Py_REFCNT(value_a) == 1);

/* value is leaked if Py_DECREF(value) is not called. */
/* Fix leak. */
Py_DECREF(value);

```

For code and tests see:

- **C:** in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyTuple_Py_BuildValue()`
- **CPython:** in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyTuple_Py_BuildValue`
- **Python:** in `tests/unit/test_c_ref_count.py`:
 - `test_PyTuple_Py_BuildValue()`.

4.3.6 Tuple Getters

There are these APIs for getting an item from a tuple:

- `PyTuple_GetItem()`, it returns a borrowed reference and will error if the supplied container is not tuple or the index is negative or out of range.
- `PyTuple_GET_ITEM()`, it returns a borrowed reference and there is no error checking for the container being a tuple or the index being in range. The type checking is performed as an assertion if Python is built in `debug` mode or with assertions. If not the results are undefined.

4.3.7 Summary

- `PyTuple_SetItem()` and `PyTuple_SET_ITEM()` *steal* references.
- `PyTuple_SetItem()` and `PyTuple_SET_ITEM()` behave differently when replacing an existing, different, value.
- `PyTuple_SetItem()` and `PyTuple_SET_ITEM()` behave differently when replacing the *same* value. In particular `PyTuple_SetItem()` can produce undefined behaviour.
- If `PyTuple_SetItem()` errors it will decrement the reference count of the given value which can produce undefined behaviour.
- `PyTuple_Pack()` and `Py_BuildValue()` increment reference counts and thus may leak.

4.4 Lists

4.4.1 PyList_SetItem() and PyList_SET_ITEM()

`PyList_SetItem()` and `PyList_SET_ITEM()` behave identically to their equivalents `PyTuple_SetItem()` (link *PyTuple_SetItem()*) and `PyTuple_SET_ITEM()` (link *PyTuple_SET_ITEM()*).

Note that, as with tuples, `PyList_SetItem()` and `PyList_SET_ITEM()` behave differently on replacement of values (see *PyTuple_SET_ITEM() Replacement*). The Python documentation on `PyList_SET_ITEM()` correctly identifies when a leak can occur (unlike `PyTuple_SET_ITEM()`).

On replacement with `PyList_SetItem()` heed the warning in *PyTuple_SetItem() Replacement* as `PyList_SetItem()` holds the same danger.

`Py_BuildValue()` also behaves identically, as far as reference counts are concerned, with Lists as it does with Tuples (see *Py_BuildValue()*).

4.4.2 PyList_Append()

`PyList_Append()` (a C function) adds an object onto the end of a list with error checking. This increments the reference count of the given value which will be decremented on container destruction. For example:

```
PyObject *container = PyList_New(0);
PyObject *value = new_unique_string(__FUNCTION__, NULL);
PyList_Append(container, value);
assert(Py_REFCNT(value) == 2);
Py_DECREF(container);
/* A leak unless decrefd. */
assert(Py_REFCNT(value) == 1);
```

`PyList_Append()` uses `PyList_SET_ITEM()` in its implementation. `PyList_Append()` can fail for two reasons:

- The given container is not a list.
- The given value is NULL.

On failure the reference count of value is unchanged and a `SystemError` is raised with the text “bad argument to internal function”.

For code and tests, including failure modes, see:

- C: `dbg_PyList_Append...` in `src/cpy/Containers/DebugContainers.c`.
- CPython: `test_PyList_Append...` in `src/cpy/RefCount/cRefCount.c`.
- Python: `tests.unit.test_c_ref_count.test_PyList_Append` etc.

4.4.3 `PyList_Insert()`

`PyList_Insert()` (a C function) inserts an object before a specific index in a list with error checking. This increments the reference count of the given value which will be decremented on container destruction.

`PyList_Insert()` is equivalent to `list.insert(...)`, for example:

```
>>> l = []
>>> l.insert(123, "Hello")
>>> l.insert(-123, "World")
>>> l
['World', 'Hello']
```

`PyList_Insert()` can fail for two reasons:

- The given container is not a list.
- The given value is NULL.

Note

`PyList_Insert()` does not fail because of any value of the index, either positive or negative (this is not identified in the Python documentation).

On failure the reference count of value is unchanged, `PyList_Insert()` returns -1, and a `SystemError` is set with the text “bad argument to internal function”.

For code and tests, including failure modes, see:

- C: `dbg_PyList_Insert...` in `src/cpy/Containers/DebugContainers.c`.
- CPython: `test_PyList_Insert...` in `src/cpy/RefCount/cRefCount.c`.
- Python: `tests.unit.test_c_ref_count.test_PyList_Insert` etc.

Note

The Python documentation does not mention (but implies) that if the index is greater than the list length then the value is appended to the list.

For example (`dbg_PyList_Insert_Is_Truncated()` in `src/cpy/Containers/DebugContainers.c`):

```
PyObject *container = PyList_New(0);
PyObject *value = new_unique_string(__FUNCTION__, NULL);
// Insert before index 4
if (PyList_Insert(container, 4L, value)) {
    assert(0);
}
```

```
assert(Py_REFCNT(value) == 2);
PyObject *get_item;
// PyList_Insert at 4 actually inserts at 0.
assert(PyList_GET_SIZE(container) == 1L);
get_item = PyList_GET_ITEM(container, 0L);
assert(get_item == value);
```

Note

Neither the Python documentation for `PyList_Insert()` or `list.insert()` does not make this clear that index can be negative in which case the index is calculated from the end. If that index calculation is less than zero it is truncated to zero.

For example (`dbg_PyList_Insert_Negative_Index()` in `src/cpy/Containers/DebugContainers.c`):

```
PyObject *container = PyList_New(0);
PyObject *value = new_unique_string(__FUNCTION__, NULL);
// Insert at end
if (PyList_Insert(container, -1L, value)) {
    assert(0);
}
assert(Py_REFCNT(value) == 2);
PyObject *get_item;
// PyList_Insert at -1 actually inserts at 0.
assert(PyList_GET_SIZE(container) == 1L);
get_item = PyList_GET_ITEM(container, 0L);
assert(get_item == value);
```

4.4.4 Py_BuildValue()

As with tuples `Py_BuildValue()` is a very convenient way to create lists. `Py_BuildValue("[0]", value);` will increment the refcount of `value` and this can, potentially, leak.

4.4.5 List Getters

There are these APIs for getting an item from a list:

- `PyList_GetItem()` This is very similar to `PyTuple_GetItem()`. It returns a borrowed reference and will error if the supplied container is not list or the index is negative or out of range.
- `PyList_GET_ITEM()` This is very similar to `PyTuple_GET_ITEM()`. It returns a borrowed reference and there is no error checking for the index being in range. The type checking is performed as an assertion if Python is built in `debug mode` or `with assertions`. If not the results are undefined.
- `PyList_GetItemRef()` [From Python 3.13 onwards]. Like `PyList_GetItem()` but this returns a new *strong* (i.e. incremented) reference to the existing object.

4.4.6 Summary

- `PyList_SetItem()` and `PyList_SET_ITEM()` *steal* references.
- `PyList_SetItem()` and `PyList_SET_ITEM()` behave differently when replacing an existing, different, value.

- `PyList_SetItem()` and `PyList_SET_ITEM()` behave differently when replacing the *same* value. In particular `PyList_SetItem()` can produce undefined behaviour.
- If `PyList_SetItem()` errors it will decrement the reference count of the given value which can produce undefined behaviour.
- `PyList_Append()` Increments the reference count of the given object and thus may leak.
- `PyList_Insert()` Increments the reference count of the given object and thus may leak.
- `Py_BuildValue()` Increments the reference count of the given object and thus may leak.

4.5 Dictionaries

This section describes how reference counts are affected when building and accessing dictionaries.

4.5.1 PyDict_SetItem()

The Python documentation for `PyDict_SetItem()` is incomplete. `PyDict_SetItem()` changes the key and value reference counts according to these rules:

- If the key exists in the dictionary then key's reference count remains the same.
- If the key does *not* exist in the dictionary then its reference count will be incremented.
- The value's reference count will always be incremented.
- If the key exists in the dictionary then the previous value reference count will be decremented before the value is replaced by the new value (and the new value reference count is incremented). See [Discarded References](#). If the key exists in the dictionary and the value is the same then this means, effectively, that reference counts of both key and value remain unchanged.

Warning

If either the key or the value are NULL this will segfault. See `dbg_PyDict_SetItem_NULL_key()` and `dbg_PyDict_SetItem_NULL_value()` in `src/cpy/Containers/DebugContainers.c`.

This code illustrates `PyDict_SetItem()` with `assert()` showing the reference count:

```
PyObject *container = PyDict_New();
PyObject *key = new_unique_string(__FUNCTION__, NULL);
PyObject *value_a = new_unique_string(__FUNCTION__, NULL);
assert(Py_REFCNT(key) == 1);
assert(Py_REFCNT(value_a) == 1);
/* Insert a new key value. */
if (PyDict_SetItem(container, key, value_a) {
    assert(0);
}
assert(Py_REFCNT(key) == 2);
assert(Py_REFCNT(value_a) == 2);
```

Now replace the value with another value:

```
/* Replace a value for the key. */
PyObject *value_b = new_unique_string(__FUNCTION__, NULL);
assert(Py_REFCNT(key) == 2);
```

(continues on next page)

(continued from previous page)

```

assert(Py_REFCNT(value_a) == 2);
assert(Py_REFCNT(value_b) == 1);
if (PyDict_SetItem(container, key, value_b)) {
    assert(0);
}
assert(Py_REFCNT(key) == 2);
assert(Py_REFCNT(value_a) == 1);
assert(Py_REFCNT(value_b) == 2);

```

Now replace the value with the same value, reference counts remain the same:

```

/* Replace with the same value for the key. */
if (PyDict_SetItem(container, key, value_b)) {
    assert(0);
}
assert(Py_REFCNT(key) == 2);
assert(Py_REFCNT(value_a) == 1);
assert(Py_REFCNT(value_b) == 2);

```

PyDict_SetItem() Failure

PyDict_SetItem() can fail for the following reasons:

- The container is not a dictionary (or a sub-class of a dictionary, see [PyDict_Check\(\)](#)).
- The key is not hashable ([PyObject_Hash\(\)](#) returns -1).
- If either the key or the value is NULL this will cause a SIGSEGV (or some other disaster). These are checked with asserts if Python is built in [debug mode](#) or [with assertions](#).

For code and tests see:

- **C:** in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyDict_SetItem_*`
- **CPython:** in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyDict_SetItem_*`
- **Python:** in `tests/unit/test_c_ref_count.py`:
 - `test_PyDict_SetItem_increments()`

Note

In `src/cpy/Containers/DebugContainers.c` there are failure tests that cause a SIGSEGV if `ACCEPT_SIGSEGV` is non zero. `ACCEPT_SIGSEGV` is defined in `src/cpy/Containers/DebugContainers.h`.

4.5.2 PyDict_SetDefault()

PyDict_SetDefault() is equivalent to the Python method `dict.setdefault()` in Python. The C function signature is:

```
PyObject *PyDict_SetDefault(PyObject *p, PyObject *key, PyObject *defaultobj);
```

The idea is that if the key exists then the appropriate value is returned and the default value is unused. If the key does *not* exist then the default value is inserted into the dictionary and returned.

If the Default Value is Unused

If the key already exists in the dictionary the reference counts of the key, existing value and default value are unchanged so the return value is a borrowed reference (see “*Borrowed*” *References*).

If the Default Value is Used

If the key does *not* exist in the dictionary the reference counts of the key and default value are incremented.

These reference count changes are not particularly clear from the official Python documentation.

For code and tests see:

- **C, in `src/cpy/Containers/DebugContainers.c`:**
 - `dbg_PyDict_SetDefault_default_unused()`
 - `dbg_PyDict_SetDefault_default_used()`
- **CPython, in `src/cpy/RefCount/cRefCount.c`.**
 - `test_PyDict_SetDefault_default_unused()`
 - `test_PyDict_SetDefault_default_used()`
- **Python, pytest, in `tests.unit.test_c_ref_count`:**
 - `test_PyDict_SetDefault_default_unused()`
 - `test_PyDict_SetDefault_default_used()`

4.5.3 `PyDict_SetDefaultRef()` [Python 3.13+]

`PyDict_SetDefaultRef()` sets a default value and retrieves the actual value. This is new in Python 3.13+. The C function signature is:

```
int PyDict_SetDefaultRef(  
    PyObject *dictionary, PyObject *key, PyObject *default_value, PyObject **result  
);
```

***result**

Any previous `*result` is always *abandoned* (see *Abandoned References*). To emphasise, there is no decrementing the reference count of the existing value (if any). This is important as the following code snippet shows:

```
PyObject *result; /* Refers to an arbitrary memory location. */  
/* Now if PyDict_SetDefaultRef() were to attempt to Py_DECREF(result)  
 * the results will be undefined.  
 */  
PyDict_SetDefaultRef(container, key, default_value, &result);
```

Key Exists

If the key already exists in the dictionary `PyDict_SetDefaultRef()` returns 1. The reference counts are changed as follows:

- key: unchanged.
- value: incremented by one
- default_value: unchanged.

`*result` is equal to the stored value.

For example:

```

1 PyObject *container = PyDict_New();
2 PyObject *key = new_unique_string(__FUNCTION__, NULL);
3 PyObject *val = new_unique_string(__FUNCTION__, NULL);
4 /* At this point the reference counts are:
5  * key: 1
6  * val: 1
7  */
8 // Set the key/value
9 PyDict_SetItem(container, key, val);
10 /* At this point the reference counts are:
11  * key: 2
12  * val: 2
13  */
14 // Create a default value and a result.
15 PyObject *default_value = new_unique_string(__FUNCTION__, NULL);
16 PyObject *result = NULL;
17 /* At this point the reference counts are:
18  * default_value: 1
19  * result: N/A
20  */
21 PyDict_SetDefaultRef(container, key, default_value, &result);
22 /* Now the reference counts are:
23  * key: 2
24  * val: 3
25  * default_value: 1
26  * result: 3 as it equals val.
27  */
28 Py_DECREF(container);
29 /* Now the reference counts are:
30  * key: 1
31  * val: 2
32  * default_value: 1
33  * result: 2 as it equals val.
34  */

```

For code and tests see:

- C, in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyDict_SetDefaultRef_default_unused()`
- CPython, in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyDict_SetDefaultRef_default_unused()`
- Python, `pytest`, in `tests.unit.test_c_ref_count`:
 - `test_PyDict_SetDefaultRef_default_unused()`

Key Does not Exist

If the key does not exist in the dictionary `PyDict_SetDefaultRef()` returns 0. The reference counts are changed as follows:

- `key`: incremented by one.
- `default_value`: incremented by *two*. The rationale is one increment as the `default_value` is inserted into the dictionary then a second increment as the `default_value` is ‘returned’ as `*result`.

*result is equal to the default_value.

For example:

```

1 PyObject *container = PyDict_New();
2 PyObject *key = new_unique_string(__FUNCTION__, NULL);
3 // Create a default value and a result.
4 PyObject *default_value = new_unique_string(__FUNCTION__, NULL);
5 PyObject *result = NULL;
6 /* At this point the reference counts are:
7  * key: 1
8  * default_value: 1
9  * result: N/A
10 */
11 PyDict_SetDefaultRef(container, key, default_value, &result);
12 /* Now the reference counts are:
13  * key: 2
14  * default_value: 3
15  * result: 3 as it equals default_value.
16 */
17 Py_DECREF(container);
18 /* Now the reference counts are:
19  * key: 1
20  * default_value: 2
21  * result: 2 as it equals default_value.
22 */

```

These reference count changes are not particularly clear from the official Python documentation.

For code and tests see:

- **C, in src/cpy/Containers/DebugContainers.c:**
 - dbg_PyDict_SetDefaultRef_default_used()
- **CPython, in src/cpy/RefCount/cRefCount.c:**
 - test_PyDict_SetDefaultRef_default_used()
- **Python, pytest, in tests.unit.test_c_ref_count:**
 - test_PyDict_SetDefaultRef_default_used()

Failure

Todo

PyDict_SetDefaultRef() failure modes.

4.5.4 PyDict_GetItem()

PyDict_GetItem() returns a borrowed reference (*“Borrowed” References*) to an existing value or NULL if the key does not exist in the dictionary.

Warning

If the key is NULL this will segfault. See `dbg_PyDict_GetItem_key_NULL()` in `src/cpy/Containers/DebugContainers.c`.

4.5.5 PyDict_GetItemRef() [Python 3.13+]

`PyDict_GetItemRef()` gets a new strong reference to a value from a dictionary.

The C signature is:

```
int PyDict_GetItemRef(PyObject *p, PyObject *key, PyObject **result);
```

Key is Present

If the key is in the dictionary then increment the reference count of the value and set `*result` to the value. The reference count of the key is unchanged. The function returns 1.

For code and tests see:

- C, in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyDict_GetItemRef()`
- CPython, in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyDict_SetDefaultRef_default_used()`
- Python, `pytest`, in `tests.unit.test_c_ref_count`:
 - `test_PyDict_SetDefaultRef_default_used()`

Failure

Todo

`PyDict_GetItemRef()` failure modes.

4.5.6 PyDict_Pop() [Python 3.13+]

`PyDict_Pop()` removes a specific value. This is new in Python 3.13+. The C function signature is:

```
int int PyDict_Pop(PyObject *p, PyObject *key, PyObject **result);
```

`*result`

Any previous `*result` is always *abandoned* (see [Abandoned References](#)). To emphasise, there is no decrementing the reference count of the existing value (if any). This is important as the following code snippet shows:

```
PyObject *result; /* Refers to an arbitrary memory location. */
/* Now if PyDict_Pop() were to attempt to Py_DECREF(result)
 * the results will be undefined.
 */
PyDict_Pop(container, key, &result);
```

Key Exists

If the key already exists in the dictionary `PyDict_Pop()` returns 1. The reference counts are changed as follows:

- key: decremented by one
- value: unchanged.

`*result` is equal to the stored value.

For example:

```
1 PyObject *container = PyDict_New();
2 PyObject *key = new_unique_string(__FUNCTION__, NULL);
3 PyObject *val = new_unique_string(__FUNCTION__, NULL);
4 /* At this point the reference counts are:
5  * key: 1
6  * val: 1
7  */
8 // Set the key/value
9 PyDict_SetItem(container, key, val);
10 /* At this point the reference counts are:
11  * key: 2
12  * val: 2
13  */
14 PyObject *result;
15 PyDict_Pop(container, key, &result);
16 /* Now the reference counts are:
17  * key: 1
18  * val: 2
19  * result: 2 as it equals val.
20  */
```

For code and tests see:

- C, in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyDict_Pop_key_present()`
- CPython, in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyDict_Pop_key_present()`
- Python, `pytest`, in `tests.unit.test_c_ref_count`:
 - `test_PyDict_Pop_key_present()`

Key Does not Exist

If the key does not exist in the dictionary `PyDict_Pop()` returns 0. The reference counts are changed as follows:

- key: unchanged.

`*result` is set to `NULL`, any previous value is *abandoned* (see *Abandoned References*).

For example:

```
PyObject *container = PyDict_New();
PyObject *key = new_unique_string(__FUNCTION__, NULL);
```

(continues on next page)

(continued from previous page)

```
// Create a default value and a result.
PyObject *dummy_value = new_unique_string(__FUNCTION__, NULL);
PyObject *result = dummy_value;
/* At this point the reference counts are:
 * key: 1
 * dummy_value: 1
 * result: 1 as it is equal to dummy_value.
 */
PyDict_Pop(container, key, &result);
/* Now the reference counts are:
 * key: 1
 * dummy_value: 1
 * result: is NULL.
 */
Py_DECREF(container);
/* No change in the reference counts. */
```

For code and tests see:

- C, in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyDict_Pop_key_absent()`
- CPython, in `src/cpy/RefCount/cRefCount.c`:
 - `test_PyDict_Pop_key_absent()`
- Python, `pytest`, in `tests.unit.test_c_ref_count`:
 - `test_PyDict_Pop_key_absent()`

Failure

Todo

Finish Dictionary `PyDict_Pop()` Failure

4.5.7 Other APIs

This section describes other dictionary APIs that are simple to describe and have no complications.

Note

There are no tests for many of these APIs in the current version of this project.

Todo

Finish Dictionary “Other APIs”

PyDict_GetItemWithError()

`PyDict_GetItemWithError()` gets a new *borrowed* reference to a value from a dictionary or NULL. Unlike `PyDict_GetItem()` this will set an exception if appropriate.

The C signature is:

```
PyObject *PyDict_GetItemWithError(PyObject *p, PyObject *key);
```

Currently, the only failure mode is if the first argument is not a dictionary.

For code and tests see:

- C, in `src/cpy/Containers/DebugContainers.c`:
 - `dbg_PyDict_GetItemWithError_fails()`

PyDict_DelItem()

`PyDict_DelItem()` removes a specific value if it exists. The reference count of the value will be decremented. The key must be hashable; if it isn't a `TypeError` is set. If key is not in the dictionary a `KeyError` is set. Returns 0 on success or -1 on failure in which case an exception will have been set.

The C function signature is:

```
int PyDict_DelItem(PyObject *p, PyObject *key);
```

Todo

Complete `PyDict_DelItem()` with code examples.

PyDict_Items()

`PyDict_Items()` returns a *new* Python list containing *new* tuples of (key, value). Each key and value will have their reference count incremented. That is to say calling `Py_DECREF` on the result will change all the reference counts within the dictionary to their previous values.

The C function signature is:

```
PyObject *PyDict_Items(PyObject *p);
```

Todo

Complete `PyDict_Items()` with code examples.

PyDict_Keys()

`PyDict_Keys()` returns a *new* Python list containing all the keys. Each key will have its reference count incremented. That is to say calling `Py_DECREF` on the result will change all the reference counts within the dictionary to their previous values.

The C function signature is:

```
PyObject *PyDict_Keys(PyObject *p);
```


 Todo

Complete PyDict_Keys() with code examples.

PyDict_Values()

`PyDict_Values()` returns a *new* Python list containing all the values. Each value will have its reference count incremented. That is to say calling `Py_DECREF` on the result will change all the reference counts within the dictionary to their previous values.

The C function signature is:

```
PyObject *PyDict_Values(PyObject *p);
```

 Todo

Complete PyDict_Values() with code examples.

PyDict_Next()

`PyDict_Next()` is the standard way of iterating through all the keys and values. Each key and value will be a *borrowed* reference. The C function signature is:

```
int PyDict_Next(PyObject *p, Py_ssize_t *ppos, PyObject **pkey, PyObject **pvalue);
```

 Todo

Complete PyDict_Next() with code examples.

Py_BuildValue()

`Py_BuildValue()` is a very convenient way to create dictionaries. `Py_BuildValue("{00}", key, value);` will increment the refcount of the key and value and this can, potentially, leak.

4.6 Sets

The set API is simple with no real difficulties for the user.

4.6.1 PySet_Add()

`PySet_Add()` is fairly straightforward. The set will increment the reference count of the value if it is *not* already in the set.

This returns 0 on success or -1 on failure in which case an exception will have been set which will be:

- A `SystemError` if set is not an instance of set or its subtype.
- A `TypeError` if the key is unhashable.
- A `MemoryError` if there is no room to grow.

Here is an example:

```
PyObject *container = PySet_New(NULL);
/* Py_REFCNT(container) is 1 */
PyObject *value = new_unique_string(__FUNCTION__, NULL);
/* Py_REFCNT(value) is 1 */
int ret_val = PySet_Add(container, value);
assert(ret_val == 0);
/* Py_REFCNT(value) is now 2 */

/* Clean up. */
Py_DECREF(container);
/* Py_REFCNT(value) is now 1 */
Py_DECREF(value);
```

When adding something that already exists in the set does *not* increment the reference count:

```
PyObject *container = PySet_New(NULL);
/* Py_REFCNT(container) is 1 */
PyObject *value = new_unique_string(__FUNCTION__, NULL);
/* Py_REFCNT(value) is 1 */
int ret_val = PySet_Add(container, value);
assert(ret_val == 0);
/* Py_REFCNT(value) is now 2 */

/* Add duplicate. */
ret_val = PySet_Add(container, value);
assert(ret_val == 0);
/* Py_REFCNT(value) is still 2 */

/* Clean up. */
Py_DECREF(container);
/* Py_REFCNT(value) is now 1 */
Py_DECREF(value);
```

4.6.2 PySet_Discard()

`PySet_Discard()` is also fairly straightforward. The set will discard (*Discarded References*) the value thus:

```
PyObject *container = PySet_New(NULL);
/* Py_REFCNT(container) is 1 */
PyObject *value = new_unique_string(__FUNCTION__, NULL);
/* Py_REFCNT(value) is 1 */
int ret_val = PySet_Add(container, value);
assert(ret_val == 0);
/* Py_REFCNT(value) is now 2 */

/* Discard. */
ret_val = PySet_Discard(container, value);
assert(ret_val == 1);
/* Py_REFCNT(value) is now 1 */

/* Clean up. */
Py_DECREF(container);
/* Py_REFCNT(value) is still 1 */
```

(continues on next page)

(continued from previous page)

```
Py_DECREF(value);
```

4.6.3 PySet_Pop()

`PySet_Pop()` will return a *new* reference to the existing object and it is up to caller to decrement the reference count appropriately.

For example, the reference counts work as follows:

```
PyObject *container = PySet_New(NULL);
/* Py_REFCNT(container) is 1 */
PyObject *value = new_unique_string(__FUNCTION__, NULL);
/* Py_REFCNT(value) is 1 */
int ret_val = PySet_Add(container, value);
assert(ret_val == 0);
/* Py_REFCNT(value) is now 2 */

/* Pop. */
PyObject *popped_value = PySet_Pop(container);
assert(popped_value == value);
/* Py_REFCNT(value) is still 2 */

/* Clean up. */
Py_DECREF(container);
/* Py_REFCNT(value) is still 2 so need to double Py_DECREF calls. */
Py_DECREF(value);
Py_DECREF(value);
```

So this is how `PySet_Pop()` might be used in practice:

```
1 void add_to_set(PyObject *container) {
2     PyObject *value = new_unique_string(__FUNCTION__, NULL);
3     /* Py_REFCNT(value) is 1 */
4     int ret_val = PySet_Add(container, value);
5     assert(ret_val == 0);
6     /* Py_REFCNT(value) is now 2 */
7     /* Decrement our local value */
8     Py_DECREF(value);
9     /* Now the container has the only reference to the value. */
10 }
11
12 void pop_from_set(PyObject *container) {
13     PyObject *value = PySet_Pop(container);
14     /* Do something with the value... */
15
16     /* Then as we 'own' value we need to free it. */
17     Py_DECREF(value);
18 }
19
20 PyObject *container = PySet_New(NULL);
21 add_to_set(container);
22 pop_from_set(container);
```

(continues on next page)

```

23  /* Clean up. */
24  Py_DECREF(container);

```

4.7 Summary

This summarises where the Python documentation is wrong, absent or misleading and where you might get into trouble.

4.7.1 Tuples

- `PyTuple_SetItem()` will lead to a SIGSEGV if the existing value is the same as the inserted value. See [PyTuple_SetItem\(\)](#).
- On failure `PyTuple_SetItem()` will decrement the reference count of the given value and this might well lead to a SIGSEGV. See [PyTuple_SetItem\(\) Failures](#).
- `PyTuple_SET_ITEM()` will lead to a memory leak when replacing an existing value as it *abandons* the previous reference. See [PyTuple_SET_ITEM\(\)](#).
- `PyTuple_SET_ITEM()` does no error checking so is capable of writing to arbitrary memory locations on error. See [PyTuple_SET_ITEM\(\) Failures](#).
- Both `PyTuple_SetItem()` and `PyTuple_SET_ITEM()` accept NULL as the value argument and behave as documented.
- `PyTuple_Pack()` will leak the given arguments. See [PyTuple_Pack\(\)](#).
- `Py_BuildValue()` will leak the given arguments when used with the "O" argument. See [Py_BuildValue\(\)](#).
- `PyTuple_GET_ITEM()` can try to access arbitrary memory locations if the arguments are wrong. See [Tuple Getters](#).

4.7.2 Lists

- `PyList_SetItem()` will lead to a SIGSEGV if the existing value is the same as the inserted value. See [PyList_SetItem\(\)](#) and [PyList_SET_ITEM\(\)](#).
- On failure `PyList_SetItem()` will decrement the reference count of the given value and this might well lead to a SIGSEGV. See [PyList_SetItem\(\)](#) and [PyList_SET_ITEM\(\)](#).
- `PyList_SET_ITEM()` will lead to a memory leak when replacing an existing value as it *abandons* the previous reference. See [PyList_SetItem\(\)](#) and [PyList_SET_ITEM\(\)](#).
- `PyList_SET_ITEM()` does no error checking so is capable of writing to arbitrary memory locations on error. See [PyList_SetItem\(\)](#) and [PyList_SET_ITEM\(\)](#).
- Both `PyList_SetItem()` and `PyList_SET_ITEM()` accept NULL as the value argument and behave as documented.
- `PyList_Append()` will increment the reference count of the given argument and so may leak. `PyList_Append()` uses `PyList_SET_ITEM()` in its implementation so those appropriate warnings apply. See [PyList_Append\(\)](#) and [PyList_SetItem\(\)](#) and [PyList_SET_ITEM\(\)](#).
- `PyList_Insert()` is poorly documented in the official Python documentation. See [PyList_Insert\(\)](#).
- `Py_BuildValue()` will leak the given arguments when used with the "O" argument. Similar to tuples, describe here: [Py_BuildValue\(\)](#).
- `PyList_GET_ITEM()` can try to access arbitrary memory locations if the arguments are wrong. See [List Getters](#).

4.7.3 Dictionaries

- `PyDict_SetItem()` generally increments the reference count of the given key/value however this API has some complicated rules about key/value reference counts that are not described in the Python documentation. See [PyDict_SetItem\(\)](#).
- With `PyDict_SetItem()` if either the key or value is NULL there will be a SIGSEGV. See [PyDict_SetItem\(\)](#).
- Otherwise `PyDict_SetItem()` has generally graceful behaviour on failure. See [PyDict_SetItem\(\) Failure](#).
- `PyDict_SetDefault()` has generally graceful behaviour on failure. See [PyDict_SetDefault\(\)](#).
- `PyDict_GetItemWithError()` is incorrectly implemented or documented. See [PyDict_GetItemWithError\(\)](#).

Todo

Complete the summary of dictionary APIs and their documentation lacunae.

4.7.4 Sets

The set API is much cleaner than the others and contains few gotchas. In summary:

- `PySet_Add()` will increment the reference count of the value if it is not already in the set. See [PySet_Add\(\)](#).
- `PySet_Discard()` will decrement the reference count of the value and returns it. It is up to the caller to decrement the reference count of the returned value. See [PySet_Discard\(\)](#).
- `PySet_Pop()` does not decrement the reference count of the returned value, it is *abandoned* (see [Abandoned References](#)). It is up to the caller to decrement the reference count of the returned value. See [PySet_Pop\(\)](#).

STRUCT SEQUENCE (A NAMEDTUPLE IN C)

A [Struct Sequence Object](#) is, more or less, the C equivalent of Python's [namedtuple](#) type¹.

As a reminder here is how named tuples work in Python:

```
>>> from collections import namedtuple
>>> nt_type = namedtuple('MyNamedTuple', ['field_one', 'field_two'])
>>> nt_type
<class '__main__.MyNamedTuple'>
>>> nt_type._fields
('field_one', 'field_two')
>>> nt = nt_type(['foo', 'bar'])
>>> nt.field_one
'foo'
>>> nt.index('bar')
1
```

The C [Struct Sequence API](#) allows you to define and create [Struct Sequence Objects](#) within C but act (almost) like `collections.namedtuple` objects. These are very useful in creating the equivalent of a C struct in Python.

5.1 Differences Between a C Struct Sequence and a Python namedtuple

Unlike a Python [namedtuple](#) a C Struct Sequence does *not* have the following functions and attributes (the official Python documentation does not point this out):

- `_make()`
- `_asdict()`
- `_replace()`
- `_fields`
- `_field_defaults`

[Struct Sequence Objects](#) also differ from [namedtuples](#) in the way that members can be accessed. [namedtuples](#) can access *all* their members either by name or by index. A [Struct Sequence Object](#) can be designed so that any attribute can be accessed by either name or index or both (or even neither!).

¹ [namedtuples](#) have been largely superseded by [dataclasses](#) and there is no direct C equivalent for them.

5.2 A Basic C Struct Sequence

Here is an example of defining a Struct Sequence in C (the code is in `src/cpy/StructSequence/cStructSequence.c`).

5.2.1 Documentation String

First create a named documentation string:

```
PyDoc_STRVAR(
    BasicNT_docstring,
    "A basic named tuple type with two fields."
);
```

5.2.2 Field Specifications

Now create the field definitions as an array of `PyStructSequence_Field`. These are just pairs of `{field_name, field_description}`:

```
static PyStructSequence_Field BasicNT_fields[] = {
    {"field_one", "The first field of the named tuple."},
    {"field_two", "The second field of the named tuple."},
    {NULL, NULL}
};
```

5.2.3 Struct Sequence Type Specification

Now create the `PyStructSequence_Desc` that is a name, documentation, fields and the number of fields visible in Python. The latter value is explained later but for the moment make it the number of declared fields.

```
static PyStructSequence_Desc BasicNT_desc = {
    "cStructSequence.BasicNT",
    BasicNT_docstring,
    BasicNT_fields,
    2,
};
```

Note

If the given number of fields (`n_in_sequence`) is greater than the length of the fields array then `PyStructSequence_NewType()` will return `NULL`.

There is a test example of this `dbg_PyStructSequence_n_in_sequence_too_large()` in `src/cpy/Containers/DebugContainers.c`.

5.2.4 Creating an Instance

Here is a function `BasicNT_create()` that creates a Struct Sequence from arguments provided from a Python session. Things to note:

- There is a static `PyTypeObject` which holds a reference to the Struct Sequence type.
- This is initialised with `PyStructSequence_NewType()` that takes the `BasicNT_desc` described above.

- Then the function `PyStructSequence_New()` is used to create a new, empty, instance of that type.
- Finally `PyStructSequence_SetItem()` is used to set each individual field from the given arguments.

Note

The careful use of `Py_INCREF` when setting the fields.

```

1 static PyObject *
2 BasicNT_create(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwargs) {
3     assert(!PyErr_Occurred());
4     static char *kwlist[] = {"field_one", "field_two", NULL};
5     PyObject *field_one = NULL;
6     PyObject *field_two = NULL;
7     static PyTypeObject *static_BasicNT_Type = NULL;
8
9     if (!PyArg_ParseTupleAndKeywords(args, kwargs, "OO", kwlist, &field_one, &field_two)) {
10         return NULL;
11     }
12     /* The two fields are PyObjects. If your design is that those arguments should be
13      * specific types then take the opportunity here to test that they are the
14      * expected types.
15      */
16
17     /* Now check that the type is available. */
18     if (!static_BasicNT_Type) {
19         static_BasicNT_Type = PyStructSequence_NewType(&BasicNT_desc);
20         if (!static_BasicNT_Type) {
21             PyErr_SetString(
22                 PyExc_MemoryError,
23                 "Can not initialise a BasicNT type with PyStructSequence_NewType()"
24             );
25             return NULL;
26         }
27     }
28     PyObject *result = PyStructSequence_New(static_BasicNT_Type);
29     if (!result) {
30         PyErr_SetString(
31             PyExc_MemoryError,
32             "Can not create a Struct Sequence with PyStructSequence_New()"
33         );
34         return NULL;
35     }
36     /* PyArg_ParseTupleAndKeywords with "O" gives a borrowed reference.
37      * https://docs.python.org/3/c-api/arg.html#other-objects
38      * "A new strong reference to the object is not created
39      * (i.e. its reference count is not increased)."
40      *
41      * So we increment as PyStructSequence_SetItem seals the reference otherwise
42      * if the callers arguments goes out of scope we will/may get undefined behaviour
43      * when accessing the named tuple fields.
44      */

```

(continues on next page)

(continued from previous page)

```

45     Py_INCREF(field_one);
46     Py_INCREF(field_two);
47     PyStructSequence_SetItem(result, 0, field_one);
48     PyStructSequence_SetItem(result, 1, field_two);
49     return result;
50 }

```

This function is then added to the `cStructSequence` module like this:

```

static PyMethodDef cStructSequence_methods[] = {
    /* More stuff here... */
    {
        "BasicNT_create",
        (PyCFunction) BasicNT_create,
        METH_VARARGS | METH_KEYWORDS,
        "Create a BasicNT from the given values."
    },
    /* More stuff here... */
    {NULL, NULL, 0, NULL} /* Sentinel */
};

```

5.2.5 Using an Instance

And can be used like this:

```

from cPyExtPatt import cStructSequence

def test_basic_nt_create():
    basic_nt = cStructSequence.BasicNT_create('foo', 'bar')
    assert str(type(basic_nt)) == "<class 'cStructSequence.BasicNT'>"

def test_basic_nt_create_attributes():
    basic_nt = cStructSequence.BasicNT_create('foo', 'bar')
    assert basic_nt.field_one == "foo"
    assert basic_nt.field_two == "bar"
    assert basic_nt.index("foo") == 0
    assert basic_nt.index("bar") == 1
    assert basic_nt.n_fields == 2
    assert basic_nt.n_sequence_fields == 2
    assert basic_nt.n_unnamed_fields == 0

```

5.3 Whether to Provide Access to the Type from Python

One decision to be made is whether to expose your Struct Sequence *type* from the module. There are only two use cases for this:

- Do you want the user to be able to create your Struct Sequence/namedtuple directly from Python? In which case then you need to expose the type of your Struct Sequence from the module. Then anyone can create these objects directly from Python.
- If the objects are created only in C then you do not need to expose the *type* in the module but you can create functions the create those Python objects (and their types) dynamically.

Firstly, exposing the Struct Sequence type to Python.

5.3.1 Exposing the Type from the CPython Module

In this case the Struct Sequence can be created *from* Python (as well as from C).

For example here is a simple Struct Sequence `cStructSequence.NTRegistered` that contains two fields.

First, creating the documentation:

```
PyDoc_STRVAR(
    NTRegistered_docstring,
    "A namedtuple type with two fields that is"
    "registered with the cStructSequence module."
);
```

Defining the fields, this is an array of `PyStructSequence_Field` which are pairs of {field_name, field_documentation} (both strings) and terminated with a NULL sentinel:

```
static PyStructSequence_Field NTRegistered_fields[] = {
    {"field_one", "The first field of the namedtuple."},
    {"field_two", "The second field of the namedtuple."},
    {NULL, NULL}
};
```

Creating the Struct Sequence description that will define the type. This is a `PyStructSequence_Desc` that consists of:

- The Struct Sequence name, this must include the module name so is of the form "module_name.struct_sequence_name":
- The documentation string.
- The fields as an array of `PyStructSequence_Field`.
- The number of fields exposed to Python.

```
static PyStructSequence_Desc NTRegistered_desc = {
    "cStructSequence.NTRegistered",
    NTRegistered_docstring,
    NTRegistered_fields,
    2,
};
```

Then the module initialisation code looks like this, this uses `PyStructSequence_NewType()` to create the type:

```
PyMODINIT_FUNC
PyInit_cStructSequence(void) {
    PyObject *m;
    m = PyModule_Create(&cStructSequence_cmodule);
    if (m == NULL) {
        return NULL;
    }
    /* Initialise NTRegisteredType */
    PyObject *NTRegisteredType = (PyObject *) PyStructSequence_NewType(
        &NTRegistered_desc
    );
    if (NTRegisteredType == NULL) {
```

(continues on next page)

(continued from previous page)

```

    Py_DECREF(m);
    return NULL;
}
Py_INCREF(NTRegisteredType);
PyModule_AddObject(m, "NTRegisteredType", NTRegisteredType);

/*
 * Other module initialisation code here.
 */

return m;
}

```

This can be used thus in Python:

```

from cPyExtPatt import cStructSequence

nt = cStructSequence.NTRegisteredType(('foo', 'bar'))
assert str(nt) == "cStructSequence.NTRegistered(field_one='foo', field_two='bar')"

```

There are tests for this in tests/unit/test_c_struct_sequence.py.

5.3.2 Hiding the Type in the Module

In this case the Struct Sequence can be *not* be created from Python, only in C. Even though the constructor is not accessible from Python the type is, as we will see.

For example here is a simple Struct Sequence `cStructSequence.NTUnRegistered` that contains two fields. It is, initially, very similar to the above.

```

PyDoc_STRVAR(
    NTUnRegistered_docstring,
    "A namedtuple type with two fields that is"
    " not registered with the cStructSequence module."
);

static PyStructSequence_Field NTUnRegistered_fields[] = {
    {"field_one", "The first field of the namedtuple."},
    {"field_two", "The second field of the namedtuple."},
    {NULL, NULL}
};

static PyStructSequence_Desc NTUnRegistered_desc = {
    "cStructSequence.NTUnRegistered",
    NTUnRegistered_docstring,
    NTUnRegistered_fields,
    2,
};

```

However as the type is not initialised in the module definition it remains static to the module C code. It is, as yet, uninitialised:

```

/* Type initailised dynamically by NTUnRegistered_create(). */
static PyTypeObject *static_NTUnRegisteredType = NULL;

```

This type, and objects created from it, can be made with a function call, in this case taking variable and keyword arguments:

```

1  /* A function that creates a cStructSequence.NTUnRegistered dynamically. */
2  PyObject *
3  NTUnRegistered_create(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwds) {
4      assert(!PyErr_Occurred());
5      static char *kwlist[] = {"field_one", "field_two", NULL};
6      PyObject *field_one = NULL;
7      PyObject *field_two = NULL;
8
9      if (!PyArg_ParseTupleAndKeywords(args, kwds, "OO", kwlist, &field_one, &field_two)) {
10         return NULL;
11     }
12     /* The two fields are PyObjects. If your design is that those arguments should be
13        * specific types then take the opportunity here to test that they are the
14        * expected types.
15        */
16
17     /* Initialise the static static_NTUnRegisteredType.
18        * Note: PyStructSequence_NewType returns a new reference.
19        */
20     if (!static_NTUnRegisteredType) {
21         static_NTUnRegisteredType = PyStructSequence_NewType(&NTUnRegistered_desc);
22         if (!static_NTUnRegisteredType) {
23             PyErr_SetString(
24                 PyExc_MemoryError,
25                 "Can not initialise a type with PyStructSequence_NewType()"
26             );
27             return NULL;
28         }
29     }
30     PyObject *result = PyStructSequence_New(static_NTUnRegisteredType);
31     if (!result) {
32         PyErr_SetString(
33             PyExc_MemoryError,
34             "Can not create a Struct Sequence with PyStructSequence_New()"
35         );
36         return NULL;
37     }
38     /* PyArg_ParseTupleAndKeywords with "O" gives a borrowed reference.
39        * https://docs.python.org/3/c-api/arg.html#other-objects
40        * "A new strong reference to the object is not created
41        * (i.e. its reference count is not increased)."
42        * So we increment as PyStructSequence_SetItem seals the reference otherwise if
43        * the callers arguments go out of scope we will/may get undefined behaviour when
44        * accessing the namedtuple fields.
45        */
46     Py_INCREF(field_one);
47     Py_INCREF(field_two);
48     PyStructSequence_SetItem(result, 0, field_one);
49     PyStructSequence_SetItem(result, 1, field_two);
50     return result;
51 }

```

And this can be used thus:

```
from cPyExtPatt import cStructSequence

ntu = cStructSequence.NTUnRegistered_create('foo', 'bar')
# Note that the type is available
assert str(type(ntu)) == "<class 'cStructSequence.NTUnRegistered'>"
```

A common use of this is converting a C struct to a Python namedtuple.

5.4 Converting a C struct to a namedtuple

A common use case for *not* exposing the namedtuple type from the module is when the data object can *only* be created in C. Suppose that we have a simple struct representing a transaction.

```
/**
 * Representation of a simple transaction.
 */
struct cTransaction {
    long id;           /* The transaction id. */
    char *reference;    /* The transaction reference. */
    double amount;     /* The transaction amount. */
};
```

And we have a C function that can recover a transaction given its ID:

```
/**
 * An example function that might recover a transaction from within C code,
 * possibly a C library.
 * In practice this will actually do something more useful than this function does!
 *
 * @param id The transaction ID.
 * @return A struct cTransaction corresponding to the transaction ID.
 */
static struct cTransaction get_transaction(long id) {
    struct cTransaction ret = {id, "Some reference.", 42.76};
    return ret;
}
```

Then we create a namedtuple type that mirrors the C struct Transaction:

```
PyDoc_STRVAR(
    cTransaction_docstring,
    "Example of a named tuple type representing a transaction created in C."
    " The type is not registered with the cStructSequence module."
);

static PyStructSequence_Field cTransaction_fields[] = {
    {"id", "The transaction id."},
    {"reference", "The transaction reference."},
    {"amount", "The transaction amount."},
    {NULL, NULL}
};
```

(continues on next page)

(continued from previous page)

```
static PyStructSequence_Desc cTransaction_desc = {
    "cStructSequence.cTransaction",
    cTransaction_docstring,
    cTransaction_fields,
    3,
};
```

This Python type is declared static and initialised dynamically when necessary. As this might be used by multiple functions so we give it an API:

```
/* Type initialised dynamically by get_cTransactionType(). */
static PyTypeObject *static_cTransactionType = NULL;

static PyTypeObject *get_cTransactionType(void) {
    if (static_cTransactionType == NULL) {
        static_cTransactionType = PyStructSequence_NewType(&cTransaction_desc);
        if (static_cTransactionType == NULL) {
            PyErr_SetString(
                PyExc_MemoryError,
                "Can not initialise a cTransaction type with PyStructSequence_NewType()"
            );
            return NULL;
        }
    }
    return static_cTransactionType;
}
```

Now the Python/C interface function:

```
/* A function that creates a cStructSequence.NTUnRegistered dynamically. */
PyObject *
cTransaction_get(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwds) {
    assert(!PyErr_Occurred());
    static char *kwlist[] = {"id", NULL};
    long id = 0;
    if (!PyArg_ParseTupleAndKeywords(args, kwds, "l", kwlist, &id)) {
        return NULL;
    }
    PyObject *result = PyStructSequence_New(get_cTransactionType());
    if (!result) {
        assert(PyErr_Occurred());
        return NULL;
    }

    struct cTransaction transaction = get_transaction(id);
    PyStructSequence_SetItem(result, 0, PyLong_FromLong(transaction.id));
    PyStructSequence_SetItem(result, 1, PyUnicode_FromString(transaction.reference));
    PyStructSequence_SetItem(result, 2, PyFloat_FromDouble(transaction.amount));
    return result;
}
```

Add to the module methods:

```
static PyMethodDef cStructSequence_methods[] = {
    /* Other stuff... */
    {
        "cTransaction_get",
        (PyCFunction) cTransaction_get,
        METH_VARARGS | METH_KEYWORDS,
        "Example of getting a transaction."
    },
    /* Other stuff... */
    {NULL, NULL, 0, NULL} /* Sentinel */
};
```

And then this can be called from Python like this:

```
nt = cStructSequence.cTransaction_get(17145)
assert nt.id == 17145
assert nt.reference == "Some reference."
assert nt.amount == 42.76
```

5.5 Controlling Member Access

Struct Sequence Objects differ from **namedtuples** in the way that members can be accessed. A **Struct Sequence Object** can be designed so that any attribute can be accessed by either name or index or both (or even neither!). This describes how to do this.

5.5.1 The Importance of the `n_in_sequence` Field

`PyStructSequence_Desc` has a field `n_in_sequence` which needs some explaining (the Python documentation is pretty silent on this). Normally `n_in_sequence` is equal to the number of fields, however what happens if it is not?

`n_in_sequence` > Number of Fields

As mentioned above if the given number of fields (`n_in_sequence`) is greater than the length of the fields array then `PyStructSequence_NewType()` will return `NULL`.

There is a test example of this `dbg_PyStructSequence_n_in_sequence_too_large()` in `src/cpy/Containers/DebugContainers.c`.

`n_in_sequence` < Number of Fields

In this case the members with an index $\geq n_in_sequence$ will raise an `IndexError`. However that same member can always be accessed from Python by name.

There some illustrative tests `test_excess_nt_*` in `tests/unit/test_c_struct_sequence.py` for this.

5.6 Unnamed Fields

The **Struct Sequence C API** provides for having *unnamed* fields. These are fields unavailable to Python code but available to C code. The rules using these are quite involved and the Python documentation is poor at describing them so I'll do my best here.

Note

Unnamed fields does not appear to work in Python versions prior to 3.11 with the following compile time error:

```
E ImportError: dlopen(... cStructSequence.cpython-310-darwin.so, 0x0002): symbol not
found in flat namespace '_PyStructSequence_UnnamedField'
```

The rules appear to be:

- Unnamed fields in the `PyStructSequence_Field` *must* follow named fields.
- Named fields in the `PyStructSequence_Field` can *not* follow unnamed fields.
- The `n_in_sequence` in the type `PyStructSequence_Desc` must be $\leq$ the number of *named* fields.

Failing to follow the rules will cause a `SystemError` when you use `repr()` or `str()` such as:

```
SystemError: In structseq_repr(), member 2 name is NULL for type module.struct_sequence_
↳ simple_with_unnamed_field
```

`PyStructSequence_UnnamedField` is the sentinel that describes the ‘name’ of the unnamed field. However you can not use it directly when declaring a `PyStructSequence_Field`:

```
static PyStructSequence_Field NTWithUnnamedField_fields[] = {
    {"field_one", "The first field of the named tuple."},
    {PyStructSequence_UnnamedField, "Documentation for an unnamed field."},
    {NULL, NULL}
};
```

This will fail with a compilation error along the lines of:

```
cStructSequence.c:391:10: fatal error: initializer element is not a compile-time constant
    {PyStructSequence_UnnamedField, "Documentation for an unnamed field."},
      ^~~~~~
1 error generated.
```

Instead declare the field name as `NULL`:

```
static PyStructSequence_Field NTWithUnnamedField_fields[] = {
    {"field_one", "The first field of the named tuple."},
    /* Use NULL then replace with PyStructSequence_UnnamedField
     * otherwise get an error "initializer element is not a compile-time constant" */
    {NULL, "Documentation for an unnamed field."},
    {NULL, NULL}
};
```

Then before the type is used the `NULL` needs to be replaced with `PyStructSequence_UnnamedField`. Here is the complete code that demonstrates initialising a static Struct Sequence type `static_NTWithUnnamedField_Type`.

5.6.1 Field Declaration

First declare the fields:

```
static PyStructSequence_Field NTWithUnnamedField_fields[] = {
    {"field_one", "The first field of the named tuple."},
```

(continues on next page)

(continued from previous page)

```

/* Use NULL then replace with PyStructSequence_UnnamedField
 * otherwise get an error "initializer element is not a compile-time constant" */
{NULL,      "Documentation for an unnamed field."},
{NULL, NULL}
};

```

Then the field description:

```

PyDoc_STRVAR(
    NTWithUnnamedField_docstring,
    "A basic named tuple type with an unnamed field."
);

static PyStructSequence_Desc NTWithUnnamedField_desc = {
    "cStructSequence.NTWithUnnamedField",
    NTWithUnnamedField_docstring,
    NTWithUnnamedField_fields,
    1, /* Of two fields only one available to Python by name. */
};

```

5.6.2 Type Declaration

Now the Struct Sequence type, for convenience the static is accessed with a *getter* that also serves as an initialisation function:

```

static PyObject *static_NTWithUnnamedField_Type = NULL;

/**
 * Initialises and returns the \c NTWithUnnamedField_Type.
 * @return The initialised type.
 */
static PyObject *get_NTWithUnnamedField_Type(void) {
    if (!static_NTWithUnnamedField_Type) {
        /* Substitute PyStructSequence_UnnamedField for NULL. */
        NTWithUnnamedField_fields[1].name = PyStructSequence_UnnamedField;
        /* Create and initialise the type. */
        static_NTWithUnnamedField_Type = PyStructSequence_NewType(
            &NTWithUnnamedField_desc
        );
        if (!static_NTWithUnnamedField_Type) {
            PyErr_SetString(
                PyExc_RuntimeError,
                "Can not initialise a NTWithUnnamedField type with PyStructSequence_
↪NewType()"
            );
            return NULL;
        }
    }
    return static_NTWithUnnamedField_Type;
}

```

5.6.3 Struct Sequence Creation

Here is the constructor that takes three arguments:

```

1  static PyObject *
2  NTWithUnnamedField_create(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwargs) {
3      assert(!PyErr_Occurred());
4      static char *kwlist[] = {"field_one", "field_two", "field_three", NULL};
5      PyObject *field_one = NULL;
6      PyObject *field_two = NULL;
7      PyObject *field_three = NULL;
8
9      if (
10         !PyArg_ParseTupleAndKeywords(
11             args, kwargs, "OOO", kwlist, &field_one, &field_two, &field_three
12         )
13     ) {
14         return NULL;
15     }
16     /* The three fields are PyObjects.
17      * If your design is that those arguments should be specific types
18      * then take the opportunity here to test that they are the expected types.
19      */
20
21     PyObject *result = PyStructSequence_New(get_NTWithUnnamedField_Type());
22     if (!result) {
23         PyErr_SetString(
24             PyExc_RuntimeError,
25             "Can not create a NTWithUnnamedField Struct"
26             " Sequence with PyStructSequence_New()"
27         );
28         return NULL;
29     }
30     /* PyArg_ParseTupleAndKeywords with "O" gives a borrowed reference.
31      * https://docs.python.org/3/c-api/arg.html#other-objects
32      * "A new strong reference to the object is not created
33      * (i.e. its reference count is not increased)."
34      * So we increment as PyStructSequence_SetItem seals the reference otherwise
35      * if the callers arguments go out of scope we will/may get undefined
36      * behaviour when accessing the named tuple fields.
37      */
38     Py_INCREF(field_one);
39     Py_INCREF(field_two);
40     Py_INCREF(field_three);
41     PyStructSequence_SetItem(result, 0, field_one);
42     PyStructSequence_SetItem(result, 1, field_two);
43     PyStructSequence_SetItem(result, 2, field_three);
44     assert(!PyErr_Occurred());
45     return result;
46 }

```

5.6.4 Access from Python

Once built this can be accessed from Python (see `tests/unit/test_c_struct_sequence.py`):

```
from cPyExtPatt import cStructSequence

ntuf = cStructSequence.NTWithUnnamedField_create('foo', 'bar', 'baz')
# All these tests are True
assert len(ntuf) == 1
assert ntuf.n_fields == 2
assert ntuf.n_sequence_fields == 1
assert ntuf.n_unnamed_fields == 1
assert tuple(ntuf) == ('foo',)
assert ntuf[0] == 'foo'
# Will raise an IndexError: 'tuple index out of range'
with pytest.raises(IndexError) as err:
    assert ntuf[1] == 'bar'
assert repr(ntuf) == "cStructSequence.NTWithUnnamedField(field_one='foo')"
assert str(ntuf) == "cStructSequence.NTWithUnnamedField(field_one='foo')"
```

EXCEPTION RAISING

This is a brief interlude on how to communicate error conditions from C code to Python. The example code for this is at `src/cpy/cExceptions.c` and the test examples are in `tests/unit/test_c_exceptions.py`.

These CPython calls are the most useful:

- `PyErr_SetString(...)` - To set an exception type with a fixed string. [PyErr_SetString\(\) documentation](#)
- `PyErr_Format(...)` - To set an exception type with a formatted string. [PyErr_Format\(\) documentation](#)
- `PyErr_Occurred()` - To check if an exception has already been set in the flow of control. This returns the current exception or NULL if nothing set. [PyErr_Occurred\(\) documentation](#)
- `PyErr_Clear()` - Clearing any set exceptions, have good reason to do this! [PyErr_Clear\(\) documentation](#)
- `PyErr_Print()` - Print a representation of the current exception then clear any set exceptions. [PyErr_Print\(\) documentation](#)

Indicating an error condition is a two stage process; your code must register an exception and then indicate failure by returning NULL. Here is a C function doing just that:

```
static PyObject *raise_error(PyObject *module) {
    PyErr_SetString(PyExc_ValueError, "Ooops.");
    return NULL;
}
```

You might want some dynamic information in the exception object, in that case `PyErr_Format` will do:

```
static PyObject *raise_error_formatted(PyObject *module) {
    PyErr_Format(PyExc_ValueError,
        "Can not read %d bytes when offset %d in byte length %d.", \
        12, 25, 32
    );
    return NULL;
}
```

If one of the two actions is missing then the exception will not be raised correctly. For example returning NULL without setting an exception type will raise a `SystemError`. For example:

```
/* Illustrate returning NULL but not setting an exception. */
static PyObject *raise_error_bad(PyObject *module) {
    return NULL;
}
```

Executing this from Python will produce a clear error message:

```
>>> cException.raise_error_bad()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
SystemError: error return without exception set
```

If the opposite error is made, that is setting an exception but not signalling then the function will succeed but leave a later runtime error:

```
static PyObject *raise_error_silent(PyObject *module) {
    PyErr_SetString(PyExc_ValueError, "ERROR: raise_error_mixup()");
    Py_RETURN_NONE;
}
```

The confusion can arise is that if a subsequent function then tests to see if an exception is set, if so signal it. It will appear to the Python interpreter that the error is coming from the second function when actually it is from the first:

```
static PyObject *raise_error_silent_test(PyObject *module) {
    if (PyErr_Occurred()) {
        return NULL;
    }
    Py_RETURN_NONE;
}
```

A common defensive pattern to use in functions to check an exception has been set by another function but not signalled is to use `assert(!PyErr_Occurred());` at the beginning of the function.

The other thing to note is that if there are multiple calls to `PyErr_SetString` only the last one counts:

```
static PyObject *raise_error_overwrite(PyObject *module) {
    /* This will be ignored. */
    PyErr_SetString(PyExc_RuntimeError, "FORGOTTEN.");
    PyErr_SetString(PyExc_ValueError, "ERROR: raise_error_overwrite()");
    assert(PyErr_Occurred());
    return NULL;
}
```

6.1 Common Exception Patterns

Here are some common use cases for raising exceptions.

6.1.1 Type Checking

A common requirement is to check the types of the arguments and raise a `TypeError` if they are wrong. Here is an example where we require a bytes object:

```
1 static PyObject*
2 function(PyObject *self, PyObject *arg) {
3     /* ... */
4     if (!PyBytes_Check(arg)) {
5         PyErr_Format(PyExc_TypeError,
6                     "Argument \"%value\" to %s must be a bytes object not a \"%s\"",
7                     __FUNCTION__, Py_TYPE(arg)->tp_name);
8         return NULL;
9     }
```

(continues on next page)

(continued from previous page)

```

9     }
10    /* ... */
11 }

```

That's fine if you have a macro such as `PyBytes_Check` and for your own types you can create a couple of suitable macros:

```

#define PyMyType_CheckExact(op) (Py_TYPE(op) == &PyMyType_Type) /* Exact match. */
#define PyMyType_Check(op) PyObject_TypeCheck(op, &PyMyType_Type) /* Exact or derived.
↪ */

```

Incidentally `PyObject_TypeCheck` is defined as:

```

#define PyObject_TypeCheck(ob, tp) \
    (Py_TYPE(ob) == (tp) || PyType_IsSubtype(Py_TYPE(ob), (tp)))

```

6.2 Creating Specialised Exceptions

Often you need to create an Exception class that is specialised to a particular module. The following C code is equivalent to the Python code:

```

class ExceptionBase(Exception):
    pass

class SpecialisedError(ExceptionBase):
    pass

```

6.2.1 Declaring Specialised Exceptions

Firstly declare the `PyObject *` exception:

```

/** Specialise exceptions base exception. */
static PyObject *ExceptionBase = NULL;
/** Specialise exceptions derived from base exception. */
static PyObject *SpecialisedError = NULL;

/* NOTE: Functions that might raise one of these exceptions will go here. See below. */

```

6.2.2 Example Module

Now define the module, `cExceptions_methods` is explained later:

```

static PyModuleDef cExceptions_module = {
    PyModuleDef_HEAD_INIT,
    "cExceptions",
    "Examples of raising exceptions.",
    -1,
    cExceptions_methods,
    NULL, NULL, NULL, NULL,
};

```

6.2.3 Initialising Specialised Exceptions

This can be done quite easily using either the `PyErr_NewException` or the `PyErr_NewExceptionWithDoc` functions. These create new exception classes that can be added to a module. For example, initialise the module, this registers the exception types and the class hierarchy:

```

1 PyMODINIT_FUNC
2 PyInit_cExceptions(void) {
3     PyObject *m = PyModule_Create(&cExceptions_module);
4     if (m == NULL) {
5         return NULL;
6     }
7     /* Initialise exceptions here.
8      *
9      * Firstly a base class exception that inherits from the builtin Exception.
10     * This is achieved by passing NULL as the PyObject* as the third argument.
11     *
12     * PyErr_NewExceptionWithDoc returns a new reference.
13     */
14     ExceptionBase = PyErr_NewExceptionWithDoc(
15         "cExceptions.ExceptionBase", /* char *name */
16         "Base exception class for the module.", /* char *doc */
17         NULL, /* PyObject *base, resolves to PyExc_Exception. */
18         NULL /* PyObject *dict */);
19     /* Error checking: this is oversimplified as it should decref
20      * anything created above such as m.
21      */
22     if (!ExceptionBase) {
23         return NULL;
24     } else {
25         Py_INCREF(ExceptionBase);
26         PyModule_AddObject(m, "ExceptionBase", ExceptionBase);
27     }
28     /* Now a subclass exception that inherits from the base exception above.
29      * This is achieved by passing non-NULL as the PyObject* as the third argument.
30      *
31      * PyErr_NewExceptionWithDoc returns a new reference.
32      */
33     SpecialisedError = PyErr_NewExceptionWithDoc(
34         "cExceptions.SpecialsiedError", /* char *name */
35         "Some specialised problem description here.", /* char *doc */
36         ExceptionBase, /* PyObject *base, declared above. */
37         NULL /* PyObject *dict */);
38     if (!SpecialisedError) {
39         return NULL;
40     } else {
41         Py_INCREF(SpecialisedError);
42         PyModule_AddObject(m, "SpecialisedError", SpecialisedError);
43     }
44     /* END: Initialise exceptions here. */
45     return m;
46 }

```


6.2.4 Raising Specialise Exceptions

To illustrate how you raise one of these exceptions suppose we have a function to test raising one of these exceptions:

```

1 static PyMethodDef Noddy_module_methods[] = {
2     // ...
3     {
4         "raise_exception_base",
5         (PyCFunction) raise_exception_base,
6         METH_NOARGS,
7         "Raises a ExceptionBase."
8     },
9     {
10        "raise_specialised_error",
11        (PyCFunction) raise_specialised_error,
12        METH_NOARGS,
13        "Raises a SpecialisedError."
14    },
15    // ...
16    {NULL, NULL, 0, NULL} /* Sentinel */
17 };

```

We can either access the exception type directly:

```

1 /** Raises a ExceptionBase. */
2 static PyObject *raise_exception_base(PyObject *Py_UNUSED(module)) {
3     if (ExceptionBase) {
4         PyErr_Format(ExceptionBase, "One %d two %d three %d.", 1, 2, 3);
5     } else {
6         PyErr_SetString(
7             PyExc_RuntimeError,
8             "Can not raise exception, module not initialised correctly"
9         );
10    }
11    return NULL;
12 }
13
14 /** Raises a SpecialisedError. */
15 static PyObject *raise_specialised_error(PyObject *Py_UNUSED(module)) {
16     if (SpecialisedError) {
17         PyErr_Format(SpecialisedError, "One %d two %d three %d.", 1, 2, 3);
18     } else {
19         PyErr_SetString(
20             PyExc_RuntimeError,
21             "Can not raise exception, module not initialised correctly"
22         );
23     }
24    return NULL;
25 }

```

Or fish it out of the module (this will be slower):

```

1 static PyObject *raise_specialised_error(PyObject *module)
2 {

```

(continues on next page)

(continued from previous page)

```
3     PyObject *err = PyDict_GetItemString(PyModule_GetDict(module), "SpecialisedError");
4     if (err) {
5         PyErr_Format(err, "One %d two %d three %d.", 1, 2, 3);
6     } else {
7         PyErr_SetString(PyExc_RuntimeError, "Can not find exception in module");
8     }
9     return NULL;
10 }
```

Here is some test code from tests/unit/test_c_exceptions.py:

```
1 from cPyExtPatt import cExceptions
2
3 def test_raise_exception_base():
4     with pytest.raises(cExceptions.ExceptionBase) as err:
5         cExceptions.raise_exception_base()
6     assert err.value.args[0] == 'One 1 two 2 three 3.'
7
8
9 def test_raise_specialised_error():
10     with pytest.raises(cExceptions.SpecialisedError) as err:
11         cExceptions.raise_specialised_error()
12     assert err.value.args[0] == 'One 1 two 2 three 3.'
```

A PYTHONIC CODING PATTERN FOR C FUNCTIONS

7.1 Principle

To avoid all the errors we have seen, particularly in *PyObjects and Reference Counting* it is useful to have a C coding pattern for handling PyObjects that does the following:

- No early returns and a single place for clean up code.
- Borrowed references incref'd and decref'd correctly.
- No stale Exception from a previous execution path.
- Exceptions set and tested.
- NULL is returned when an exception is set.
- Non-NULL is returned when no exception is set.

The basic pattern in C is similar to Python's try/except/finally pattern:

```
try:
    /* Do fabulous stuff here. */
except:
    /* Handle every abnormal condition and clean up. */
finally:
    /* Clean up under normal conditions and return an appropriate value. */
```

7.2 Coding the Function

Firstly we set any local PyObject (s) and the return value to NULL:

```
static PyObject *function(PyObject *arg_1) {
    PyObject *obj_a    = NULL;
    PyObject *ret      = NULL;
```

Then we have a little bit of Pythonic C - this can be omitted:

```
    goto try; /* Pythonic 'C' ;-) */
try:
```

Check that there are no lingering Exceptions:

```
assert(! PyErr_Occurred());
```

An alternative check for no lingering Exceptions with non-debug builds:

```
if(PyErr_Occurred()) {
    PyObject_Print(PyErr_Occurred(), stdout, Py_PRINT_RAW);
    goto except;
}
```

Now we assume that any argument is a “Borrowed” reference so we increment it (we need a matching `Py_DECREF` before function exit, see below). The first pattern assumes a non-NULL argument.

```
assert(arg_1);
Py_INCREF(arg_1);
```

If you are willing to accept NULL arguments then this pattern would be more suitable:

```
if (arg_1) {
    Py_INCREF(arg_1);
}
```

Or just use `Py_XINCREF`.

Now we create any local objects, if they are “Borrowed” references we need to incref them. With any abnormal behaviour we do a local jump straight to the cleanup code.

```
/* Local object creation. */
/* obj_a = ...; */
if (! obj_a) {
    PyErr_SetString(PyExc_ValueError, "Ooops.");
    goto except;
}
/* If obj_a is a borrowed reference rather than a new reference. */
Py_INCREF(obj_a);
```

Create the return value and deal with abnormal behaviour in the same way:

```
/* More of your code to do stuff with arg_1 and obj_a. */
/* Return object creation, ret should be a new reference otherwise you are in trouble. */
/* ret = ...; */
if (! ret) {
    PyErr_SetString(PyExc_ValueError, "Ooops again.");
    goto except;
}
```

You might want to check the contents of the return value here. On error jump to `except`: otherwise jump to `finally`:

```
/* Any return value checking here. */

/* If success then check exception is clear,
 * then goto finally; with non-NULL return value. */
assert(! PyErr_Occurred());
assert(ret);
goto finally;
```

This is the except block where we cleanup any local objects and set the return value to NULL.

```

except:
    /* Failure so Py_XDECREF the return value here. */
    Py_XDECREF(ret);
    /* Check a Python error is set somewhere above. */
    assert(PyErr_Occurred());
    /* Signal failure. */
    ret = NULL;

```

Notice the `except:` block falls through to the `finally:` block.

```

finally:
    /* All _local_ PyObject's declared at the entry point are Py_XDECREF'd here.
     * For new references this will free them. For borrowed references this
     * will return them to their previous refcount.
     */
    Py_XDECREF(obj_a);
    /* Decrement the ref count of externally supplied the arguments here.
     * If you allow arg_1 == NULL then Py_XDECREF(arg_1). */
    Py_DECREF(arg_1);
    /* And return... */
    return ret;
}

```

[Continued on the next page]

7.3 The Function Code as One

Here is the complete code with minimal comments:

```
static PyObject *function(PyObject *arg_1) {
    PyObject *obj_a    = NULL;
    PyObject *ret       = NULL;

    goto try;
try:
    assert(! PyErr_Occurred());
    assert(arg_1);
    Py_INCREF(arg_1);

    /* Create obj_a = ...; */

    if (! obj_a) {
        PyErr_SetString(PyExc_ValueError, "Ooops.");
        goto except;
    }
    /* Only do this if obj_a is a borrowed reference. */
    Py_INCREF(obj_a);

    /* More of your code to do stuff with obj_a. */

    /* Return object creation, ret must be a new reference. */
    /* ret = ...; */
    if (! ret) {
        PyErr_SetString(PyExc_ValueError, "Ooops again.");
        goto except;
    }
    assert(! PyErr_Occurred());
    assert(ret);
    goto finally;
except:
    Py_XDECREF(ret);
    assert(PyErr_Occurred());
    ret = NULL;
finally:
    /* Only do this if obj_a is a borrowed reference. */
    Py_XDECREF(obj_a);
    Py_DECREF(arg_1);
    return ret;
}
```

PARSING PYTHON ARGUMENTS

This section describes how you write functions that accept Python `*args` and `**kwargs` arguments and how to extract `PyObject` or C fundamental types from them.

8.1 Specifying the Function Arguments

Python has a myriad of ways of function arguments and the C API reflects that. Two important features of CPython C functions are:

- Declaring them correctly with the right signature and flags.
- Parsing the arguments and checking their types.

These are described below.

8.1.1 C Function Declaration

The C function signature must be declared correctly depending on the arguments it is expected to work with. Here is a small summary of the required declaration.

In all cases the function has a *context* which is the first argument.

- For free functions the first argument is the module within which the function is declared. This allows you to access other attributes or functions in the module.
- For member functions (methods) the first argument is the object within which the function is declared (`self`). This allows you to access other properties or functions in the object.

If the argument is unused then the `Py_UNUSED` can be used to suppress a compiler warning or error thus:

```
static PyObject *  
parse_args_kwargs(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwargs);
```

8.1.2 Setting the `ml_flags` Field

The `ml_flags` field in `PyMethodDef` specifies the form of the arguments.

No Arguments

- The flags will be `METH_NOARGS`
- The C Function Signature will be `PyObject *PyCFunction(PyObject *self, PyObject *args);`
- The second argument will be `NULL`.

One Argument

- The flags will be `METH_O`
- The C Function Signature will be `PyObject *PyCFunction(PyObject *self, PyObject *args);`
- The second argument will be the single argument.

Multiple Positional Arguments

- The flags will be `METH_VARARGS`
- The C Function Signature will be `PyObject *PyCFunction(PyObject *self, PyObject *args);`
- Second value will be a tuple of arguments.
- `PyArg_ParseTuple()` is used to unpack the arguments.

Positional and Keyword Arguments

- The flags will be `METH_NOARGS | METH_KEYWORDS`
- The C Function Signature will be `PyObject *PyCFunctionWithKeywords(PyObject *self, PyObject *args, PyObject *kwargs);`
- Second value will be a sequence of arguments, the third the dictionary of arguments.
- `PyArg_ParseTupleAndKeywords()` is used to unpack the arguments.

Documentation:

- No arguments, single or multiple arguments: `PyCFunction`
- Multiple argument and keywords `PyCFunctionWithKeywords`

Note

I don't cover the use of `METH_FASTCALL` in this tutorial.

Note

`METH_KEYWORDS` can only be used in combination with other flags.

Example

A function that takes positional and keyword arguments would be declared as:

```
static PyObject *  
parse_args_kwargs(PyObject *module, PyObject *args, PyObject *kwargs);
```

And this would be added to the module, say, by using:

```
static PyMethodDef cParseArgs_methods[] = {  
    /* ... */  
    {  
        "parse_args_kwargs",  
        (PyCFunction) parse_args_kwargs,  
        METH_VARARGS | METH_KEYWORDS,  
    },  
};
```

(continues on next page)

(continued from previous page)

```

        "Documentation for parse_args_kwargs()."
    },
    /* ... */
    {NULL, NULL, 0, NULL} /* Sentinel */
};

```

Failure

If the `ml_flags` are in conflict then a `SystemError` will be raised on import.

For example, the conflict with `METH_NOARGS` | `METH_KEYWORDS`:

```

static PyMethodDef cParseArgs_methods[] = {
    /* ... */
    {
        "clear",
        (PyCFunction) clear,
        METH_NOARGS | METH_KEYWORDS,
        "Documentation for clear()."
    },
    /* ... */
    {NULL, NULL, 0, NULL} /* Sentinel */
};

```

On import this gives:

```

>>> import cParseArgs
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
SystemError: clear() method: bad call flags

```

Which identifies the problematic function but not the actual problem.

8.2 Parsing the Arguments

Once we have the C function correctly declared then the arguments have to be parsed according to their types and, if required, converted to C types (so-called “unboxing”). This is done using the `PyArg_ParseTuple()` and `PyArg_ParseTupleAndKeywords()` (ignoring “old-style” functions which use `PyArg_Parse`).

These use formatting strings that can become bewilderingly complex so this tutorial uses examples as an introduction. The reference documentation is excellent: [argument parsing and building values](#).

Warning

Error messages can be slightly misleading as the argument index is the *C* index not the *Python* index.

For example a C function using `METH_VARARGS` declared as:

```
static PyObject *parse_args(PyObject *module, PyObject *args);
```

Which expects the Python `args[0]` to be a bytes object and the Python `args[1]` to be an integer by using:

```
PyArg_ParseTuple(args, "Si", &arg0, &arg1)
```

Calling this from Python with `parse_args(21, 22)` will give:

`TypeError: argument 1 must be bytes, not int.`

The “1” here refers to the C index not the Python index. The correct call would be to change the 0th Python argument to:

```
cParseArgs.parse_args(b'21', 22)
```

Also consider the signature:

```
def parse_args(a: bytes, b: int, c: str = '') -> int:
```

Called with `parse_args(b'bytes', '456')` gives the error:

```
''str' object cannot be interpreted as an integer"
```

without specifying which argument it is referring to.

8.3 Examples

These examples are in `src/cpy/cParseArgs.c` and their tests are in `tests/unit/test_c_parse_args.py`.

8.3.1 No Arguments

The simplest form is a global function in a module that takes no arguments at all:

```
static PyObject *parse_no_args(PyObject *Py_UNUSED(module)) {
    /* Your code here... */

    Py_RETURN_NONE;
}
```

This function is added to the module methods with the `METH_NOARGS` value. The Python interpreter will raise a `TypeError` on any arguments are offered to the function.

```
static PyMethodDef cParseArgs_methods[] = {
    /* Other functions here... */
    {
        "parse_no_args",
        (PyCFunction)parse_no_args,
        METH_NOARGS,
        "No arguments."
    },
    /* Other functions here... */
    {NULL, NULL, 0, NULL} /* Sentinel */
};
```

8.3.2 One Argument

There is no parsing required here, a single `PyObject` is expected:

```
static PyObject *parse_one_arg(PyObject *Py_UNUSED(module), PyObject *Py_UNUSED(arg)) {
    /* arg as a borrowed reference and the general rule is that you Py_INCREF them
     * whilst you have an interest in them.
     */
}
```

(continues on next page)

(continued from previous page)

```

// Py_INCREF(arg);

/* Your code here... */

/* If we were to treat arg as a borrowed reference and had Py_INCREF'd above we
 * should do this. See below. */
// Py_DECREF(arg);
Py_RETURN_NONE;
}

```

This function can be added to the module with the METH_O flag:

```

static PyMethodDef cParseArgs_methods[] = {
    /* Other functions here... */
    {
        "parse_one_arg",
        (PyCFunction) parse_one_arg,
        METH_O,
        "One argument."
    },
    /* Other functions here... */
    {NULL, NULL, 0, NULL} /* Sentinel */
};

```

Arguments as Borrowed References

There is some subtlety here as indicated by the comments. `*arg` is not our reference, it is a borrowed reference so why don't we increment it at the beginning of this function and decrement it at the end? After all we are trying to protect against calling into some malicious/badly written code that could hurt us. For example:

```

static PyObject *foo(PyObject *module, PyObject *arg) {
    /* arg has a minimum recount of 1. */
    call_malicious_code_that_decrefs_by_one_this_argument(arg);
    /* arg potentially could have had a ref count of 0 and been deallocated. */
    /* ... */
    /* So now doing something with arg could be undefined. */
}

```

A solution would be, since `arg` is a 'borrowed' reference and borrowed references should always be incremented whilst in use and decremented when done with. This would suggest the following:

```

static PyObject *foo(PyObject *module, PyObject *arg) {
    /* arg has a minimum recount of 1. */
    Py_INCREF(arg);
    /* arg now has a minimum recount of 2. */
    call_malicious_code_that_decrefs_by_one_this_argument(arg);
    /* arg can not have a ref count of 0 so is safe to use. */
    /* Use arg to your hearts content... */
    /* Do a matching decref. */
    Py_DECREF(arg);
    /* But now arg could have had a ref count of 0 so is unsafe to use by the caller. */
}

```

But now we have just pushed the burden onto our caller. They created `arg` and passed it to us in good faith and whilst we have protected ourselves have not protected the caller and they can fail unexpectedly. So it is best to fail fast, as near the error site, that dastardly `call_malicious_code_that_decrefs_by_one_this_argument()`.

Side note: Of course this does not protect you from malicious/badly written code that decrements by more than one :-)

The function will be called with two arguments, the module and a `PyTupleObject` that contains a tuple of arguments. You can either parse this list yourself or use a helper method to parse it into Python and C types.

In the following code we are expecting a bytes object, an integer and an optional string whose default value is 'default_string'. For demonstration purposes, this returns the same three arguments. In Python the equivalent function signature would be:

```
def parse_args(a: bytes, b: int, c: str = 'default_string') \
    -> typing.Tuple[bytes, int, str]:
```

Here is the C code, note the string that describes the argument types passed to `PyArg_ParseTuple`, if these types are not present a `ValueError` will be set.

```
static PyObject *parse_args(PyObject *Py_UNUSED(module), PyObject *args) {
    PyObject *arg_0 = NULL;
    int arg_1;
    char *arg_2 = "default_string";

    if (!PyArg_ParseTuple(args, "Si|s", &arg_0, &arg_1, &arg_2)) {
        return NULL;
    }

    /* Your code here... */

    return Py_BuildValue("Ois", arg_0, arg_1, arg_2);
}
```

This function can be added to the module with the `METH_VARARGS` flag:

```
static PyMethodDef cParseArgs_methods[] = {
    /* Other functions here... */
    {
        "parse_args",
        (PyCFunction) parse_args,
        METH_VARARGS,
        "parse_args() documentation"
    },
    /* Other functions here... */
    {NULL, NULL, 0, NULL} /* Sentinel */
};
```

This code can be seen in `src/cpy/cParseArgs.c`. It is tested in `tests.unit.test_c_parse_args.test_parse_args`. Failure modes, when the wrong arguments are passed are tested in `tests.unit.test_c_parse_args.test_parse_args_raises`. Note the wide variety of error messages that are obtained.

8.3.3 Variable Number of Arguments and Keyword Arguments

The function will be called with three arguments, the module, a PyTupleObject that contains a tuple of arguments and a PyDictObject that contains a dictionary of keyword arguments. The keyword arguments can be NULL if there are no keyword arguments. You can either parse these yourself or use a helper method to parse it into Python and C types.

In the following code we are expecting a sequence and an optional integer count defaulting to 1. It returns the *sequence* repeated *count* times. In Python the equivalent function declaration would be:

```
def parse_args_kwargs(sequence=typing.Sequence[typing.Any], count: = 1) \
    -> typing.Sequence[typing.Any]:
    return sequence * count
```

Here is the C code, note the string "O|i" that describes the argument types passed to PyArg_ParseTupleAndKeywords, if these types are not present a TypeError will be set.

```
1 static PyObject *
2 parse_args_kwargs(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwargs) {
3     PyObject *ret = NULL;
4     PyObject *py_sequence = NULL;
5     int count = 1; /* Default. */
6     static char *kwlist[] = {
7         "sequence", /* A sequence object, str, list, tuple etc. */
8         "count", /* Python int converted to a C int. */
9         NULL,
10    };
11
12    if (!PyArg_ParseTupleAndKeywords(
13        args, kwargs, "O|i", kwlist, &py_sequence, &count
14    )) {
15        goto except;
16    }
17
18    /* Your code here... */
19
20    ret = PySequence_Repeat(py_sequence, count);
21    if (ret == NULL) {
22        goto except;
23    }
24    assert(!PyErr_Occurred());
25    goto finally;
26 except:
27    assert(PyErr_Occurred());
28    Py_XDECREF(ret);
29    ret = NULL;
30 finally:
31    return ret;
32 }
```

This function must be added to the module with both the METH_VARARGS and METH_KEYWORDS flags set:

```
static PyMethodDef cParseArgs_methods[] = {
    /* Other functions here... */
    {
```

(continues on next page)

(continued from previous page)

```

    "parse_args_kwargs",
    (PyCFunction) parse_args_kwargs,
    METH_VARARGS | METH_KEYWORDS,
    "parse_args_kwargs() documentation"
},
/* Other functions here... */
{NULL, NULL, 0, NULL} /* Sentinel */
};

```

This code can be seen in `src/cpy/cParseArgs.c`. It is tested in `tests.unit.test_c_parse_args.test_parse_args_kwargs` which shows the variety of ways this can be called. Failure modes, when the wrong arguments are passed are tested in `tests.unit.test_c_parse_args.test_parse_args_kwargs_raises`.

All arguments are keyword arguments so this function can be called in a number of ways, all of the following are equivalent:

```

cParseArgs.parse_args_kwargs([1, 2, 3], 2)
cParseArgs.parse_args_kwargs([1, 2, 3], count=2)
cParseArgs.parse_args_kwargs(sequence=[1, 2, 3], count=2)

```

Keyword Arguments and C++11

C++11 compilers warn when creating non-const `char*` from string literals as we have done with the keyword array above. The solution is to declare these `const char*` however `PyArg_ParseTupleAndKeywords()` expects a `char**`. The solution is to cast away `const` in the call:

```

/* ... */
static const char *kwlist[] = { "foo", "bar", "baz", NULL };
if (!PyArg_ParseTupleAndKeywords(args, kwds, "000",
                                const_cast<char**>(kwlist),
                                &foo, &bar, &baz)) {

    return NULL;
}
/* ... */

```

8.3.4 Default String and Bytes Arguments

The recommended way to accept binary data is to parse the argument using the `"y*"` formatting string and supply a `Py_Buffer` argument. This also applies to strings, using the `"s*"` formatting string, where they might contain `'\0'` characters.

Typically this would be done with C code such as this:

```

Py_buffer arg;

if (!PyArg_ParseTupleAndKeywords(args, kwargs, "y*", kwlist, &arg)) {
    assert(PyErr_Occurred());
    return NULL;
}
/* arg.buf has the byte data of length arg.len */

```

However this will likely segfault if it is used as a default argument using `"|y*"` formatting string as the `Py_Buffer` is uninitialized. Here is the fix for using a default value of `b''`:

```
Py_buffer arg;
arg.buf = NULL;
arg.len = 0;

if (!PyArg_ParseTupleAndKeywords(args, kwargs, "|y*", kwlist, &arg)) {
    assert(PyErr_Occurred());
    return NULL;
}
```

For a different default value, say `b'default'`, then this will work. The Python signature is:

```
def parse_default_bytes_object(b: bytes = b"default") -> bytes:
```

The complete C code is:

```
static PyObject *
parse_default_bytes_object(PyObject *Py_UNUSED(module), PyObject *args,
                          PyObject *kwargs) {
    static const char *arg_default = "default";
    Py_buffer arg;
    arg.buf = (void *)arg_default;
    arg.len = strlen(arg_default);
    static char *kwlist[] = {
        "b",
        NULL,
    };
    if (!PyArg_ParseTupleAndKeywords(args, kwargs, "|y*", kwlist, &arg)) {
        assert(PyErr_Occurred());
        return NULL;
    }
    return Py_BuildValue("y#", arg.buf, arg.len);
}
```

See `tests/unit/test_c_parse_args.py` for some Python uses of this code.

8.3.5 Positional Only and Keyword Only Arguments

This section shows how to achieve [positional only](#) and [keyword only](#) arguments in a C extension. These are described in the Python documentation for [Special parameters](#) Specifically [positional only parameters](#) and [keyword only arguments](#).

Suppose we want the functional equivalent of the Python function signature (reproducing <https://docs.python.org/3/tutorial/controlflow.html#special-parameters>):

```
def parse_pos_only_kwd_only(pos1: str, pos2: int, /, pos_or_kwd: bytes, *,
                           kwd1: float = 256.0, kwd2: int = -421):
    return None
```

This is achieved by combining two techniques:

- Positional only: The strings in the `*kwlist` passed to `PyArg_ParseTupleAndKeywords()` are empty.
- Keyword only: The formatting string passed to `PyArg_ParseTupleAndKeywords()` uses the `'$'` character.

A function using either positional only or keyword only arguments must use the flags `METH_VARARGS` | `METH_KEYWORDS` and uses `PyArg_ParseTupleAndKeywords()`. Currently, all keyword-only arguments must also be optional arguments, so `'|'` must always be specified before `'$'` in the format string.

Here is the C code:

```

1 static PyObject *
2 parse_pos_only_kwd_only(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwargs) {
3     /* Arguments, first three are required. */
4     Py_buffer pos1;
5     int pos2;
6     Py_buffer pos_or_kwd;
7     /* Last two are optional. */
8     double kwd1 = 256.0;
9     int kwd2 = -421;
10    static char *kwlist[] = {
11        "", /* pos1 is positional only. */
12        "", /* pos2 is positional only. */
13        "pos_or_kwd", /* pos_or_kwd can be positional or keyword argument. */
14        /* NOTE: As '$' is in format string the next two are keyword only. */
15        "kwd1", /* kwd1 is keyword only argument. */
16        "kwd2", /* kwd2 is keyword only argument. */
17        NULL,
18    };
19
20    if (!PyArg_ParseTupleAndKeywords(args, kwargs, "s*iy*$di", kwlist, &pos1, &pos2,
21                                     &pos_or_kwd, &kwd1, &kwd2)) {
22        assert(PyErr_Occurred());
23        return NULL;
24    }
25    /* Return the parsed arguments.
26     * NOTE the asymmetry between "s*iy*$di" above and "s#iy#di" here. */
27    return Py_BuildValue("s#iy#di", pos1.buf, pos1.len, pos2, pos_or_kwd.buf,
28                         pos_or_kwd.len, kwd1, kwd2);
29 }

```

8.3.6 Parsing Arguments With Functional Conversion to C

Often you want to convert a Python argument to a C value(s) in a way that is not covered by the format strings provided by the Python C API. To do this you can provide a special conversion function in C and give it to `PyArg_ParseTuple()` or `PyArg_ParseTupleAndKeywords()`.

In this example we are expecting the Python argument to be a list of integers and we want the sum of them as a C long. First create a C function that takes a Python list, checks it and sums the values. The function returns 1 on success or 0 on error and, in that case, an exception is expected to be set. On success the result will be written into the opaque pointer, here called `address`:

```

1 int sum_list_of_longs(PyObject *list_longs, void *address) {
2     PyObject *item = NULL;
3
4     /* Note: PyList_Check allows sub-types. */
5     if (!list_longs || !PyList_Check(list_longs)) {
6         PyErr_Format(PyExc_TypeError,
7                     "check_list_of_longs(): First argument is not a list"
8                     );
9         return 0;
10    }
11    long result = 0L;

```

(continues on next page)

(continued from previous page)

```

12     for (Py_ssize_t i = 0; i < PyList_GET_SIZE(list_longs); ++i) {
13         item = PyList_GetItem(list_longs, i);
14         if (!PyLong_CheckExact(item)) {
15             PyErr_Format(PyExc_TypeError,
16                          "check_list_of_longs(): Item %d is not a Python integer.", i
17                          );
18             return 0;
19         }
20         /* PyLong_AsLong() must always succeed because of check above. */
21         result += PyLong_AsLong(item);
22     }
23     long *p_long = (long *) address;
24     *p_long = result;
25     return 1; /* Success. */
26 }

```

Now we can pass this function to `PyArg_ParseTuple` with the "O&" formatting string that takes two arguments, the Python list and the C conversion function. On success `PyArg_ParseTuple` writes the value to the target, `result`.

In this case the function just returns the sum of the integers in the list. Here is the C code.

```

1  static PyObject *
2  parse_args_with_function_conversion_to_c(PyObject *Py_UNUSED(module), PyObject *args) {
3      PyObject *ret = NULL;
4      long result;
5
6      if (!PyArg_ParseTuple(args, "O&", sum_list_of_longs, &result)) {
7          /* NOTE: If check_list_of_numbers() returns 0 an error should be set. */
8          assert(PyErr_Occurred());
9          goto except;
10     }
11
12     /* Your code here... */
13     ret = PyLong_FromLong(result);
14     if (ret == NULL) {
15         goto except;
16     }
17     assert(!PyErr_Occurred());
18     goto finally;
19 except:
20     assert(PyErr_Occurred());
21     Py_XDECREF(ret);
22     ret = NULL;
23 finally:
24     return ret;
25 }

```

8.4 Being Pythonic with Default Mutable Arguments

If the arguments default to some C fundamental type the code above is fine. However if the mutable arguments default to Python objects then a little more work is needed.

Note

See *Handling Default Arguments* for a way of simplifying this with C++.

Here is a function that has an object and as default argument as a list, appends the object to the list and returns the list. The Python signature is:

```
def function(arg_0, arg_1=[]):
    arg_1.append(arg_0)
    return arg_1
```

The first argument is immutable, the second is mutable. We need to mimic the well known behaviour of Python with mutable arguments where default arguments are evaluated once only at function definition time and then becomes a (mutable) property of the function.

For example:

```
>>> function(1)
[1]
>>> function(2)
[1, 2]
>>> my_list = []
>>> f(10, my_list)
[10]
>>> f(10)
[1, 2, 10]
```

In C we can get this behaviour by treating the mutable argument as static, the immutable argument does not need to be static but it will do no harm if it is (if non-static it will have to be initialised on every function call).

My advice: Always make all PyObject* references to default arguments static.

So first we declare a static PyObject* for each default argument:

```
1  /** Parse the args where we are simulating mutable default of an empty list.
2  *
3  * This is equivalent to:
4  *
5  * def parse_args_with_mutable_defaults_macro_helper(obj, default_list=[]):
6  *     default_list.append(obj)
7  *     return default_list
8  *
9  * See also parse_args_with_mutable_defaults_macro_helper() in cParseArgsHelper.cpp
10 * This adds the object to the list and returns None.
11 *
12 * This imitates the Python way of handling defaults.
13 */
14 static PyObject *parse_args_with_mutable_defaults(PyObject *Py_UNUSED(module),
15                                                    PyObject *args) {
16     PyObject *ret = NULL;
17     /* Pointers to the non-default argument, initialised by PyArg_ParseTuple below. */
18     PyObject *arg_0 = NULL;
19     /* Pointers to the default argument, initialised below. */
20     static PyObject *arg_1_default = NULL;
```

(continues on next page)

(continued from previous page)

```

21  /* Set defaults for argument 1. */
22  if (!arg_1_default) {
23      arg_1_default = PyList_New(0);
24  }

```

Then we declare a `PyObject*` for each argument that will either reference the default or the passed in argument. It is important that these `pyObjArg...` pointers are `NULL` so that we can subsequently detect if `PyArg_ParseTuple` has set them non-`NULL`.

```

/* This pointer is the one we use in the body of the function, it
 * either points at the supplied argument or the default (static) argument.
 */
PyObject *arg_1 = NULL;

```

Now parse the given arguments to see what, if anything, is there. `PyArg_ParseTuple` will set each working pointer non-`NULL` if the argument is present. As we set the working pointers `NULL` prior to this call we can now tell if any argument is present.

```

if (!PyArg_ParseTuple(args, "O|O", &arg_0, &arg_1)) {
    goto except;
}

```

Now switch our working pointers to the default argument if no argument is given. We also treat these as “borrowed” references regardless of whether they are default or supplied so increment the refcount (we must decrement the refcount when done).

```

/* If optional argument absent then switch to defaults. */
if (!arg_1) {
    arg_1 = arg_1_default;
}

```

Now write the main body of your function and that must be followed by this clean up code:

```

/* Your code here... */

/* Append the first argument to the second.
 * PyList_Append increments the reference count of arg_0. */
if (PyList_Append(arg_1, arg_0)) {
    PyErr_SetString(PyExc_RuntimeError, "Can not append to list!");
    goto except;
}
/* Success. */
assert(!PyErr_Occurred());
/* Increments the default or the given argument. */
Py_INCREF(arg_1);
ret = arg_1;
goto finally;

```

Now the two blocks `except` and `finally`.

```

except:
    assert(PyErr_Occurred());
    Py_XDECREF(ret);

```

(continues on next page)

(continued from previous page)

```

    ret = NULL;
finally:
    return ret;
}

```

Here is the complete C code:

```

1  /** Parse the args where we are simulating mutable default of an empty list.
2  *
3  * This is equivalent to:
4  *
5  * def parse_args_with_mutable_defaults_macro_helper(obj, default_list=[]):
6  *     default_list.append(obj)
7  *     return default_list
8  *
9  * See also parse_args_with_mutable_defaults_macro_helper() in cParseArgsHelper.cpp
10 * This adds the object to the list and returns None.
11 *
12 * This imitates the Python way of handling defaults.
13 */
14 static PyObject *parse_args_with_mutable_defaults(PyObject *Py_UNUSED(module),
15                                                    PyObject *args) {
16     PyObject *ret = NULL;
17     /* Pointers to the non-default argument, initialised by PyArg_ParseTuple below. */
18     PyObject *arg_0 = NULL;
19     /* Pointers to the default argument, initialised below. */
20     static PyObject *arg_1_default = NULL;
21     /* Set defaults for argument 1. */
22     if (!arg_1_default) {
23         arg_1_default = PyList_New(0);
24     }
25     /* This pointer is the one we use in the body of the function, it
26     * either points at the supplied argument or the default (static) argument.
27     */
28     PyObject *arg_1 = NULL;
29
30     if (!PyArg_ParseTuple(args, "O|O", &arg_0, &arg_1)) {
31         goto except;
32     }
33     /* If optional argument absent then switch to defaults. */
34     if (!arg_1) {
35         arg_1 = arg_1_default;
36     }
37
38     /* Your code here...*/
39
40     /* Append the first argument to the second.
41     * PyList_Append increments the reference count of arg_0. */
42     if (PyList_Append(arg_1, arg_0)) {
43         PyErr_SetString(PyExc_RuntimeError, "Can not append to list!");
44         goto except;
45     }

```

(continues on next page)

(continued from previous page)

```

46  /* Success. */
47  assert(!PyErr_Occurred());
48  /* Increments the default or the given argument. */
49  Py_INCREF(arg_1);
50  ret = arg_1;
51  goto finally;
52  except:
53      assert(PyErr_Occurred());
54      Py_XDECREF(ret);
55      ret = NULL;
56  finally:
57      return ret;
58  }
59

```

The code can be found in `src/cpy/ParseArgs/cParseArgs.c`.

Tests are in `test_parse_args_with_mutable_defaults()` in `tests/unit/test_c_parse_args.py`

8.5 Helper Macros

Some macros can make this easier. These are in `src/cpy/ParseArgs/cParseArgsHelper.cpp`. Firstly a macro to declare the static default object:

```

#define PY_DEFAULT_ARGUMENT_INIT(name, value, ret) \
    PyObject *name = NULL; \
    static PyObject *default_##name = NULL; \
    if (! default_##name) { \
        default_##name = value; \
        if (! default_##name) { \
            PyErr_SetString( \
                PyExc_RuntimeError, \
                "Can not create default value for " #name \
            ); \
            return ret; \
        } \
    } \
}

```

Warning

When using this macro in a source file then make sure each given “name” argument is unique within the translation unit.

And a macro to set it:

```

#define PY_DEFAULT_ARGUMENT_SET(name) \
    if (! name) { \
        name = default_##name; \
    } \
}

```

And a macro to check the type of the argument:

```

#define PY_DEFAULT_CHECK(name, check_function, type) \
    if (!check_function(name)) { \
        PyErr_Format( \
            PyExc_TypeError, \
            #name " must be " #type ", not \"%s\"", \
            Py_TYPE(name)->tp_name \
        ); \
        return NULL; \
    }

```

8.5.1 Immutable Arguments

These can be used thus. This is equivalent to the Python function:

```

def parse_defaults_with_helper_macro(
    encoding_m: str = "utf-8",
    the_id_m: int = 1024,
    log_interval_m: float = 8.0):
    return encoding_m, the_id_m, log_interval_m

```

Here it is in C:

```

1 static PyObject *
2 parse_defaults_with_helper_macro(PyObject *Py_UNUSED(module), PyObject *args, PyObject *
   ↪ *kwds) {
3     PyObject *ret = NULL;
4     /* Initialise default arguments. Note: these might cause an early return. */
5     PY_DEFAULT_ARGUMENT_INIT(encoding_m, PyUnicode_FromString("utf-8"), NULL);
6     PY_DEFAULT_ARGUMENT_INIT(the_id_m, PyLong_FromLong(DEFAULT_ID), NULL);
7     PY_DEFAULT_ARGUMENT_INIT(log_interval_m, PyFloat_FromDouble(DEFAULT_FLOAT), NULL);
8
9     static const char *kwlist[] = {"encoding", "the_id", "log_interval", NULL};
10    if (!PyArg_ParseTupleAndKeywords(args, kwds, "|000",
11                                     const_cast<char **>(kwlist),
12                                     &encoding_m, &the_id_m, &log_interval_m)) {
13        goto except;
14    }
15    /*
16     * Assign absent arguments to defaults and increment the reference count.
17     * Don't forget to decrement the reference count before returning!
18     */
19    PY_DEFAULT_ARGUMENT_SET(encoding_m);
20    PY_DEFAULT_ARGUMENT_SET(the_id_m);
21    PY_DEFAULT_ARGUMENT_SET(log_interval_m);
22
23    /* Check the types of the given or default arguments. */
24    PY_DEFAULT_CHECK(encoding_m, PyUnicode_Check, "str");
25    PY_DEFAULT_CHECK(the_id_m, PyLong_Check, "int");
26    PY_DEFAULT_CHECK(log_interval_m, PyFloat_Check, "float");
27
28    /*
29     * Use 'encoding': Python str, 'the_id': C long, 'must_log': C long from here on...
30     */

```

(continues on next page)

(continued from previous page)

```

31  /* Py_BuildValue("O") increments the reference count. */
32  ret = Py_BuildValue("000", encoding_m, the_id_m, log_interval_m);
33  assert(!PyErr_Occurred());
34  assert(ret);
35  goto finally;
36
37 except:
38     assert(PyErr_Occurred());
39     Py_XDECREF(ret);
40     ret = NULL;
41 finally:
42     return ret;
43 }

```

The code is:

- Function: `parse_defaults_with_helper_macro()`
- Location: `src/cpy/ParseArgs/cParseArgsHelper.cpp`

The tests are in `tests/unit/test_c_parse_args_helper.py`.

If you are in a C++ environment then the section on *Handling Default Arguments* can help.

8.5.2 Mutable Arguments

These macros also work with mutable arguments.

The following C code emulates this Python code:

```

def parse_mutable_defaults_with_helper_macro(obj, default_list=[]):
    default_list.append(obj)
    return default_list

```

Here is the C code:

```

1  /** Parse the args where we are simulating mutable default of an empty list.
2   * This uses the helper macros.
3   *
4   * This is equivalent to:
5   *
6   * def parse_mutable_defaults_with_helper_macro(obj, default_list=[]):
7   *     default_list.append(obj)
8   *     return default_list
9   *
10  * This adds the object to the list and returns None.
11  *
12  * This imitates the Python way of handling defaults.
13  */
14 static PyObject *parse_mutable_defaults_with_helper_macro(PyObject *Py_UNUSED(module),
15                                                         PyObject *args) {
16     PyObject *ret = NULL;
17     /* Pointers to the non-default argument, initialised by PyArg_ParseTuple below. */
18     PyObject *arg_0 = NULL;
19     /* Pointers to the default argument, initialised below. */

```

(continues on next page)

(continued from previous page)

```

20  /* Initialise default arguments. Note: these might cause an early return. */
21  PY_DEFAULT_ARGUMENT_INIT(list_argument_m, PyList_New(0), NULL);
22
23  if (!PyArg_ParseTuple(args, "O|O", &arg_0, &list_argument_m)) {
24      goto except;
25  }
26  /* If optional argument absent then switch to defaults. */
27  PY_DEFAULT_ARGUMENT_SET(list_argument_m);
28  PY_DEFAULT_CHECK(list_argument_m, PyList_Check, "list");
29
30  /* Your code here...*/
31
32  /* Append the first argument to the second.
33   * PyList_Append() increments the refcount of arg_0. */
34  if (PyList_Append(list_argument_m, arg_0)) {
35      PyErr_SetString(PyExc_RuntimeError, "Can not append to list!");
36      goto except;
37  }
38
39  /* Success. */
40  assert(!PyErr_Occurred());
41  /* This increments the default or the given argument. */
42  Py_INCREF(list_argument_m);
43  ret = list_argument_m;
44  goto finally;
45 except:
46     assert(PyErr_Occurred());
47     Py_XDECREF(ret);
48     ret = NULL;
49 finally:
50     return ret;
51 }

```

Here is some test code from tests/unit/test_c_parse_args_helper.py.

First a test to establish the Python behaviour:

```

1  def test_parse_mutable_defaults_with_helper_macro_python():
2      """A local Python equivalent of cParseArgsHelper.parse_mutable_defaults_with_helper_
   ↪ macro()."""
3
4      def parse_mutable_defaults_with_helper_macro(obj, default_list=[]):
5          default_list.append(obj)
6          return default_list
7
8      result = parse_mutable_defaults_with_helper_macro(1)
9      assert sys.getrefcount(result) == 3
10     assert result == [1, ]
11     result = parse_mutable_defaults_with_helper_macro(2)
12     assert sys.getrefcount(result) == 3
13     assert result == [1, 2]
14     result = parse_mutable_defaults_with_helper_macro(3)
15     assert sys.getrefcount(result) == 3

```

(continues on next page)

(continued from previous page)

```

16     assert result == [1, 2, 3]
17
18     local_list = []
19     assert sys.getrefcount(local_list) == 2
20     assert parse_mutable_defaults_with_helper_macro(10, local_list) == [10]
21     assert sys.getrefcount(local_list) == 2
22     assert parse_mutable_defaults_with_helper_macro(11, local_list) == [10, 11]
23     assert sys.getrefcount(local_list) == 2
24
25     result = parse_mutable_defaults_with_helper_macro(4)
26     assert result == [1, 2, 3, 4]
27     assert sys.getrefcount(result) == 3

```

Now a similar test to establish the C behaviour:

```

1  from cPyExtPatt import cParseArgsHelper
2
3  def test_parse_mutable_defaults_with_helper_macro_c():
4      result = cParseArgsHelper.parse_mutable_defaults_with_helper_macro(1)
5      assert sys.getrefcount(result) == 3
6      assert result == [1, ]
7      result = cParseArgsHelper.parse_mutable_defaults_with_helper_macro(2)
8      assert sys.getrefcount(result) == 3
9      assert result == [1, 2]
10     result = cParseArgsHelper.parse_mutable_defaults_with_helper_macro(3)
11     assert sys.getrefcount(result) == 3
12     assert result == [1, 2, 3]
13
14     local_list = []
15     assert sys.getrefcount(local_list) == 2
16     assert cParseArgsHelper.parse_mutable_defaults_with_helper_macro(10, local_list) == [
17 ↪ 10]
18     assert sys.getrefcount(local_list) == 2
19     assert cParseArgsHelper.parse_mutable_defaults_with_helper_macro(11, local_list) == [
20 ↪ 10, 11]
21     assert sys.getrefcount(local_list) == 2
22
23     result = cParseArgsHelper.parse_mutable_defaults_with_helper_macro(4)
24     assert result == [1, 2, 3, 4]
25     assert sys.getrefcount(result) == 3

```


CREATING NEW TYPES

The creation of new extension types (AKA ‘classes’) is pretty well described in the Python documentation [tutorial](#) and [reference](#). This section is a cookbook of tricks and examples.

9.1 Properties

9.1.1 Referencing Existing Properties

If the property is part of the extension type then it is fairly easy to make it directly accessible as [described here](#)

Note

Terminology

In this section “property”, “attribute” and “field” are used interchangeably.

For example the Noddy struct has a Python object (a Python string) and a C object (an C int):

```
typedef struct {
    PyObject_HEAD
    PyObject *first; /* first name */
    /* ... */
    int number;
} Noddy;
```

These can be exposed by identifying them as members with an array of PyMemberDef like this:

```
static PyMemberDef Noddy_members[] = {
    {"first", T_OBJECT_EX, offsetof(Noddy, first), 0,
     "first name"},
    /* ... */
    {"number", T_INT, offsetof(Noddy, number), 0,
     "noddy number"},
    {NULL} /* Sentinel */
};
```

And the type struct must reference this array of PyMemberDef thus:

```
static PyTypeObject NoddyType = {
    /* ... */
    Noddy_members,          /* tp_members */
```

(continues on next page)

(continued from previous page)

```

    /* ... */
};

```

Reference to `PyMemberDef`.

9.1.2 Created Properties

If the properties are not directly accessible, for example they might need to be created, then an array of `PyGetSetDef` structures is used in the `PyTypeObject.tp_getset` slot.

```

1 static PyObject*
2 Foo_property_getter(Foo* self, void * /* closure */) {
3     return /* ... */;
4 }
5
6 int
7 Foo_property_setter(Foo* self, PyObject *value) {
8     return /* 0 on success, -1 on failure with error set. */;
9 }
10
11 static PyGetSetDef Foo_properties[] = {
12     {"id", (getter) Foo_property_getter, (setter) Foo_property_setter,
13      "The property documentation.", NULL },
14     {NULL} /* Sentinel */
15 };

```

And the type struct must reference this array of `PyMemberDef` thus:

```

static PyTypeObject FooType = {
    /* ... */
    Foo_properties,          /* tp_getset */
    /* ... */
};

```

Reference to `PyGetSetDef`.

9.2 Subclassing

This large subject gets its own chapter: *Subclassing and Using super()*.

9.3 Examples

See `src/cpy/cObject.c` for some examples, the tests for these are in `tests/unit/test_c_object.py`:

- `Null` is a basic class that does nothing.
- `Str` is a subclass of the builtin `str` class.

9.3.1 Setting, Getting and Deleting Attributes Dynamically

In `src/cpy/cObject.c` there is an example, `ObjectWithAttributes`, which is a class that can set, get, delete attributes dynamically.

Firstly the object declaration:

```
typedef struct {
    PyObject_HEAD
    /* Attributes dictionary, NULL on construction,
     * will be populated by MyObj_getattro. */
    PyObject *x_attr;
} ObjectWithAttributes;
```

Then some type checking that requires a forward declaration:

```
/** Forward declaration. */
static PyTypeObject ObjectWithAttributes_Type;
#define ObjectWithAttributes_Check(v) (Py_TYPE(v) == &ObjectWithAttributes_Type)
```

The `__new__` and `__del__` methods:

```
1 static ObjectWithAttributes *
2 ObjectWithAttributes_new(PyObject *Py_UNUSED(arg)) {
3     ObjectWithAttributes *self;
4     self = PyObject_New(ObjectWithAttributes, &ObjectWithAttributes_Type);
5     if (self == NULL) {
6         return NULL;
7     }
8     self->x_attr = NULL;
9     return self;
10 }
11
12 /* ObjectWithAttributes methods */
13 static void
14 ObjectWithAttributes_dealloc(ObjectWithAttributes *self) {
15     Py_XDECREF(self->x_attr);
16     PyObject_Del(self);
17 }
```

Add an empty method for demonstration:

```
1 static PyObject *
2 ObjectWithAttributes_demo(ObjectWithAttributes *Py_UNUSED(self), PyObject *args) {
3     if (!PyArg_ParseTuple(args, ":demo")) {
4         return NULL;
5     }
6     Py_INCREF(Py_None);
7     return Py_None;
8 }
9
10 static PyMethodDef ObjectWithAttributes_methods[] = {
11     {"demo", (PyCFunction) ObjectWithAttributes_demo, METH_VARARGS,
12         PyDoc_STR("demo() -> None")},
13     {NULL, NULL, 0, NULL} /* sentinel */
14 };
```

Now the methods to get and set attribute:

```
1 static PyObject *
2 ObjectWithAttributes_getattro(ObjectWithAttributes *self, PyObject *name) {
    (continues on next page)
```

(continued from previous page)

```

3     if (self->x_attr != NULL) {
4         PyObject *v = PyDict_GetItem(self->x_attr, name);
5         if (v != NULL) {
6             Py_INCREF(v);
7             return v;
8         }
9     }
10    return PyObject_GenericGetAttr((PyObject *) self, name);
11}
12
13static int
14ObjectWithAttributes_setattr(ObjectWithAttributes *self, char *name, PyObject *v) {
15    if (self->x_attr == NULL) {
16        self->x_attr = PyDict_New();
17        if (self->x_attr == NULL)
18            return -1;
19    }
20    if (v == NULL) {
21        int rv = PyDict_DelItemString(self->x_attr, name);
22        if (rv < 0)
23            PyErr_SetString(PyExc_AttributeError,
24                            "delete non-existing ObjectWithAttributes attribute");
25        return rv;
26    } else
27        /* v is a borrowed reference,
28         * then PyDict_SetItemString() does NOT steal it
29         * so nothing to do. */
30        return PyDict_SetItemString(self->x_attr, name, v);
31}

```

Finally the type declaration. Note this is the complete type declaration with compiler declarations to suit Python versions 3.6 to 3.13:

```

1 static PyTypeObject ObjectWithAttributes_Type = {
2     /* The ob_type field must be initialized in the module init function
3      * to be portable to Windows without using C++. */
4     PyVarObject_HEAD_INIT(NULL, 0)
5     "cObject.ObjectWithAttributes", /*tp_name*/
6     sizeof(ObjectWithAttributes), /*tp_basicsize*/
7     0, /*tp_itemsize*/
8     /* methods */
9     (destructor) ObjectWithAttributes_dealloc, /*tp_dealloc*/
10    #if PY_MINOR_VERSION < 8
11        0, /*tp_print*/
12    #else
13        0, /* Py_ssize_t tp_vectorcall_offset; */
14    #endif
15    (getattrfunc) 0, /*tp_getattr*/
16    (setattrfunc) ObjectWithAttributes_setattr, /*tp_setattr*/
17    0, /*tp_reserved*/
18    0, /*tp_repr*/
19    0, /*tp_as_number*/

```

(continues on next page)

(continued from previous page)

```

20         0,                                /*tp_as_sequence*/
21         0,                                /*tp_as_mapping*/
22         0,                                /*tp_hash*/
23         0,                                /*tp_call*/
24         0,                                /*tp_str*/
25         (getattrfunc) ObjectWithAttributes_getattro, /*tp_getattro*/
26         0,                                /*tp_setattro*/
27         0,                                /*tp_as_buffer*/
28         Py_TPFLAGS_DEFAULT,               /*tp_flags*/
29         0,                                /*tp_doc*/
30         0,                                /*tp_traverse*/
31         0,                                /*tp_clear*/
32         0,                                /*tp_richcompare*/
33         0,                                /*tp_weaklistoffset*/
34         0,                                /*tp_iter*/
35         0,                                /*tp_iternext*/
36         ObjectWithAttributes_methods,     /*tp_methods*/
37         0,                                /*tp_members*/
38         0,                                /*tp_getset*/
39         0,                                /*tp_base*/
40         0,                                /*tp_dict*/
41         0,                                /*tp_descr_get*/
42         0,                                /*tp_descr_set*/
43         0,                                /*tp_dictoffset*/
44         0,                                /*tp_init*/
45         0,                                /*tp_alloc*/
46         // PyType_GenericNew,             /*tp_new*/
47         (newfunc) ObjectWithAttributes_new, /*tp_new*/
48         0,                                /*tp_free*/
49         0,                                /*tp_is_gc*/
50         NULL,                             /* tp_bases */
51         NULL,                             /* tp_mro */
52         NULL,                             /* tp_cache */
53         NULL,                             /* tp_subclasses */
54         NULL,                             /* tp_weaklist */
55         NULL,                             /* tp_del */
56         0,                                /* tp_version_tag */
57         NULL,                             /* tp_finalize */
58         #if PY_MINOR_VERSION > 7
59         NULL,                             /* tp_vectorcall */
60     #endif
61     #if PY_MINOR_VERSION == 8
62         0,                                /*tp_print*/
63     #endif
64     #if PY_MINOR_VERSION >= 12
65         '\0',                             /* unsigned char tp_watched */
66     #if PY_MINOR_VERSION >= 13
67         0,                                /* uint16_t tp_versions_used */
68     #endif
69     #endif
70     };

```

Finally add this to the module (partial code):

```

1 static struct PyModuleDef cObject = {
2     PyModuleDef_HEAD_INIT,
3     "cObject",
4     module_doc,
5     -1,
6     cObject_functions,
7     NULL,
8     NULL,
9     NULL,
10    NULL
11 };
12
13 PyMODINIT_FUNC
14 PyInit_cObject(void) {
15     PyObject *m = NULL;
16
17     /* Create the module and add the functions */
18     m = PyModule_Create(&cObject);
19     if (m == NULL) {
20         goto fail;
21     }
22     /* Add some symbolic constants to the module */
23     if (ErrorObject == NULL) {
24         ErrorObject = PyErr_NewException("cObject.error", NULL, NULL);
25         if (ErrorObject == NULL)
26             goto fail;
27     }
28     Py_INCREF(ErrorObject);
29     if (PyModule_AddObject(m, "error", ErrorObject)) {
30         goto fail;
31     }
32     /* Finalize the type object including setting type of the new type
33      * object; doing it here is required for portability, too. */
34     if (PyType_Ready(&ObjectWithAttributes_Type) < 0) {
35         goto fail;
36     }
37     if (PyModule_AddObject(m, "ObjectWithAttributes",
38                           (PyObject *) &ObjectWithAttributes_Type)) {
39         goto fail;
40     }
41     /* More here ... */
42     return m;
43 fail:
44     Py_XDECREF(m);
45     return NULL;
46 }

```

This can be tested thus, in tests/unit/test_c_object.py:

```

1 import pytest
2
3 from cPyExtPatt import cObject
4

```

(continues on next page)

(continued from previous page)

```

5 def test_ObjectWithAttributes_set_and_get():
6     obj = cObject.ObjectWithAttributes()
7     obj.some_attr = 'Some attribute'
8     assert hasattr(obj, 'some_attr')
9     assert obj.some_attr == 'Some attribute'
10
11
12 def test_ObjectWithAttributes_set_and_del():
13     obj = cObject.ObjectWithAttributes()
14     obj.some_attr = 'Some attribute'
15     assert hasattr(obj, 'some_attr')
16     delattr(obj, 'some_attr')
17     assert not hasattr(obj, 'some_attr')
18     with pytest.raises(AttributeError) as err:
19         obj.some_attr
20     assert err.value.args[0] == "'cObject.ObjectWithAttributes' object has no attribute
    ↪ 'some_attr'"

```

9.4 Emulating Sequence Types

This section describes how to make an object act like a sequence using `tp_as_sequence`. See also [Sequence Object Structures](#). This allows your objects to have set and get methods like a list.

As an example here is an extension that can represent a sequence of longs in C with a CPython sequence interface. The code is in `src/cpy/object/cSeqObject.c`.

First the class declaration:

```

#define PY_SSIZE_T_CLEAN
#include <Python.h>
#include "structmember.h"

typedef struct {
    PyObject_HEAD
    long *array_long;
    ssize_t size;
} SequenceLongObject;

```

Then the equivalent of `__new__`:

```

1 static PyObject *
2 SequenceLongObject_new(PyTypeObject *type, PyObject *Py_UNUSED(args),
3     PyObject *Py_UNUSED(kwds)) {
4     SequenceLongObject *self;
5     self = (SequenceLongObject *) type->tp_alloc(type, 0);
6     if (self != NULL) {
7         assert(!PyErr_Occurred());
8         self->size = 0;
9         self->array_long = NULL;
10    }
11    return (PyObject *) self;
12 }

```

And the equivalent of `__init__` that takes a Python sequence of ints:

```

1 static int
2 SequenceLongObject_init(SequenceLongObject *self, PyObject *args, PyObject *kwds) {
3     static char *kwlist[] = {"sequence", NULL};
4     PyObject *sequence = NULL;
5
6     if (!PyArg_ParseTupleAndKeywords(args, kwds, "O", kwlist, &sequence)) {
7         return -1;
8     }
9     if (!PySequence_Check(sequence)) {
10        return -2;
11    }
12    self->size = PySequence_Length(sequence);
13    self->array_long = malloc(self->size * sizeof(long));
14    if (!self->array_long) {
15        return -3;
16    }
17    for (Py_ssize_t i = 0; i < self->size; ++i) {
18        // New reference.
19        PyObject *py_value = PySequence_GetItem(sequence, i);
20        if (PyLong_Check(py_value)) {
21            self->array_long[i] = PyLong_AsLong(py_value);
22            Py_DECREF(py_value);
23        } else {
24            PyErr_Format(
25                PyExc_TypeError,
26                "Argument [%zd] must be a int, not type %s",
27                i,
28                Py_TYPE(sequence)->tp_name
29            );
30            // Clean up on error.
31            free(self->array_long);
32            self->array_long = NULL;
33            Py_DECREF(py_value);
34            return -4;
35        }
36    }
37    return 0;
38 }

```

And de-allocation:

```

static void
SequenceLongObject_dealloc(SequenceLongObject *self) {
    free(self->array_long);
    Py_TYPE(self)->tp_free((PyObject *) self);
}

```

9.4.1 The Sequence Function Table

The sequence functions are a table of function pointers in the `PySequenceMethods` table. Here is a summary of the fields, each is described in more detail below:

Table 1: Sequence Methods

Member	Description
<code>sq_length</code>	Returns the length of the sequence.
<code>sq_concat</code>	Takes two sequences and returns a new third one with the first and second concatenated.
<code>sq_repeat</code>	Returns a new sequence with the old one repeated <i>n</i> times.
<code>sq_item</code>	Returns a <i>new</i> reference to the <i>n</i> 'th item in the sequence. Negative indexes are handled appropriately. Used by <code>PySequence_GetItem()</code> . This is a fairly crucial implementation for a sequence as <code>PySequence_Check()</code> detects this to decide if the object is a sequence.
<code>sq_ass_item</code>	Sets the <i>n</i> 'th item in the sequence. If the value is NULL the item is deleted and the sequence concatenated (thus called by <code>PyObject_DelItem()</code>). Negative indexes are handled appropriately. Used by <code>PyObject_SetItem()</code> .
<code>sq_ass_contains</code>	Returns non-zero if the sequence contains the given object. Used by <code>PySequence_Contains()</code> . This slot may be left to NULL, in this case <code>PySequence_Contains()</code> simply traverses the sequence until it finds a match.
<code>sq_inplace_concat</code>	Provides in-place concatenation, for example <code>+=</code> . If this slot is NULL then <code>PySequence_Concat()</code> will be used returning a new object.
<code>sq_inplace_repeat</code>	Provides in-place concatenation, for example <code>+=</code> . Returns the existing sequence repeated <i>n</i> times. If this slot is NULL then <code>PySequence_Repeat()</code> will be used returning a new object.

Here are the individual functions, their implementation and the test code.

9.4.2 `sq_length`

Table 2: Sequence Methods: `sq_length`

Member	<code>sq_length</code>
Function type	<code>lenfunc</code>
Function signature	<code>Py_ssize_t (*lenfunc)(PyObject*)</code>
Description	Returns the length of the sequence.

Implementation

In `src/cpy/object/cSeqObject.c`:

```
static Py_ssize_t
SequenceLongObject_sq_length(PyObject *self) {
    return ((SequenceLongObject *) self)->size;
}
```

Tests

Tests are in `tests/unit/test_c_seqobject.py`:

```
from cPyExtPatt import cSeqObject

def test_SequenceLongObject_len():
    obj = cSeqObject.SequenceLongObject([7, 4, 1, ])
    assert len(obj) == 3
```

9.4.3 sq_concat

Table 3: Sequence Methods: sq_concat

Member	<code>sq_concat</code>
Function type	<code>binaryfunc</code>
Function signature	<code>PyObject *(*binaryfunc)(PyObject*, PyObject*)</code>
Description	Takes two sequences and returns a new third one with the first and second concatenated. This is used by the + Python operator and the <code>PySequence_Concat()</code> C API.

Implementation

Note the use of forward references for the type checker as we need to check of the second argument is of the correct type. In `src/cpy/Object/cSeqObject.c`:

```

1 // Forward references
2 static PyTypeObject SequenceLongObjectType;
3 static int is_sequence_of_long_type(PyObject *op);
4
5 /** Returns a new SequenceLongObject composed of self + other. */
6 static PyObject *
7 SequenceLongObject_sq_concat(PyObject *self, PyObject *other) {
8     if (!is_sequence_of_long_type(other)) {
9         PyErr_Format(
10             PyExc_TypeError,
11             "%s(): argument 1 must have type \"SequenceLongObject\" not %s",
12             Py_TYPE(other)->tp_name
13         );
14         return NULL;
15     }
16     PyObject *ret = SequenceLongObject_new(&SequenceLongObjectType, NULL, NULL);
17     if (!ret) {
18         assert(PyErr_Occurred());
19         return NULL;
20     }
21     /* For convenience. */
22     SequenceLongObject *ret_as_slo = (SequenceLongObject *) ret;
23     ret_as_slo->size = ((SequenceLongObject *) self)->size \
24         + ((SequenceLongObject *) other)->size;
25     ret_as_slo->array_long = malloc(ret_as_slo->size * sizeof(long));
26     if (!ret_as_slo->array_long) {
27         PyErr_Format(
28             PyExc_MemoryError, "%s(): Can not create new object.", __FUNCTION__
29         );
30         Py_DECREF(ret);
31         return NULL;
32     }
33
34     ssize_t i = 0;
35     ssize_t ub = ((SequenceLongObject *) self)->size;
36     while (i < ub) {
37         ret_as_slo->array_long[i] = ((SequenceLongObject *) self)->array_long[i];
38         i++;

```

(continues on next page)

(continued from previous page)

```

39 }
40 ssize_t j = 0;
41 ub = ((SequenceLongObject *) other)->size;
42 while (j < ub) {
43     ret_as_slo->array_long[i] = ((SequenceLongObject *) other)->array_long[j];
44     i++;
45     j++;
46 }
47 return ret;
48 }

```

Tests

Tests are in tests/unit/test_c_seqobject.py:

```

def test_SequenceLongObject_concat():
    obj_a = cSeqObject.SequenceLongObject([7, 4, 1, ])
    obj_b = cSeqObject.SequenceLongObject([70, 40, 100, ])
    assert id(obj_a) != id(obj_b)
    obj = obj_a + obj_b
    assert id(obj) != id(obj_a)
    assert id(obj) != id(obj_b)
    assert len(obj) == 6
    assert list(obj) == [7, 4, 1, ] + [70, 40, 100, ]

```

9.4.4 sq_repeat

Table 4: Sequence Methods: sq_repeat

Member	sq_repeat
Function type	ssizeargfunc
Function signature	PyObject *(*ssizeargfunc)(PyObject*, Py_ssize_t)
Description	Returns a new sequence with the old one repeated the given number of times. This is used by the * Python operator and the <code>PySequence_Repeat()</code> C API.

Implementation

The implementation is fairly straightforward in src/cpy/Object/cSeqObject.c. Note that count can be zero or negative:

```

1  /** Return a new sequence which contains the old one repeated count times. */
2  static PyObject *
3  SequenceLongObject_sq_repeat(PyObject *self, Py_ssize_t count) {
4      PyObject *ret = SequenceLongObject_new(&SequenceLongObjectType, NULL, NULL);
5      if (!ret) {
6          assert(PyErr_Occurred());
7          return NULL;
8      }
9      assert(ret != self);
10     if (((SequenceLongObject *) self)->size > 0 && count > 0) {
11         /* For convenience. */

```

(continues on next page)

(continued from previous page)

```

12     SequenceLongObject *self_as_slo = (SequenceLongObject *) self;
13     SequenceLongObject *ret_as_slo = (SequenceLongObject *) ret;
14     ret_as_slo->size = self_as_slo->size * count;
15     assert(ret_as_slo->size > 0);
16     ret_as_slo->array_long = malloc(ret_as_slo->size * sizeof(long));
17     if (!ret_as_slo->array_long) {
18         PyErr_Format(
19             PyExc_MemoryError, "%s(): Can not create new object.", __FUNCTION__
20         );
21         Py_DECREF(ret);
22         return NULL;
23     }
24     Py_ssize_t ret_index = 0;
25     for (Py_ssize_t i = 0; i < count; ++i) {
26         for (Py_ssize_t j = 0; j < self_as_slo->size; ++j) {
27             ret_as_slo->array_long[ret_index] = self_as_slo->array_long[j];
28             ++ret_index;
29         }
30     }
31 } else {
32     /* Empty sequence. */
33 }
34 return ret;
35 }

```

Tests

Tests are in tests/unit/test_c_seqobject.py:

```

1 @pytest.mark.parametrize(
2     'initial_sequence, count, expected',
3     (
4         ([], 1, []),
5         ([7, 4, 1, ], 0, []),
6         ([7, 4, 1, ], -1, []),
7         ([7, 4, 1, ], 1, [7, 4, 1, ]),
8         ([7, 4, 1, ], 2, [7, 4, 1, 7, 4, 1, ]),
9         ([7, 4, 1, ], 3, [7, 4, 1, 7, 4, 1, 7, 4, 1, ]),
10    )
11 )
12 def test_SequenceLongObject_repeat(initial_sequence, count, expected):
13     obj_a = cSeqObject.SequenceLongObject(initial_sequence)
14     obj = obj_a * count
15     print()
16     assert id(obj_a) != id(obj)
17     assert list(obj) == expected
18     assert list(obj) == (list(obj_a) * count)

```

9.4.5 sq_item

`sq_item` gives read access to an indexed member.

Table 5: Sequence Methods: `sq_item`

Member	<code>sq_item</code>
Function type	<code>ssizeargfunc</code>
Function signature	<code>PyObject *(*ssizeargfunc)(PyObject*, Py_ssize_t)</code>
Description	Returns a <i>new</i> reference to the n'th item in the sequence. Negative indexes are handled appropriately. Used by <code>PySequence_GetItem()</code> . This is a fairly crucial implementation for a sequence as <code>PySequence_Check()</code> detects this to decide if the object is a sequence.

Implementation

In `src/cpy/Object/cSeqObject.c`:

```

1  /** Returns a new reference to an indexed item in a sequence. */
2  static PyObject *
3  SequenceLongObject_sq_item(PyObject *self, Py_ssize_t index) {
4      Py_ssize_t my_index = index;
5      if (my_index < 0) {
6          my_index += SequenceLongObject_sq_length(self);
7      }
8      // Corner case example: len(self) == 0 and index < 0
9      if (my_index < 0 || my_index >= SequenceLongObject_sq_length(self)) {
10         PyErr_Format(
11             PyExc_IndexError,
12             "Index %ld is out of range for length %ld",
13             index,
14             SequenceLongObject_sq_length(self)
15         );
16         return NULL;
17     }
18     return PyLong_FromLong(((SequenceLongObject *) self)->array_long[my_index]);
19 }

```

Tests

Tests are in `tests/unit/test_c_seqobject.py`:

```

1  from cPyExtPatt import cSeqObject
2
3  @pytest.mark.parametrize(
4      'initial_sequence, index, expected',
5      (
6          ([7, 4, 1, ], 0, 7,),
7          ([7, 4, 1, ], 1, 4,),
8          ([7, 4, 1, ], 2, 1,),
9          ([7, 4, 1, ], -1, 1,),
10         ([7, 4, 1, ], -2, 4,),
11         ([7, 4, 1, ], -3, 7,),
12     )
13 )

```

(continues on next page)

(continued from previous page)

```

14 def test_SequenceLongObject_item(initial_sequence, index, expected):
15     obj = cSeqObject.SequenceLongObject(initial_sequence)
16     assert obj[index] == expected

```

And failure modes:

```

1 from cPyExtPatt import cSeqObject
2
3 @pytest.mark.parametrize(
4     'initial_sequence, index, expected',
5     (
6         ([], 0, 'Index 0 is out of range for length 0',),
7         ([], -1, 'Index -1 is out of range for length 0',),
8         ([1, ], 2, 'Index 2 is out of range for length 1',),
9     )
10 )
11 def test_SequenceLongObject_item_raises(initial_sequence, index, expected):
12     obj = cSeqObject.SequenceLongObject(initial_sequence)
13     with pytest.raises(IndexError) as err:
14         obj[index]
15     assert err.value.args[0] == expected

```

9.4.6 sq_ass_item

`sq_ass_item` gives write and delete access to an indexed member.

Table 6: Sequence Methods: `sq_ass_item`

Member	<code>sq_ass_item</code>
Function type	<code>ssizeobjargproc</code>
Function signature	<code>int (*ssizeobjargproc)(PyObject*, Py_ssize_t, PyObject*)</code>
Description	Sets the the n'th item in the sequence. Returns 0 on success, -1 on failure. If the value is NULL the item is deleted and the sequence concatenated (thus called by <code>PyObject_DelItem()</code>). Negative indexes are handled appropriately. Used by <code>PyObject_SetItem()</code> .

Implementation

Warning

There is an undocumented feature when using `sq_ass_item` from `PyObject_SetItem()` and `PyObject_DelItem()` when using negative indexes and when the index is *out of range*.

In that case, before the `sq_ass_item` function is called the index will have had the sequence length added to it.

For example if the sequence length is 3 and the given index is -4 then the index that the `sq_ass_item` function receives is -1. If the given index is -5 then the index that the `sq_ass_item` function receives is -2.

Thus the slightly odd code below to fix this problem. Failing to do this will mean out of range errors will not be detected by the `sq_ass_item` function and any error message will be wrong.

In `src/cpy/Object/cSeqObject.c`:


```

1 static int
2 SequenceLongObject_sq_ass_item(PyObject *self, Py_ssize_t index, PyObject *value) {
3     /* See warning above. */
4     if (index < 0) {
5         index -= SequenceLongObject_sq_length(self);
6     }
7
8     Py_ssize_t my_index = index;
9     if (my_index < 0) {
10         my_index += SequenceLongObject_sq_length(self);
11     }
12     // Corner case example: len(self) == 0 and index < 0
13     if (my_index < 0 || my_index >= SequenceLongObject_sq_length(self)) {
14         PyErr_Format(
15             PyExc_IndexError,
16             "Index %ld is out of range for length %ld",
17             index,
18             SequenceLongObject_sq_length(self)
19         );
20         return -1;
21     }
22     if (value != NULL) {
23         /* Just set the value. */
24         if (!PyLong_Check(value)) {
25             PyErr_Format(
26                 PyExc_TypeError,
27                 "sq_ass_item value needs to be an int, not type %s",
28                 Py_TYPE(value)->tp_name
29             );
30             return -1;
31         }
32         ((SequenceLongObject *) self)->array_long[my_index] = PyLong_AsLong(value);
33     } else {
34         /* Delete the value. */
35         /* For convenience. */
36         SequenceLongObject *self_as_slo = (SequenceLongObject *) self;
37         /* Special case: deleting the only item in the array. */
38         if (self_as_slo->size == 1) {
39             free(self_as_slo->array_long);
40             self_as_slo->array_long = NULL;
41             self_as_slo->size = 0;
42         } else {
43             /* Delete the value and re-compose the array. */
44             long *new_array = malloc((self_as_slo->size - 1) * sizeof(long));
45             if (!new_array) {
46                 PyErr_Format(
47                     PyExc_MemoryError,
48                     "sq_ass_item can not allocate new array. %s#%d",
49                     __FILE__, __LINE__
50                 );
51                 return -1;
52             }
53             /* Copy up to the index. */

```

(continues on next page)

(continued from previous page)

```

54     Py_ssize_t index_new_array = 0;
55     for (Py_ssize_t i = 0; i < my_index; ++i, ++index_new_array) {
56         new_array[index_new_array] = self_as_slo->array_long[i];
57     }
58     /* Copy past the index. */
59     for (
60         Py_ssize_t i = my_index + 1;
61         i < self_as_slo->size;
62         ++i, ++index_new_array) {
63         new_array[index_new_array] = self_as_slo->array_long[i];
64     }
65
66     free(self_as_slo->array_long);
67     self_as_slo->array_long = new_array;
68     --self_as_slo->size;
69 }
70 }
71 return 0;
72 }

```

Tests

Tests are in tests/unit/test_c_seqobject.py which includes failure modes. First setting a value:

```

1  from cPyExtPatt import cSeqObject
2
3  @pytest.mark.parametrize(
4      'initial_sequence, index, value, expected',
5      (
6          ([7, 4, 1, ], 0, 14, [14, 4, 1, ],),
7          ([7, 4, 1, ], -1, 14, [7, 4, 14, ],),
8          ([7, 4, 1, ], -2, 14, [7, 14, 1, ],),
9          ([7, 4, 1, ], -3, 14, [14, 4, 1, ],),
10     )
11 )
12 def test_SequenceLongObject_setitem(initial_sequence, index, value, expected):
13     obj = cSeqObject.SequenceLongObject(initial_sequence)
14     obj[index] = value
15     assert list(obj) == expected

```

Setting a value with an out of range index:

```

1  from cPyExtPatt import cSeqObject
2
3  @pytest.mark.parametrize(
4      'initial_sequence, index, expected',
5      (
6          ([7, 4, 1, ], 3, 'Index 3 is out of range for length 3',),
7          ([7, 4, 1, ], -4, 'Index -4 is out of range for length 3',),
8      )
9  )
10 def test_SequenceLongObject_setitem_raises(initial_sequence, index, expected):

```

(continues on next page)

(continued from previous page)

```

11     obj = cSeqObject.SequenceLongObject(initial_sequence)
12     with pytest.raises(IndexError) as err:
13         obj[index] = 100
14     assert err.value.args[0] == expected

```

Deleting a value:

```

1  from cPyExtPatt import cSeqObject
2
3  @pytest.mark.parametrize(
4      'initial_sequence, index, expected',
5      (
6          ([7, ], 0, []),
7          ([7, ], -1, []),
8          ([7, 4, 1, ], 1, [7, 1, ]),
9          ([7, 4, ], 0, [4, ]),
10         ([7, 4, 1, ], -1, [7, 4, ]),
11         ([7, 4, 1, ], -2, [7, 1, ]),
12         ([7, 4, 1, ], -3, [4, 1, ]),
13     )
14 )
15 def test_SequenceLongObject_delitem(initial_sequence, index, expected):
16     obj = cSeqObject.SequenceLongObject(initial_sequence)
17     del obj[index]
18     assert list(obj) == expected

```

Deleting a value with an out of range index:

```

1  from cPyExtPatt import cSeqObject
2
3  @pytest.mark.parametrize(
4      'initial_sequence, index, expected',
5      (
6          ([], 0, 'Index 0 is out of range for length 0'),
7          ([], -1, 'Index -1 is out of range for length 0'),
8          ([7, ], 1, 'Index 1 is out of range for length 1'),
9          ([7, ], -3, 'Index -3 is out of range for length 1'),
10     )
11 )
12 def test_SequenceLongObject_delitem_raises(initial_sequence, index, expected):
13     obj = cSeqObject.SequenceLongObject(initial_sequence)
14     with pytest.raises(IndexError) as err:
15         del obj[index]
16     assert err.value.args[0] == expected

```

9.4.7 sq_ass_contains

Table 7: Sequence Methods: sq_ass_contains

Member	sq_ass_contains
Function type	objobjproc
Function signature	int (*objobjproc)(PyObject*, PyObject*)
Description	Returns non-zero if the sequence contains the given object. If an item in o is equal to value, return 1, otherwise return 0. On error, return -1. This is equivalent to the Python expression <code>value in o</code> . Used by <code>PySequence_Contains()</code> . This slot may be left to NULL, in this case <code>PySequence_Contains()</code> simply traverses the sequence until it finds a match.

Implementation

In `src/cpy/Object/cSeqObject.c`:

```

1  /** If an item in self is equal to value, return 1, otherwise return 0.
2   * On error, return -1. */
3  static int
4  SequenceLongObject_sq_contains(PyObject *self, PyObject *value) {
5      fprintf(
6          stdout, "%s()#%d: self=%p value=%p\n",
7          __FUNCTION__, __LINE__, (void *) self, (void *) value
8      );
9      if (!PyLong_Check(value)) {
10         /* Alternates: Could raise TypeError or return -1.
11          * Here we act benignly! */
12         return 0;
13     }
14     long c_value = PyLong_AsLong(value);
15     /* For convenience. */
16     SequenceLongObject *self_as_slo = (SequenceLongObject *) self;
17     for (Py_ssize_t i = 0; i < SequenceLongObject_sq_length(self); ++i) {
18         if (self_as_slo->array_long[i] == c_value) {
19             return 1;
20         }
21     }
22     return 0;
23 }
```

Note

Whilst `SequenceLongObject_sq_contains()` returns 0 or 1 however Python code such as `value in obj` converts this to False and True respectively

Tests

Tests are in `tests/unit/test_c_seqobject.py`:

```

1  from cPyExtPatt import cSeqObject
2
3  @pytest.mark.parametrize(
```

(continues on next page)

(continued from previous page)

```

4     'initial_sequence, value, expected',
5     (
6         ([7, ], 0, False,),
7         ([7, ], 7, True,),
8         ([1, 4, 7, ], 7, True,),
9     )
10 )
11 def test_SequenceLongObject_contains(initial_sequence, value, expected):
12     obj = cSeqObject.SequenceLongObject(initial_sequence)
13     result = value in obj
14     assert result == expected

```

9.4.8 sq_inplace_concat

Table 8: Sequence Methods: sq_inplace_concat

Member	<code>sq_inplace_concat</code>
Function type	<code>binaryfunc</code>
Function signature	<code>PyObject *(*binaryfunc)(PyObject*, PyObject*)</code>
Description	Provides in-place concatenation, for example <code>+=</code> . If this slot is NULL then <code>PySequence_InPlaceConcat()</code> will use <code>PySequence_Concat()</code> .

Implementation

If `sq_concat` is implemented then this need not be implemented In `src/cpy/Object/cSeqObject.c` this is not implemented to show this mechanism at work.

Tests

Tests are in `tests/unit/test_c_seqobject.py`:

```

from cPyExtPatt import cSeqObject

def test_SequenceLongObject_concat_inplace():
    obj_a = cSeqObject.SequenceLongObject([7, 4, 1, ])
    obj_b = cSeqObject.SequenceLongObject([70, 40, 100, ])
    assert id(obj_a) != id(obj_b)
    obj_a += obj_b
    assert len(obj_a) == 6
    assert list(obj_a) == [7, 4, 1, ] + [70, 40, 100, ]

```

9.4.9 sq_inplace_repeat

Table 9: Sequence Methods: sq_inplace_repeat

Member	<code>sq_inplace_repeat</code>
Function type	<code>ssizeargfunc</code>
Function signature	<code>PyObject *(*ssizeargfunc)(PyObject*, Py_ssize_t)</code>
Description	Provides in-place concatenation, for example <code>+=</code> . Returns the existing sequence repeated <code>n</code> times. If this slot is NULL then <code>PySequence_Repeat()</code> will be used returning a new object.

Implementation

If `sq_repeat` is implemented then this need not be implemented In `src/cpy/Object/cSeqObject.c` this is not implemented to show this mechanism at work.

Tests

Tests are in `tests/unit/test_c_seqobject.py`:

```

1  from cPyExtPatt import cSeqObject
2
3  @pytest.mark.parametrize(
4      'initial_sequence, count, expected',
5      (
6          ([], 1, []),
7          ([7, 4, 1, ], 0, []),
8          ([7, 4, 1, ], -1, []),
9          ([7, 4, 1, ], 1, [7, 4, 1, ]),
10         ([7, 4, 1, ], 2, [7, 4, 1, 7, 4, 1, ]),
11         ([7, 4, 1, ], 3, [7, 4, 1, 7, 4, 1, 7, 4, 1, ]),
12     )
13 )
14 def test_SequenceLongObject_repeat_inplace(initial_sequence, count, expected):
15     obj = cSeqObject.SequenceLongObject(initial_sequence)
16     obj *= count
17     assert list(obj) == expected
18     assert list(obj) == (initial_sequence * count)

```

9.4.10 The Function Table

All these functions are gathered together in a `PySequenceMethods` table:

```

1  static PySequenceMethods SequenceLongObject_sequence_methods = {
2      .sq_length = (lenfunc)SequenceLongObject_sq_length,
3      .sq_concat = (binaryfunc)SequenceLongObject_sq_concat,
4      .sq_repeat = (ssizeargfunc)SequenceLongObject_sq_repeat,
5      .sq_item = (ssizeargfunc)SequenceLongObject_sq_item,
6      .sq_ass_item = (ssizeobjargproc)SequenceLongObject_sq_ass_item,
7      .sq_contains = (objobjproc)SequenceLongObject_sq_contains,
8      .sq_inplace_concat = (binaryfunc)NULL,           // Not implemented. See above.
9      .sq_inplace_repeat = (ssizeargfunc)NULL,        // Not implemented. See above.
10 };

```

And the `SequenceLongObjectType` type is declared with the `tp_as_sequence` field referring to this table:

```

1  static PyTypeObject SequenceLongObjectType = {
2      PyVarObject_HEAD_INIT(NULL, 0)
3      .tp_name = "SequenceLongObject",
4      .tp_basicsize = sizeof(SequenceLongObject),
5      .tp_itemsize = 0,
6      .tp_dealloc = (destructor) SequenceLongObject_dealloc,
7      .tp_as_sequence = &SequenceLongObject_sequence_methods,
8      .tp_str = (reprfunc) SequenceLongObject__str__,
9      .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,

```

(continues on next page)

(continued from previous page)

```

10     .tp_doc = "Sequence of long integers.",
11     .tp_methods = SequenceLongObject_methods,
12     .tp_init = (initproc) SequenceLongObject_init,
13     .tp_new = SequenceLongObject_new,
14 };

```

9.4.11 Module Initialisation

This is as usual:

```

1  static PyModuleDef sequence_object_cmodule = {
2      PyModuleDef_HEAD_INIT,
3      .m_name = "cSeqObject",
4      .m_doc = (
5          "Example module that creates an extension type with sequence methods"
6      ),
7      .m_size = -1,
8      .m_methods = cIterator_methods,
9  };
10
11  PyMODINIT_FUNC
12  PyInit_cSeqObject(void) {
13      PyObject *m;
14      m = PyModule_Create(&sequence_object_cmodule);
15      if (m == NULL) {
16          return NULL;
17      }
18
19      if (PyType_Ready(&SequenceLongObjectType) < 0) {
20          Py_DECREF(m);
21          return NULL;
22      }
23      Py_INCREF(&SequenceLongObjectType);
24      if (PyModule_AddObject(
25          m,
26          "SequenceLongObject",
27          (PyObject *) &SequenceLongObjectType) < 0
28      ) {
29          Py_DECREF(&SequenceLongObjectType);
30          Py_DECREF(m);
31          return NULL;
32      }
33      return m;
34  }

```

9.5 TODOs:

Todo

“Creating New Types”: Add a section on making an object act like a number using `tp_as_number`. See also [Number](#)

Object Structures

 Todo

“Creating New Types”: Add a section on making an object act like a mapping object using `tp_as_mapping`. See also [Mapping Object Structures](#)

NOTE: This is rarely used in the Python stdlib, really only for dictionaries.

SETTING AND GETTING MODULE GLOBALS

This section describes how you create and access module globals from Python C Extensions.

In this module, written as a Python extension in C, we are going to have a string, int, list, tuple and dict in global scope. In the C code we firstly define names for them:

```
const char *NAME_INT = "INT";
const char *NAME_STR = "STR";
const char *NAME_LST = "LST";
const char *NAME_TUP = "TUP";
const char *NAME_MAP = "MAP";
```

These are the names of the objects that will appear in the Python module:

```
>>> import cModuleGlobals
>>> dir(cModuleGlobals)
['INT', 'LST', 'MAP', 'STR', 'TUP', '__doc__', '__file__', '__loader__', '__name__', '__package__', 'print']
```

10.1 Initialising Module Globals

This is the module declaration, it will be called `cModuleGlobals` and has just one function; `print()` that will access the module globals from C:

```
1 static PyMethodDef cModuleGlobals_methods[] = {
2     {"print", (PyCFunction)print_globals, METH_NOARGS,
3      "Access and print out the globals."},
4     },
5     {NULL, NULL, 0, NULL} /* Sentinel */
6 };
7
8
9 static PyModuleDef cModuleGlobals_module = {
10     PyModuleDef_HEAD_INIT,
11     "cModuleGlobals",
12     "Examples of global values in a module.",
13     -1,
14     cModuleGlobals_methods, /* cModuleGlobals_methods */
15     NULL, /* inquiry m_reload */
16     NULL, /* traverseproc m_traverse */
17     NULL, /* inquiry m_clear */
```

(continues on next page)

(continued from previous page)

```

18     NULL, /* freefunc m_free */
19 };

```

The module initialisation code is next, this uses the Python C API to create the various global objects:

```

1 PyMODINIT_FUNC
2 PyInit_cModuleGlobals(void)
3 {
4     PyObject *m = NULL;
5
6     m = PyModule_Create(&cModuleGlobals_module);
7
8     if (m == NULL) {
9         goto except;
10    }
11    /* Adding module globals */
12    if (PyModule_AddIntConstant(m, NAME_INT, 42)) {
13        goto except;
14    }
15    if (PyModule_AddStringConstant(m, NAME_STR, "String value")) {
16        goto except;
17    }
18    if (PyModule_AddObject(m, NAME_TUP, Py_BuildValue("iii", 66, 68, 73))) {
19        goto except;
20    }
21    if (PyModule_AddObject(m, NAME_LST, Py_BuildValue("[iii]", 66, 68, 73))) {
22        goto except;
23    }
24    /* An invented convenience function for this dict. See below. */
25    if (add_map_to_module(m)) {
26        goto except;
27    }
28    goto finally;
29 except:
30     Py_XDECREF(m);
31     m = NULL;
32 finally:
33     return m;
34 }

```

The dict is added in a separate C function merely for readability:

```

1 /* Add a dict of {str : int, ...}.
2  * Returns 0 on success, 1 on failure.
3  */
4 int add_map_to_module(PyObject *module) {
5     int ret = 0;
6     PyObject *pMap = NULL;
7     PyObject *key = NULL;
8     PyObject *val = NULL;
9
10    pMap = PyDict_New();

```

(continues on next page)

(continued from previous page)

```

11     if (!pMap) {
12         goto except;
13     }
14     /* Load map. */
15     key = PyBytes_FromString("66");
16     val = PyLong_FromLong(66);
17     if (PyDict_SetItem(pMap, key, val)) {
18         goto except;
19     }
20     Py_XDECREF(key);
21     Py_XDECREF(val);
22     key = PyBytes_FromString("123");
23     val = PyLong_FromLong(123);
24     if (PyDict_SetItem(pMap, key, value)) {
25         goto except;
26     }
27     Py_XDECREF(key);
28     Py_XDECREF(val);
29     /* Add map to module. */
30     if (PyModule_AddObject(module, NAME_MAP, pMap)) {
31         goto except;
32     }
33     ret = 0;
34     goto finally;
35 except:
36     Py_XDECREF(pMap);
37     Py_XDECREF(key);
38     Py_XDECREF(val);
39     ret = 1;
40 finally:
41     return ret;
42 }

```

10.2 Getting and Setting

10.2.1 From Python

Once the module is built we can access the globals from Python as usual:

```

1  >>> import cModuleGlobals
2  >>> dir(cModuleGlobals)
3  ['INT', 'LST', 'MAP', 'STR', 'TUP', '__doc__', '__file__', '__loader__', '__name__', '__
   ↪package__', 'print']
4  >>> cModuleGlobals.STR
5  'String value'
6  >>> cModuleGlobals.STR = 'F'
7  >>> cModuleGlobals.STR
8  'F'
9  >>> cModuleGlobals.MAP
10 {b'123': 123, b'66': 66}
11 >>> cModuleGlobals.MAP[b'asd'] = 9

```

(continues on next page)

(continued from previous page)

```

12 >>> cModuleGlobals.MAP
13 {b'123': 123, b'asd': 9, b'66': 66}

```

10.2.2 Getting Module Globals From C

Accessing Python module globals from C is a little bit more tedious as we are getting borrowed references from the modules `__dict__` and we should increment and decrement them appropriately. Here we print out the global INT as both a Python object and a 'C' long:

```

1  static PyObject *print_global_INT(PyObject *pMod) {
2      PyObject *ret = NULL;
3      PyObject *pItem = NULL;
4      long val;
5
6      /* Sanity check. */
7      assert(pMod);
8      assert(PyModule_CheckExact(pMod));
9      assert(! PyErr_Occurred());
10
11     /* NOTE: PyModule_GetDict(pMod); never fails and returns a borrowed
12      * reference. pItem is NULL or a borrowed reference.
13      */
14     pItem = PyDict_GetItemString(PyModule_GetDict(pMod), NAME_INT);
15     if (! pItem) {
16         PyErr_Format(PyExc_AttributeError,
17                     "Module '%s' has no attribute '%s'.", \
18                     PyModule_GetName(pMod), NAME_INT
19                     );
20         goto except;
21     }
22     Py_INCREF(pItem);
23     fprintf(stdout, "Integer: \"%s\" ", NAME_INT);
24     PyObject_Print(pItem, stdout, 0);
25     val = PyLong_AsLong(pItem);
26     fprintf(stdout, " C long: %ld ", val);
27     fprintf(stdout, "\n");
28
29     assert(! PyErr_Occurred());
30     Py_INCREF(Py_None);
31     ret = Py_None;
32     goto finally;
33 except:
34     assert(PyErr_Occurred());
35     Py_XDECREF(ret);
36     ret = NULL;
37 finally:
38     Py_DECREF(pItem);
39     return ret;
40 }

```

From Python we would see this (C's `print_global_INT()` is mapped to Python's `cModuleGlobals.printINT()`):

```
>>> import cModuleGlobals
>>> cModuleGlobals.printINT()
Module:
<module 'cModuleGlobals' from './cModuleGlobals.so'>
Integer: "INT" 42 C long: 42
```

10.2.3 Setting Module Globals From C

This is similar to the get code above but using `int PyDict_SetItemString(PyObject *p, const char *key, PyObject *val)` where `val` will **not** be a *stolen* reference thus must be created as a temporary and subsequently decref'd:

```
1 static PyObject *some_set_function(PyObject *pMod) {
2     PyObject *ret = NULL;
3     long val = ...; /* Some computed value. */
4
5     PyObject *py_long = PyLong_FromLong(val);
6     if (! py_long) {
7         goto except;
8     }
9     /* PyDict_SetItemString does not steal a reference to py_long
10    so we have to decref the temporary. */
11    if (PyDict_SetItemString(PyModule_GetDict(pMod), NAME_INT, py_long)) {
12        PyErr_Format(PyExc_AttributeError,
13                    "Can not set Module '%s' attribute '%s'.", \
14                    PyModule_GetName(pMod), NAME_INT
15                    );
16        goto except;
17    }
18
19    assert(! PyErr_Occurred());
20    Py_INCREF(Py_None);
21    ret = Py_None;
22    goto finally;
23 except:
24    assert(PyErr_Occurred());
25    Py_XDECREF(ret);
26    ret = NULL;
27 finally:
28    /* See comment above about PyDict_SetItemString(). */
29    Py_XDECREF(py_long);
30    return ret;
31 }
```


LOGGING AND FRAMES

This chapter describes how to use the Python logging interface for C and how to access Python frames from C.

11.1 Logging From C

This presents a recipe for using the Python logging module in C.

Many thanks to [nnathan](#) for starting this.

We import the module and define C equivalent logging functions that are compatible with the **printf* family.

11.1.1 Logging C Declarations

First define the log levels, `#define` is used so the that switch case does not issue a compile time non-const error:

```
1 #define PPY_SSIZE_T_CLEAN
2 #include <Python.h>
3 /* For va_start, va_end */
4 #include <stdarg.h>
5
6 /* logging levels defined by logging module
7  * From: https://docs.python.org/3/library/logging.html#logging-levels */
8 #define LOGGING_DEBUG 10
9 #define LOGGING_INFO 20
10 #define LOGGING_WARNING 30
11 #define LOGGING_ERROR 40
12 #define LOGGING_CRITICAL 50
13 #define LOGGING_EXCEPTION 60
```

11.1.2 Logging C Globals

Then two globals, the first is the imported logging module, the next the current logger:

```
/* This modules globals */
static PyObject *g_logging_module = NULL; /* Initialise by PyInit_cLogging() below. */
static PyObject *g_logger = NULL;
```

11.1.3 Logging C Functions

Now a function to get a logger object from the logging module:

```

1 static PyObject *py_get_logger(char *logger_name) {
2     assert(g_logging_module);
3     PyObject *logger = NULL;
4
5     logger = PyObject_CallMethod(g_logging_module, "getLogger", "s", logger_name);
6     if (logger == NULL) {
7         const char *err_msg = "failed to call logging.getLogger";
8         PyErr_SetString(PyExc_RuntimeError, err_msg);
9     }
10    /*
11    fprintf(stdout, "%s()#%d logger=0x%p\n", __FUNCTION__, __LINE__, (void *)logger);
12    */
13    return logger;
14 }

```

Now the main interface to logging function:

```

1 static PyObject *
2 py_log_msg(int log_level, char *printf_fmt, ...) {
3     assert(g_logger);
4     assert(!PyErr_Occurred());
5     PyObject *log_msg = NULL;
6     PyObject *ret = NULL;
7     va_list fmt_args;
8
9     va_start(fmt_args, printf_fmt);
10    log_msg = PyUnicode_FromFormatV(printf_fmt, fmt_args);
11    va_end(fmt_args);
12
13    if (log_msg == NULL) {
14        /* fail. */
15        ret = PyObject_CallMethod(
16            g_logger,
17            "critical",
18            "0", "Unable to create log message."
19        );
20    } else {
21        /* call function depending on loglevel */
22        switch (log_level) {
23            case LOGGING_DEBUG:
24                ret = PyObject_CallMethod(g_logger, "debug", "0", log_msg);
25                break;
26            case LOGGING_INFO:
27                ret = PyObject_CallMethod(g_logger, "info", "0", log_msg);
28                break;
29            case LOGGING_WARNING:
30                ret = PyObject_CallMethod(g_logger, "warning", "0", log_msg);
31                break;
32            case LOGGING_ERROR:
33                ret = PyObject_CallMethod(g_logger, "error", "0", log_msg);
34                break;
35            case LOGGING_CRITICAL:
36                ret = PyObject_CallMethod(g_logger, "critical", "0", log_msg);

```

(continues on next page)

(continued from previous page)

```

37         break;
38     default:
39         ret = PyObject_CallMethod(g_logger, "critical", "0", log_msg);
40         break;
41     }
42     assert(!PyErr_Occurred());
43 }
44 Py_DECREF(log_msg);
45 return ret;
46 }

```

A function to set a log level:

```

1 static PyObject *
2 py_log_set_level(PyObject *Py_UNUSED(module), PyObject *args) {
3     assert(g_logger);
4     PyObject *py_log_level;
5
6     if (!PyArg_ParseTuple(args, "O", &py_log_level)) {
7         return NULL;
8     }
9     return PyObject_CallMethod(g_logger, "setLevel", "0", py_log_level);
10 }

```

And the main function to log a message:

```

1 static PyObject *
2 py_log_message(PyObject *Py_UNUSED(module), PyObject *args) {
3     int log_level;
4     char *message;
5
6     if (!PyArg_ParseTuple(args, "iz", &log_level, &message)) {
7         return NULL;
8     }
9     return py_log_msg(log_level, "%s", message);
10 }

```

11.1.4 cLogging Module

Setup the module functions:

```

1 static PyMethodDef logging_methods[] = {
2     /* ... */
3     {
4         "py_log_set_level",
5         (PyCFunction) py_log_set_level,
6         METH_VARARGS,
7         "Set the logging level."
8     },
9     {
10        "log",
11        (PyCFunction) py_log_message,

```

(continues on next page)

(continued from previous page)

```

12     METH_VARARGS,
13     "Log a message."
14 },
15 /* ... */
16 {NULL, NULL, 0, NULL} /* Sentinel */
17 };

```

The module definition:

```

static PyModuleDef cLogging = {
    PyModuleDef_HEAD_INIT,
    .m_name = "cLogging",
    .m_doc = "Logging module.",
    .m_size = -1,
    .m_methods = logging_methods,
};

```

The module initialisation, this is where the logging module is imported with `PyImport_ImportModule()`:

```

1 PyMODINIT_FUNC PyInit_cLogging(void) {
2     PyObject *m = PyModule_Create(&cLogging);
3     if (!m) {
4         goto except;
5     }
6     g_logging_import = PyImport_ImportModule("logging");
7     if (!g_logging_import) {
8         const char *err_msg = "failed to import 'logging'";
9         PyErr_SetString(PyExc_ImportError, err_msg);
10        goto except;
11    }
12    g_logger = py_get_logger("cLogging");
13    if (!g_logger) {
14        goto except;
15    }
16    /* Adding module globals */
17    /* logging levels defined by logging module.
18     * Note: In Python logging FATAL = CRITICAL */
19    if (PyModule_AddIntConstant(m, "INFO", LOGGING_INFO)) {
20        goto except;
21    }
22    if (PyModule_AddIntConstant(m, "WARNING", LOGGING_WARNING)) {
23        goto except;
24    }
25    if (PyModule_AddIntConstant(m, "ERROR", LOGGING_ERROR)) {
26        goto except;
27    }
28    if (PyModule_AddIntConstant(m, "FATAL", LOGGING_FATAL)) {
29        goto except;
30    }
31    if (PyModule_AddIntConstant(m, "CRITICAL", LOGGING_FATAL)) {
32        goto except;
33    }
34 }

```

(continues on next page)

(continued from previous page)

```

34     if (PyModule_AddIntConstant(m, "DEBUG", LOGGING_DEBUG)) {
35         goto except;
36     }
37     if (PyModule_AddIntConstant(m, "EXCEPTION", LOGGING_EXCEPTION)) {
38         goto except;
39     }
40
41     goto finally;
42 except:
43     /* abnormal cleanup */
44     /* cleanup logger references */
45     Py_XDECREF(g_logging_import);
46     Py_XDECREF(g_logger);
47 finally:
48     return m;
49 }

```

Create in the setup.py:

```

Extension(name=f"{PACKAGE_NAME}.Logging.cLogging",
        include_dirs=[],
        sources=["src/cpy/Logging/cLogging.c", ],
        extra_compile_args=extra_compile_args_c,
        language='c',
),

```

And run `python setup.py develop`.

11.1.5 Using and Testing

Using From C

To simply use the interface defined in the above function, use it like the *printf* family of functions:

```
py_log_msg(WARNING, "error code: %d", 10);
```

Using From Python

```

from cPyExtPatt.Logging import cLogging

cLogging.log(cLogging.ERROR, "Test log message")

```

There are various tests in `tests/unit/test_c_logging.py`. As pytest swallows logging messages there is a `main()` function in that script so you can run it from the command line:

```

1 def main():
2     logger.setLevel(logging.DEBUG)
3     logger.info('main')
4     logger.warning('Test warning message XXXX')
5     logger.debug('Test debug message XXXX')
6     logger.info('_test_logging')
7     test_logging()
8     print()

```

(continues on next page)

(continued from previous page)

```

9     print(cLogging)
10    print(dir(cLogging))
11    print()
12    logger.info('cLogging.log():')
13    cLogging.py_log_set_level(10)
14    cLogging.log(cLogging.ERROR, "cLogging.log(): Test log message")
15
16    return 0
17
18
19 if __name__ == '__main__':
20     exit(main())

```

Here is an example output:

```

1 $python tests/unit/test_c_logging.py
2 2025-03-07 11:49:23,994 7064 INFO      main
3 2025-03-07 11:49:23,994 7064 WARNING  Test warning message XXXX
4 2025-03-07 11:49:23,994 7064 DEBUG   Test debug message XXXX
5 2025-03-07 11:49:23,994 7064 INFO      _test_logging
6 2025-03-07 11:49:23,994 7064 WARNING  Test warning message XXXX
7 2025-03-07 11:49:23,994 7064 DEBUG   Test debug message XXXX
8
9 <module 'cPyExtPatt.Logging.cLogging' from 'PythonExtensionPatterns/cPyExtPatt/Logging/
  ↳ cLogging.cpython-313-darwin.so'>
10 ['CRITICAL', 'DEBUG', 'ERROR', 'EXCEPTION', 'INFO', 'WARNING', '__doc__', '__file__', '__
  ↳ loader__', '__name__', '__package__', '__spec__', 'c_file_line_function', 'log', 'py_
  ↳ file_line_function', 'py_log_set_level']
11
12 2025-03-07 11:49:23,994 7064 INFO      cLogging.log():
13 2025-03-07 11:49:23,994 7064 ERROR    cLogging.log(): Test log message

```

11.2 Frames

This describes how to extract the currently executing Python *frame* (or call stack if you wish).

11.2.1 The Python Frame

The crucial call is `PyEval_GetFrame()` that gets the current executing frame as a `PyFrameObject`. From Python 3.11 onwards the `PyFrameObject` has no public members instead there are many functions in the `Frame API` to extract the data we need. Prior to that we could extract the data directly.

The example code is in `src/cpy/Logging/cLogging.c`.

```

1 /**
2  * Returns a tuple of the file, line and function of the current Python frame.
3  * Returns (None, 0, None) on failure.
4  * @param _unused_module
5  * @return PyObject *, a tuple of three values.
6  */
7 static PyObject *
8 py_file_line_function(PyObject *Py_UNUSED(module)) {

```

(continues on next page)

(continued from previous page)

```

9     const unsigned char *file_name = NULL;
10    const char *func_name = NULL;
11    int line_number = 0;
12
13    PyFrameObject *frame = PyEval_GetFrame();
14    if (frame) {
15        /* Get the file name. */
16        #if PY_MAJOR_VERSION == 3 && PY_MINOR_VERSION >= 11
17            /* See:
18             * https://docs.python.org/3.11/whatsnew/3.11.html#pyframeobject-3-11-hiding
19             */
20            file_name = PyUnicode_1BYTE_DATA(PyFrame_GetCode(frame)->co_filename);
21        #else
22            file_name = PyUnicode_1BYTE_DATA(frame->f_code->co_filename);
23        #endif // PY_MAJOR_VERSION == 3 && PY_MINOR_VERSION >= 11
24        /* Get the line number. */
25        line_number = PyFrame_GetLineNumber(frame);
26        /* Get the function name. */
27        #if PY_MAJOR_VERSION == 3 && PY_MINOR_VERSION >= 11
28            /* See:
29             * https://docs.python.org/3.11/whatsnew/3.11.html#pyframeobject-3-11-hiding
30             */
31            func_name = (const char *) PyUnicode_1BYTE_DATA(PyFrame_GetCode(frame)->co_name);
32        #else
33            func_name = (const char *) PyUnicode_1BYTE_DATA(frame->f_code->co_name);
34        #endif // PY_MAJOR_VERSION == 3 && PY_MINOR_VERSION >= 11
35    }
36    /* Build the return value. */
37    /* Use 'z' that makes Python None if the string is NULL. */
38    return Py_BuildValue("ziz", file_name, line_number, func_name);
39 }

```

And this function is registered thus:

```

1  static PyMethodDef logging_methods[] = {
2      /* ... */
3      {
4          "py_file_line_function",
5          (PyCFunction) py_file_line_function,
6          METH_NOARGS,
7          "Return the file, line and function name from the current Python frame."
8      },
9      /* ... */
10     {NULL, NULL, 0, NULL} /* Sentinel */
11 };

```

And can be used thus (test code is in tests/unit/test_c_logging.py):

```

1  from cPyExtPatt.Logging import cLogging
2
3  def test_py_file_line_function_file():
4      file, _line, _function = cLogging.py_file_line_function()

```

(continues on next page)

(continued from previous page)

```

5     assert file == __file__
6
7
8 def test_py_file_line_function_line():
9     _file, line, _function = cLogging.py_file_line_function()
10    assert line == 50
11
12
13 def test_py_file_line_function_function():
14     _file, _line, function = cLogging.py_file_line_function()
15    assert function == 'test_py_file_line_function_function'

```

11.2.2 The C “Frame”

It is also useful to be able to extract the C “frame” (actually the code location at compile time) and present that as the same kind of Python tuple.

A simple macro helps:

```
#define C_FILE_LINE_FUNCTION Py_BuildValue("sis", __FILE__, __LINE__, __FUNCTION__)
```

And is used thus:

```

static PyObject *
c_file_line_function(PyObject *Py_UNUSED(module)) {
    return C_FILE_LINE_FUNCTION;
}

```

The function is registered thus:

```

1 static PyMethodDef logging_methods[] = {
2     /* ... */
3     {
4         "c_file_line_function",
5         (PyCFunction) c_file_line_function,
6         METH_NOARGS,
7         "Return the file, line and function name from the current C code."
8     },
9     /* ... */
10    {NULL, NULL, 0, NULL} /* Sentinel */
11 };

```

And can be used thus (test code is in tests/unit/test_c_logging.py):

```

def test_c_file_line_function_file():
    file, line, function = cLogging.c_file_line_function()
    assert file == 'src/cpy/Logging/cLogging.c'
    assert line == 148
    assert function == 'c_file_line_function'

```

FILE PATHS AND FILES

This chapter describes reading and writing files from CPython C extensions.

12.1 File Paths

There are several builtin functions that allow conversion between Python and C described in the [File System Encoding API](#) which uses the [filesystem encoding and error handler](#), see also [File Objects](#)

In summary:

- `PyUnicode_FSConverter()` Converts a Python a `str` or *path-like* object to a Python bytes object.
- `PyUnicode_FSDecoder()` Converts a Python bytes object to a Python `str`.
- `PyUnicode_DecodeFSDefaultAndSize()` Takes a C string and length and returns a Python `str`.
- `PyUnicode_DecodeFSDefault()` Takes a null terminated C string and length and returns a Python `str`.
- `PyUnicode_EncodeFSDefault()` Takes a Python `str` and return a Python bytes object.

The example code is in:

- `src/cpy/cFile.cpp`
- `src/cpy/PythonFileWrapper.h`
- `src/cpy/PythonFileWrapper.cpp`

The Python tests are in `tests/unit/test_c_file.py`.

12.1.1 Parsing File Paths as Arguments

The Python API provides functionality for converting Python file paths (a `str` or *path-like* object) to C file paths (`char *`). From Python to C; `PyUnicode_FSConverter()` and the reverse from C to Python `PyUnicode_DecodeFSDefaultAndSize()`

Here is an example of taking a Python Unicode string representing a file path, converting it to C and then back to Python. The stages are:

- Use `PyArg_ParseTupleAndKeywords()` and `PyUnicode_FSConverter()` to convert the path-like Python object to a Python bytes object. Note the use of the `"O&"` formatting string that takes a Python object and a conversion function.
- Extract the raws bytes to use as a C path.
- Take a C path and convert it to a Python Unicode `str` and return it.

The Python signature is:

```
def parse_filesystem_argument(path: typing.Union[str, pathlib.Path]) -> str:
```

Here is the C code:

```
1 static PyObject *
2 parse_filesystem_argument(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwargs)
3 ↪{
4     assert(!PyErr_Occurred());
5     assert(args || kwargs);
6
7     PyBytesObject *py_path = NULL;
8     char *c_path = NULL;
9     Py_ssize_t path_size;
10    PyObject *ret = NULL;
11
12    /* Parse arguments */
13    static char *kwlist[] = {"path", NULL};
14    if (!PyArg_ParseTupleAndKeywords(args, kwargs, "O&", kwlist, PyUnicode_FSConverter,
15                                     &py_path)) {
16        goto except;
17    }
18    /* Check arguments. */
19    assert(py_path);
20    /* Grab a reference to the internal bytes buffer. */
21    if (PyBytes_AsStringAndSize((PyObject *) py_path, &c_path, &path_size)) {
22        /* Should have a TypeError or ValueError. */
23        assert(PyErr_Occurred());
24        assert(PyErr_ExceptionMatches(PyExc_TypeError)
25               || PyErr_ExceptionMatches(PyExc_ValueError));
26        goto except;
27    }
28    assert(c_path);
29    /* Use the C path. */
30
31    /* Now convert the C path to a Python object, a string. */
32    ret = PyUnicode_DecodeFSDefaultAndSize(c_path, path_size);
33    if (!ret) {
34        goto except;
35    }
36    assert(!PyErr_Occurred());
37    goto finally;
38except:
39    assert(PyErr_Occurred());
40    Py_XDECREF(ret);
41    ret = NULL;
42finally:
43    // Decref temporary locals.
44    Py_XDECREF(py_path);
45    return ret;
46}
```


12.2 Files

This section describes how to interoperate between Python files, C FILE* and C++ iostream objects.

12.2.1 Reading a Python File

Here is an example of reading from a Python file in C. The Python signature is:

```
def read_python_file_to_c(file_object: typing.IO, size: int = -1) -> bytes:
```

The technique is to get the read() method from the file object with PyObject_GetAttrString then call it with the appropriate arguments using PyObject_Call. Here is the C code:

```
1 static PyObject *
2 read_python_file_to_c(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwargs) {
3     assert(!PyErr_Occurred());
4     static const char *kwlist[] = {"file_object", "size", NULL};
5     PyObject *py_file_object = NULL;
6     Py_ssize_t bytes_to_read = -1;
7     PyObject *py_read_meth = NULL;
8     PyObject *py_read_args = NULL;
9     PyObject *py_read_data = NULL;
10    char *c_bytes_data = NULL;
11    PyObject *ret = NULL;
12
13    if (!PyArg_ParseTupleAndKeywords(args, kwargs, "O|n", (char **) (kwlist),
14                                     &py_file_object, &bytes_to_read)) {
15        return NULL;
16    }
17    // Check that this is a readable file, well does it have a read method?
18    /* Get the read method of the passed object */
19    py_read_meth = PyObject_GetAttrString(py_file_object, "read"); // New reference
20    if (py_read_meth == NULL) {
21        PyErr_Format(PyExc_ValueError,
22                     "Argument of type %s does not have a read() method.",
23                     Py_TYPE(py_file_object)->tp_name);
24        goto except;
25    }
26    if (!PyCallable_Check(py_read_meth)) {
27        PyErr_Format(PyExc_ValueError,
28                     "read attribute of type %s is not callable.",
29                     Py_TYPE(py_file_object)->tp_name);
30        goto except;
31    }
32    // Call read(VisibleRecord::NUMBER_OF_HEADER_BYTES) to get a Python bytes object.
33    py_read_args = Py_BuildValue("(i)", bytes_to_read);
34    if (!py_read_args) {
35        goto except;
36    }
37    // This should advance that readable file pointer.
38    py_read_data = PyObject_Call(py_read_meth, py_read_args, NULL);
39    if (py_read_data == NULL) {
40        goto except;
41    }
42 }
```

(continues on next page)

(continued from previous page)

```

42  /* Check for EOF */
43  if (bytes_to_read >= 0 && PySequence_Length(py_read_data) != bytes_to_read) {
44      assert(PyErr_Occurred());
45      PyErr_Format(PyExc_IOError,
46                  "Reading file object gives EOF. Requested bytes %ld, got %ld.",
47                  bytes_to_read, PySequence_Length(py_read_data));
48      goto except;
49  }
50  c_bytes_data = PyBytes_AsString(py_read_data);
51  if (c_bytes_data == NULL) {
52      // TypeError already set.
53      goto except;
54  }
55  ret = py_read_data;
56  goto finally;
57 except:
58     /* Handle every abnormal condition and clean up. */
59     assert(PyErr_Occurred());
60     ret = NULL;
61 finally:
62     /* Clean up under normal conditions and return an appropriate value. */
63     Py_XDECREF(py_read_meth);
64     Py_XDECREF(py_read_args);
65     return ret;
66 }

```

12.2.2 Writing to a Python File

A similar technique can be used to write to a file, however there are a couple of C functions for writing directly to a Python file:

PyFile_WriteObject()

This writes a Python object to a Python file using the objects `__str__` method (if `Py_PRINT_RAW` is given as the flags argument or the objects `__repr__` method if flags is zero.

PyFile_WriteString()

This will write a C char `*` to a Python file.

Note

`PyFile_WriteString()` creates a unicode string and then calls `PyFile_WriteObject()` so the Python file object must be capable of writing strings.

Here is an example of taking a Python bytes object, extracting the char `*` C buffer and writing that to a Python file. The Python function signature is:

```
def write_bytes_to_python_file(bytes_to_write: bytes, file_object: typing.IO) -> int:
```

Here is the C code:

```

1 static PyObject *
2 write_bytes_to_python_file(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwds) {
3     assert(!PyErr_Occurred());
4     static const char *kwlist[] = {"bytes_to_write", "file_object", NULL};
5     PyObject *py_file_object = NULL;
6     Py_buffer c_buffer;
7     PyObject *ret = NULL;
8
9     if (!PyArg_ParseTupleAndKeywords(args, kwds, "y*0", (char **) (kwlist),
10                                     &c_buffer, &py_file_object)) {
11         return NULL;
12     }
13     /* NOTE: PyFile_WriteString() creates a unicode string and then
14      * calls PyFile_WriteObject() so the py_file_object must be
15      * capable of writing strings. */
16     int result = PyFile_WriteString((char *)c_buffer.buf, py_file_object);
17     if (result != 0) {
18         goto except;
19     }
20     ret = Py_BuildValue("n", c_buffer.len);
21     goto finally;
22 except:
23     assert(PyErr_Occurred());
24     ret = NULL;
25 finally:
26     return ret;
27 }

```

A C++ Python File Wrapper

In `src/cpy/PythonFileWrapper.h` and `src/cpy/PythonFileWrapper.cpp` there is a C++ class that takes a Python file and extracts the `read()`, `write()`, `seek()` and `tell()` methods that can then be used to read and write to the Python file from C++.

Here is the class:

```

1  /// Class that is created with a PyObject* that looks like a Python File.
2  /// This can then read from that file object and write to a user provided C++ stream or
3  → read from a user provided C++
4  /// stream and write to the give Python file like object.
5  class PythonFileObjectWrapper {
6  public:
7      explicit PythonFileObjectWrapper(PyObject *python_file_object);
8
9      /// Read from a Python file and write to the C++ stream.
10     /// Return zero on success, non-zero on failure.
11     int read_py_write_cpp(Py_ssize_t number_of_bytes, std::iostream &ios);
12
13     /// Read from a C++ stream and write to a Python file.
14     /// Return zero on success, non-zero on failure.
15     int read_cpp_write_py(std::iostream &ios, Py_ssize_t number_of_bytes);
16
17     /// Read a number of bytes from a Python file and load them into the result.

```

(continues on next page)

(continued from previous page)

```

17  /// Return zero on success, non-zero on failure.
18  int read(Py_ssize_t number_of_bytes, std::vector<char> &result);
19
20  /// Write a number of bytes to a Python file.
21  /// Return zero on success, non-zero on failure.
22  int write(const char *buffer, Py_ssize_t number_of_bytes);
23
24  /// Move the file pointer to the given position.
25  /// whence is:
26  /// 0 - start of the stream (the default); offset should be zero or positive.
27  /// 1 - current stream position; offset may be negative.
28  /// 2 - end of the stream; offset is usually negative.
29  /// Returns the new absolute position.
30  long seek(Py_ssize_t pos, int whence = 0);
31
32  /// Returns the current absolute position.
33  long tell();
34  /// Returns a multi-line string that describes the class state.
35  std::string str_pointers();
36  /// Returns a Python multi-line bytes object that describes the class state.
37  PyObject *py_str_pointers();
38  /// Destructor, this decrements the held references.
39  virtual ~PythonFileObjectWrapper();
40
41 protected:
42  PyObject *m_python_file_object = NULL;
43  PyObject *m_python_read_method = NULL;
44  PyObject *m_python_write_method = NULL;
45  PyObject *m_python_seek_method = NULL;
46  PyObject *m_python_tell_method = NULL;
47  };

```

Some example code is in src/cpy/cFile.cpp:

```

1  /**
2   * Wraps a Python file object.
3   */
4  static PyObject *
5  wrap_python_file(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwds) {
6      assert(!PyErr_Occurred());
7      static const char *kwlist[] = {"file_object", NULL};
8      PyObject *py_file_object = NULL;
9
10     if (!PyArg_ParseTupleAndKeywords(args, kwds, "O", (char **) (kwlist),
11                                     &py_file_object)) {
12         return NULL;
13     }
14     PythonFileObjectWrapper py_file_wrapper(py_file_object);
15
16     /* Exercise this wrapper by writing, reading etc. */
17     py_file_wrapper.write("Test write to python file", 25);
18     return py_file_wrapper.py_str_pointers();

```

(continues on next page)

(continued from previous page)

19 }

And some tests are in tests/unit/test_c_file.py:

```
1 def test_wrap_python_file():
2     file = io.BytesIO()
3     result = cFile.wrap_python_file(file)
4     print()
5     print(' Result '.center(75, '-'))
6     print(result.decode('ascii'))
7     print(' Result DONE '.center(75, '-'))
8     print(' file.getvalue() '.center(75, '-'))
9     get_value = file.getvalue()
10    print(get_value)
11    print(' file.getvalue() DONE '.center(75, '-'))
12    assert get_value == b'Test write to python file'
```


SUBCLASSING AND USING SUPER()

This chapter describes how to subclass existing types and how to call `super()` in C where necessary.

13.1 Basic Subclassing

In this example we are going to subclass the built in list object and just add a new attribute `state`. The code is based on [Python documentation on subclassing](#). The full code is in `src/cpy/SubClass/sublist.c` and the tests are in `tests/unit/test_c_subclass.py`.

13.1.1 Writing the C Extension

First the declaration of the `SubListObject`:

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>
#include "structmember.h"

typedef struct {
    PyListObject list; // The superclass.
    int state;          // Our new attribute.
} SubListObject;
```

The `__init__` method:

```
static int
SubList_init(SubListObject *self, PyObject *args, PyObject *kwds) {
    if (PyList_Type.tp_init((PyObject *) self, args, kwds) < 0) {
        return -1;
    }
    self->state = 0;
    return 0;
}
```

Now add an `increment()` method and the method table:

```
1 static PyObject *
2 SubList_increment(SubListObject *self, PyObject *Py_UNUSED(unused)) {
3     self->state++;
4     return PyLong_FromLong(self->state);
5 }
6
```

(continues on next page)

(continued from previous page)

```

7 static PyMethodDef SubList_methods[] = {
8     {"increment", (PyCFunction) SubList_increment, METH_NOARGS,
9         PyDoc_STR("increment state counter")},
10    {NULL, NULL, 0, NULL},
11 };

```

Add the state attribute:

```

static PyMemberDef SubList_members[] = {
    {"state", T_INT, offsetof(SubListObject, state), 0,
        "Value of the state."},
    {NULL, 0, 0, 0, NULL} /* Sentinel */
};

```

Declare the type.

Note that we do not initialise `tp_base` just yet. The reason is that C99 requires the initializers to be “address constants”. This is best described in the [tp_base documentation](#).

```

1 static PyObject SubListType = {
2     PyVarObject_HEAD_INIT(NULL, 0)
3     .tp_name = "sublist.SubList",
4     .tp_doc = PyDoc_STR("SubList objects"),
5     .tp_basicsize = sizeof(SubListObject),
6     .tp_itemsize = 0,
7     .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
8     .tp_init = (initproc) SubList_init,
9     .tp_methods = SubList_methods,
10    .tp_members = SubList_members,
11 };

```

Declare the module:

```

static PyModuleDef sublistmodule = {
    PyModuleDef_HEAD_INIT,
    .m_name = "sublist",
    .m_doc = "Module that contains a subclass of a list.",
    .m_size = -1,
};

```

Initialise the module, this is where we set `tp_base`:

```

1 PyMODINIT_FUNC
2 PyInit_sublist(void) {
3     PyObject *m;
4     SubListType.tp_base = &PyList_Type;
5     if (PyType_Ready(&SubListType) < 0) {
6         return NULL;
7     }
8     m = PyModule_Create(&sublistmodule);
9     if (m == NULL) {
10        return NULL;
11    }

```

(continues on next page)

(continued from previous page)

```

12     Py_INCREF(&SubListType);
13     if (PyModule_AddObject(m, "SubList", (PyObject *) &SubListType) < 0) {
14         Py_DECREF(&SubListType);
15         Py_DECREF(m);
16         return NULL;
17     }
18     return m;
19 }

```

13.1.2 Setup and Build

In the `setup.py` add an Extension such as:

```

1 Extension(name=f"cpyExtPatt.SubClass.sublist",
2           include_dirs=[
3               '/usr/local/include',
4           ],
5           sources=[
6               "src/cpy/SubClass/sublist.c",
7           ],
8           extra_compile_args=extra_compile_args_c,
9           language='c',
10          ),

```

The extension can be built with `python setup.py develop`.

13.1.3 Test

The extension can be used like this:

```

from cpyExtPatt.SubClass import sublist

obj = sublist.SubList()
assert obj.state == 0
obj.increment()
assert obj.state == 1

```

This is fine for subclasses that just add some additional functionality however if you want to overload the super class you need to be able to call `super()` from C which is described next.

13.2 Calling `super()` from C

I needed to call `super()` from a C extension and I couldn't find a good description of how to do this online so I am including this here. The ability to call `super()` is needed when you want to modify the behaviour of inherited classes.

Suppose we wanted to subclass a list and record how many times `append()` was called. This is simple enough in pure Python:

```

class SubList(list):
    def __init__(self, *args, **kwargs):
        self.appendes = 0
        super().__init__(*args, **kwargs)

```

(continues on next page)

(continued from previous page)

```
def append(self, v):
    self.append += 1
    return super().append(v)
```

To do it in C is a bit trickier. Taking as our starting point the [example of sub-classing a list](#) in the Python documentation (amended a little bit for our example).

Our type contains an integer count of the number of appends. That is set to zero on construction and can be accessed like a normal member.

```
1 typedef struct {
2     PyListObject list;
3     int appends;
4 } SubListObject;
5
6
7 static int
8 SubListObject_init(SubListObject *self, PyObject *args, PyObject *kwds)
9 {
10     if (PyList_Type.tp_init((PyObject *)self, args, kwds) < 0) {
11         return -1;
12     }
13     self->appends = 0;
14     return 0;
15 }
16
17 static PyMemberDef SubListObject_members[] = {
18     ...
19     {"appends", T_INT, offsetof(SubListObject, appends), 0,
20      "Number of append operations."},
21     ...
22     {NULL, 0, 0, 0, NULL} /* Sentinel */
23 };
```

We now need to create the `append()` function, which will call the superclass `append()` and then increment the `appends` counter:

```
static PyMethodDef SubListObject_methods[] = {
    ...
    {"append", (PyCFunction)SubListObject_append, METH_VARARGS,
     PyDoc_STR("Append to the list")},
    ...
    {NULL, NULL, 0, NULL},
};
```

This is where it gets tricky, how do we implement `SubListObject_append`?

13.2.1 The Obvious Way is Wrong

A first attempt might do something like a method call on the `PyListObject`:

```

1 typedef struct {
2     PyListObject list;
3     int appends;
4 } SubListObject;
5
6 /* Other stuff here. */
7
8 static PyObject *
9 SubListObject_append(SubListObject *self, PyObject *args) {
10     PyObject *result = PyObject_CallMethod((PyObject *)&self->list, "append", "O", args);
11     if (result) {
12         self->appends++;
13     }
14     return result;
15 }

```

This leads to infinite recursion as the address of the first element of a C struct (`list`) is the address of the struct so `self` is the same as `&self->list`. This function is recursive with no base case.

13.2.2 Doing it Right

Our `append` method needs to use *super* to search our super-classes for the “append” method and call that.

Here are a couple of ways of calling `super()` correctly:

- Construct a super object directly and call that.
- Extract the super object from the builtins module and call that.

The full code is in `src/cpy/Util/py_call_super.h` and `src/cpy/Util/py_call_super.c`.

Construct a super object directly

The plan is to do this:

- Create the arguments to initialise an instance of the class `super`.
- Call `super.__new__` with those arguments.
- Call `super.__init__` with those arguments.
- With that `super` object then search for the method we want to call. This is `append` in our case. This calls the `super_getattro` method that performs the search and returns the Python function.
- Call that Python function and return the result.

Our function is defined thus, for simplicity there is no error checking here. For the full function see below:

```

1 /* Call func_name on the super classes of self with the arguments and
2  * keyword arguments.
3  *
4  * Equivalent to getattr(super(type(self), self), func_name)(*args, **kwargs)
5  *
6  * func_name is a Python string.
7  * The implementation creates a new super object on each call.
8  */
9 PyObject *
10 call_super_pyname(PyObject *self, PyObject *func_name,

```

(continues on next page)

(continued from previous page)

```

11         PyObject *args, PyObject *kwargs) {
12     PyObject *super_func = NULL;
13     PyObject *super_args = NULL;
14     PyObject *func = NULL;
15     PyObject *result = NULL;
16
17     // Will be decremented when super_args is decremented if Py_BuildValue succeeds.
18     Py_INCREF(self->ob_type);
19     Py_INCREF(self);
20     super_args = Py_BuildValue("OO", (PyObject *) self->ob_type, self);
21     super_func = PyType_GenericNew(&PySuper_Type, super_args, NULL);
22     // Use tuple as first arg, super() second arg (i.e. kwargs) should be NULL
23     super_func->ob_type->tp_init(super_func, super_args, NULL);
24     func = PyObject_GetAttr(super_func, func_name);
25     result = PyObject_Call(func, args, kwargs);
26     Py_XDECREF(super_func);
27     Py_XDECREF(super_args);
28     Py_XDECREF(func);
29     return result;
30 }

```

We can make this function quite general to be used in the CPython type system. For convenience we can create two functions, one calls the super function by a C NTS, the other by a PyObject string. The following code is essentially the same as above but with error checking.

The header file might be `py_call_super.h` which just declares our two functions:

```

1  #ifndef __PythonSubclassList__py_call_super__
2  #define __PythonSubclassList__py_call_super__
3
4  #include <Python.h>
5
6  extern PyObject *
7  call_super_pyname(PyObject *self, PyObject *func_name,
8                  PyObject *args, PyObject *kwargs);
9  extern PyObject *
10 call_super_name(PyObject *self, const char *func_cname,
11                PyObject *args, PyObject *kwargs);
12
13 #endif /* defined(__PythonSubclassList__py_call_super__) */

```

And the implementation file would be `py_call_super.c`, this is the code above with full error checking:

```

1  /* Call func_name on the super classes of self with the arguments and
2   * keyword arguments.
3   *
4   * Equivalent to getattr(super(type(self), self), func_name)(*args, **kwargs)
5   *
6   * func_name is a Python string.
7   * The implementation creates a new super object on each call.
8   */
9  PyObject *
10 call_super_pyname(PyObject *self, PyObject *func_name,

```

(continues on next page)

(continued from previous page)

```

11         PyObject *args, PyObject *kwargs) {
12     PyObject *super_func = NULL;
13     PyObject *super_args = NULL;
14     PyObject *func = NULL;
15     PyObject *result = NULL;
16
17     // Error check input
18     if (!PyUnicode_Check(func_name)) {
19         PyErr_Format(PyExc_TypeError,
20                     "super() must be called with unicode attribute not %s",
21                     Py_TYPE(func_name)->tp_name);
22     }
23     // Will be decremented when super_args is decremented if Py_BuildValue succeeds.
24     Py_INCREF(self->ob_type);
25     Py_INCREF(self);
26     super_args = Py_BuildValue("OO", (PyObject *) self->ob_type, self);
27     if (!super_args) {
28         Py_DECREF(self->ob_type);
29         Py_DECREF(self);
30         PyErr_SetString(PyExc_RuntimeError, "Could not create arguments for super().");
31         goto except;
32     }
33     super_func = PyType_GenericNew(&PySuper_Type, super_args, NULL);
34     if (!super_func) {
35         PyErr_SetString(PyExc_RuntimeError, "Could not create super().");
36         goto except;
37     }
38     // Use tuple as first arg, super() second arg (i.e. kwargs) should be NULL
39     super_func->ob_type->tp_init(super_func, super_args, NULL);
40     if (PyErr_Occurred()) {
41         goto except;
42     }
43     func = PyObject_GetAttr(super_func, func_name);
44     if (!func) {
45         assert(PyErr_Occurred());
46         goto except;
47     }
48     if (!PyCallable_Check(func)) {
49         PyErr_Format(PyExc_AttributeError,
50                     "super() attribute \"%S\" is not callable.", func_name);
51         goto except;
52     }
53     result = PyObject_Call(func, args, kwargs);
54     if (!result) {
55         assert(PyErr_Occurred());
56         goto except;
57     }
58     assert(!PyErr_Occurred());
59     goto finally;
60 except:
61     assert(PyErr_Occurred());
62     Py_XDECREF(result);

```

(continues on next page)

(continued from previous page)

```

63     result = NULL;
64 finally:
65     Py_XDECREF(super_func);
66     Py_XDECREF(super_args);
67     Py_XDECREF(func);
68     return result;
69 }

```

Extract the super object from the builtins

Another way to do this is to fish out the *super* class from the *builtins* module and use that. Incidentally this is how Cython does it.

The steps are:

1. Get the *builtins* module.
2. Get the *super* class from the *builtins* module.
3. Create a tuple of the arguments to pass to the super class.
4. Create the *super* object with the arguments.
5. Use this *super* object to call the function with the appropriate function arguments.

Again this code has no error checking for simplicity:

```

1  extern PyObject *
2  call_super_pyname_lookup(PyObject *self, PyObject *func_name,
3                          PyObject *args, PyObject *kwargs) {
4      PyObject *result = NULL;
5      PyObject *builtins = NULL;
6      PyObject *super_type = NULL;
7      PyObject *super = NULL;
8      PyObject *super_args = NULL;
9      PyObject *func = NULL;
10
11     builtins = PyImport_AddModule("builtins");
12     // Borrowed reference
13     Py_INCREF(builtins);
14     super_type = PyObject_GetAttrString(builtins, "super");
15     // Will be decremented when super_args is decremented if Py_BuildValue succeeds.
16     Py_INCREF(self->ob_type);
17     Py_INCREF(self);
18     super_args = Py_BuildValue("OO", (PyObject *) self->ob_type, self);
19     super = PyObject_Call(super_type, super_args, NULL);
20     // The following code is the same as call_super_pyname()
21     func = PyObject_GetAttr(super, func_name);
22     result = PyObject_Call(func, args, kwargs);
23     Py_XDECREF(builtins);
24     Py_XDECREF(super_args);
25     Py_XDECREF(super_type);
26     Py_XDECREF(super);
27     Py_XDECREF(func);
28     return result;
29 }

```

Here is the function with full error checking:

```

1  /* Call func_name on the super classes of self with the arguments and
2  * keyword arguments.
3  *
4  * Equivalent to getattr(super(type(self), self), func_name)(*args, **kwargs)
5  *
6  * func_name is a Python string.
7  * The implementation uses the builtin super().
8  */
9  extern PyObject *
10 call_super_pyname_lookup(PyObject *self, PyObject *func_name,
11                          PyObject *args, PyObject *kwargs) {
12     PyObject *result = NULL;
13     PyObject *builtins = NULL;
14     PyObject *super_type = NULL;
15     PyObject *super = NULL;
16     PyObject *super_args = NULL;
17     PyObject *func = NULL;
18
19     builtins = PyImport_AddModule("builtins");
20     if (!builtins) {
21         assert(PyErr_Occurred());
22         goto except;
23     }
24     // Borrowed reference
25     Py_INCREF(builtins);
26     super_type = PyObject_GetAttrString(builtins, "super");
27     if (!super_type) {
28         assert(PyErr_Occurred());
29         goto except;
30     }
31     // Will be decremented when super_args is decremented if Py_BuildValue succeeds.
32     Py_INCREF(self->ob_type);
33     Py_INCREF(self);
34     super_args = Py_BuildValue("OO", (PyObject *) self->ob_type, self);
35     if (!super_args) {
36         Py_DECREF(self->ob_type);
37         Py_DECREF(self);
38         PyErr_SetString(PyExc_RuntimeError, "Could not create arguments for super().");
39         goto except;
40     }
41     super = PyObject_Call(super_type, super_args, NULL);
42     if (!super) {
43         assert(PyErr_Occurred());
44         goto except;
45     }
46     // The following code is the same as call_super_pyname()
47     func = PyObject_GetAttr(super, func_name);
48     if (!func) {
49         assert(PyErr_Occurred());
50         goto except;
51     }

```

(continues on next page)

(continued from previous page)

```

52     if (!PyCallable_Check(func)) {
53         PyErr_Format(PyExc_AttributeError,
54                     "super() attribute \"%S\" is not callable.", func_name);
55         goto except;
56     }
57     result = PyObject_Call(func, args, kwargs);
58     if (!result) {
59         assert(PyErr_Occurred());
60         goto except;
61     }
62     assert(!PyErr_Occurred());
63     goto finally;
64 except:
65     assert(PyErr_Occurred());
66     Py_XDECREF(result);
67     result = NULL;
68 finally:
69     Py_XDECREF(builtins);
70     Py_XDECREF(super_args);
71     Py_XDECREF(super_type);
72     Py_XDECREF(super);
73     Py_XDECREF(func);
74     return result;
75 }

```

13.3 Subclassing with Overloading

Lets revisit our subclass of the builtin list that counts how many time `append()` is called and we will use the `C` `super()` API described above.

The full code is in `src/cpy/SubClass/sublist.c` and the tests are in `tests/unit/test_c_subclass.py`.

13.3.1 Writing the C Extension

First the declaration and initialisation of `SubListObject` that has an `appends` counter. Note the inclusion of `py_call_super.h`:

```

1  #define PY_SSIZE_T_CLEAN
2  #include <Python.h>
3  #include "structmember.h"
4
5  #include "py_call_super.h"
6
7  typedef struct {
8      PyListObject list;
9      int appends;
10 } SubListObject;
11
12 static int
13 SubList_init(SubListObject *self, PyObject *args, PyObject *kws) {
14     if (PyList_Type.tp_init((PyObject *) self, args, kws) < 0) {
15         return -1;

```

(continues on next page)

(continued from previous page)

```

16     }
17     self->appends = 0;
18     return 0;
19 }

```

Now the implementation of `append` that makes the `super()` call and then increments the `appends` attribute and returning the value of the `super()` call:

```

static PyObject *
SubList_append(SubListObject *self, PyObject *args) {
    PyObject *result = call_super_name((PyObject *)self, "append",
                                       args, NULL);

    if (result) {
        self->appends++;
    }
    return result;
}

```

The declaration of methods, members and the type:

```

1 static PyMethodDef SubList_methods[] = {
2     {"append", (PyCFunction) SubList_append, METH_VARARGS,
3         PyDoc_STR("append an item")},
4     {NULL, NULL, 0, NULL},
5 };
6
7 static PyMemberDef SubList_members[] = {
8     {"appends", T_INT, offsetof(SubListObject, appends), 0,
9         "Number of append operations."},
10    {NULL, 0, 0, 0, NULL} /* Sentinel */
11 };
12
13 static PyTypeObject SubListType = {
14     PyVarObject_HEAD_INIT(NULL, 0)
15     .tp_name = "sublist.SubList",
16     .tp_doc = PyDoc_STR("SubList objects"),
17     .tp_basicsize = sizeof(SubListObject),
18     .tp_itemsize = 0,
19     .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
20     .tp_init = (initproc) SubList_init,
21     .tp_methods = SubList_methods,
22     .tp_members = SubList_members,
23 };

```

Finally the module definition which is very similar to before:

```

1 static PyModuleDef sublistmodule = {
2     PyModuleDef_HEAD_INIT,
3     .m_name = "sublist",
4     .m_doc = "Module that contains a subclass of a list.",
5     .m_size = -1,
6 };
7

```

(continues on next page)

(continued from previous page)

```

8 PyMODINIT_FUNC
9 PyInit_sublist(void) {
10     PyObject *m;
11     SubListType.tp_base = &PyList_Type;
12     if (PyType_Ready(&SubListType) < 0) {
13         return NULL;
14     }
15     m = PyModule_Create(&sublistmodule);
16     if (m == NULL) {
17         return NULL;
18     }
19     Py_INCREF(&SubListType);
20     if (PyModule_AddObject(m, "SubList", (PyObject *) &SubListType) < 0) {
21         Py_DECREF(&SubListType);
22         Py_DECREF(m);
23         return NULL;
24     }
25     return m;
26 }

```

13.3.2 Setup and Build

In the `setup.py` add an Extension such as:

```

1 Extension(name=f"cPyExtPatt.SubClass.sublist",
2           include_dirs=[
3               '/usr/local/include',
4           ],
5           sources=[
6               "src/cpy/SubClass/sublist.c",
7           ],
8           extra_compile_args=extra_compile_args_c,
9           language='c',
10          ),

```

The extension can be built with `python setup.py develop`.

13.3.3 Test

The extension can be used like this:

```

from cPyExtPatt.SubClass import sublist

obj = sublist.SubList()
assert obj.appendends == 0
obj.append(42)
assert obj.appendends == 1
assert obj == [42, ]

```

13.3.4 Another Example of Using this API

Here is a real example of using this see overloading `replace()` when subclassing a datetime in *Using an Existing Capsule* from the chapter *Capsules*.

The code here calls the `super()` function then raises if the given arguments are unacceptable (trying to set the `tzinfo` property to `None`):

```
static PyObject *
DateTimeTZ_replace(PyObject *self, PyObject *args, PyObject *kwargs) {
    PyObject *result = call_super_name(self, "replace", args, kwargs);
    if (result) {
        result = (PyObject *) raise_if_no_tzinfo((DateTimeTZ *) result);
    }
    return result;
}
```


CAPSULES

Usually C extension code is used solely by the extension itself, for that reason all functions are declared `static` however there are cases where the C extension code is useful to another C extension. When modules are used as shared libraries the symbols in one extension might not be visible to another extension.

Capsules are a means by which this can be achieved by passing C pointers from one extension module to another.

14.1 A Simple Example

This takes the example given in the Python documentation and makes it complete. The code is in `src/cpy/Capsules/spam*`.

14.1.1 Basic Extension

Here is the basic example of an Extension that can make a system call. The code is in `src/cpy/Capsules/spam.c`.

```
1 #define PY_SSIZE_T_CLEAN
2 #include <Python.h>
3
4 static PyObject *
5 spam_system(PyObject *Py_UNUSED(self), PyObject *args) {
6     const char *command;
7     int sts;
8
9     if (!PyArg_ParseTuple(args, "s", &command))
10         return NULL;
11     sts = system(command);
12     return PyLong_FromLong(sts);
13 }
14
15 static PyMethodDef SpamMethods[] = {
16     /* ... */
17     {"system", spam_system, METH_VARARGS,
18      "Execute a shell command."},
19     /* ... */
20     {NULL, NULL, 0, NULL} /* Sentinel */
21 };
22
23 static struct PyModuleDef spammodule = {
24     PyModuleDef_HEAD_INIT,
25     "spam", /* name of module */
```

(continues on next page)

(continued from previous page)

```

26     PyDoc_STR("Documentation for the spam module"),
27     -1,          /* size of per-interpreter state of the module,
28                  or -1 if the module keeps state in global variables. */
29     SpamMethods,
30     NULL, NULL, NULL, NULL,
31 };
32
33 PyMODINIT_FUNC
34 PyInit_spam(void) {
35     return PyModule_Create(&spammodule);
36 }

```

This would be built with this entry in `setup.py`:

```

Extension(f"{PACKAGE_NAME}.Capsules.spam", sources=['src/cpy/Capsules/spam.c'],
        include_dirs=['/usr/local/include', ],
        library_dirs=[os.getcwd(), ],
        extra_compile_args=extra_compile_args_c,
        language='c',
        ),

```

This can be tested with the code in `tests/unit/test_c_capsules.py`:

```

from cPyExtPatt.Capsules import spam

def test_spam():
    result = spam.system("ls -l")
    assert result == 0

```

14.1.2 An Exportable Extension

To make a version that exports its API in a Capsule we will make some changes:

- Introduce a function `PySpam_System` that actually does the system call. We will export this.
- Introduce a header file `src/cpy/Capsules/spam_capsule.h` that contains the exports.
- Introduce a macro `SPAM_CAPSULE` that controls whether we are exporting or importing the API.
- Change the module initialisation to initialise the API.

The header file in `src/cpy/Capsules/spam_capsule.h` looks like this:

```

1  #ifndef Py_SPAM_CAPSULE_H
2  #define Py_SPAM_CAPSULE_H
3  #ifdef __cplusplus
4  extern "C" {
5  #endif
6
7  #include <Python.h>
8
9  /* Header file for spammodule */
10
11 /* C API functions */
12 #define PySpam_System_NUM 0

```

(continues on next page)

(continued from previous page)

```

13 #define PySpam_System_RETURN int
14 #define PySpam_System_PROTO (const char *command)
15
16 /* Total number of C API pointers */
17 #define PySpam_API_pointers 1
18
19 #ifndef SPAM_CAPSULE
20
21 /* This section is used when compiling spam_capsule.c */
22 static PySpam_System_RETURN PySpam_System PySpam_System_PROTO;
23
24 #else
25 /* This section is used in modules that use spam_capsule's API */
26 static void **PySpam_API;
27
28 #define PySpam_System \
29     (*(PySpam_System_RETURN (*)(PySpam_System_PROTO) PySpam_API[PySpam_System_NUM])
30
31 /* Return -1 on error, 0 on success.
32  * PyCapsule_Import will set an exception if there's an error.
33  */
34 static int
35 import_spam_capsule(void) {
36     PySpam_API = (void **)PyCapsule_Import("cPyExtPatt.Capsules.spam_capsule._C_API", 0);
37     return (PySpam_API != NULL) ? 0 : -1;
38 }
39 #endif
40 #ifdef __cplusplus
41 }
42 #endif
43 #endif /* !defined(Py_SPAM_CAPSULE_H) */

```

The full code is in `src/cpy/Capsules/spam_capsule.c`:

```

1 // Implements: https://docs.python.org/3/extending/extending.html#extending-simpleexample
2 // as a capsule: https://docs.python.org/3/extending/extending.html#providing-a-c-api-
3 // for-an-extension-module
4 // Includes specific exception.
5 // Lightly edited.
6
7 #define PY_SSIZE_T_CLEAN
8
9 #include <Python.h>
10
11 #define SPAM_CAPSULE
12
13 #include "spam_capsule.h"
14
15 static int
16 PySpam_System(const char *command) {
17     return system(command);
18 }

```

(continues on next page)

(continued from previous page)

```

18
19 static PyObject *
20 spam_system(PyObject *Py_UNUSED(self), PyObject *args) {
21     const char *command;
22     int sts;
23
24     if (!PyArg_ParseTuple(args, "s", &command))
25         return NULL;
26     sts = PySpam_System(command);
27     return PyLong_FromLong(sts);
28 }
29
30 static PyMethodDef SpamMethods[] = {
31     /* ... */
32     {"system", spam_system, METH_VARARGS,
33      "Execute a shell command."},
34     /* ... */
35     {NULL, NULL, 0, NULL}          /* Sentinel */
36 };
37
38 static struct PyModuleDef spammodule = {
39     PyModuleDef_HEAD_INIT,
40     "spam_capsule", /* name of module */
41     PyDoc_STR("Documentation for the spam module"),
42     -1, /* size of per-interpreter state of the module,
43        or -1 if the module keeps state in global variables. */
44     SpamMethods,
45     NULL, NULL, NULL, NULL,
46 };
47
48 PyMODINIT_FUNC
49 PyInit_spam_capsule(void) {
50     PyObject *m;
51     static void *PySpam_API[PySpam_API_pointers];
52     PyObject *c_api_object;
53
54     m = PyModule_Create(&spammodule);
55     if (m == NULL)
56         return NULL;
57
58     /* Initialize the C API pointer array */
59     PySpam_API[PySpam_System_NUM] = (void *) PySpam_System;
60
61     /* Create a Capsule containing the API pointer array's address */
62     c_api_object = PyCapsule_New((void *) PySpam_API, "cPyExtPatt.Capsules.spam_capsule._
63     ↪C_API", NULL);
64
65     if (PyModule_AddObject(m, "_C_API", c_api_object) < 0) {
66         Py_XDECREF(c_api_object);
67         Py_DECREF(m);
68         return NULL;
69     }

```

(continues on next page)

(continued from previous page)

```

69     return m;
70 }

```

This can be built by adding this Extension to `setup.py`:

```

Extension(f"{PACKAGE_NAME}.Capsules.spam_capsule",
        sources=['src/cpy/Capsules/spam_capsule.c'],
        include_dirs=['/usr/local/include', 'src/cpy/Capsules'],
        library_dirs=[os.getcwd(), ],
        extra_compile_args=extra_compile_args_c,
        language='c',
        ),

```

This can be tested with the code in `tests/unit/test_c_capsules.py`:

```

from cPyExtPatt.Capsules import spam_capsule

def test_spam_capsule():
    result = spam_capsule.system("ls -l")
    assert result == 0

```

14.1.3 A Client Extension

This can now be used by another extension, say `spam_client` in `src/cpy/Capsules/spam_client.c`. Note the use of:

- `#include "spam_capsule.h"`
- The call to the imported function `PySpam_System`.
- Calling `import_spam_capsule()` in the module initialisation.

Here is the complete C code:

```

1  // Implements: https://docs.python.org/3/extending/extending.html#extending-simpleexample
2  // but using a capsule exported by spam_capsule.h/.c
3  // Excludes specific exception.
4  // Lightly edited.
5
6  #define PY_SSIZE_T_CLEAN
7
8  #include <Python.h>
9  #include "spam_capsule.h"
10
11 static PyObject *
12 spam_system(PyObject *Py_UNUSED(self), PyObject *args) {
13     const char *command;
14     int sts;
15
16     if (!PyArg_ParseTuple(args, "s", &command))
17         return NULL;
18     sts = PySpam_System(command);
19     return PyLong_FromLong(sts);
20 }

```

(continues on next page)

(continued from previous page)

```

21
22 static PyMethodDef SpamMethods[] = {
23     /* ... */
24     {"system", spam_system, METH_VARARGS,
25      "Execute a shell command."},
26     /* ... */
27     {NULL, NULL, 0, NULL}          /* Sentinel */
28 };
29
30 static struct PyModuleDef spam_clientmodule = {
31     PyModuleDef_HEAD_INIT,
32     "spam_client",    /* name of module */
33     PyDoc_STR("Documentation for the spam module"),
34     -1,               /* size of per-interpreter state of the module,
35                      or -1 if the module keeps state in global variables. */
36     SpamMethods,
37     NULL, NULL, NULL, NULL,
38 };
39
40 PyMODINIT_FUNC
41 PyInit_spam_client(void) {
42     PyObject *m;
43
44     m = PyModule_Create(&spam_clientmodule);
45     if (m == NULL)
46         return NULL;
47     if (import_spam_capsule() < 0)
48         return NULL;
49     /* additional initialization can happen here */
50     return m;
51 }

```

This can be built by adding this Extension to setup.py:

```

Extension(f"{PACKAGE_NAME}.Capsules.spam_client",
        sources=['src/cpy/Capsules/spam_client.c'],
        include_dirs=['/usr/local/include', 'src/cpy/Capsules'],
        library_dirs=[os.getcwd(), ],
        extra_compile_args=extra_compile_args_c,
        language='c',
        ),

```

This can be tested with the code in tests/unit/test_c_capsules.py:

```

from cPyExtPatt.Capsules import spam_client

def test_spam_client():
    result = spam_client.system("ls -l")
    assert result == 0

```

14.2 Using an Existing Capsule

Here is an example of using an existing Capsule, the `datetime` C API. In this case we want to create a subclass of the `datetime.datetime` object that always has a time zone i.e. no *naive* datetimes.

Here is the C Extension code to create a `datetimetz` module and a `datetimetz.datetimetz` object. This code is lightly edited for clarity and works with Python 3.10+ (using the modern API). The actual code is in `src/cpy/Capsules/datetimetz.c` (which works with the Python 3.9 API as well) and the tests are in `tests/unit/test_c_capsules.py`.

14.2.1 Writing the Code for the Object

Firstly the declaration of the timezone aware datetime, it just inherits from `datetime.datetime` :

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>
#include "datetime.h"

typedef struct {
    PyDateTime_DateTime datetime;
} DateTimeTZ;
```

Then create a function that sets an error if the `DateTimeTZ` lacks a `tzinfo`. This will be used in a couple of places.

```
1  /* This function sets an error if a tzinfo is not set and returns NULL.
2   * In practice this would use Python version specific calls.
3   * For simplicity this uses Python 3.10+ code.
4   */
5  static DateTimeTZ *
6  raise_if_no_tzinfo(DateTimeTZ *self) {
7      if (!_PyDateTime_HAS_TZINFO(&self->datetime)) {
8          PyErr_SetString(PyExc_TypeError, "No time zone provided.");
9          Py_DECREF(self);
10         self = NULL;
11     }
12     return self;
13 }
```

Now the code for creating a new instance:

```
1  static PyObject *
2  DateTimeTZ_new(PyTypeObject *type, PyObject *args, PyObject *kwargs) {
3      DateTimeTZ *self = (DateTimeTZ *) PyDateTimeAPI->DateTimeType->tp_new(
4          type, args, kwargs
5      );
6      if (self) {
7          self = raise_if_no_tzinfo(self);
8      }
9      return (PyObject *) self;
10 }
```

So far a new `datetimetz` object must be created with a `tzinfo` but the `datetime.datetime` has an API `replace()` that creates a new `datetime` with different properties, including `tzinfo`. We need to guard against the user trying to change the timezone to `None`. To do this we call the super class function and then check, and raise, if a `tzinfo` is absent.

This uses the utility function that call Python's `super()` function. That code is in `src/cpy/Util/py_call_super.h` and `src/cpy/Util/py_call_super.c`:

```
static PyObject *
DateTimeTZ_replace(PyObject *self, PyObject *args, PyObject *kwargs) {
    PyObject *result = call_super_name(self, "replace", args, kwargs);
    if (result) {
        result = (PyObject *) raise_if_no_tzinfo((DateTimeTZ *) result);
    }
    return result;
}
```

Finally the module code:

```
1 static PyMethodDef DateTimeTZ_methods[] = {
2     {
3         "replace",
4         (PyCFunction) DateTimeTZ_replace,
5         METH_VARARGS | METH_KEYWORDS,
6         PyDoc_STR("Return datetime with new specified fields.")
7     },
8     {NULL, NULL, 0, NULL} /* Sentinel */
9 };
10
11 static PyTypeObject DatetimeTZType = {
12     PyVarObject_HEAD_INIT(NULL, 0)
13     .tp_name = "datetime.datetime",
14     .tp_doc = "A datetime that requires a time zone.",
15     .tp_basicsize = sizeof(DateTimeTZ),
16     .tp_itemsize = 0,
17     .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
18     .tp_new = DateTimeTZ_new,
19     .tp_methods = DateTimeTZ_methods,
20 };
21
22 static PyModuleDef datetimetzmodule = {
23     PyModuleDef_HEAD_INIT,
24     .m_name = "datetime",
25     .m_doc = (
26         "Module that contains a datetime,"
27         "a datetime.datetime with a mandatory time zone."
28     ),
29     .m_size = -1,
30 };
```

Initialise the module, this is when we use the existing capsule:

```
1 PyMODINIT_FUNC
2 PyInit_datetime(void) {
3     PyObject *m = PyModule_Create(&datetimetzmodule);
4     if (m == NULL) {
5         return NULL;
6     }
7     // datetime.datetime_CAPI
```

(continues on next page)

(continued from previous page)

```

8   PyDateTime_IMPORT;
9   if (!PyDateTimeAPI) {
10      Py_DECREF(m);
11      return NULL;
12  }
13  // Set inheritance.
14  DatetimeTZType.tp_base = PyDateTimeAPI->DateTimeType;
15  if (PyType_Ready(&DatetimeTZType) < 0) {
16      Py_DECREF(m);
17      return NULL;
18  }
19  Py_INCREF(&DatetimeTZType);
20  PyModule_AddObject(m, "datetimetz", (PyObject *) &DatetimeTZType);
21  /* additional initialization can happen here */
22  return m;
23 }

```

The extension is created with this in `setup.py`:

```

1  Extension(f"{PACKAGE_NAME}.Capsules.datetimetz",
2          sources=[
3              'src/cpy/Capsules/datetimetz.c',
4              'src/cpy/Util/py_call_super.c',
5          ],
6          include_dirs=[
7              '/usr/local/include',
8              'src/cpy/Capsules',
9              'src/cpy/Util',
10         ],
11         library_dirs=[os.getcwd(), ],
12         extra_compile_args=extra_compile_args_c,
13         language='c',
14     ),

```

Extensive tests are in `tests/unit/test_c_capsules.py`.

14.2.2 Building

Running `python setup.py develop` will build the extension(s) and then `datetimetz` can be imported:

```
from cPyExtPatt.Capsules import datetimetz
```

14.2.3 Testing

All the tests are in `tests/unit/test_c_capsules.py`, but here is a relevant selection. All depend on:

```

import datetime
import zoneinfo

import pytest

from cPyExtPatt.Capsules import datetimetz

```

A check on the `__mro__`:

```
def test_datetimetz_datetimetz_mro():
    mro = datetimetz.datetimetz.__mro__
    assert [str(v) for v in mro] == [
        "<class 'datetimetz.datetimetz'>",
        "<class 'datetime.datetime'>",
        "<class 'datetime.date'>",
        "<class 'object'>",
    ]
```

A check on construction, first with a timezone, then without:

```
1 @pytest.mark.parametrize(
2     'args, kwargs, expected',
3     (
4         (
5             (2024, 7, 15, 10, 21, 14),
6             {'tzinfo': zoneinfo.ZoneInfo('Europe/London')},
7             '2024-07-15 10:21:14+01:00',
8         ),
9     )
10 )
11 def test_datetimetz_datetimetz_str(args, kwargs, expected):
12     d = datetimetz.datetimetz(*args, **kwargs)
13     assert str(d) == expected
14
15
16 @pytest.mark.parametrize(
17     'args, kwargs, expected',
18     (
19         (
20             (2024, 7, 15, 10, 21, 14),
21             {},
22             'No time zone provided.',
23         ),
24         (
25             (2024, 7, 15, 10, 21, 14),
26             {'tzinfo': None, },
27             'No time zone provided.',
28         ),
29     )
30 )
31 def test_datetimetz_datetimetz_raises(args, kwargs, expected):
32     with pytest.raises(TypeError) as err:
33         d = datetimetz.datetimetz(*args, **kwargs)
34     assert err.value.args[0] == expected
```

Check the `repr()`. Note how this uses inheritance correctly whilst getting the type right:

```
1 @pytest.mark.parametrize(
2     'args, kwargs, expected',
3     (
4         (
```

(continues on next page)

(continued from previous page)

```

5         (2024, 7, 15, 10, 21, 14),
6         {'tzinfo': zoneinfo.ZoneInfo('Europe/London')},
7         (
8             "datetimetz.datetimetz(2024, 7, 15, 10, 21, 14,"
9             " tzinfo=zoneinfo.ZoneInfo(key='Europe/London'))"
10        ),
11    ),
12 )
13 )
14 def test_datetimetz_datetimetz_repr(args, kwargs, expected):
15     d = datetimetz.datetimetz(*args, **kwargs)
16     assert repr(d) == expected

```

Here is a test for setting the tzinfo directly. The error is handled correctly by the superclass.

```

def test_datetimetz_datetimetz_set_tzinfo_raises():
    d = datetimetz.datetimetz(
        2024, 7, 15, 10, 21, 14,
        tzinfo=zoneinfo.ZoneInfo('Europe/London')
    )
    with pytest.raises(AttributeError) as err:
        d.tzinfo = None
    assert err.value.args[0] == "attribute 'tzinfo' of 'datetime.datetime' objects is_
↳not writable"

```

Check that `.replace()` works as expected with tzinfo:

```

def test_datetimetz_datetimetz_replace_raises_tzinfo():
    d = datetimetz.datetimetz(
        2024, 7, 15, 10, 21, 14,
        tzinfo=zoneinfo.ZoneInfo('Europe/London')
    )
    with pytest.raises(TypeError) as err:
        d.replace(tzinfo=None)
    assert err.value.args[0] == 'No time zone provided.'

```

Some equality tests. We want to fail when comparing our `datetimetz` with a naive `datetime` object.

```

1 def test_datetimetz_datetimetz_equal():
2     d_tz = datetimetz.datetimetz(
3         2024, 7, 15, 10, 21, 14,
4         tzinfo=zoneinfo.ZoneInfo('Europe/London'))
5     d = datetime.datetime(
6         2024, 7, 15, 10, 21, 14,
7         tzinfo=zoneinfo.ZoneInfo('Europe/London')
8     )
9     assert d_tz == d
10
11
12 def test_datetime_datetime_equal_naive():
13     d = datetime.datetime(
14         2024, 7, 15, 10, 21, 14,
15         tzinfo=zoneinfo.ZoneInfo('Europe/London')

```

(continues on next page)

(continued from previous page)

```

16     )
17     d_no_tz = datetime.datetime(2024, 7, 15, 10, 21, 14)
18     assert d_no_tz != d

```

Some datetime subtraction tests that show our `datetimetz` inter-operates correctly with itself and a `datetime` object.

```

1  @pytest.mark.parametrize(
2      'd_tz, d, expected',
3      (
4          (
5              datetimetz.datetimetz(
6                  2024, 7, 15, 10, 21, 14,
7                  tzinfo=zoneinfo.ZoneInfo('Europe/London')
8              ),
9              datetime.datetime(
10                 2024, 7, 15, 10, 21, 14,
11                 tzinfo=zoneinfo.ZoneInfo('Europe/London')
12             ),
13             datetime.timedelta(0),
14         ),
15         (
16             datetimetz.datetimetz(
17                 2024, 7, 15, 10, 21, 14,
18                 tzinfo=zoneinfo.ZoneInfo('Europe/London')
19             ),
20             datetime.datetime(
21                 2024, 7, 15, 10, 21, 14,
22                 tzinfo=zoneinfo.ZoneInfo('America/New_York')
23             ),
24             datetime.timedelta(seconds=-5 * 60 * 60),
25         ),
26         (
27             datetimetz.datetimetz(
28                 2024, 7, 15, 10, 21, 14,
29                 tzinfo=zoneinfo.ZoneInfo('America/New_York')
30             ),
31             datetime.datetime(
32                 2024, 7, 15, 10, 21, 14,
33                 tzinfo=zoneinfo.ZoneInfo('Europe/London')
34             ),
35             datetime.timedelta(seconds=5 * 60 * 60),
36         ),
37     )
38 )
39 def test_datetimetz_datetimetz_subtract(d_tz, d, expected):
40     assert (d_tz - d) == expected
41
42
43 @pytest.mark.parametrize(
44     'd_tz, d, expected',
45     (
46         (

```

(continues on next page)

(continued from previous page)

```
47         datetimetz.datetimetz(  
48             2024, 7, 15, 10, 21, 14,  
49             tzinfo=zoneinfo.ZoneInfo('Europe/London')  
50         ),  
51         datetime.datetime(2024, 7, 15, 10, 21, 14),  
52         '',  
53     ),  
54 )  
55 )  
56 def test_datetimetz_datetimetz_subtract_raises(d_tz, d, expected):  
57     with pytest.raises(TypeError) as err:  
58         d_tz - d  
59     assert err.value.args[0] == "can't subtract offset-naive and offset-aware datetimes"
```


ITERATORS AND GENERATORS

This chapter describes how to write iterators for your C objects. These iterators allow your objects to be used with a [Generator](#).

15.1 Iterators

The iterator concept is actually fairly straight forward:

- You have some object that contains some data.
- You have some iterator that traverses the data.

That iterator:

- Has a strong reference to the originating object, thus its data. This strong reference keeps the originating object alive as long as the iterator is alive.
- It has a notion of *state*, in other words ‘where I was before so know where to go next’.

Warning

The strong reference to the underlying data structure keeps it alive but what happens if the underlying structure is altered *during* iteration?

Here is an example:

```
lst = list(range(8))
for i, value in enumerate(lst):
    print(f'i={i} value={value}')
    del lst[i]
```

This gives the sequence:

```
i=0 value=0
i=1 value=2
i=2 value=4
i=3 value=6
```

Which may not be what you want. It is hard to make a ‘good’ design to cope with this (defer the `del`? raise?) so the general advice is: do not alter the underlying structure whilst iterating.

15.1.1 Example of a Sequence

In this example we create a module that has an object which holds a sequence of C long s. The complete code is in `src/cpy/Iterators/cIterator.c` here are just the essential parts. The test code is in `tests/unit/test_c_iterators.py`.

Essentially in Python this means I will be able to do this:

```
from cPyExtPatt.Iterators import cIterator

sequence = cIterator.SequenceOfLong([1, 7, 4])
result = [v for v in sequence]
assert result == [1, 7, 4]
```

Note

Because of the entwined nature of the sequence object and the iterator object the code in `src/cpy/Iterators/cIterator.c` occasionally appears out of order.

Firstly, here is the C declaration of the `SequenceOfLong` struct:

```
1 #define PY_SSIZE_T_CLEAN
2
3 #include <Python.h>
4 #include "structmember.h"
5
6 typedef struct {
7     PyObject_HEAD
8     long *array_long;
9     size_t size;
10 } SequenceOfLong;
```

This will be instantiated with a Python sequence of integers:

```
1 static PyObject *
2 SequenceOfLong_new(PyTypeObject *type, PyObject *Py_UNUSED(args),
3                   PyObject *Py_UNUSED(kwds)) {
4     SequenceOfLong *self;
5     self = (SequenceOfLong *) type->tp_alloc(type, 0);
6     if (self != NULL) {
7         assert(!PyErr_Occurred());
8         self->size = 0;
9         self->array_long = NULL;
10    }
11    return (PyObject *) self;
12 }
```

And initialised with a Python sequence of integers:

Note

The use of the [Sequence Protocol API](#) such as `PySequence_Length()` and `PySequence_GetItem()`

```

1 static int
2 SequenceOfLong_init(SequenceOfLong *self, PyObject *args, PyObject *kwargs) {
3     static char *kwlist[] = {"sequence", NULL};
4     PyObject *sequence = NULL;
5
6     if (!PyArg_ParseTupleAndKeywords(args, kwargs, "O", kwlist, &sequence)) {
7         return -1;
8     }
9     if (!PySequence_Check(sequence)) {
10        return -2;
11    }
12    self->size = PySequence_Length(sequence);
13    self->array_long = malloc(self->size * sizeof(long));
14    if (!self->array_long) {
15        return -3;
16    }
17    for (Py_ssize_t i = 0; i < PySequence_Length(sequence); ++i) {
18        // New reference.
19        PyObject *py_value = PySequence_GetItem(sequence, i);
20        if (PyLong_Check(py_value)) {
21            self->array_long[i] = PyLong_AsLong(py_value);
22            Py_DECREF(py_value);
23        } else {
24            PyErr_Format(
25                PyExc_TypeError,
26                "Argument [%zd] must be a int, not type %s",
27                i,
28                Py_TYPE(sequence)->tp_name
29            );
30            // Clean up on error.
31            free(self->array_long);
32            self->array_long = NULL;
33            Py_DECREF(py_value);
34            return -4;
35        }
36    }
37    return 0;
38 }

```

And the de-allocation function frees the dynamically allocated memory:

```

static void
SequenceOfLong_dealloc(SequenceOfLong *self) {
    free(self->array_long);
    Py_TYPE(self)->tp_free((PyObject *) self);
}

```

We provide a single method `size()` for the length of the sequence and a `__str__` method:

```

1 SequenceOfLong_size(SequenceOfLong *self, PyObject *Py_UNUSED(ignored)) {
2     return Py_BuildValue("n", self->size);
3 }
4

```

(continues on next page)

(continued from previous page)

```

5 static PyMethodDef SequenceOfLong_methods[] = {
6     {
7         "size",
8         (PyCFunction) SequenceOfLong_size,
9         METH_NOARGS,
10        "Return the size of the sequence."
11    },
12    {NULL, NULL, 0, NULL} /* Sentinel */
13 };
14
15 static PyObject *
16 SequenceOfLong__str__(SequenceOfLong *self, PyObject *Py_UNUSED(ignored)) {
17     assert(!PyErr_Occurred());
18     return PyUnicode_FromFormat("<SequenceOfLong sequence size: %ld>", self->size);
19 }

```

The type declaration then becomes:

```

1 static PyTypeObject SequenceOfLongType= {
2     PyVarObject_HEAD_INIT(NULL, 0)
3     .tp_name = "SequenceOfLong",
4     .tp_basicsize = sizeof(SequenceOfLong),
5     .tp_itemsize = 0,
6     .tp_dealloc = (destructor) SequenceOfLong_dealloc,
7     .tp_str = (reprfunc) SequenceOfLong__str__,
8     .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
9     .tp_doc = "Sequence of long integers.",
10    .tp_methods = SequenceOfLong_methods,
11    .tp_init = (initproc) SequenceOfLong_init,
12    .tp_new = SequenceOfLong_new,
13 };

```

This will be used thus:

```

from cPyExtPatt.Iterators import cIterator

sequence = cIterator.SequenceOfLong([1, 7, 4])

```

But we can't (yet) iterate across the sequence. To do that we need to add an iterator.

15.1.2 Adding an Iterator

Here is the iterator that takes a reference to the SequenceOfLong and maintains an index:

```

typedef struct {
    PyObject_HEAD
    PyObject *sequence;
    size_t index;
} SequenceOfLongIterator;

```

Here are the __new__, __init__ and de-allocation methods:

```

1 static PyObject *
2 SequenceOfLongIterator_new(PyTypeObject *type, PyObject *Py_UNUSED(args),
3                             PyObject *Py_UNUSED(kwds)) {
4     SequenceOfLongIterator *self;
5     self = (SequenceOfLongIterator *) type->tp_alloc(type, 0);
6     if (self != NULL) {
7         assert(!PyErr_Occurred());
8     }
9     return (PyObject *) self;
10 }
11
12 // Forward reference
13 static int is_sequence_of_long_type(PyObject *op);
14 // Defined later to be:
15 // static int
16 // is_sequence_of_long_type(PyObject *op) {
17 //     return Py_TYPE(op) == &SequenceOfLongType;
18 // }

```

Here is the initialisation function, note the line `Py_INCREF(sequence);` that keeps the original sequence alive.

```

1 static int
2 SequenceOfLongIterator_init(SequenceOfLongIterator *self, PyObject *args,
3                             PyObject *kwds) {
4     static char *kwlist[] = {"sequence", NULL};
5     PyObject *sequence = NULL;
6     if (!PyArg_ParseTupleAndKeywords(args, kwds, "O", kwlist, &sequence)) {
7         return -1;
8     }
9     if (!is_sequence_of_long_type(sequence)) {
10         PyErr_Format(
11             PyExc_ValueError,
12             "Argument must be a SequenceOfLongType, not type %s",
13             Py_TYPE(sequence)->tp_name
14         );
15         return -2;
16     }
17     // Borrowed reference
18     // Keep the sequence alive as long as the iterator is alive.
19     // Decrement on iterator de-allocation.
20     Py_INCREF(sequence);
21     self->sequence = sequence;
22     self->index = 0;
23     return 0;
24 }

```

Here is the de-allocation function once the iterator is deleted. Note the line `Py_XDECREF(self->sequence);` that allows the original sequence to be free'd.

```

static void
SequenceOfLongIterator_dealloc(SequenceOfLongIterator *self) {
    // Decrement borrowed reference.
    Py_XDECREF(self->sequence);
}

```

(continues on next page)

(continued from previous page)

```
Py_TYPE(self)->tp_free((PyObject *) self);
}
```

Here is the `__next__` method. This returns the next object in the sequence, or `NULL` if the sequence is exhausted. It updates the internal counter on each call:

```
1 static PyObject *
2 SequenceOfLongIterator_next(SequenceOfLongIterator *self) {
3     size_t size = ((SequenceOfLong *) self->sequence)->size;
4     if (self->index < size) {
5         PyObject *ret = PyLong_FromLong(
6             ((SequenceOfLong *) self->sequence)->array_long[self->index]
7         );
8         self->index += 1;
9         return ret;
10    }
11    // End iteration.
12    return NULL;
13 }
```

Here is the iterator type declaration, note the use of `tp_iter` and `tp_iternext`.

- `tp_iter` signals that this object is an iterable and its result is the iterator. The use of `PyObject_SelfIter` merely says “I am an iterator”.
- `tp_iternext` is the function to call with the iterator as its sole argument and this returns the next item in the sequence.

Note

`PyObject_SelfIter()` is a supported CPython API and is implemented thus:

```
PyObject *
PyObject_SelfIter(PyObject *obj)
{
    Py_INCREF(obj);
    return obj;
}
```

Here is the type declaration for the iterator, the iterator is both iterable and has a `__next__` method so both `tp_iter` and `tp_iternext` are defined:

```
1 static PyTypeObject SequenceOfLongIteratorType = {
2     PyVarObject_HEAD_INIT(NULL, 0)
3     .tp_name = "SequenceOfLongIterator",
4     .tp_basicsize = sizeof(SequenceOfLongIterator),
5     .tp_itemsize = 0,
6     .tp_dealloc = (destructor) SequenceOfLongIterator_dealloc,
7     .tp_str = (reprfunc) SequenceOfLongIterator__str__,
8     .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
9     .tp_doc = "SequenceOfLongIterator object.",
10    .tp_iter = PyObject_SelfIter,
```

(continues on next page)

(continued from previous page)

```

11     .tp_iternext = (iternextfunc) SequenceOfLongIterator_next,
12     .tp_init = (initproc) SequenceOfLongIterator_init,
13     .tp_new = SequenceOfLongIterator_new,
14 };

```

Now change the type declaration of the `SequenceOfLongType` to add iteration. This defines `tp_iter` as it is iterable but does *not* define the `tp_iternext` method as it does *not* have a `__next__` method, the iterator instance provides that:

```

1 static PyObject SequenceOfLongType= {
2     PyVarObject_HEAD_INIT(NULL, 0)
3     .tp_name = "SequenceOfLong",
4     .tp_basicsize = sizeof(SequenceOfLong),
5     .tp_itemsize = 0,
6     .tp_dealloc = (destructor) SequenceOfLong_dealloc,
7     .tp_str = (reprfunc) SequenceOfLong__str__,
8     .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
9     .tp_doc = "Sequence of long integers.",
10    // I am iterable, this function gives you an iterator on me.
11    .tp_iter = (getiterfunc) SequenceOfLong_iter,
12    .tp_methods = SequenceOfLong_methods,
13    .tp_init = (initproc) SequenceOfLong_init,
14    .tp_new = SequenceOfLong_new,
15 };

```

15.1.3 A Note on Module Initialisation

The module initialisation looks like this:

```

PyMODINIT_FUNC
PyInit_cIterator(void) {
    PyObject *m;
    m = PyModule_Create(&iterator_cmodule);
    if (m == NULL) {
        return NULL;
    }
    // ...
}

```

Naturally enough we have to include the initialisation of `SequenceOfLongType`:

```

1 if (PyType_Ready(&SequenceOfLongType) < 0) {
2     Py_DECREF(m);
3     return NULL;
4 }
5 Py_INCREF(&SequenceOfLongType);
6 if (PyModule_AddObject(
7     m,
8     "SequenceOfLong",
9     (PyObject *) &SequenceOfLongType) < 0
10 ) {
11     Py_DECREF(&SequenceOfLongType);

```

(continues on next page)

(continued from previous page)

```

12     Py_DECREF(m);
13     return NULL;
14 }

```

We *must* include the initialisation of `SequenceOfLongIteratorType`:

```

if (PyType_Ready(&SequenceOfLongIteratorType) < 0) {
    Py_DECREF(m);
    return NULL;
}
Py_INCREF(&SequenceOfLongIteratorType);

```

However the following is optional, as the comment suggests:

```

1 // Not strictly necessary unless you need to expose this type.
2 // For type checking for example.
3 if (PyModule_AddObject(
4     m,
5     "SequenceOfLongIterator",
6     (PyObject *) &SequenceOfLongIteratorType) < 0
7 ) {
8     Py_DECREF(&SequenceOfLongType);
9     Py_DECREF(&SequenceOfLongIteratorType);
10    Py_DECREF(m);
11    return NULL;
12 }

```

If you omit that the code will work just fine, the iterator is instantiated dynamically, it is just that the type is not exposed from the module.

15.1.4 Iterating a Python Object in C

In `src/cpy/Iterators/cIterator.c` there is an example of iterating a Python object using the [Iterator Protocol](#). There is a function `iterate_and_print` that takes an object supporting the Iterator Protocol and iterates across it printing out each item.

The equivalent Python code is:

```

def iterate_and_print(sequence: typing.Iterable) -> None:
    print('iterate_and_print:')
    for i, item in enumerate(sequence):
        print(f'[{i}]: {item}')
    print('iterate_and_print: DONE')

```

The C code looks like this:

```

1 static PyObject *
2 iterate_and_print(PyObject *Py_UNUSED(module), PyObject *args, PyObject *kwds) {
3     assert(!PyErr_Occurred());
4     static char *kwlist[] = {"sequence", NULL};
5     PyObject *sequence = NULL;
6
7     if (!PyArg_ParseTupleAndKeywords(args, kwds, "O", kwlist, &sequence)) {

```

(continues on next page)

(continued from previous page)

```

8     return NULL;
9 }
10 PyObject *iterator = PyObject_GetIter(sequence);
11 if (iterator == NULL) {
12     /* propagate error */
13     assert(PyErr_Occurred());
14     return NULL;
15 }
16 PyObject *item = NULL;
17 long index = 0;
18 fprintf(stdout, "%s:\n", __FUNCTION__ );
19 while ((item = PyIter_Next(iterator))) {
20     /* do something with item */
21     fprintf(stdout, "[%ld]: ", index);
22     if (PyObject_Print(item, stdout, Py_PRINT_RAW) == -1) {
23         /* Handle error. */
24         Py_DECREF(item);
25         Py_DECREF(iterator);
26         if (!PyErr_Occurred()) {
27             PyErr_Format(PyExc_RuntimeError,
28                          "Can not print an item of type %s",
29                          Py_TYPE(sequence)->tp_name);
30         }
31         return NULL;
32     }
33     fprintf(stdout, "\n");
34     ++index;
35     /* release reference when done */
36     Py_DECREF(item);
37 }
38 Py_DECREF(iterator);
39 if (PyErr_Occurred()) {
40     /* propagate error */
41     return NULL;
42 }
43 fprintf(stdout, "%s: DONE\n", __FUNCTION__ );
44 assert(!PyErr_Occurred());
45 Py_RETURN_NONE;
46 }

```

This function is added to the cIterator module thus:

```

1 static PyMethodDef cIterator_methods[] = {
2     {"iterate_and_print", (PyCFunction) iterate_and_print, METH_VARARGS,
3      "Iterate through the argument printing the values."},
4     {NULL, NULL, 0, NULL} /* Sentinel */
5 };
6
7 static PyModuleDef iterator_cmodule = {
8     PyModuleDef_HEAD_INIT,
9     .m_name = "cIterator",
10    .m_doc = (

```

(continues on next page)

(continued from previous page)

```

11         "Example module that creates an extension type"
12         "that has forward and reverse iterators."
13     ),
14     .m_size = -1,
15     .m_methods = cIterator_methods,
16 };

```

An example of using this is shown below.

15.1.5 Examples

Now we can import the module, and create a sequence:

```

from cPyExtPatt.Iterators import cIterator

sequence = cIterator.SequenceOfLong([1, 7, 4])

```

And these calls now work:

```

result = [v for v in sequence]
assert result == [1, 7, 4]

```

Delete the underlying object, the iteration still works:

```

iterator = iter(sequence)
del sequence
result = [v for v in iterator]
assert result == [1, 7, 4]

```

Using the builtin `next()`:

```

iterator = iter(sequence)
assert next(iterator) == 1
assert next(iterator) == 7
assert next(iterator) == 4
# next() will raise a StopIteration.

```

Using the builtin `sorted()`:

```

sequence = cIterator.SequenceOfLong([1, 7, 4])
result = sorted(sequence)
assert result == [1, 4, 7, ]

```

And to iterate across a Python object:

```

cIterator.iterate_and_print('abc')

```

Result in the stdout:

```

iterate_and_print:
[0]: a
[1]: b
[2]: c
iterate_and_print: DONE

```

Note

If you are running under pytest you can capture the output to stdout from C using the capfd fixture:

```

1 import pytest
2
3 @pytest.mark.parametrize(
4     'arg, expected',
5     (
6         (
7             'abc',
8             """iterate_and_print:
9 [0]: a
10 [1]: b
11 [2]: c
12 iterate_and_print: DONE
13 """
14         ),
15     )
16 )
17 def test_iterate_and_print(arg, expected, capfd):
18     cIterator.iterate_and_print(arg)
19     captured = capfd.readouterr()
20     assert captured.out == expected

```

15.1.6 Reverse Iterators

If we try and use the `reversed()` function on our current `SequenceOfLong` we will get an error:

```
TypeError: 'SequenceOfLong' object is not reversible
```

Reverse iterators are slightly unusual, the `reversed()` function calls the object `__reversed__()` method if available or falls back on using `__len__()` and `__getitem__()`.

`reversed()` acts like this python code:

```

1 def reversed(obj: object):
2     if hasattr(obj, '__reversed__'):
3         yield from obj.__reversed__()
4     elif hasattr(obj, '__len__') and hasattr(obj, '__getitem__'):
5         i = len(obj) - 1
6         while i >= 0:
7             yield obj[i]
8             i -= 1
9     else:
10        raise TypeError(f'{type(object)} is not reversible')

```

To support this in C we can implement `__len__()` and `__getitem__()` using the `PySequenceMethods` method table.

First the implementation of `__len__()` in C:

```

static Py_ssize_t
SequenceOfLong_sq_length(PyObject *self) {

```

(continues on next page)

(continued from previous page)

```

return ((SequenceOfLong *)self)->size;
}

```

Then the implementation of `__getitem__()`, note here that we support negative indexes and set and exception if the index is out of range:

```

1 static PyObject *
2 SequenceOfLong_sq_item(PyObject *self, Py_ssize_t index) {
3     Py_ssize_t my_index = index;
4     if (my_index < 0) {
5         my_index += SequenceOfLong_sq_length(self);
6     }
7     if (my_index > SequenceOfLong_sq_length(self)) {
8         PyErr_Format(
9             PyExc_IndexError,
10            "Index %ld is out of range for length %ld",
11            index,
12            SequenceOfLong_sq_length(self)
13        );
14        return NULL;
15    }
16    return PyLong_FromLong(((SequenceOfLong *)self)->array_long[my_index]);
17 }

```

Create `PySequenceMethods` method table:

```

PySequenceMethods SequenceOfLong_sequence_methods = {
    .sq_length = &SequenceOfLong_sq_length,
    .sq_item = &SequenceOfLong_sq_item,
};

```

Add this method table into the type specification:

```

static PyTypeObject SequenceOfLongType = {
    PyVarObject_HEAD_INIT(NULL, 0)
    /* Other stuff. */
    .tp_dealloc = (destructor) SequenceOfLong_dealloc,
    .tp_as_sequence = &SequenceOfLong_sequence_methods,
    /* Other stuff. */
};

```

And we can test it thus:

```

def test_c_iterator_reversed():
    sequence = cIterator.SequenceOfLong([1, 7, 4])
    result = reversed(sequence)
    assert list(result) == [4, 7, 1]

```

15.2 Generators

Iterators are a requirement for [Generators](#), the secret weapon in Python's toolbox. If you don't believe me then ask David Beazley who has done some very fine and informative [presentations on Generators](#)

15.2.1 Our Iterator as a Generator

Now we have an iterator we can write generator:

```
def yield_from_an_iterator_times_two(iterator):  
    for value in iterator:  
        yield 2 * value
```

And test it:

```
def test_c_iterator_yield_forward():  
    sequence = cIterator.SequenceOfLong([1, 7, 4])  
    iterator = iter(sequence)  
    result = []  
    for v in yield_from_an_iterator_times_two(iterator):  
        result.append(v)  
    assert result == [2, 14, 8]
```

And create a generator expression:

```
sequence = cIterator.SequenceOfLong([1, 7, 4])  
result = sum(v * 4 for v in sequence)  
assert result == 4 * (1 + 7 + 4)
```


CONTEXT MANAGERS

This chapter describes how to write for your C objects.

16.1 C Functions

This is a summary of what is required for the C functions implementing a [context manger](#). There is no specific `tp_...` slot for the context manager functions `__enter__` and `__exit__`, instead they are added to the object as named, looked up, Python methods.

16.1.1 `__enter__`

The C function must, at least, increment the reference count of `self` and return `self`:

```
static PyObject *
ContextManager_enter(ContextManager *self, PyObject *Py_UNUSED(args)) {
    /* Stuff here. */
    Py_INCREF(self);
    return (PyObject *)self;
}
```

16.1.2 `__exit__`

The `__exit__()` function is declared thus. It takes three arguments so `METH_VARARGS` is used. The three arguments are each `None` if no exception has been raised within the `with` block. If an exception *has* been raised within the `with` block then the three arguments are the exception type, value and the traceback object.

The return value of the `__exit__` method tells the interpreter whether any exception should be suppressed. If the function returns `False` then the exception should be propagated. This is the common case. If the function returns `True` then the exception should be suppressed and execution continues with the statement immediately after the `with` statement.

The `exit` method is defined in C thus. Note that there is no change to the reference count, that is all done appropriately by the CPython interpreter:

```
static PyObject *
ContextManager_exit(ContextManager *self, PyObject *args) {
    /* Stuff. */
    Py_RETURN_FALSE;
}
```

16.1.3 Method declarations

Note that `__enter__()` is declared with `METH_NOARGS` and `__exit__()` is declared with `METH_VARARGS`:

```
static PyMethodDef ContextManager_methods[] = {
    /* ... */
    {"__enter__", (PyCFunction) ContextManager_enter, METH_NOARGS,
     PyDoc_STR("__enter__() -> ContextManager")},
    {"__exit__", (PyCFunction) ContextManager_exit, METH_VARARGS,
     PyDoc_STR("__exit__(exc_type, exc_value, exc_tb) -> bool")},
    /* ... */
    {NULL, NULL, 0, NULL} /* sentinel */
};
```

16.2 Understanding the Context Manager

What is worth understanding is the way that reference counts are incremented and decremented and the interplay between your C code and the CPython interpreter.

16.2.1 A Context Manager Without a Target

Take this simple code:

```
from cPyExtPatt import cCtxMgr

with cCtxMgr.ContextManager():
    pass
```

The sequence of reference count changes are as follows:

1. Creating the `cCtxMgr.ContextManager()` calls `ContextManager_new` which makes the reference count 1.
2. The `with` statement causes the CPython interpreter to increment the reference count (to 2) and then call `__enter__` that is implemented in our C function `ContextManager_enter`.
3. Our `ContextManager_enter` function increments the reference count, so it is now 3.
4. As the context manager ends the `with` statement the CPython interpreter decrements the reference count *twice* to the value 1. The logic is:
 1. Decrement the reference count once as we are exiting the `with` statement. The reference count is now 2.
 2. Did the `with` statement have a target? If not, as in this case, then decrement the reference count once more. The reference count is now 1.
5. after the `pass` statement the CPython interpreter then calls `__exit__` which is implemented in our function `ContextManager_exit`. This does not change the reference count which remains at 1.
6. As the context manager goes out of scope the CPython interpreter decrements the reference count to 0 and then calls our C function `ContextManager_dealloc` with a reference count of 0 and that frees the object.

16.2.2 A Context Manager With a Target

The importance of the `with` statement having a target is because a context manager can be used like this:

```
from cPyExtPatt import cCtxMgr
```

(continues on next page)

(continued from previous page)

```

with cCtxMgr.ContextManager() as context:
    pass
# context survives here with a reference count of 1.
# This will be decremented when context goes out of scope.
# For example on a function return.

```

In this case the `context` survives the `with` statement and is available to any following code. So step 4 above would only decrement the reference count once leaving it with a reference count of 2. The `context` variable reference count will be finally decremented when it goes out of scope, for example on a function return.

The sequence of reference count changes are now as follows:

1. As above, the reference count becomes 1.
2. As above, the reference count becomes 2.
3. As above, the reference count becomes 3.
4. As the context manager ends the `with` statement the CPython interpreter decrements the reference count just *once* to the value 2 as there *is* a target, called `context` in this case.
5. After the `pass` statement the CPython interpreter then calls `__exit__` which is implemented in our function `ContextManager_exit`. This does not change the reference count which remains at 2.
6. As the context manager goes out of scope the CPython interpreter decrements the reference count to 1. This ensures the survival of `context` after the `with` block.
7. When `context` goes out of scope, say on a function return or a `del` statement the CPython interpreter decrements the reference count to 0 and then calls our C function `ContextManager_dealloc` which frees the object.

16.3 Minimal Context Manager in C

Here is the minimal, complete, C code that implements context manager. There is example code in `src/cpy/CtxMgr/cCtxMgr.c` and tests in `tests/unit/test_c_ctxmgr.py`

First the object declaration, allocation and de-allocation functions:

```

1  #define PY_SSIZE_T_CLEAN
2
3  #include "Python.h"
4
5  typedef struct {
6      PyObject_HEAD
7  } ContextManager;
8
9  static ContextManager *
10 ContextManager_new(PyObject *Py_UNUSED(arg)) {
11     return PyObject_New(ContextManager, &ContextManager_Type);
12 }
13
14 static void
15 ContextManager_dealloc(ContextManager *self) {
16     PyObject_Del(self);
17 }

```

16.3.1 `__enter__` and `__exit__` Methods

The `__enter__` and `__exit__` methods:

```

1  static PyObject *
2  ContextManager_enter(ContextManager *self, PyObject *Py_UNUSED(args)) {
3      Py_INCREF(self);
4      return (PyObject *)self;
5  }
6
7  static PyObject *
8  ContextManager_exit(ContextManager *Py_UNUSED(self), PyObject *Py_UNUSED(args)) {
9      Py_RETURN_FALSE;
10 }
11
12 static PyMethodDef ContextManager_methods[] = {
13     {"__enter__", (PyCFunction) ContextManager_enter, METH_VARARGS,
14         PyDoc_STR("__enter__() -> ContextManager")},
15     {"__exit__", (PyCFunction) ContextManager_exit, METH_VARARGS,
16         PyDoc_STR("__exit__() -> bool")},
17     {NULL, NULL, 0, NULL} /* sentinel */
18 };

```

16.3.2 Type Declaration

The type declaration:

```

static PyTypeObject ContextManager_Type = {
    PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "cObject.ContextManager",
    .tp_basicsize = sizeof(ContextManager),
    .tp_dealloc = (destructor) ContextManager_dealloc,
    .tp_flags = Py_TPFLAGS_DEFAULT,
    .tp_methods = ContextManager_methods,
    .tp_new = (newfunc) ContextManager_new,
};

```

16.3.3 Module

Finally the module:

```

1  PyDoc_STRVAR(module_doc, "Example of a context manager.");
2
3  static struct PyModuleDef cCtxMgr = {
4      PyModuleDef_HEAD_INIT,
5      .m_name = "cCtxMgr",
6      .m_doc = module_doc,
7      .m_size = -1,
8  };
9
10 PyMODINIT_FUNC
11 PyInit_cCtxMgr(void) {
12     PyObject *m = NULL;
13     m = PyModule_Create(&cCtxMgr);

```

(continues on next page)

(continued from previous page)

```

14     if (m == NULL) {
15         goto fail;
16     }
17     if (PyType_Ready(&ContextManager_Type) < 0) {
18         goto fail;
19     }
20     if (PyModule_AddObject(m, "ContextManager", (PyObject *) &ContextManager_Type)) {
21         goto fail;
22     }
23     return m;
24 fail:
25     Py_XDECREF(m);
26     return NULL;
27 }

```

16.3.4 Setup

This code is added to the setup.py file:

```

Extension(f"{PACKAGE_NAME}.cCtxMgr", sources=['src/cpy/CtxMgr/cCtxMgr.c', ],
        include_dirs=['/usr/local/include', ],
        library_dirs=[os.getcwd(), ],
        extra_compile_args=extra_compile_args_c,
        language='c',
        ),

```

And can be used thus:

```

from cPyExtPatt import cCtxMgr

with cCtxMgr.ContextManager():
    pass

```

16.3.5 Testing

The actual code in src/cpy/CtxMgr/cCtxMgr.c contains extra trace reporting that confirms the reference counts and (no) memory leakage.

This can be run with:

```
$ pytest tests/unit/test_c_ctxmgr.py -vs
```

This test:

```

def test_very_simple():
    print()
    with cCtxMgr.ContextManager():
        pass

```

Gives this output:

```

tests/unit/test_c_ctxmgr.py::test_very_simple
    ContextManager_new DONE REFCNT = 1

```

(continues on next page)

(continued from previous page)

```

ContextManager_enter  STRT REFCNT = 2
ContextManager_enter  DONE REFCNT = 3
ContextManager_exit   STRT REFCNT = 1
ContextManager_exit   DONE REFCNT = 1
ContextManager_dealloc STRT REFCNT = 0
ContextManager_dealloc DONE REFCNT = 4546088432

```

This test:

```

1 def test_simple():
2     print()
3     with cCtxMgr.ContextManager() as context:
4         assert sys.getrefcount(context) == 3
5         assert context.len_buffer_lifetime() == cCtxMgr.BUFFER_LENGTH
6         assert context.len_buffer_context() == cCtxMgr.BUFFER_LENGTH
7     assert sys.getrefcount(context) == 2
8     assert context.len_buffer_lifetime() == cCtxMgr.BUFFER_LENGTH
9     assert context.len_buffer_context() == 0
10    del context

```

Gives this output:

```

tests/unit/test_c_ctxmgr.py::test_simple
ContextManager_new  DONE REFCNT = 1
ContextManager_enter STRT REFCNT = 2
ContextManager_enter DONE REFCNT = 3
ContextManager_exit  STRT REFCNT = 2
ContextManager_exit  DONE REFCNT = 2
ContextManager_dealloc STRT REFCNT = 0
ContextManager_dealloc DONE REFCNT = 4546096048

```

This test:

```

1 def test_memory():
2     proc = psutil.Process()
3     print()
4     print(f'RSS START: {proc.memory_info().rss:12,d}')
5     for i in range(8):
6         print(f'RSS START {i:5d}: {proc.memory_info().rss:12,d}')
7         with cCtxMgr.ContextManager() as context:
8             print(f'RSS START CTX: {proc.memory_info().rss:12,d}')
9             # Does not work in the debugger due to introspection.
10            # assert sys.getrefcount(context) == 3
11            assert context.len_buffer_lifetime() == cCtxMgr.BUFFER_LENGTH
12            assert context.len_buffer_context() == cCtxMgr.BUFFER_LENGTH
13            print(f'RSS END CTX: {proc.memory_info().rss:12,d}')
14            # Does not work in the debugger due to introspection.
15            # assert sys.getrefcount(context) == 2
16            assert context.len_buffer_lifetime() == cCtxMgr.BUFFER_LENGTH
17            assert context.len_buffer_context() == 0
18            del context
19            print(f'RSS END {i:5d}: {proc.memory_info().rss:12,d}')
20    print(f'RSS END: {proc.memory_info().rss:12,d}')

```

Gives this output:

```

1 tests/unit/test_c_ctxmgr.py::test_memory
2 RSS START: 300,032,000
3 RSS START 0: 300,048,384
4     ContextManager_new DONE REFCNT = 1
5     ContextManager_enter STRT REFCNT = 2
6     ContextManager_enter DONE REFCNT = 3
7 RSS START CTX: 300,048,384
8 RSS END CTX: 300,048,384
9     ContextManager_exit STRT REFCNT = 2
10    ContextManager_exit DONE REFCNT = 2
11    ContextManager_dealloc STRT REFCNT = 0
12    ContextManager_dealloc DONE REFCNT = 4546096048
13 RSS END 0: 300,048,384
14 RSS START 1: 300,048,384
15     ContextManager_new DONE REFCNT = 1
16     ContextManager_enter STRT REFCNT = 2
17     ContextManager_enter DONE REFCNT = 3
18 RSS START CTX: 300,048,384
19 RSS END CTX: 300,048,384
20     ContextManager_exit STRT REFCNT = 2
21     ContextManager_exit DONE REFCNT = 2
22     ContextManager_dealloc STRT REFCNT = 0
23     ContextManager_dealloc DONE REFCNT = 4546096048
24 RSS END 1: 300,048,384
25 RSS START 2: 300,048,384
26     ContextManager_new DONE REFCNT = 1
27     ContextManager_enter STRT REFCNT = 2
28     ContextManager_enter DONE REFCNT = 3
29 RSS START CTX: 300,048,384
30 RSS END CTX: 300,048,384
31     ContextManager_exit STRT REFCNT = 2
32     ContextManager_exit DONE REFCNT = 2
33     ContextManager_dealloc STRT REFCNT = 0
34     ContextManager_dealloc DONE REFCNT = 4546096048
35 RSS END 2: 300,048,384
36 RSS START 3: 300,048,384
37     ContextManager_new DONE REFCNT = 1
38     ContextManager_enter STRT REFCNT = 2
39     ContextManager_enter DONE REFCNT = 3
40 RSS START CTX: 300,048,384
41 RSS END CTX: 300,048,384
42     ContextManager_exit STRT REFCNT = 2
43     ContextManager_exit DONE REFCNT = 2
44     ContextManager_dealloc STRT REFCNT = 0
45     ContextManager_dealloc DONE REFCNT = 4546096048
46 RSS END 3: 300,048,384
47 RSS START 4: 300,048,384
48     ContextManager_new DONE REFCNT = 1
49     ContextManager_enter STRT REFCNT = 2
50     ContextManager_enter DONE REFCNT = 3
51 RSS START CTX: 300,048,384

```

(continues on next page)

(continued from previous page)

```

52 RSS   END CTX:  300,048,384
53     ContextManager_exit  STRT REFCNT = 2
54     ContextManager_exit  DONE REFCNT = 2
55     ContextManager_dealloc STRT REFCNT = 0
56     ContextManager_dealloc DONE REFCNT = 4546096048
57 RSS   END      4:  300,048,384
58 RSS  START      5:  300,048,384
59     ContextManager_new  DONE REFCNT = 1
60     ContextManager_enter STRT REFCNT = 2
61     ContextManager_enter DONE REFCNT = 3
62 RSS  START CTX:  300,048,384
63 RSS   END CTX:  300,048,384
64     ContextManager_exit  STRT REFCNT = 2
65     ContextManager_exit  DONE REFCNT = 2
66     ContextManager_dealloc STRT REFCNT = 0
67     ContextManager_dealloc DONE REFCNT = 4546096048
68 RSS   END      5:  300,048,384
69 RSS  START      6:  300,048,384
70     ContextManager_new  DONE REFCNT = 1
71     ContextManager_enter STRT REFCNT = 2
72     ContextManager_enter DONE REFCNT = 3
73 RSS  START CTX:  300,048,384
74 RSS   END CTX:  300,048,384
75     ContextManager_exit  STRT REFCNT = 2
76     ContextManager_exit  DONE REFCNT = 2
77     ContextManager_dealloc STRT REFCNT = 0
78     ContextManager_dealloc DONE REFCNT = 4546096048
79 RSS   END      6:  300,048,384
80 RSS  START      7:  300,048,384
81     ContextManager_new  DONE REFCNT = 1
82     ContextManager_enter STRT REFCNT = 2
83     ContextManager_enter DONE REFCNT = 3
84 RSS  START CTX:  300,048,384
85 RSS   END CTX:  300,048,384
86     ContextManager_exit  STRT REFCNT = 2
87     ContextManager_exit  DONE REFCNT = 2
88     ContextManager_dealloc STRT REFCNT = 0
89     ContextManager_dealloc DONE REFCNT = 4546096048
90 RSS   END      7:  300,048,384
91 RSS  END:  300,048,384

```


PICKLING C EXTENSION TYPES

If you need to provide support for pickling your specialised types from your C extension then you need to implement some special functions.

This example shows you how to provide pickle support for the `custom2.Custom` type described in the C extension tutorial in the [Python documentation](#). This defines an `CustomObject` object that has three fields; a first name, a last name and a number. The `CustomObject` definition that needs to be pickled and un-pickled looks like this in C:

```
typedef struct {
    PyObject_HEAD
    PyObject *first; /* first name */
    PyObject *last;  /* last name */
    int number;
} CustomObject;
```

- The example C code is in `src/cpy/Pickle/cCustomPickle.c`.
- The test code is in `tests/unit/test_c_custom_pickle.py`.

17.1 Pickle Version Control

Since the whole point of `pickle` is persistence then pickled objects can hang around in databases, file systems, data from the `shelve` module and whatnot for a long time. It is entirely possible that when un-pickled, sometime in the future, that your C extension has moved on and then things become awkward.

It is *strongly* recommended that you add some form of version control to your pickled objects. In this example I just have a single integer version number which I write to the pickled object. If the number does not match on unpickling then I raise an exception. When I change the type API I would, judiciously, change this version number.

Clearly more sophisticated strategies are possible by supporting older versions of the pickled object in some way but this will do for now.

We add some simple pickle version information to the C extension:

```
static const char* PICKLE_VERSION_KEY = "_pickle_version";
static int PICKLE_VERSION = 1;
```

Now we can implement `__getstate__` and `__setstate__`, think of these as symmetric operations.

First `__getstate__`.

17.2 Implementing `__getstate__`

`__getstate__` pickles the object. `__getstate__` is expected to return a dictionary of the internal state of the Custom object. Note that a Custom object has two Python objects (`first` and `last`) and a C integer (`number`) that need to be converted to a Python object. We also need to add the version information.

Here is the C implementation:

```

1  /* Pickle the object */
2  static PyObject *
3  Custom__getstate__(CustomObject *self, PyObject *Py_UNUSED(ignored)) {
4      PyObject *ret = Py_BuildValue("{sOs0sisi}",
5                                   "first", self->first,
6                                   "last", self->last,
7                                   "number", self->number,
8                                   PICKLE_VERSION_KEY, PICKLE_VERSION);
9      return ret;
10 }
```

Note

Note the careful use of `Py_BuildValue()`. "s" and "i" causes a new PyObject to be created in the dict. "O" increments the reference count of an existing PyObject which is inserted into the dict.

See *Warning on "Stolen" References With Containers*

17.3 Implementing `__setstate__`

The implementation of `__setstate__` un-pickles the object. This is a little more complicated as there is quite a lot of error checking going on. We are being passed an arbitrary Python object and need to check:

- It is a Python dictionary.
- It has a version key and the version value is one that we can deal with.
- It has the required keys and values to populate our Custom object.

Note that our `__new__` method (`Custom_new()`) has already been called on `self`. Before setting any member value we need to de-allocate the existing value set by `Custom_new()` otherwise we will have a memory leak.

17.3.1 Error Checking

```

1  /* Un-pickle the object */
2  static PyObject *
3  Custom__setstate__(CustomObject *self, PyObject *state) {
4      /* Error check. */
5      if (!PyDict_CheckExact(state)) {
6          PyErr_SetString(PyExc_ValueError, "Pickled object is not a dict.");
7          return NULL;
8      }
9      /* Version check. */
10     /* Borrowed reference but no need to increment as we create a C long
11      * from it. */
```

(continues on next page)

(continued from previous page)

```

12 PyObject *temp = PyDict_GetItemString(state, PICKLE_VERSION_KEY);
13 if (temp == NULL) {
14     /* PyDict_GetItemString does not set any error state so we have to. */
15     PyErr_Format(PyExc_KeyError, "No \"%s\" in pickled dict.",
16                 PICKLE_VERSION_KEY);
17     return NULL;
18 }
19 if (!PyLong_Check(temp)) {
20     PyErr_Format(
21         PyExc_KeyError,
22         "Pickled dict version key \"%s\" is not a long but type \"%s\".",
23         PICKLE_VERSION_KEY,
24         temp->ob_type->tp_name
25     );
26     return NULL;
27 }
28 int pickle_version = (int) PyLong_AsLong(temp);
29 if (pickle_version != PICKLE_VERSION) {
30     PyErr_Format(PyExc_ValueError,
31                 "Pickle version mismatch. Got version %d but expected version %d.",
32                 pickle_version, PICKLE_VERSION);
33     return NULL;
34 }

```

17.3.2 Set the first Member

```

1  /* NOTE: Custom_new() will have been invoked so self->first and self->last
2   * will have been allocated so we have to de-allocate them. */
3  temp = PyDict_GetItemString(state, "first"); /* Borrowed reference. */
4  if (temp == NULL) {
5      /* PyDict_GetItemString does not set any error state so we have to. */
6      PyErr_SetString(PyExc_KeyError, "No \"first\" in pickled dict.");
7      return NULL;
8  }
9  if (!PyUnicode_Check(temp)) {
10     PyErr_Format(
11         PyExc_KeyError,
12         "Pickled dict key \"first\" is not a str but type \"%s\".",
13         temp->ob_type->tp_name
14     );
15     return NULL;
16 }
17 Py_DECREF(self->first);
18 self->first = temp;
19 Py_INCREF(self->first);

```

17.3.3 Set the last Member

This code is very similar to the code for the first member above.

```

1 temp = PyDict_GetItemString(state, "last"); /* Borrowed reference. */
2 if (temp == NULL) {
3     /* PyDict_GetItemString does not set any error state so we have to. */
4     PyErr_SetString(PyExc_KeyError, "No \"last\" in pickled dict.");
5     return NULL;
6 }
7 if (!PyUnicode_Check(temp)) {
8     PyErr_Format(
9         PyExc_KeyError,
10        "Pickled dict key \"last\" is not a str but type \"%s\".",
11        temp->ob_type->tp_name
12    );
13    return NULL;
14 }
15 Py_DECREF(self->last);
16 self->last = temp;
17 Py_INCREF(self->last);

```

17.3.4 Set the number Member

This is a C fundamental type so the code is slightly different:

```

1 /* Borrowed reference but no need to incref as we create a C long from it. */
2 PyObject *number = PyDict_GetItemString(state, "number");
3 if (number == NULL) {
4     /* PyDict_GetItemString does not set any error state so we have to. */
5     PyErr_SetString(PyExc_KeyError, "No \"number\" in pickled dict.");
6     return NULL;
7 }
8 if (!PyLong_Check(number)) {
9     PyErr_Format(
10        PyExc_KeyError,
11        "Pickled dict key \"number\" is not an int but type \"%s\".",
12        number->ob_type->tp_name
13    );
14    return NULL;
15 }
16 self->number = (int) PyLong_AsLong(number);

```

And we are done.

```

Py_RETURN_NONE;
}

```

17.3.5 __setstate__ in Full

```

1 static PyObject *
2 Custom__setstate__(CustomObject *self, PyObject *state) {
3
4     if (!PyDict_CheckExact(state)) {
5         PyErr_SetString(PyExc_ValueError, "Pickled object is not a dict.");
6         return NULL;

```

(continues on next page)

(continued from previous page)

```

7  }
8  /* Version check. */
9  /* Borrowed reference but no need to increment as we create a C long from it. */
10 PyObject *temp = PyDict_GetItemString(state, PICKLE_VERSION_KEY);
11 if (temp == NULL) {
12     /* PyDict_GetItemString does not set any error state so we have to. */
13     PyErr_Format(PyExc_KeyError, "No \"%s\" in pickled dict.", PICKLE_VERSION_KEY);
14     return NULL;
15 }
16 if (!PyLong_Check(temp)) {
17     PyErr_Format(
18         PyExc_KeyError,
19         "Pickled dict version key \"%s\" is not a long but type \"%s\".",
20         PICKLE_VERSION_KEY,
21         temp->ob_type->tp_name
22     );
23     return NULL;
24 }
25 int pickle_version = (int) PyLong_AsLong(temp);
26 if (pickle_version != PICKLE_VERSION) {
27     PyErr_Format(
28         PyExc_ValueError,
29         "Pickle version mismatch. Got version %d but expected version %d.",
30         pickle_version, PICKLE_VERSION
31     );
32     return NULL;
33 }
34
35 temp = PyDict_GetItemString(state, "first"); /* Borrowed reference. */
36 if (temp == NULL) {
37     /* PyDict_GetItemString does not set any error state so we have to. */
38     PyErr_SetString(PyExc_KeyError, "No \"first\" in pickled dict.");
39     return NULL;
40 }
41 if (!PyUnicode_Check(temp)) {
42     PyErr_Format(
43         PyExc_KeyError,
44         "Pickled dict key \"first\" is not a str but type \"%s\".",
45         temp->ob_type->tp_name
46     );
47     return NULL;
48 }
49 Py_DECREF(self->first);
50 self->first = temp;
51 Py_INCREF(self->first);
52
53 temp = PyDict_GetItemString(state, "last"); /* Borrowed reference. */
54 if (temp == NULL) {
55     /* PyDict_GetItemString does not set any error state so we have to. */
56     PyErr_SetString(PyExc_KeyError, "No \"last\" in pickled dict.");
57     return NULL;
58 }

```

(continues on next page)

(continued from previous page)

```

59     if (!PyUnicode_Check(temp)) {
60         PyErr_Format(
61             PyExc_KeyError,
62             "Pickled dict key \"%last\" is not a str but type \"%s\".",
63             temp->ob_type->tp_name
64         );
65         return NULL;
66     }
67     Py_DECREF(self->last);
68     self->last = temp;
69     Py_INCREF(self->last);
70
71     /* Borrowed reference but no need to increment as we create a C long from it. */
72     PyObject *number = PyDict_GetItemString(state, "number");
73     if (number == NULL) {
74         /* PyDict_GetItemString does not set any error state so we have to. */
75         PyErr_SetString(PyExc_KeyError, "No \"%number\" in pickled dict.");
76         return NULL;
77     }
78     if (!PyLong_Check(number)) {
79         PyErr_Format(
80             PyExc_KeyError,
81             "Pickled dict key \"%number\" is not an int but type \"%s\".",
82             number->ob_type->tp_name
83         );
84         return NULL;
85     }
86     self->number = (int) PyLong_AsLong(number);
87
88     Py_RETURN_NONE;
89 }

```

17.4 Add the Special Methods

Now we need to add these two special methods to the methods table which now looks like this:

```

1  static PyMethodDef Custom_methods[] = {
2      /* Existing methods here... */
3      {"__getstate__", (PyCFunction) Custom___getstate__, METH_NOARGS,
4          "Pickle the Custom object"
5      },
6      {"__setstate__", (PyCFunction) Custom___setstate__, METH_0,
7          "Un-pickle the Custom object"
8      },
9      {NULL} /* Sentinel */
10 };

```

17.5 Pickling a custom2.Custom Object

We can test this with code like this that pickles one `custom2.Custom` object then creates another `custom2.Custom` object from that pickle. Here is some Python code that exercises our module (tests are in `tests/unit/test_c_custom_pickle.py`):

```

1 import io
2 import pickle
3 import pickletools
4 import sys
5
6 import pytest
7
8 from cPyExtPatt import cPickle
9
10
11 def test_module_dir():
12     assert dir(cPickle) == [
13         'Custom', '__doc__', '__file__', '__loader__',
14         '__name__', '__package__', '__spec__',
15     ]
16
17
18 ARGS_FOR_CUSTOM_CLASS = ('FIRST', 'LAST', 11)
19 PICKLE_BYTES_FOR_CUSTOM_CLASS = (
20     b'\x80\x04\x95f\x00\x00\x00\x00\x00\x00\x00\x00\x0c\x12cPyExtPatt.cPickle\x94'
21     b'\x8c\x06Custom\x94\x93\x94)\x81\x94}\x94(\x8c\x05first\x94\x8c\x05FIRST'
22     b'\x94\x8c\x04last\x94\x8c\x04LAST\x94\x8c\x06number\x94K\x0b\x8c\x0f_pickle_'
23     b'version\x94K\x01ub.'
24 )
25
26 def test_pickle_getstate():
27     custom = cPickle.Custom(*ARGS_FOR_CUSTOM_CLASS)
28     pickled_value = pickle.dumps(custom)
29     print()
30     print(f'Pickled original is {pickled_value}')
31     assert pickled_value == PICKLE_BYTES_FOR_CUSTOM_CLASS
32     # result = pickle.loads(pickled_value)
33
34
35 def test_pickle_setstate():
36     custom = pickle.loads(PICKLE_BYTES_FOR_CUSTOM_CLASS)
37     assert custom.first == 'FIRST'
38     assert custom.last == 'LAST'
39     assert custom.number == 11
40
41 def test_pickle_round_trip():
42     custom = cPickle.Custom(*ARGS_FOR_CUSTOM_CLASS)
43     pickled_value = pickle.dumps(custom)
44     result = pickle.loads(pickled_value)
45     assert id(result) != id(custom)

```

So we have pickled one object and recreated a different, but equivalent, instance from the pickle of the original object which is what we set out to do.

17.6 The Pickled Object in Detail

If you are curious about the contents of the pickled object the Python standard library provides the `pickletools` module. This allows you to inspect the pickled object. Here is a test for that:

```
1 import io
2 import pickle
3 import pickletools
4 import sys
5
6 import pytest
7
8 from cPyExtPatt import cPickle
9
10 def test_pickletools():
11     outfile = io.StringIO()
12     pickletools.dis(PICKLE_BYTES_FOR_CUSTOM_CLASS, out=outfile, annotate=1)
13     result = outfile.getvalue()
14     expected = '<See below>'
15     assert result == expected
```

The expected output will be something like this:

17.6. The Pickled Object in Detail 199

17.7 Pickling Objects with External State

This is just a simple example, if your object relies on external state such as open files, databases and the like you need to be careful, and knowledgeable about your state management. There is some useful information in [Handling Stateful Objects](#)

Note

Marshalling support for use with the `marshal` module is given by the [C Marshall API](#)

17.8 References

- Python API documentation for `__setstate__`
- Python API documentation for `__getstate__`
- Useful documentation for [Handling Stateful Objects](#)
- Python [pickle](#) module
- Python [shelve](#) module

WATCHERS [PYTHON 3.12+]

From Python 3.12 onwards *watchers* have been added¹. This allows registering a callback function on specific dict, type, code or function object. The callback is called with any event that occurs on the specific object. This can be a powerful debugging technique.

Here is an example of a dictionary watcher.

18.1 Dictionary Watchers

Here is a context manager `cWatchers.PyDictWatcher` that wraps the low level CPython code with a watcher that reports every dictionary operation to stdout. The code is in `src/cpy/Watchers/watcher_example.py`.

```
1  """Example of using watchers."""
2
3  from cPyExtPatt import cWatchers
4
5
6  def dict_watcher_demo() -> None:
7      print('dict_watcher_demo():')
8      d = {}                                # The dictionary we are going to watch.
9      with cWatchers.PyDictWatcher(d):
10         dd = {'age': 17, }
11         d.update(dd)                       # Generates event: PyDict_EVENT_CLONED
12         d['age'] = 42                       # Generates event: PyDict_EVENT_MODIFIED
13         del d['age']                       # Generates event: PyDict_EVENT_DELETED
14         d['name'] = 'Python'               # Generates event: PyDict_EVENT_ADDED
15         d.clear()                         # Generates event: PyDict_EVENT_CLEARED
16         del d
17
18
19  if __name__ == '__main__':
20      dict_watcher_demo()
```

And the output would be something like this, it reports the Python file, line number, function, event and then detail about the arguments used to manipulate the dictionary:

[Continued on the next page]

¹ This change was not done with any PEP that I can find. Instead it was done during the ordinary pace of development. For example this change has example tests: <https://github.com/python/cpython/pull/31787/files> and is tracked with this issue: <https://github.com/python/cpython/issues/91052>

```
dict_watcher_demo():
Dict @ 0x0x10fb53c00: watcher_example.py 11 dict_watcher_demo PyDict_EVENT_CLONED
    Dict: {}
    Key (dict): {'age': 17}
    New value : NULL
Dict @ 0x0x10fb53c00: watcher_example.py 12 dict_watcher_demo PyDict_EVENT_MODIFIED
    Dict: {'age': 17}
    Key (str): age
    New value (int): 42
Dict @ 0x0x10fb53c00: watcher_example.py 13 dict_watcher_demo PyDict_EVENT_DELETED
    Dict: {'age': 42}
    Key (str): age
    New value : NULL
Dict @ 0x0x10fb53c00: watcher_example.py 14 dict_watcher_demo PyDict_EVENT_ADDED
    Dict: {}
    Key (str): name
    New value (str): Python
Dict @ 0x0x10fb53c00: watcher_example.py 15 dict_watcher_demo PyDict_EVENT_CLEARED
    Dict: {'name': 'Python'}
    Key : NULL
    New value : NULL
```

There are some obvious variations here:

- Add some prefix to each watcher output line to discriminate it from the rest of stdout.
- Different outputs, such as JSON.

But how does this watcher work?

18.1.1 Low Level C Implementation

We need some low level C code that interacts with the CPython watcher API. First a header file that provides the interface to our dictionary watcher code. This declares two functions:

- `dict_watcher_verbose_add()` this adds a watcher to a dictionary and returns the watcher ID.
- `dict_watcher_verbose_remove()` this removes a watcher ID from a dictionary.

The actual code is in `src/cpy/Watchers/DictWatcher.h`. It looks like this, note the Python version guard to ensure this only works with Python 3.12+:

```
#define PPY_SSIZE_T_CLEAN
#include "Python.h"

#if PY_VERSION_HEX < 0x030C0000
#error "Required version of Python is 3.12+ (PY_VERSION_HEX >= 0x030C0000)"
#else

int dict_watcher_verbose_add(PyObject *dict);
int dict_watcher_verbose_remove(int watcher_id, PyObject *dict);

#endif // #if PY_VERSION_HEX >= 0x030C0000
```

So there are several moving parts in the implementation in `src/cpy/Watchers/DictWatcher.c`. First we have some general purpose functions that extract the file name, function name and line number from a Python frame. Note that the Python frame API changed in Python 3.11.

First up, getting the Python file name:

```
#include "DictWatcher.h"

static const unsigned char *
get_python_file_name(PyFrameObject *frame) {
    if (frame) {
        // Python 3.11+ specific code.
        #if PY_VERSION_HEX >= 0x030B0000
            /* See:
             * https://docs.python.org/3.11/whatsnew/3.11.html#pyframeobject-3-11-hiding
             */
            const unsigned char *file_name = PyUnicode_1BYTE_DATA(
                PyFrame_GetCode(frame)->co_filename
            );
        #else
            const unsigned char *file_name = PyUnicode_1BYTE_DATA(
                frame->f_code->co_filename
            );
        #endif // #if PY_VERSION_HEX >= 0x030B0000
        return file_name;
    }
    return "";
}
```

Now, getting the Python function name:

```
static const char *
get_python_function_name(PyFrameObject *frame) {
    const char *func_name = NULL;
    if (frame) {
        // Python 3.11+ specific code.
        #if PY_VERSION_HEX >= 0x030B0000
            /* See:
             * https://docs.python.org/3.11/whatsnew/3.11.html#pyframeobject-3-11-hiding
             */
            func_name = (const char *) PyUnicode_1BYTE_DATA(
                PyFrame_GetCode(frame)->co_name
            );
        #else
            func_name = (const char *) PyUnicode_1BYTE_DATA(frame->f_code->co_name);
        #endif // #if PY_VERSION_HEX >= 0x030B0000
        return func_name;
    }
    return "";
}
```

[Continued on the next page]

Then, getting the Python line number:

```
int get_python_line_number(PyFrameObject *frame) {
    if (frame) {
        return PyFrame_GetLineNumber(frame);
    }
    return 0;
}
```

We bring these together to print a summary of the frame state to a file, such as stdout:

```
static void
write_frame_data_to_outfile(FILE *outfile, PyFrameObject *frame) {
    if (frame) {
        fprintf(outfile,
            "%-80s %6d %-24s",
            get_python_file_name(frame),
            get_python_line_number(frame),
            get_python_function_name(frame)
        );
    } else {
        fprintf(outfile, "No Python frame available.");
    }
}
```

Then there is a simple little helper function that returns a string based on the event type:

```
static const char *watch_event_name(PyDict_WatchEvent event) {
    switch (event) {
        case PyDict_EVENT_ADDED:
            return "PyDict_EVENT_ADDED";
            break;
        case PyDict_EVENT_MODIFIED:
            return "PyDict_EVENT_MODIFIED";
            break;
        case PyDict_EVENT_DELETED:
            return "PyDict_EVENT_DELETED";
            break;
        case PyDict_EVENT_CLONED:
            return "PyDict_EVENT_CLONED";
            break;
        case PyDict_EVENT_CLEARED:
            return "PyDict_EVENT_CLEARED";
            break;
        case PyDict_EVENT_DEALLOCATED:
            return "PyDict_EVENT_DEALLOCATED";
            break;
        default:
            Py_UNREACHABLE();
            break;
    }
    return "PyDict_EVENT_UNKNOWN";
}
```

Now we define the callback function that reports the dictionary event to stdout. This calls all the functions as above

and uses the CPython API `PyObject_Print` to print the representation of each object to `stdout`. This function has to respect NULL arguments:

```
static int dict_watcher_verbose(PyDict_WatchEvent event, PyObject *dict,
                               PyObject *key, PyObject *new_value) {
    fprintf(stdout, "Dict @ 0x%p: ", (void *)dict);
    write_frame_data_to_outfile(stdout, PyEval_GetFrame());
    fprintf(stdout, " Event: %s", watch_event_name(event));
    fprintf(stdout, "\n");
    if (dict) {
        fprintf(stdout, "    Dict: ");
        PyObject_Print(dict, stdout, Py_PRINT_RAW);
    } else {
        fprintf(stdout, "    Dict: NULL");
    }
    fprintf(stdout, "\n");
    if (key) {
        fprintf(stdout, "    Key (%s): ", Py_TYPE(key)->tp_name);
        PyObject_Print(key, stdout, Py_PRINT_RAW);
    } else {
        fprintf(stdout, "    Key : NULL");
    }
    fprintf(stdout, "\n");
    if (new_value) {
        fprintf(stdout, "    New value (%s): ", Py_TYPE(new_value)->tp_name);
        PyObject_Print(new_value, stdout, Py_PRINT_RAW);
    } else {
        fprintf(stdout, "    New value : NULL");
    }
    fprintf(stdout, "\n");
    return 0;
}
```

Finally we have the two implementations that register and unregister the callback using the low level Python C API. This uses `PyDict_AddWatcher()` and `PyDict_Watch()`. The first registers the callback, returning the watcher ID (error handling code omitted):

```
// Set watcher.
int dict_watcher_verbose_add(PyObject *dict) {
    int watcher_id = PyDict_AddWatcher(&dict_watcher_verbose);
    PyDict_Watch(watcher_id, dict);
    return watcher_id;
}
```

The second de-registers the callback, with the watcher ID and the dictionary in question (error handling code omitted). This uses `PyDict_Unwatch()` and `PyDict_ClearWatcher()`.

```
// Remove watcher.
int dict_watcher_verbose_remove(int watcher_id, PyObject *dict) {
    PyDict_Unwatch(watcher_id, dict);
    PyDict_ClearWatcher(watcher_id);
    return 0;
}
```

18.1.2 Exposing This to CPython

Now we create a Python module `cWatchers` that exposes this low level C code to CPython. This code is in `src/cpy/Watchers/cWatchers.c`.

First some module level CPython wrappers around our underlying C code:

```

1  #define PPY_SSIZE_T_CLEAN
2  #include "Python.h"
3
4  #include "DictWatcher.h"
5
6  static PyObject *
7  py_dict_watcher_verbose_add(PyObject *Py_UNUSED(module), PyObject *arg) {
8      if (!PyDict_Check(arg)) {
9          PyErr_Format(
10             PyExc_TypeError,
11             "Argument must be a dict not type %s",
12             Py_TYPE(arg)->tp_name
13         );
14         return NULL;
15     }
16     long watcher_id = dict_watcher_verbose_add(arg);
17     return Py_BuildValue("l", watcher_id);
18 }
19
20 static PyObject *
21 py_dict_watcher_verbose_remove(PyObject *Py_UNUSED(module), PyObject *args) {
22     long watcher_id;
23     PyObject *dict = NULL;
24
25     if (!PyArg_ParseTuple(args, "lO", &watcher_id, &dict)) {
26         return NULL;
27     }
28
29     if (!PyDict_Check(dict)) {
30         PyErr_Format(
31             PyExc_TypeError,
32             "Argument must be a dict not type %s",
33             Py_TYPE(dict)->tp_name
34         );
35         return NULL;
36     }
37     long result = dict_watcher_verbose_remove(watcher_id, dict);
38     return Py_BuildValue("l", result);
39 }

```

Now create the table of module methods:

```

static PyMethodDef module_methods[] = {
    {"py_dict_watcher_verbose_add",
     (PyCFunction) py_dict_watcher_verbose_add,
     METH_O,
     "Adds watcher to a dictionary. Returns the watcher ID."
    },

```

(continues on next page)

(continued from previous page)

```

    {"py_dict_watcher_verbose_remove",
     (PyCFunction) py_dict_watcher_verbose_remove,
     METH_VARARGS,
     "Removes the watcher ID from the dictionary."
    },
    {NULL, NULL, 0, NULL} /* Sentinel */
};

```

18.1.3 Creating the Context Manager

These are all fine but to be Pythonic it would be helpful to create a Context Manager (see *Context Managers*) in C. The context manager holds a reference to the dictionary and the watcher ID. Here is the definition which holds a watcher ID and a reference to the dictionary:

```

typedef struct {
    PyObject_HEAD
    int watcher_id;
    PyObject *dict;
} PyDictWatcher;

```

Here is the creation code:

```

static PyDictWatcher *
PyDictWatcher_new(PyObject *Py_UNUSED(arg)) {
    PyDictWatcher *self;
    self = PyObject_New(PyDictWatcher, &PyDictWatcher_Type);
    if (self == NULL) {
        return NULL;
    }
    self->watcher_id = -1;
    self->dict = NULL;
    return self;
}

static PyObject *
PyDictWatcher_init(PyDictWatcher *self, PyObject *args) {
    if (!PyArg_ParseTuple(args, "O", &self->dict)) {
        return NULL;
    }
    if (!PyDict_Check(self->dict)) {
        PyErr_Format(
            PyExc_TypeError,
            "Argument must be a dictionary not a %s",
            Py_TYPE(self->dict)->tp_name
        );
        return NULL;
    }
    Py_INCREF(self->dict);
    return (PyObject *)self;
}

```

The destruction code just decrements the reference count of the dictionary:

```
static void
PyDictWatcher_dealloc(PyDictWatcher *self) {
    Py_DECREF(self->dict);
    PyObject_Del(self);
}
```

Now the code that provides the context manager's `__enter__` and `__exit__` methods. First the `__enter__` function, this uses the low level C function `dict_watcher_verbose_add()` to mark the dictionary as watched and hold the watcher ID:

```
static PyObject *
PyDictWatcher_enter(PyDictWatcher *self, PyObject *Py_UNUSED(args)) {
    self->watcher_id = dict_watcher_verbose_add(self->dict);
    Py_INCREF(self);
    return (PyObject *)self;
}
```

Now the `__exit__` function, this uses the low level C function `dict_watcher_verbose_remove()` to remove the watcher from the dictionary:

```
static PyObject *
PyDictWatcher_exit(PyDictWatcher *self, PyObject *Py_UNUSED(args)) {
    int result = dict_watcher_verbose_remove(self->watcher_id, self->dict);
    if (result) {
        PyErr_Format(
            PyExc_RuntimeError,
            "dict_watcher_verbose_remove() returned %d",
            result
        );
        Py_RETURN_TRUE;
    }
    Py_RETURN_FALSE;
}
```

Now we define the context manager methods and type:

```
static PyMethodDef PyDictWatcher_methods[] = {
    {"__enter__", (PyCFunction) PyDictWatcher_enter, METH_VARARGS,
     PyDoc_STR("__enter__() -> PyDictWatcher")},
    {"__exit__", (PyCFunction) PyDictWatcher_exit, METH_VARARGS,
     PyDoc_STR("__exit__(exc_type, exc_value, exc_tb) -> bool")},
    {NULL, NULL, 0, NULL} /* sentinel */
};

static PyTypeObject PyDictWatcher_Type = {
    PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "cWatchers.PyDictWatcher",
    .tp_basicsize = sizeof(PyDictWatcher),
    .tp_dealloc = (destructor) PyDictWatcher_dealloc,
    .tp_flags = Py_TPFLAGS_DEFAULT,
    .tp_methods = PyDictWatcher_methods,
    .tp_new = (newfunc) PyDictWatcher_new,
    .tp_init = (initproc) PyDictWatcher_init
};
```

18.1.4 Module, Setup and Test

Now we create the cWatchers module,

```
static PyModuleDef cWatchers = {
    PyModuleDef_HEAD_INIT,
    .m_name = "cWatchers",
    .m_doc = "Dictionary and type watchers.",
    .m_size = -1,
    .m_methods = module_methods,
};

PyMODINIT_FUNC PyInit_cWatchers(void) {
    PyObject *m = PyModule_Create(&cWatchers);
    if (!m) {
        goto fail;
    }
    if (PyType_Ready(&PyDictWatcher_Type) < 0) {
        goto fail;
    }
    if (PyModule_AddObject(m, "PyDictWatcher", (PyObject *) &PyDictWatcher_Type)) {
        goto fail;
    }
    return m;
fail:
    Py_XDECREF(m);
    return NULL;
}
```

And then in setup.py we add the extension:

```
Extension(
    name=f"{PACKAGE_NAME}.cWatchers",
    include_dirs=['src/cpy', ],
    sources=[
        "src/cpy/Watchers/cWatchers.c",
        "src/cpy/Watchers/DictWatcher.c",
        "src/cpy/pyextpatt_util.c",
    ],
    extra_compile_args=extra_compile_args_c,
    language='c',
),
```

And it can be used like this:

```
from cPyExtPatt import cWatchers

d = {}
with cWatchers.PyDictWatcher(d):
    d['age'] = 42
```

And the result on stdout is something like:

```
Dict @ 0x0x10fb53c00: watcher_example.py 12 dict_watcher_demo PyDict_EVENT_ADDED
Dict: {}
```

(continues on next page)

(continued from previous page)

```
Key (str): age  
New value (int): 42
```

18.1.5 Without the Context Manager

If you are putting in some debugging code then a context manager might not be convenient. `cWatchers` provides two functions, `py_dict_watcher_verbose_add()` and `py_dict_watcher_verbose_remove` that achieve the same aim:

```
from cPyExtPatt import cWatchers  
  
d = {}  
watcher_id = cWatchers.py_dict_watcher_verbose_add(d)  
d['age'] = 42  
cWatchers.py_dict_watcher_verbose_remove(watcher_id, d)
```

18.2 Type Watchers

These allow a callback when a type is modified.

This beyond the scope of this version of this document.

More information can be found in <https://docs.python.org/3/c-api/type.html>

18.3 Function Watchers

These allow a callback when a function is created and destroyed.

This beyond the scope of this version of this document.

More information can be found in <https://docs.python.org/3/c-api/function.html>

18.4 Code Watchers

These allow a callback when code is created and destroyed.

This beyond the scope of this version of this document.

More information can be found in <https://docs.python.org/3/c-api/code.html>

SETTING COMPILER FLAGS

It is sometimes difficult to decide what flags to set for the compiler and the best advice is to use the same flags that the version of Python you are using was compiled with. Here are a couple of ways to do that.

19.1 From the Command Line

In the Python install directory there is a *pythonX.Y-config* executable that can be used to extract the compiler flags where X is the major version and Y the minor version. For example (output is wrapped here for clarity):

```
$ which python3
/Library/Frameworks/Python.framework/Versions/3.13/bin/python3
$ python3 -VV
Python 3.13.0b3 (v3.13.0b3:7b413952e8, Jun 27 2024, 09:57:31) [Clang 15.0.0 (clang-1500.
↪3.9.4)]
$ /Library/Frameworks/Python.framework/Versions/3.13/bin/python3-config --cflags
-I/Library/Frameworks/Python.framework/Versions/3.13/include/python3.13
-I/Library/Frameworks/Python.framework/Versions/3.13/include/python3.13
-fno-strict-overflow -Wsign-compare -Wunreachable-code -fno-common -dynamic -DNDEBUG
-g -O3 -Wall -arch arm64 -arch x86_64 -g
```

19.2 Programmatically from Within a Python Process

The `sysconfig` module contains information about the build environment for the particular version of Python:

```
1 >>> import sysconfig
2 >>> sysconfig.get_config_var('CFLAGS')
3 '-fno-strict-overflow -Wsign-compare -Wunreachable-code -fno-common -dynamic -DNDEBUG -g
↪-O3 -Wall -arch arm64 -arch x86_64 -g'
4 >>> import pprint
5 >>> pprint.pprint(sysconfig.get_paths())
6 {'data': '/Library/Frameworks/Python.framework/Versions/3.13',
7  'include': '/Library/Frameworks/Python.framework/Versions/3.13/include/python3.13',
8  'platinclude': '/Library/Frameworks/Python.framework/Versions/3.13/include/python3.13',
9  'platlib': '/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-
↪packages',
10 'platstdlib': '/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13',
11 'purelib': '/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-
↪packages',
12 'scripts': '/Library/Frameworks/Python.framework/Versions/3.13/bin',
13 'stdlib': '/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13'}
```

(continues on next page)

(continued from previous page)

```

14 >>> sysconfig.get_paths()['include']
15 '/Library/Frameworks/Python.framework/Versions/3.13/include/python3.13'

```

19.3 From the Command Line using sysconfig

This very verbose output will give you a complete picture of your environment:

```

1 $ python3 -m sysconfig
2 Platform: "macosx-10.13-universal2"
3 Python version: "3.13"
4 Current installation scheme: "posix_prefix"
5
6 Paths:
7     data = "/Library/Frameworks/Python.framework/Versions/3.13"
8     include = "/Library/Frameworks/Python.framework/Versions/3.13/include/python3.13"
9     platinclude = "/Library/Frameworks/Python.framework/Versions/3.13/include/python3.13"
10    platlib = "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-
    ↳ packages"
11    platstdlib = "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13"
12    purelib = "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-
    ↳ packages"
13    scripts = "/Library/Frameworks/Python.framework/Versions/3.13/bin"
14    stdlib = "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13"
15
16 Variables:
17     ABIFLAGS = ""
18     AC_APPLE_UNIVERSAL_BUILD = "0"
19     AIX_BUILDDATE = "0"
20     AIX_GENUINE_CPLUSPLUS = "0"
21     ALIGNOF_LONG = "8"
22     ALIGNOF_MAX_ALIGN_T = "8"
23     ALIGNOF_SIZE_T = "8"
24     ALT_SOABI = "0"
25     ANDROID_API_LEVEL = "0"
26     AR = "/usr/bin/xcrun ar"
27     ...

```

19.4 Setting Flags Automatically in setup.py

The sysconfig module allows you to create a generic setup.py script for Python C extensions, something along these lines:

```

1 from distutils.core import setup, Extension
2 import os
3 import sysconfig
4
5 _DEBUG = False
6 # Generally I write code so that if DEBUG is defined as 0 then all optimisations
7 # are off and asserts are enabled. Typically run times of these builds are x2 to x10
8 # release builds.

```

(continues on next page)

(continued from previous page)

```

9  # If DEBUG > 0 then extra code paths are introduced such as checking the integrity of
10 # internal data structures. In this case the performance is by no means comparable
11 # with release builds.
12 _DEBUG_LEVEL = 0
13
14 # Common flags for both release and debug builds.
15 extra_compile_args = sysconfig.get_config_var('CFLAGS').split()
16 extra_compile_args += ["-std=c++11", "-Wall", "-Wextra"]
17 if _DEBUG:
18     extra_compile_args += ["-g3", "-O0", "-DDEBUG=%s" % _DEBUG_LEVEL, "-UNDEBUG"]
19 else:
20     extra_compile_args += ["-DNDEBUG", "-O3"]
21
22 setup(
23     name                = '...',
24     version              = '...',
25     author               = '...',
26     author_email         = '...',
27     maintainer           = '...',
28     maintainer_email     = '...',
29     description          = '...',
30     long_description     = """...
31 """,
32     platforms            = ['Mac OSX', 'POSIX',],
33     classifiers           = [
34         '...',
35     ],
36     license               = 'GNU Lesser General Public License v2 or later (LGPLv2+)',
37     ext_modules=[
38         Extension("MyExtension",
39             sources=[
40                 '...',
41             ],
42             include_dirs=[
43                 '.',
44                 '..',
45                 os.path.join(os.getcwd(), 'include'),
46                 sysconfig.get_paths()['include'],
47             ],
48             library_dirs = [os.getcwd(),], # path to .a or .so file(s)
49             extra_compile_args=extra_compile_args,
50             language='c++11',
51         ),
52     ]
53 )

```

19.5 Getting C/C++ Flags from a CMake Build

If your project can be built under CMake (such as this one can be) then the compiler flags can be obtained by looking at the generated file `cmake-build-debug/CMakeFiles/<PROJECT>.dir/flags.make` (debug) or `cmake-build-release/CMakeFiles/<PROJECT>.dir/flags.make` (release).

For example, for this project the file is `cmake-build-debug/CMakeFiles/PythonExtensionPatterns.dir/flags.make`.

This looks something like this (wrapped for clarity and replaced user with `<USER>`) :

```

1 # CMAKE generated file: DO NOT EDIT!
2 # Generated by "Unix Makefiles" Generator, CMake Version 3.28
3
4 # compile C with \
5 # /Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/bin/
6 ↪ cc
7 # compile CXX with \
8 # /Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/bin/
9 ↪ C++
10 C_DEFINES =
11
12 C_INCLUDES = -I/Library/Frameworks/Python.framework/Versions/3.13/include/python3.13 \
13 -I/Users/<USER>/CLionProjects/PythonExtensionPatterns/src/debugging/XcodeExample/
14 ↪ PythonSubclassList/PythonSubclassList \
15 -I/Users/<USER>/CLionProjects/PythonExtensionPatterns/src/cpy \
16 -I/Users/<USER>/CLionProjects/PythonExtensionPatterns/src/cpy/Containers \
17 -I/Users/<USER>/CLionProjects/PythonExtensionPatterns/src/cpy/Watchers
18
19 C_FLAGSarm64 = -g -std=gnu11 -arch arm64 -isysroot \
20 /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/
21 ↪ MacOSX13.3.sdk \
22 -fcolor-diagnostics -Wall -Wextra -Wpedantic -Werror -Wfatal-errors \
23 -Wno-unused-variable -Wno-unused-parameter -fexceptions \
24 -Wno-c99-extensions -Wno-c++11-extensions -O0 -g3 -ggdb \
25 -Wno-unused-function
26
27 C_FLAGS = -g -std=gnu11 -arch arm64 -isysroot \
28 /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/
29 ↪ MacOSX13.3.sdk \
30 -fcolor-diagnostics -Wall -Wextra -Wpedantic -Werror -Wfatal-errors \
31 -Wno-unused-variable -Wno-unused-parameter -fexceptions \
32 -Wno-c99-extensions -Wno-c++11-extensions -O0 -g3 -ggdb \
33 -Wno-unused-function
34
35 CXX_DEFINES =
36
37 CXX_INCLUDES = -I/Library/Frameworks/Python.framework/Versions/3.13/include/python3.13 \
38 -I/Users/<USER>/CLionProjects/PythonExtensionPatterns/src/debugging/XcodeExample/
39 ↪ PythonSubclassList/PythonSubclassList \
40 -I/Users/<USER>/CLionProjects/PythonExtensionPatterns/src/cpy \
41 -I/Users/<USER>/CLionProjects/PythonExtensionPatterns/src/cpy/Containers \
42 -I/Users/<USER>/CLionProjects/PythonExtensionPatterns/src/cpy/Watchers
43
44 CXX_FLAGSarm64 = -g -std=gnu++17 -arch arm64 -isysroot \
45 /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/
46 ↪ MacOSX13.3.sdk \
47 -fcolor-diagnostics -Wall -Wextra -Wpedantic -Werror -Wfatal-errors \
48 -Wno-unused-variable -Wno-unused-parameter -fexceptions \
49 -Wno-c99-extensions -Wno-c++11-extensions -O0 -g3 -ggdb \

```

(continues on next page)

(continued from previous page)

```
43     -Wno-unused-function
44
45 CXX_FLAGS = -g -std=gnu++17 -arch arm64 -isysroot \
46     /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/
47     ↪MacOSX13.3.sdk \
48     -fcolor-diagnostics -Wall -Wextra -Wpedantic -Werror -Wfatal-errors \
49     -Wno-unused-variable -Wno-unused-parameter -fexceptions \
50     -Wno-c99-extensions -Wno-c++11-extensions -O0 -g3 -ggdb \
    -Wno-unused-function
```

This would be fairly easy to parse, perhaps by `setup.py`.

19.6 Other Resources

- Advice from openssf.org

DEBUGGING

This is very much work in progress. I will add to it/correct it as I develop new techniques.

20.1 Debugging Tools

First create your toolbox, in this one we have:

- Debug version of Python - great for finding out more detail of your Python code as it executes.
- Valgrind - the goto tool for memory leaks. It is a little tricky to get working but should be in every developers toolbox.
- OS memory monitoring - this is a quick and simple way of identifying whether memory leaks are happening or not. An example is given below: *A Simple Memory Monitor*

20.1.1 Build a Debug Version of Python

There are a large combination of debug builds of Python that you can create and each one will give you extra information when you either:

- Invoke Python with a command line option.
 - Example: a Py_DEBUG build invoking Python with `python -X showrefcount`
- Set an environment variable.
 - Example: a Py_DEBUG build invoking Python with `PYTHONMALLOCSTATS=1 python`
- Additional functions that are added to the `sys` module that can give useful information.
 - Example: a Py_DEBUG build an calling `sys.getobjects(...)`.

See here *Building and Using a Debug Version of Python* for instructions on how to do this.

20.1.2 Valgrind

See here *Building Python for Valgrind* for instructions on how to build Valgrind.

See here *Using Valgrind* for instructions on how to use Valgrind.

Here *Finding Where the Leak is With Valgrind* is an example of finding a leak with Valgrind.

20.1.3 A Simple Memory Monitor

A useful technique is to monitor the memory usage of a Python program. Here is a simple process memory monitor using the `psutil` library. See the *Memory Leaks* chapter for a more comprehensive approach, in particular *pymemtrace*.

```

1 import sys
2 import time
3
4 import psutil
5
6 def memMon(pid, freq=1.0):
7     proc = psutil.Process(pid)
8     print(proc.memory_info_ex())
9     prev_mem = None
10    while True:
11        try:
12            mem = proc.memory_info().rss / 1e6
13            if prev_mem is None:
14                print('{:10.3f} [Mb]'.format(mem))
15            else:
16                print('{:10.3f} [Mb] {:+10.3f} [Mb]'.format(mem, mem - prev_mem))
17            prev_mem = mem
18            time.sleep(freq)
19        except KeyboardInterrupt:
20            try:
21                input(' Pausing memMon, <cr> to continue, ^C to end...')
22            except KeyboardInterrupt:
23                print('\n')
24                return
25
26 if __name__ == '__main__':
27     if len(sys.argv) < 2:
28         print('Usage: python pidmon.py <PID>')
29         sys.exit(1)
30     pid = int(sys.argv[1])
31     memMon(pid)
32     sys.exit(0)

```

Lets test it. In one shell fire up Python and find its PID:

```

>>> import os
>>> os.getpid()
13360

```

In a second shell fire up pidmon.py with this PID:

```

$ python3 pidmon.py 13360
pextmem(rss=7364608, vms=2526482432, pfaults=9793536, pageins=24576)
 7.365 [Mb]
 7.365 [Mb]      +0.000 [Mb]
 7.365 [Mb]      +0.000 [Mb]
...

```

Pause pidmon.py with Ctrl-C:

```

^C Pausing memMon, <cr> to continue, ^C to end...

```

Go back to the first shell and create a large string (1Gb):

```
>>> s = ' ' * 1024**3
```

In the second shell continue pidmon.py with <cr> and we see the memory usage:

```
1077.932 [Mb] +1070.567 [Mb]
1077.932 [Mb] +0.000 [Mb]
1077.932 [Mb] +0.000 [Mb]
...
```

Go back to the first shell and delete the string:

```
>>> del s
```

In the second shell we see the memory usage drop:

```
1077.953 [Mb] +0.020 [Mb]
1077.953 [Mb] +0.000 [Mb]
272.679 [Mb] -805.274 [Mb]
4.243 [Mb] -268.435 [Mb]
4.243 [Mb] +0.000 [Mb]
...
```

In the second shell halt pidmon with two Ctrl-C commands:

```
^C Pausing memMon, <cr> to continue, ^C to end...^C
```

So we can observe the total memory usage of another process simply and cheaply. This is often the first test to do when examining processes for memory leaks.

20.2 Building and Using a Debug Version of Python

There is a spectrum of debug builds of Python that you can create. This chapter describes how to create them.

20.2.1 Building a Standard Debug Version of Python

Download check and and unpack the Python source into the directory of your choice:

```
1 $ curl -o Python-3.13.2.tgz https://www.python.org/ftp/python/3.13.2/Python-3.13.2.tgz
2 # Get the Gzipped source tarball md5 from:
3 # https://www.python.org/downloads/release/python-3132/ "6192ce4725d9c9fc0e8a1cd38410b417"
4 $ md5 Python-3.13.2.tgz | grep 6192ce4725d9c9fc0e8a1cd38410b417
5 MD5 (Python-3.13.2.tgz) = 6192ce4725d9c9fc0e8a1cd38410b417
6 # No output would be a md5 mismatch.
7 $ tmp echo $?
8 0
9 # 1 would be a md5 mismatch.
10 $ tar -xzf Python-3.13.2.tgz
11 $ cd Python-3.13.2
```

Then in the source directory create a debug directory for the debug build:

```
$ mkdir debug
$ cd debug
$ ../configure --with-pydebug
$ make
$ make test
```

20.2.2 Specifying Macros

They can be specified at the configure stage, this works:

```
$ ../configure CFLAGS='-DPy_DEBUG -DPy_TRACE_REFS' --with-pydebug
$ make
```

However the python documentation suggests the alternative way of specifying them when invoking make:

```
$ ../configure --with-pydebug
$ make EXTRA_CFLAGS="-DPy_REF_DEBUG"
```

I don't know why one way would be regarded as better than the other.

20.2.3 The Debug Builds

The builds are controlled by the following macros. The third column shows if CPython C extensions have to be rebuilt against that version of Python:

Table 1: Debug Macros

Macro	Description	Rebuild EXT?
Py_DEBUG	A standard debug build. Py_DEBUG sets LLTRACE, Py_REF_DEBUG, Py_TRACE_REFS, and PYMALLOC_DEBUG (if WITH_PYMALLOC is enabled).	Yes
Py_REF_DEBUG	Turn on aggregate reference counting which will be displayed in the interactive interpreter when invoked with <code>-X showrefcount</code> on the command line. If you are not keeping references to objects and the count is increasing there is probably a leak. Also adds <code>sys.gettotalrefcount()</code> to the <code>sys</code> module and this returns the total number of references.	No
Py_TRACE_REFS	Turns on reference tracing. Sets Py_REF_DEBUG.	Yes
WITH_PYMALLOC	Enables Python's small memory allocator. For Valgrind this must be disabled, if using Python's malloc debugger (using PYMALLOC_DEBUG) this must be enabled. See: Python's Memory Allocator	No
PYMALLOC_DEBUG	Enables Python's malloc debugger that annotates memory blocks. Requires WITH_PYMALLOC. See: Python's Memory Allocator	No

Here is the description of other debug macros that are set by one of the macros above:

Macro	Description
LLTRACE	Low level tracing. See <code>Python/ceval.c</code> .

20.2.4 Python's Memory Allocator

A normal build of Python gives CPython a special memory allocator 'PyMalloc'. When enabled this mallocs largish chunks of memory from the OS and then uses this pool for the actual PyObjects. With PyMalloc active Valgrind can not see all allocations and deallocations.

There are two Python builds of interest to help solve memory problems:

- Disable PyMalloc so that Valgrind can analyse the memory usage.
- Enable PyMalloc in debug mode, this creates memory blocks with special bit patterns and adds debugging information on each end of any dynamically allocated memory. This pattern is checked on every alloc/free and if found to be corrupt a diagnostic is printed and the process terminated.

To make a version of Python with its memory allocator suitable for use with Valgrind:

```
$ ../configure --with-pydebug --without-pymalloc
$ make
```

See *Using Valgrind* for using Valgrind.

To make a version of Python with its memory allocator using Python's malloc debugger either:

```
$ ../configure CFLAGS='-DPYALLOC_DEBUG' --with-pydebug
$ make
```

Or:

```
$ ../configure --with-pydebug
$ make EXTRA_CFLAGS="-DPYALLOC_DEBUG"
```

This builds Python with the WITH_PYALLOC and PYALLOC_DEBUG macros defined.

Finding Access after Free With PYALLOC_DEBUG

TODO:

Python built with PYALLOC_DEBUG is the most effective way of detecting access after free. For example if we have this CPython code:

```
1 static PyObject *access_after_free(PyObject *Py_UNUSED(module)) {
2     PyObject *pA = PyLong_FromLong(1024L * 1024L);
3     fprintf(
4         stdout,
5         "%s(): Before Py_DECREF(0x%p) Ref count: %zd\n",
6         __FUNCTION__, (void *)pA, Py_REFCNT(pA)
7     );
8     PyObject_Print(pA, stdout, Py_PRINT_RAW);
9     fprintf(stdout, "\n");
10
11     Py_DECREF(pA);
12
13     fprintf(
14         stdout,
15         "%s(): After Py_DECREF(0x%p) Ref count: %zd\n",
16         __FUNCTION__, (void *)pA, Py_REFCNT(pA)
17     );
18     PyObject_Print(pA, stdout, Py_PRINT_RAW);
```

(continues on next page)

(continued from previous page)

```

19     fprintf(stdout, "\n");
20
21     Py_RETURN_NONE;
22 }

```

And we call this from the interpreter what we get is undefined and may vary from Python version to version. Here is Python 3.9:

```

1  # $ python
2  # Python 3.9.7 (v3.9.7:1016ef3790, Aug 30 2021, 16:39:15)
3  # [Clang 6.0 (clang-600.0.57)] on darwin
4  # Type "help", "copyright", "credits" or "license" for more information.
5  >>> from cPyExtPatt import cPyRefs
6  >>> cPyRefs.access_after_free()
7  access_after_free(): Before Py_DECREF(0x0x7fe55f1e2fb0) Ref count: 1
8  1048576
9  access_after_free(): After Py_DECREF(0x0x7fe55f1e2fb0) Ref count: 140623120051984
10 1048576
11 >>>

```

And Python 3.13:

```

1  # $ python
2  # Python 3.13.1 (v3.13.1:06714517797, Dec 3 2024, 14:00:22) [Clang 15.0.0 (clang-1500.3.
   ↪ 9.4)] on darwin
3  # Type "help", "copyright", "credits" or "license" for more information.
4  >>> from cPyExtPatt import cPyRefs
5  >>> cPyRefs.access_after_free()
6  access_after_free(): Before Py_DECREF(0x0x102465890) Ref count: 1
7  1048576
8  access_after_free(): After Py_DECREF(0x0x102465890) Ref count: 4333131856
9  0
10 >>>

```

And we call this from the (debug) Python 3.13 interpreter we get a diagnostic:

```

1  # (PyExtPatt_3.13.2_Debug) PythonExtensionPatterns git:(develop) $ python
2  # Python 3.13.2 (main, Mar 9 2025, 11:01:02) [Clang 14.0.3 (clang-1403.0.22.14.1)] on
   ↪ darwin
3  # Type "help", "copyright", "credits" or "license" for more information.
4  >>> from cPyExtPatt import cPyRefs
5  >>> cPyRefs.access_after_free()
6  access_after_free(): Before Py_DECREF(0x0x10a984e00) Ref count: 1
7  1048576
8  access_after_free(): After Py_DECREF(0x0x10a984e00) Ref count: -2459565876494606883
9  <refcnt -2459565876494606883 at 0x10a984e00>
10 >>>

```


Getting Statistics on PyMalloc

If the environment variable `PYTHONMALLOCSTATS` exists when running Python built with `WITH_PYMALLOC`+`PYMALLOC_DEBUG` then a (detailed) report of pymalloc activity is output on stderr whenever a new 'arena' is allocated.

```
$ PYTHONMALLOCSTATS=1 python
```

I have no special knowledge about the output you see when running Python this way which looks like this:

```
1 >>> from cPyExtPatt import cPyRefs
2 >>> cPyRefs.leak_new_reference(1000, 10000)
3 loose_new_reference: value=1000 count=10000
4 Small block threshold = 512, in 32 size classes.
5
6 class    size    num pools    blocks in use    avail blocks
7 -----
8     1      32         1           83           427
9     2      48         1          146          194
10    3      64        48         12240           0
11    4      80        82         16690           38
12    5      96        74         12409          171
13    6     112        22          3099           91
14    7     128        10          1156          114
15    8     144        12          1346           10
16    9     160         4           389           19
17   10     176         4           366            2
18   11     192        32          2649           71
19   12     208         3           173           61
20   13     224         2           132           12
21   14     240        13           860           24
22   15     256         5           281           34
23   16     272         6           344           16
24   17     288         7           348           44
25   18     304         8           403           21
26   19     320         4           198            6
27   20     336         4           183            9
28   21     352         5           198           32
29   22     368         3           119           13
30   23     384         3           104           22
31   24     400         3           106           14
32   25     416         3            98           19
33   26     432         8           295            1
34   27     448         2            64            8
35   28     464         3            70           35
36   29     480         2            44           24
37   30     496         2            59            5
38   31     512         2            45           17
39
40 # arenas allocated total      =           6
41 # arenas reclaimed            =           0
42 # arenas highwater mark      =           6
43 # arenas allocated current    =           6
44 6 arenas * 1048576 bytes/arena = 6,291,456
```

(continues on next page)

(continued from previous page)

```

45
46 # bytes in allocated blocks      =          5,927,280
47 # bytes in available blocks      =          224,832
48 0 unused pools * 16384 bytes    =              0
49 # bytes lost to pool headers     =          18,144
50 # bytes lost to quantization     =          22,896
51 # bytes lost to arena alignment  =          98,304
52 Total                           =        6,291,456
53
54 arena map counts
55 # arena map mid nodes           =              1
56 # arena map bot nodes           =              1
57
58 # bytes lost to arena map root   =          262,144
59 # bytes lost to arena map mid    =          262,144
60 # bytes lost to arena map bot    =          131,072
61 Total                           =          655,360
62 loose_new_reference: DONE

```

20.2.5 Identifying the Python Build Configuration from the Runtime

The module `sysconfig` allows you access to the configuration of the Python runtime. At its simplest, and most verbose, this can be used thus:

```

1 $ ./python.exe -m sysconfig
2 Platform: "macosx-10.9-x86_64"
3 Python version: "3.4"
4 Current installation scheme: "posix_prefix"
5
6 Paths:
7     data = "/usr/local"
8     ...
9     stdlib = "/usr/local/lib/python3.4"
10
11 Variables:
12     ABIFLAGS = "dm"
13     AC_APPLE_UNIVERSAL_BUILD = "0"
14     AIX_GENUINE_CPLUSPLUS = "0"
15     AR = "ar"
16     ...
17     py_version = "3.4.3"
18     py_version_nodot = "34"
19     py_version_short = "3.4"

```

Importing `sysconfig` into an interpreter session gives two useful functions are `get_config_var(...)` which gets the setting for a particular macro and `get_config_vars()` which gets a dict of `{macro : value, ...}`. For example:

```

>>> import sysconfig
>>> sysconfig.get_config_var('Py_DEBUG')
1

```

For advanced usage you can parse any `pyconfig.h` into a dict by opening that file and passing it to `sysconfig`.

`parse_config_h(f)` as a file object. `sysconfig.get_config_h_filename()` will give you the configuration file for the runtime (assuming it still exists). So:

```
>>> with open(sysconfig.get_config_h_filename()) as f:
    cfg = sysconfig.parse_config_h(f)
```

20.3 Valgrind

This is about how to build Valgrind, a Valgrind friendly version of Python and finally how to use and interpret Valgrind.

Note

These instructions have been tested on Mac OS X 10.9 (Mavericks). They may or may not work on other OS's

20.3.1 Building Valgrind

This should be fairly straightforward:

```
svn co svn://svn.valgrind.org/valgrind
cd valgrind
./autogen.sh
./configure
make
make install
```

20.3.2 Building Python for Valgrind

Prepare the source by uncommenting `Py_USING_MEMORY_DEBUGGER` in `Objects/obmalloc.c` around line 1082 or so.

Configuring

`configure` takes the following arguments:

Argument	
<code>--enable-framework</code>	Installs it in <code>/Library/Frameworks/Python.framework/Versions/</code>
<code>--with-pydebug</code>	Debug build of Python. See <code>Misc/SpecialBuilds.txt</code>
<code>--without-pymalloc</code>	With Valgrind support <code>Misc/README.valgrind</code>

To make a framework install:

```
./configure --enable-framework --with-pydebug --without-pymalloc --with-valgrind
sudo make frameworkinstall
```

To make a local version cd to the source tree and we will build a Valgrind version of Python in the `valgrind/` directory:

```
mkdir valgrind
cd valgrind
../configure --with-pydebug --without-pymalloc --with-valgrind
make
```

Check debug build

```
$ python3 -X showrefcount
```

```

1 Python 3.4.3 (default, May 26 2015, 19:54:01)
2 [GCC 4.2.1 Compatible Apple LLVM 6.0 (clang-600.0.51)] on darwin
3 Type "help", "copyright", "credits" or "license" for more information.
4 >>> 23
5 23
6 [54793 refs, 0 blocks]
7 >>> import sys
8 [54795 refs, 0 blocks]
9 >>> sys.gettotalrefcount()
10 54817
11 [54795 refs, 0 blocks]
12 >>> import sysconfig
13 >>> sysconfig.get_config_var('Py_DEBUG')
14 1

```

20.3.3 Using Valgrind

In the <Python source>/Misc directory there is a valgrind-python.supp file that suppresses some Valgrind spurious warnings. I find that this needs editing so:

```
cp <Python source>/Misc/valgrind-python.supp ~/valgrind-python.supp
vi ~/valgrind-python.supp
```

Uncomment PyObject_Free and PyObject_Realloc in the valgrind suppression file.

Invoking the Python interpreter with Valgrind:

```
valgrind --tool=memcheck --dsymutil=yes --track-origins=yes --show-leak-kinds=all --
↪ trace-children=yes --suppressions=$HOME/.valgrind-python.supp <Python source>/valgrind/
↪ python.exe -X showrefcount
```

An example of using Valgrind to detect leaks is here: *Finding Where the Leak is With Valgrind*.

20.4 Leaked New References

This shows what happens if create new Python objects and leak them and how we might detect that.

20.4.1 A Leak Example

Here is an example function that deliberately creates leaks with new references. It takes an integer value to be created (and leaked) and a count of the number of times that this should happen. We can then call this from the Python interpreter and observe what happens.

```

1 static PyObject *leak_new_reference(PyObject *pModule,
2                                     PyObject *args, PyObject *kwargs) {
3     PyObject *ret = NULL;
4     int value, count;
5     static char *kwlist[] = {"value", "count", NULL};
6

```

(continues on next page)

(continued from previous page)

```

7     if (!PyArg_ParseTupleAndKeywords(args, kwargs, "ii", kwlist, &value, &count)) {
8         goto except;
9     }
10    fprintf(stdout, "loose_new_reference: value=%d count=%d\n", value, count);
11    for (int i = 0; i < count; ++i) {
12        PyLong_FromLong(value);    /* New reference, leaked. */
13    }
14
15    Py_INCREF(Py_None);
16    ret = Py_None;
17    goto finally;
18 except:
19    Py_XDECREF(ret);
20    ret = NULL;
21 finally:
22    fprintf(stdout, "loose_new_reference: DONE\n");
23    return ret;
24 }

```

And we add this to the cPyRefs module function table as the python function leakNewRefs:

```

static PyMethodDef cPyRefs_methods[] = {
    /* Other functions here
     * ...
     */
    {"leakNewRefs", (PyCFunction)leak_new_reference,
     METH_VARARGS | METH_KEYWORDS, "Leaks new references to longs."},

    {NULL, NULL, 0, NULL} /* Sentinel */
};

```

In Python first we check what the size of a long is then we call the leaky function with the value 1000 (not the values -5 to 255 which are interned) one million times and there should be a leak of one million times the size of a long:

This is with a debug version of Python 3.13:

```

$ python
Python 3.13.2 (main, Mar 9 2025, 13:27:38) [Clang 15.0.0 (clang-1500.0.40.1)] on darwin
Type "help", "copyright", "credits" or "license" for more information.

```

```

>>> import sys
>>> sys.getsizeof(1000)
44
>>> from cPyExtPatt import cPyRefs
>>> cPyRefs.leakNewRefs(1000, 1000000)
loose_new_reference: value=1000 count=1000000
loose_new_reference: DONE
>>>

```

This should generate a leak of 44Mb or thereabouts.

20.4.2 Recognising Leaked New References

Leaked references can lay unnoticed for a long time, especially if they are small. The ways that you might detect that they are happening are:

- Noticing an unanticipated, ever increasing memory usage at the OS level (by using `top` for example).
- If you run a debug build of Python with `Py_REF_DEBUG` defined you might notice a very high level of total reference counts by either invoking Python with `-X showrefcount` or calling `sys.gettotalrefcount()`.
- Other types of debug build can be useful too.
- Examining Valgrind results.

Observing the Memory Usage

In the debug tools section there is a *A Simple Memory Monitor* that can examine the memory of another running process.

Lets test it. In one shell fire up Python and find its PID:

```
>>> import os
>>> os.getpid()
14488
```

In a second shell fire up `pidmon.py` with this PID:

```
$ python3 pidmon.py 14488
pextmem(rss=7659520, vms=2475937792, pfaults=8380416, pageins=2617344)
 7.660 [Mb]
 7.660 [Mb]      +0.000 [Mb]
 7.660 [Mb]      +0.000 [Mb]
 7.660 [Mb]      +0.000 [Mb]
```

Go back to the first shell and import `cPyRefs`:

```
>>> from cPyExtPatt import cPyRefs
>>> cPyRefs.leakNewRefs(1000, 1000000)
loose_new_reference: value=1000 count=1000000
loose_new_reference: DONE
>>>
```

In the second shell `pidmon.py` shows the sudden jump in memory usage:

```
1 $ python3 pidmon.py 14488
2 pextmem(rss=7659520, vms=2475937792, pfaults=8380416, pageins=2617344)
3   7.660 [Mb]
4   7.660 [Mb]      +0.000 [Mb]
5   7.660 [Mb]      +0.000 [Mb]
6   7.660 [Mb]      +0.000 [Mb]
7   ...
8   7.684 [Mb]      +0.000 [Mb]
9   7.684 [Mb]      +0.000 [Mb]
10  56.443 [Mb]     +48.759 [Mb]
11  56.443 [Mb]      +0.000 [Mb]
12  56.443 [Mb]      +0.000 [Mb]
13  56.443 [Mb]      +0.000 [Mb]
```

(continues on next page)

(continued from previous page)

```

14 56.443 [Mb]      +0.000 [Mb]
15 ...

```

Observing the Total Reference Counts

If you have a debug build of Python with `Py_REF_DEBUG` defined you might notice a very high level of total reference counts by either invoking Python with `-X showrefcount` or calling `sys.gettotalrefcount()`.

For example:

```

1 >>> import sys
2 >>> from cPyExtPatt import cPyRefs
3 >>> dir()
4 ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'cPyRefs',
  ↳ 'sys']
5 >>> sys.gettotalrefcount()
6 55019
7 >>> cPyRefs.leakNewRefs(1000, 10000000)
8 loose_new_reference: value=1000 count=10000000
9 loose_new_reference: DONE
10 >>> dir()
11 ['__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'cPyRefs',
  ↳ 'sys']
12 >>> sys.gettotalrefcount()
13 1055019

```

Notice that `cPyRefs.leakNewRefs(1000, 10000000)` does not add anything to the result of `dir()` but adds 1m reference counts somewhere.

And those references are not collectable:

```

>>> import gc
>>> gc.collect()
0
>>> sys.gettotalrefcount()
1055519

```

20.4.3 Finding Where the Leak is With Valgrind

Now that we have noticed that there is a problem, then where, exactly, is the problem?

Lets run our debug version of Python with Valgrind and see if we can spot the leak (assumes `.valgrind-python.supp` is in your `$HOME` directory and `./Python.3.4.3D` is the debug executable):

```

valgrind --tool=memcheck --trace-children=yes --dsymutil=yes --leak-check=full --show-
↳ leak-kinds=all --suppressions=~/.valgrind-python.supp ./Python.3.4.3D

```

Note

You need to use the option `--show-leak-kinds=all` for the next bit to work otherwise you see in the summary that memory has been leaked but not where from.

Then run this code:

```
>>> from cPyExtPatt import cPyRefs
>>> cPyRefs.leakNewRefs(1000, 1000000)
loose_new_reference: value=1000 count=1000000
loose_new_reference: DONE
>>> ^D
```

In the Valgrind output you should see this in the summary at the end:

```
==13042== LEAK SUMMARY:
==13042==      definitely lost: 6,925 bytes in 26 blocks
==13042==      indirectly lost: 532 bytes in 5 blocks
==13042==      possibly lost: 329,194 bytes in 709 blocks
==13042==      still reachable: 44,844,200 bytes in 1,005,419 blocks
```

The “still reachable” value is a clue that something has gone awry.

In the body of the Valgrind output you should find something like this - the important lines are highlighted:

```
1 ==13042== 44,000,000 bytes in 1,000,000 blocks are still reachable in loss record 2,325_
   ↳ of 2,325
2 ==13042==    at 0x47E1: malloc (vg_replace_malloc.c:300)
3 ==13042==    by 0x41927E: _PyMem_RawMalloc (in /Library/Frameworks/Python.framework/
   ↳ Versions/3.4/Python)
4 ==13042==    by 0x418D80: PyObject_Malloc (in /Library/Frameworks/Python.framework/
   ↳ Versions/3.4/Python)
5 ==13042==    by 0x3DFBA8: _PyLong_New (in /Library/Frameworks/Python.framework/Versions/
   ↳ 3.4/Python)
6 ==13042==    by 0x3DFF8D: PyLong_FromLong (in /Library/Frameworks/Python.framework/
   ↳ Versions/3.4/Python)
7 ==13042==    by 0x11BFA40: leak_new_reference (cPyRefs.c:149)
8 ==13042==    by 0x40D81C: PyCFunction_Call (in /Library/Frameworks/Python.framework/
   ↳ Versions/3.4/Python)
9 ==13042==    by 0x555C15: call_function (in /Library/Frameworks/Python.framework/
   ↳ Versions/3.4/Python)
10 ==13042==    by 0x54E02B: PyEval_EvalFrameEx (in /Library/Frameworks/Python.framework/
   ↳ Versions/3.4/Python)
11 ==13042==    by 0x53A7B4: PyEval_EvalCodeEx (in /Library/Frameworks/Python.framework/
   ↳ Versions/3.4/Python)
12 ==13042==    by 0x539414: PyEval_EvalCode (in /Library/Frameworks/Python.framework/
   ↳ Versions/3.4/Python)
13 ==13042==    by 0x5A605E: run_mod (in /Library/Frameworks/Python.framework/Versions/3.4/
   ↳ Python)
14 ==13042==
```

Which is exactly what we are looking for.

20.5 Debugging Tactics

So what is the problem that you are trying to solve?

20.5.1 Access After Free

Problem

You suspect that you are accessing a Python object after it has been free'd. Code such as this:

```
static PyObject *access_after_free(PyObject *pModule) {
    PyObject *pA = PyLong_FromLong(1024L);
    Py_DECREF(pA);
    PyObject_Print(pA, stdout, 0);
    Py_RETURN_NONE;
}
```

pA has been freed before PyObject_Print is called.

Solution

```
Python 3.4.3 (default, Sep 16 2015, 16:56:10)
[GCC 4.2.1 Compatible Apple LLVM 6.0 (clang-600.0.51)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> from cPyExtPatt import cPyRefs
>>> cPyRefs.afterFree()
<refcnt -2604246222170760229 at 0x10a474130>
>>>
```

20.6 Using gcov for C/C++ Code Coverage

This is about how to run your C/C++ code using gcov to gather coverage metrics.

Note

These instructions have been tested on Mac OS X 10.9 (Mavericks). They may or may not work on other OS's

20.6.1 gcov under Mac OS X

Configuring Xcode

This [document](#) tells you how to configure Xcode for gcov (actually llvm-cov).

Running and Analysing Code Coverage

In Xcode select Product->Run then once that has completed note the GCov_build directory in Xcode by selecting Products-><project> in the left hand pane. In the right hand pane under 'Identity and Type' you should see 'Full Path' of something like:

```
/Users/$USER/Library/Developer/Xcode/DerivedData/<project name and hash>/Build/Products/
↳ GCov_Build/<project>
```

In the Terminal navigate to the 'Build' directory:

```
$ cd /Users/$USER/Library/Developer/Xcode/DerivedData/<project name and hash>/Build/
```

Now navigate to the Intermediates/ directory and in there you should find the code coverage data in a path such as this:

```
cd Intermediates/<project>.build/GCov_Build/<project>.build/Objects-normal/x86_64
```

In there the interesting file has a .gcno extension. To convert this into something readable we need to run gcov on it. We can use xcrun to find gcov and this gives an overall code coverage number for each file (and its includes):

```
$ xcrun gcov TimeBDT.gcno
...
File 'TimeBDT.cpp'
Lines executed:87.18% of 78
TimeBDT.cpp:creating 'TimeBDT.cpp.gcov'
...
```

This has now generated a detailed file TimeBDT.cpp.gcov that contains line by line coverage:

```
1  $ ls -l TimeBDT.*
2  -rw-r--r--  1 paulross  staff    14516  6 Oct 18:48 TimeBDT.cpp.gcov
3  -rw-r--r--  1 paulross  staff      248  6 Oct 18:38 TimeBDT.d
4  -rw-r--r--  1 paulross  staff    1120  6 Oct 18:38 TimeBDT.dia
5  -rw-r--r--  1 paulross  staff   32252  6 Oct 18:45 TimeBDT.gcda
6  -rw-r--r--  1 paulross  staff  121444  6 Oct 18:38 TimeBDT.gcno
7  -rw-r--r--  1 paulross  staff    3671  6 Oct 18:48 TimeBDT.h.gcov
8  -rw-r--r--  1 paulross  staff  310496  6 Oct 18:38 TimeBDT.o
9  $ cat TimeBDT.cpp.gcov | less
10 ...
11 -: 208:/* Returns the total number of seconds in this particular year. */
12 -: 209:time_t TimeBDTMapEntry::secsInThisYear() const {
13 60000000: 210:     return _isLeap ? SECONDS_IN_YEAR[1] : SECONDS_IN_YEAR[0];
14 -: 211:}
15 ...
16 -: 289:         } else {
17 #####: 290:             if (pTimeBDT->ttYearStart() > tt) {
18 #####: 291:                 --year;
19 #####: 292:             } else {
20 #####: 293:                 ++year;
21 -: 294:             }
22 -: 295:         }
```

The first snippet shows that line 210 is executed 60000000 times. In the second snippet the ##### shows that lines 290-293 have not executed at all.

20.6.2 Using gcov on CPython Extensions

Whilst it is common to track code coverage in Python test code it gets a bit more tricky with Cpython extensions as Python code coverage tools can not track C/C++ extension code. The solution is to use gcov and run the tests in a C/C++ process by embedding the Python interpreter.

```
1  #include <Python.h>
2
3  int test_cpython_module_foo_functions(PyObject *pModule) {
4      assert(pModule);
5      int fail = 0;
6  try:
7      /* foo.function(...) */
```

(continues on next page)

(continued from previous page)

```

8     pFunc = PyObject_GetAttrString(pModule, "function");
9     if (pFunc && PyCallable_Check(pFunc)) {
10
11         pValue = PyObject_CallObject(pFunc, pArgs);
12     }
13     /*
14      * Test other Foo functions here
15     */
16 except:
17     assert(fail != 0);
18 finally:
19     return fail;
20 }
21
22 int test_cpython_module_foo() {
23     int fail = 0;
24     PyObject *pName = NULL;
25 try:
26     pName = PyUnicode_FromString("foo");
27     if (! pName) {
28         fail = 1;
29         goto except;
30     }
31     pModule = PyImport_Import(pName);
32     if (! pModule) {
33         fail = 2;
34         goto except;
35     }
36     /* foo.function(...) */
37     pFunc = PyObject_GetAttrString(pModule, "function");
38     if (pFunc && PyCallable_Check(pFunc)) {
39
40         pValue = PyObject_CallObject(pFunc, pArgs);
41     }
42     /*
43      * Test other Foo functions here
44     */
45 except:
46     assert(fail != 0);
47 finally:
48     Py_XDECREF(pName);
49     Py_XDECREF(pModule);
50     return fail;
51 }
52
53 void test_cpython_code() {
54     Py_SetProgramName("TestCPythonExtensions"); /* optional, recommended */
55     Py_Initialize();
56     test_cpython_module_foo();
57     /*
58      * Test other modules here
59     */

```

(continues on next page)

(continued from previous page)

```

60     Py_Finalize();
61 }
62
63 int main(int argc, const char * argv[]) {
64     std::cout << "Testing starting" << std::endl;
65     test_cpython_code();
66     /*
67      * Non-CPython code tests here...
68      */
69     std::cout << "Testing DONE" << std::endl;
70     return 0;
71 }

```

20.7 Debugging Python C Extensions in an IDE

`gdb` and `lldb` work well with Python extensions but if you want to step through your C extension in an IDE here is one way to do it.

The basic idea is to compile/link your C extension in your IDE and get `main()` to call a function `int import_call_execute(int argc, const char *argv[])` that embeds the Python interpreter which then imports a Python module, say a unit test, that exercises your C extension code.

This `import_call_execute()` entry point is fairly generic and takes the standard arguments to `main()` so it can be in its own `.h/c` file.

20.7.1 Creating a Python Unit Test to Execute

Suppose you have a Python extension `ScList` that sub-classes a list and counts the number of times `.append(...)` was called making this count available as a `.appends` property. You have a unit test called `test_sclist.py` that looks like this with a single function `test()`:

```

import ScList

def test():
    s = ScList.ScList()
    assert s.appends == 0
    s.append(8)
    assert s.appends == 1

```

20.7.2 Writing a C Function to call any Python Unit Test

We create the `import_call_execute()` function that takes that same arguments as `main()` which can forward its arguments. `import_call_execute()` expects 4 arguments:

- `argc[0]` - Name of the executable.
- `argc[1]` - Path to the directory that the Python module is in.
- `argc[2]` - Name of the Python module to be imported. This could be a unit test module for example.
- `argc[3]` - Name of the Python function in the Python module (no arguments will be supplied, the return value is ignored). This could be a particular unit test.

The `import_call_execute()` function does this:

1. Check the arguments and initialises the Python interpreter

2. Add the path to the `argv[1]` to `sys.paths`.
3. Import `argv[2]`.
4. Find the function `argv[3]` in module `argv[2]` and call it.
5. Clean up.

Code Walk Through

`import_call_execute()` is quite generic and could be implemented in, say, `py_import_call_execute.c`.

Here is a walk through of the implementation code:

Step 1: Check the arguments and initialise the Python interpreter

```

1  /** This imports a Python module and calls a specific function in it.
2   * It's arguments are similar to main():
3   * argc - Number of strings in argv
4   * argv - Expected to be 4 strings:
5   *     - Name of the executable.
6   *     - Path to the directory that the Python module is in.
7   *     - Name of the Python module.
8   *     - Name of the function in the module.
9   *
10  * The Python interpreter will be initialised and the path to the Python module
11  * will be added to sys.paths then the module will be imported.
12  * The function will be called with no arguments and its return value will be
13  * ignored.
14  *
15  * This returns 0 on success, non-zero on failure.
16  */
17 int import_call_execute(int argc, const char *argv[]) {
18     int return_value = 0;
19     PyObject *pModule = NULL;
20     PyObject *pFunc = NULL;
21     PyObject *pResult = NULL;
22
23     if (argc != 4) {
24         fprintf(stderr,
25             "Wrong arguments!"
26             " Usage: %s package_path module function\n", argv[0]);
27         return_value = -1;
28         goto except;
29     }
30     Py_SetProgramName((wchar_t*)argv[0]);
31     Py_Initialize();

```

Step 2: Add the path to the `test_sclist.py` to `sys.paths`. For convenience we have a separate function `add_path_to_sys_module()` to do this.

```

if (add_path_to_sys_module(argv[1])) {
    return_value = -2;
    goto except;
}

```

Here is the implementation of `add_path_to_sys_module()`:

```

1  /** Takes a path and adds it to sys.paths by calling PyRun_SimpleString.
2  * This does rather laborious C string concatenation so that it will work in
3  * a primitive C environment.
4  *
5  * Returns 0 on success, non-zero on failure.
6  */
7  int add_path_to_sys_module(const char *path) {
8      int ret = 0;
9      const char *prefix = "import sys\nsys.path.append(\"";
10     const char *suffix = "\\")\n";
11     char *command = (char*)malloc(strlen(prefix)
12                               + strlen(path)
13                               + strlen(suffix)
14                               + 1);
15
16     if (! command) {
17         return -1;
18     }
19     strcpy(command, prefix);
20     strcat(command, path);
21     strcat(command, suffix);
22     ret = PyRun_SimpleString(command);
23 #ifdef DEBUG
24     printf("Calling PyRun_SimpleString() with:\n");
25     printf("%s", command);
26     printf("PyRun_SimpleString() returned: %d\n", ret);
27     fflush(stdout);
28 #endif
29     free(command);
30     return ret;
31 }

```

Step 3: Import test_sclist.

```

pModule = PyImport_ImportModule(argv[2]);
if (! pModule) {
    fprintf(stderr,
            EXECUTABLE_NAME ": Failed to load module \"%s\"\n", argv[2]);
    return_value = -3;
    goto except;
}

```

Step 4: Find the function test() in test_sclist and call it.

```

1  pFunc = PyObject_GetAttrString(pModule, argv[3]);
2  if (! pFunc) {
3      fprintf(stderr,
4              EXECUTABLE_NAME ": Can not find function \"%s\"\n", argv[3]);
5      return_value = -4;
6      goto except;
7  }
8  if (! PyCallable_Check(pFunc)) {
9      fprintf(stderr,
10             EXECUTABLE_NAME ": Function \"%s\" is not callable\n", argv[3]);

```

(continues on next page)

(continued from previous page)

```

11     return_value = -5;
12     goto except;
13 }
14 pResult = PyObject_CallObject(pFunc, NULL);
15 if (! pResult) {
16     fprintf(stderr, EXECUTABLE_NAME ": Function call failed\n");
17     return_value = -6;
18     goto except;
19 }
20 #ifdef DEBUG
21     printf(EXECUTABLE_NAME ": PyObject_CallObject() succeeded\n");
22 #endif
23     assert(! PyErr_Occurred());
24     goto finally;

```

Step 5: Clean up.

```

1  except:
2      assert(PyErr_Occurred());
3      PyErr_Print();
4  finally:
5      Py_XDECREF(pFunc);
6      Py_XDECREF(pModule);
7      Py_XDECREF(pResult);
8      Py_Finalize();
9      return return_value;
10 }

```

And then we need `main()` to call this, thus:

```

#include "py_import_call_execute.h"

int main(int argc, const char *argv[]) {
    return import_call_execute(argc, argv);
}

```

Complete Code

The complete code for `py_import_call_execute.c` is here:

```

1  #include "py_import_call_execute.h"
2
3  #include <Python.h>
4
5  #ifdef __cplusplus
6  extern "C" {
7  #endif
8
9  /** Takes a path and adds it to sys.paths by calling PyRun_SimpleString.
10   * This does rather laborious C string concatenation so that it will work in
11   * a primitive C environment.
12   *

```

(continues on next page)

(continued from previous page)

```

13  * Returns 0 on success, non-zero on failure.
14  */
15  int add_path_to_sys_module(const char *path) {
16      int ret = 0;
17      const char *prefix = "import sys\nsys.path.append(\"";
18      const char *suffix = "\")\n";
19      char *command = (char*)malloc(strlen(prefix)
20                               + strlen(path)
21                               + strlen(suffix)
22                               + 1);
23      if (! command) {
24          return -1;
25      }
26      strcpy(command, prefix);
27      strcat(command, path);
28      strcat(command, suffix);
29      ret = PyRun_SimpleString(command);
30  #ifdef DEBUG
31      printf("Calling PyRun_SimpleString() with:\n");
32      printf("%s", command);
33      printf("PyRun_SimpleString() returned: %d\n", ret);
34      fflush(stdout);
35  #endif
36      free(command);
37      return ret;
38  }
39
40  /** This imports a Python module and calls a specific function in it.
41  * It's arguments are similar to main():
42  * argc - Number of strings in argv
43  * argv - Expected to be 4 strings:
44  *     - Name of the executable.
45  *     - Path to the directory that the Python module is in.
46  *     - Name of the Python module.
47  *     - Name of the function in the module.
48  * 
49  * The Python interpreter will be initialised and the path to the Python module
50  * will be added to sys.paths then the module will be imported.
51  * The function will be called with no arguments and its return value will be
52  * ignored.
53  * 
54  * This returns 0 on success, non-zero on failure.
55  */
56  int import_call_execute(int argc, const char *argv[]) {
57      int return_value = 0;
58      PyObject *pModule = NULL;
59      PyObject *pFunc = NULL;
60      PyObject *pResult = NULL;
61
62      if (argc != 4) {
63          fprintf(stderr,
64              "Wrong arguments!"

```

(continues on next page)

(continued from previous page)

```

65         " Usage: %s package_path module function\n", argv[0]);
66     return_value = -1;
67     goto except;
68 }
69 Py_SetProgramName((wchar_t*)argv[0]);
70 Py_Initialize();
71 if (add_path_to_sys_module(argv[1])) {
72     return_value = -2;
73     goto except;
74 }
75 pModule = PyImport_ImportModule(argv[2]);
76 if (! pModule) {
77     fprintf(stderr,
78         "%s: Failed to load module \"%s\"\n", argv[0], argv[2]);
79     return_value = -3;
80     goto except;
81 }
82 pFunc = PyObject_GetAttrString(pModule, argv[3]);
83 if (! pFunc) {
84     fprintf(stderr,
85         "%s: Can not find function \"%s\"\n", argv[0], argv[3]);
86     return_value = -4;
87     goto except;
88 }
89 if (! PyCallable_Check(pFunc)) {
90     fprintf(stderr,
91         "%s: Function \"%s\" is not callable\n", argv[0], argv[3]);
92     return_value = -5;
93     goto except;
94 }
95 pResult = PyObject_CallObject(pFunc, NULL);
96 if (! pResult) {
97     fprintf(stderr, "%s: Function call failed\n", argv[0]);
98     return_value = -6;
99     goto except;
100 }
101 #ifdef DEBUG
102     printf("%s: PyObject_CallObject() succeeded\n", argv[0]);
103 #endif
104     assert(! PyErr_Occurred());
105     goto finally;
106 except:
107     assert(PyErr_Occurred());
108     PyErr_Print();
109 finally:
110     Py_XDECREF(pFunc);
111     Py_XDECREF(pModule);
112     Py_XDECREF(pResult);
113     Py_Finalize();
114     return return_value;
115 }
116

```

(continues on next page)

(continued from previous page)

```

117 #ifdef __cplusplus
118 // extern "C" {
119 #endif

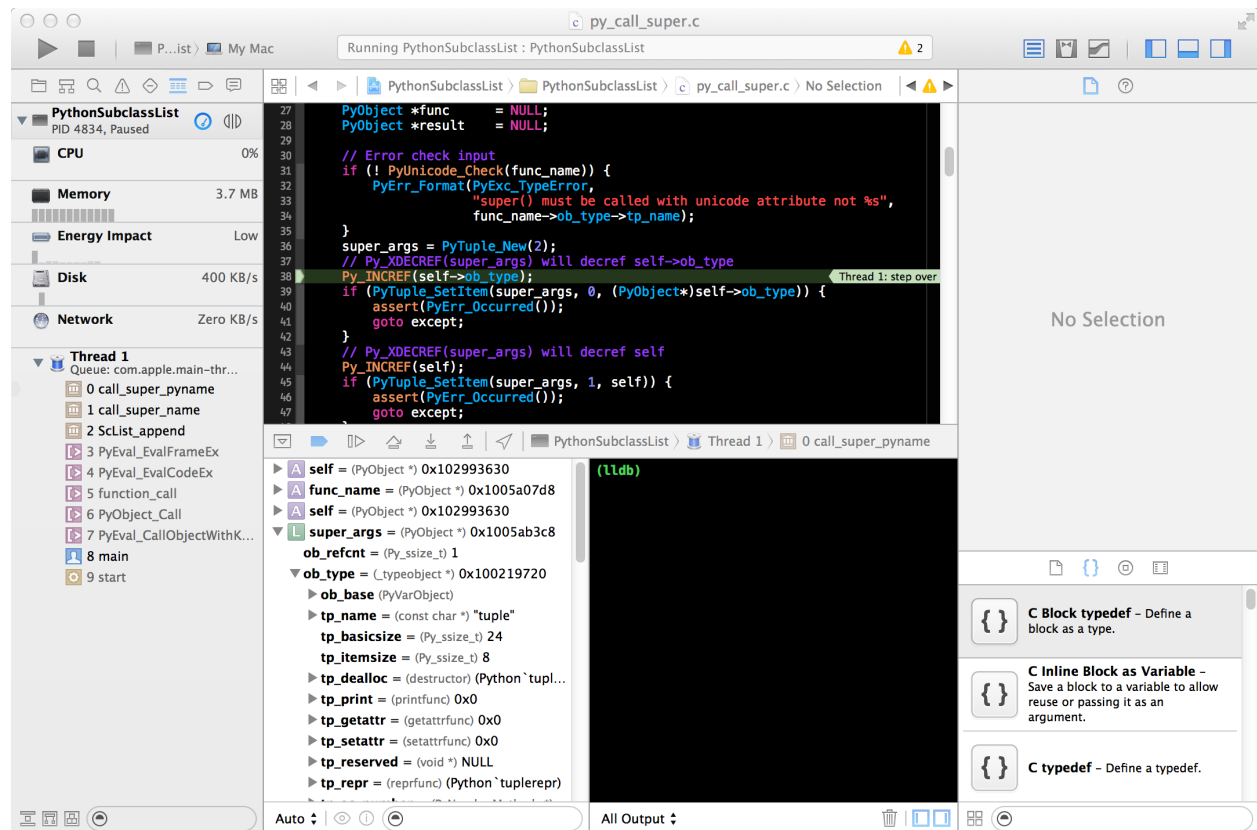
```

20.7.3 Debugging Python C Extensions in Xcode

Build this code in Xcode, set break points in your extension, and run with the command line arguments:

```
<path to test_sclist.py> test_sclist test
```

And you should get something like this:



The full code for this is in `src/debugging/XcodeExample/PythonSubclassList/`.

20.7.4 Using a Debug Version of Python C with Xcode

To get Xcode to use a debug version of Python first build Python from source assumed here to be `<SOURCE_DIR>` with, as a minimum, `--with-pydebug`. This example is using Python 3.6:

```

cd <SOURCE_DIR>
mkdir debug-framework
cd debug-framework/
./configure --with-pydebug --without-pymalloc --with-valgrind --enable-framework
make

```

Then in Xcode select the project and “Add files to ...” and add:

- `<SOURCE_DIR>/debug-framework/Python.framework/Versions/3.6/Python`

- `<SOURCE_DIR>/debug-framework/libpython3.6d.a`

In “Build Settings”:

- add `/Library/Frameworks/Python.framework/Versions/3.6/include/python3.6m/` to “Header Search Paths”. Alternatively add both `<SOURCE_DIR>/Include` and `<SOURCE_DIR>/debug-framework` to “Header Search Paths”, the latter is needed for `pyconfig.h`.
- add `<SOURCE_DIR>/debug-framework` to “Library Search Paths”.

Now you should be able to step into the CPython code.

20.7.5 Debugging Python C Extensions in Eclipse

The same `main()` can be used.

TODO: The intern can do this.

20.8 Instrumenting the Python Process for Your Structures

Some debugging problems can be solved by instrumenting your C extensions for the duration of the Python process and reporting what happened when the process terminates. The data could be: the number of times classes were instantiated, functions called, memory allocations/deallocations or anything else that you wish.

To take a simple case, suppose we have a class that implements a up/down counter and we want to count how often each `inc()` and `dec()` function is called during the entirety of the Python process. We will create a C extension that has a class that has a single member (an integer) and two functions that increment or decrement that number. If it was in Python it would look like this:

```
class Counter:
    def __init__(self, count=0):
        self.count = count

    def inc(self):
        self.count += 1

    def dec(self):
        self.count -= 1
```

What we would like to do is to count how many times `inc()` and `dec()` are called on *all* instances of these objects and summarise them when the Python process exits¹.

There is an interpreter hook `Py_AtExit()` that allows you to register C functions that will be executed as the Python interpreter exits. This allows you to dump information that you have gathered about your code execution.

20.8.1 An Implementation of a Counter

First here is the module `pyatexit` with the class `pyatexit.Counter` with no instrumentation (it is equivalent to the Python code above). We will add the instrumentation later:

```
1 #include <Python.h>
2 #include "structmember.h"
3
4 #include <stdio.h>
```

(continues on next page)

¹ The `atexit` module in Python can be used to similar effect however registered functions are called at a different stage of interpreted teardown than `Py_AtExit`.

(continued from previous page)

```

5
6 typedef struct {
7     PyObject_HEAD int number;
8 } Py_Counter;
9
10 static void Py_Counter_dealloc(Py_Counter* self) {
11     Py_TYPE(self)->tp_free((PyObject*)self);
12 }
13
14 static PyObject* Py_Counter_new(PyTypeObject* type, PyObject* args,
15                                 PyObject* kwds) {
16     Py_Counter* self;
17     self = (Py_Counter*)type->tp_alloc(type, 0);
18     if (self != NULL) {
19         self->number = 0;
20     }
21     return (PyObject*)self;
22 }
23
24 static int Py_Counter_init(Py_Counter* self, PyObject* args, PyObject* kwds) {
25     static char* kwlist[] = { "number", NULL };
26     if (!PyArg_ParseTupleAndKeywords(args, kwds, "|i", kwlist, &self->number)) {
27         return -1;
28     }
29     return 0;
30 }
31
32 static PyMemberDef Py_Counter_members[] = {
33     { "count", T_INT, offsetof(Py_Counter, number), 0, "count value" },
34     { NULL, 0, 0, 0, NULL } /* Sentinel */
35 };
36
37 static PyObject* Py_Counter_inc(Py_Counter* self) {
38     self->number++;
39     Py_RETURN_NONE;
40 }
41
42 static PyObject* Py_Counter_dec(Py_Counter* self) {
43     self->number--;
44     Py_RETURN_NONE;
45 }
46
47 static PyMethodDef Py_Counter_methods[] = {
48     { "inc", (PyCFunction)Py_Counter_inc, METH_NOARGS, "Increments the counter" },
49     { "dec", (PyCFunction)Py_Counter_dec, METH_NOARGS, "Decrements the counter" },
50     { NULL, NULL, 0, NULL } /* Sentinel */
51 };
52
53 static PyTypeObject Py_CounterType = {
54     PyVarObject_HEAD_INIT(NULL, 0) "pyatexit.Counter", /* tp_name */
55     sizeof(Py_Counter), /* tp_basicsize */
56     0, /* tp_itemsize */

```

(continues on next page)

(continued from previous page)

```

57     (destructor)Py_Counter_dealloc, /* tp_dealloc */
58     0, /* tp_print */
59     0, /* tp_getattr */
60     0, /* tp_setattr */
61     0, /* tp_reserved */
62     0, /* tp_repr */
63     0, /* tp_as_number */
64     0, /* tp_as_sequence */
65     0, /* tp_as_mapping */
66     0, /* tp_hash */
67     0, /* tp_call */
68     0, /* tp_str */
69     0, /* tp_getattro */
70     0, /* tp_setattro */
71     0, /* tp_as_buffer */
72     Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE, /* tp_flags */
73     "Py_Counter objects", /* tp_doc */
74     0, /* tp_traverse */
75     0, /* tp_clear */
76     0, /* tp_richcompare */
77     0, /* tp_weaklistoffset */
78     0, /* tp_iter */
79     0, /* tp_iternext */
80     Py_Counter_methods, /* tp_methods */
81     Py_Counter_members, /* tp_members */
82     0, /* tp_getset */
83     0, /* tp_base */
84     0, /* tp_dict */
85     0, /* tp_descr_get */
86     0, /* tp_descr_set */
87     0, /* tp_dictoffset */
88     (initproc)Py_Counter_init, /* tp_init */
89     0, /* tp_alloc */
90     Py_Counter_new, /* tp_new */
91     0, /* tp_free */
92 };
93
94 static PyModuleDef pyexitmodule = {
95     PyModuleDef_HEAD_INIT, "pyatexit",
96     "Extension that demonstrates the use of Py_AtExit().",
97     -1, NULL, NULL, NULL, NULL,
98     NULL
99 };
100
101 PyMODINIT_FUNC PyInit_pyatexit(void) {
102     PyObject* m;
103
104     if (PyType_Ready(&Py_CounterType) < 0) {
105         return NULL;
106     }
107     m = PyModule_Create(&pyexitmodule);
108     if (m == NULL) {

```

(continues on next page)

(continued from previous page)

```

109     return NULL;
110 }
111 Py_INCREF(&Py_CounterType);
112 PyModule_AddObject(m, "Counter", (PyObject*)&Py_CounterType);
113 return m;
114 }

```

If this was a file `Py_AtExitDemo.c` then a Python `setup.py` file might look like this:

```

from distutils.core import setup, Extension
setup(
    ext_modules=[
        Extension("pyatexit", sources=['Py_AtExitDemo.c']),
    ]
)

```

Building this with `python3 setup.py build_ext --inplace` we can check everything works as expected:

```

1 >>> import pyatexit
2 >>> c = pyatexit.Counter(8)
3 >>> c.inc()
4 >>> c.inc()
5 >>> c.dec()
6 >>> c.count
7 9
8 >>> d = pyatexit.Counter()
9 >>> d.dec()
10 >>> d.dec()
11 >>> d.count
12 -2
13 >>> ^D

```

20.8.2 Instrumenting the Counter

To add the instrumentation we will declare a macro `COUNT_ALL_DEC_INC` to control whether the compilation includes instrumentation.

```
#define COUNT_ALL_DEC_INC
```

In the global area of the file declare some global counters and a function to write them out on exit. This must be a void function taking no arguments:

```

1 #ifndef COUNT_ALL_DEC_INC
2 /* Counters for operations and a function to dump them at Python process end. */
3 static size_t count_inc = 0;
4 static size_t count_dec = 0;
5
6 static void dump_inc_dec_count(void) {
7     fprintf(stdout, "==== dump_inc_dec_count() ====\\n");
8     fprintf(stdout, "Increments: %" PY_FORMAT_SIZE_T "d\\n", count_inc);
9     fprintf(stdout, "Decrements: %" PY_FORMAT_SIZE_T "d\\n", count_dec);
10    fprintf(stdout, "== dump_inc_dec_count() END ==\\n");

```

(continues on next page)

(continued from previous page)

```

11 }
12 #endif

```

In the `Py_Counter_new` function we add some code to register this function. This must be only done once so we use the static `has_registered_exit_function` to guard this:

```

1 static PyObject* Py_Counter_new(PyTypeObject* type, PyObject* args,
2                                 PyObject* kwds) {
3     Py_Counter* self;
4     #ifdef COUNT_ALL_DEC_INC
5         static int has_registered_exit_function = 0;
6         if (!has_registered_exit_function) {
7             if (Py_AtExit(dump_inc_dec_count)) {
8                 return NULL;
9             }
10            has_registered_exit_function = 1;
11        }
12    #endif
13    self = (Py_Counter*)type->tp_alloc(type, 0);
14    if (self != NULL) {
15        self->number = 0;
16    }
17    return (PyObject*)self;
18 }

```

Note

`Py_AtExit` can take, at most, 32 functions. If the function can not be registered then `Py_AtExit` will return -1.

Warning

Since Python's internal finalization will have completed before the cleanup function, no Python APIs should be called by any registered function.

Now we modify the `inc()` and `dec()` functions thus:

```

1 static PyObject* Py_Counter_inc(Py_Counter* self) {
2     self->number++;
3     #ifdef COUNT_ALL_DEC_INC
4         count_inc++;
5     #endif
6     Py_RETURN_NONE;
7 }
8
9 static PyObject* Py_Counter_dec(Py_Counter* self) {
10    self->number--;
11    #ifdef COUNT_ALL_DEC_INC
12        count_dec++;
13    #endif

```

(continues on next page)

(continued from previous page)

```
14     Py_RETURN_NONE;  
15 }
```

Now when we build this extension and run it we see the following:

```
1  >>> import pyatexit  
2  >>> c = pyatexit.Counter(8)  
3  >>> c.inc()  
4  >>> c.inc()  
5  >>> c.dec()  
6  >>> c.count  
7  9  
8  >>> d = pyatexit.Counter()  
9  >>> d.dec()  
10 >>> d.dec()  
11 >>> d.count  
12 -2  
13 >>> ^D  
14 ==== dump_inc_dec_count() ====  
15 Increments: 2  
16 Decrements: 3  
17 == dump_inc_dec_count() END ==
```

Todo

Review the Debugging chapter as some of it might be obsolete.

MEMORY LEAKS

This chapter describes some techniques for detecting and fixing memory leaks. Much of this is taken from the [pymemtrace project](#)

21.1 Introduction

This describes tools and techniques that can identify memory leaks in long running Python programs.

21.1.1 Is it a Leak?

Rising memory is not necessarily a leak, it can be internal data structures that grow naturally. A common strategy for hash tables, arrays and the like is that when they reach capacity they reallocate themselves into twice the original memory and this can look, superficially like a memory leak.

Python data structures are not particularly efficient, an `int` is typically 24 bytes, a `datetime` 48 bytes and so on.

A further source of ‘leaks’, or code that can mask memory leaks, are caches, in-memory databases and so on.

21.1.2 Sources of Leaks

Here is a non-exhaustive list in rough order of popularity:

- **Classic C/C++ leaks:**
 - `malloc` without corresponding `free`.
 - `new` without corresponding `delete`.
 - Static data structures that use ever increasing heap storage.
- Reference counting errors in C/C++ extensions used by Python.
- Bugs in C/C++ wrappers such as Cython or pybind11.
- Bugs in Python.

21.1.3 A Bit About (C)Python Memory Management

Python objects are allocated on the heap with their parent references on the stack. When the stack unwinds the reference goes out of scope and, without any other action, the heap allocated would be leaked. Python uses a couple of techniques to prevent this; reference counting and Garbage Collection. Bear in mind that Python is quite old and the Garbage Collector reflects that¹.

¹ This chapter discusses Python's memory management system however the design of that may change over the range of Python versions that this project covers. Therefore some descriptions may be inaccurate or obsolete depending on the Python version under consideration.

Reference Counts

The primary technique that Python uses is reference counting, when an object is created it is given a reference count of 1. When another object refers to it the reference count increases by one. When an object goes out of scope that refers to the object the reference count is decremented by one. When the reference count becomes zero the object is de-allocated and the memory can be re-used immediately.

In C/C++ extensions you have to manage these reference counts manually and correctly. An important point about reference counts in C/C++ extensions is that:

- If they are too low the object might get de-allocated prematurely whilst there are still valid references. Then those references might try to access the deleted object and that may, or may not, result in a segfault.
- If the reference count is incremented unnecessarily the object will never get de-allocated and there will be a memory leak.

The latter is often regarded as the lesser of the two problems and the temptation is to err on the side of increasing reference counts for ‘safety’. This swaps an easy to solve problem (segfault) for a harder to solve one (memory leak).

You can find the reference count of any object by calling `sys.getrefcount()` with the Python object as the argument. The count is one higher than you might expect as it includes the (temporary) reference to the `sys.getrefcount()`.

Reference counting is always switched on in Python.

Note

Some objects are *interned*, that is their reference count never goes to zero so that they are, in effect, permanent. This is done for performance and includes most builtins and the integers -5 to 255.

For example:

```
>>> sys.getrefcount(None)
14254
>>> sys.getrefcount(0)
2777
>>> sys.getrefcount(400)
2
```

Reference counts have one major problem, cyclic references. Consider this:

```
class A: pass
a = A()
b = A()
a.next = b
b.next = a
```

a references b and b references a so you can not delete either without deleting the other. To get round this problem Python uses a simple garbage collector.

Garbage Collection

The only job of Python’s garbage collector (GC) is to discover unreachable objects that have cyclic references. The Python garbage collector is fairly simple and rather old. You can use the `gc` module to inspect and control the garbage collector. The garbage collector can be switched off and this is often done in high performance systems.

In particular:

- The GC will not reclaim objects that are not tracked. This includes many objects created in C/C++ extensions. See `gc.is_tracked()` to see if an object is being tracked by the GC.

- The GC only looks at unreachable objects.
- The GC only deals with cyclic references.
- The GC is easily defeated, even inadvertently, for example if objects implement `__del__`.
- A real restriction on the GC is due to C/C++ extensions. An unreachable C/C++ object from Python code with a zero reference count can not be deleted as there is no way of knowing if some C/C++ code might have a reference to it. In Java this is easier as the VM controls the whole estate and can safely delete unreachable objects.

The Big Picture

Here is a visualisation of memory allocators from top to bottom (from the Python source `Objects/obmalloc.c`):

```

+3 | [ int ] [ dict ] [ list ] ... [ string ]      Python core      |
    | <----- Object-specific memory -----> | <-- Non-object memory --> |
    |                                         |                   |
    | [ Python's object allocator ]           |                   |
+2 | ##### Object memory ##### | <----- Internal buffers -----> |
    |                                         |                   |
    | [ Python's raw memory allocator (PyMem_ API) ] |                   |
+1 | <----- Python memory (under PyMem manager's control) -----> |
    |                                         |                   |
    | [ Underlying general-purpose allocator (ex: C library malloc) ] |
0  | <----- Virtual memory allocated for the python process -----> |

=====

    | [ OS-specific Virtual Memory Manager (VMM) ] |
-1 | <--- Kernel dynamic storage allocation & management (page-based) ---> |
    |                                         |
    | [ ] [ ] |
-2 | <-- Physical memory: ROM/RAM --> | | <-- Secondary storage (swap) --> |

```

Layer +2 is significant, it is the CPython's Object Allocator (`pymalloc`).

CPython's Object Allocator (`pymalloc`)

Standard CPython uses an in-memory pool for small items (≤ 512 bytes) to reduce the cost of going to the OS for memory allocations. One consequence of this is that small memory leaks will be hidden when observing the *overall* memory usage of a process. Another consequence is that tools such as Valgrind are rendered nearly useless for detecting memory leaks when the `pymalloc` is in use. `pymalloc` can be disabled with a special build of Python. Requests > 512 bytes are allocated without `pymalloc` and routed to the platform's allocator (usually the C `malloc()` function).

A summary of `pymalloc`:

- `pymalloc` consists of a set of *Arena*'s.
- An *Arena* is a 256kB (262,144 bytes) chunk of memory divided up into *Pool*'s.
- A *Pool* is a chunk of memory the size of a OS page, usually 4096 bytes.
- A *Pool* is subdivided into *Block*'s which all have the same size for that Pool.
- A *Block* is memory sized between 8 and 512 (modulo 8).

To understand this better try:

```
import sys
sys._debugmallocstats()
```

An you will get something like:

Small block threshold = 512, in 64 size classes.

class	size	num pools	blocks in use	avail blocks
0	8	2	551	461
1	16	1	82	171
2	24	2	186	150
...				
62	504	10	73	7
63	512	19	132	1

# arenas allocated total	=	95
# arenas reclaimed	=	46
# arenas highwater mark	=	49
# arenas allocated current	=	49
49 arenas * 262144 bytes/arena	=	12,845,056
# bytes in allocated blocks	=	12,129,080
# bytes in available blocks	=	174,784
59 unused pools * 4096 bytes	=	241,664
# bytes lost to pool headers	=	147,696
# bytes lost to quantization	=	151,832
# bytes lost to arena alignment	=	0
Total	=	12,845,056

14 free PyCFunctionObjects * 48 bytes each	=	672
78 free PyDictObjects * 48 bytes each	=	3,744
7 free PyFloatObjects * 24 bytes each	=	168
3 free PyFrameObjects * 384 bytes each	=	1,152
80 free PyListObjects * 40 bytes each	=	3,200
17 free PyMethodObjects * 40 bytes each	=	680
25 free 1-sized PyTupleObjects * 32 bytes each	=	800
1446 free 2-sized PyTupleObjects * 40 bytes each	=	57,840
...		
1 free 19-sized PyTupleObjects * 176 bytes each	=	176

There are five sections:

- The first line states the small object limit (512) and how this is divided: $512 / 8 = 64$ 'class's. Each of these 'class's handle memory allocations of a specific size.
- The second section shows how many pools and blocks are in use for each 'class' (specific size of memory allocation).
- The third section is about *Arena*'s, there are currently 49 at 262,144 bytes each.
- The fourth section summarises the total memory usage, in particular the amount of memory consumed by the pymalloc administration.
- The fifth section is a summary of the memory consumed by particular Python type. NOTE: This is not an exclusive list, many types such as `int`, `set` are absent.

In summary:

- 49 Arenas of 256kB (262,144 bytes) is 12,845,056 in total.
 - Each Arena is divided into 64 pools of 4096 bytes each, thus $49 \times 64 = 3136$ pools (the sum of 'num pools') above.
 - Each Pool of 4096 bytes is allocated a fixed size between 8 and 512 bytes and divided by that into Blocks. So there are between 512×8 byte blocks and 8×512 byte blocks in a Pool.

See *pymemtrace Debug Malloc Stats* for examples of `pymemtrace.debug_malloc_stats` that can make this information much more useful.

Memory De-allocation

- If the object is >512 bytes it is not under control of `pymalloc` and the memory is returned to the OS immediately.
- A *Pool* is free'd when all the blocks are empty.
- An *Arena* is free'd when all the *Pool*'s are empty.
- There is no attempt to reorganise `pymalloc` periodically reduce the memory use such as a copying garbage collector might do.

This means that pools and arenas can exist for a very long time.

21.2 Tools for Detecting Memory Leaks

Tools for analysing memory can be characterised by:

Table 1: Tool Characteristics

Characteristic	Description
Availability	Does it come with the platform? Is it within Python or the standard library? Does it need third party library installation or requires a special Python build of some sort?
Memory Granularity	How detailed is the memory measurement? Somewhere between every <code>malloc</code> or the overall memory usage as seen by the OS.
Execution Granularity	How detailed is the execution measurement? Per line, per function, for Python or C code?
Memory Cost	What is the extra memory consumption is introduced by using this tool?
Execution Cost	What is the extra runtime introduced by using this tool?
Developer Cost	How hard it is to use the tool?

Each tool makes trade offs between each of these characteristics.

21.2.1 Platform Tools

These tools come ready with the platform. They give a good overall picture of the memory usage.

Characteristic	Description
Availability	Always.
Memory Granularity	Usually the total memory usage by the process.
Execution Granularity	Generally periodic at low frequency, typically of the order of seconds.
Memory Cost	Usually none.
Execution Cost	Usually none.
Developer Cost	Easy.

Here are some tools available on different platforms:

Windows

A weakness of Windows, especially in corporate environments, is that the OS is usually severely locked down. At best this usually vastly extends the time it takes to find a leak, at worst this hopelessly limits the tools that can be installed or run on the platform. In this case some leaks can never be found and fixed.

Windows Task Manager

This is the basic tool for reviewing process memory. The columns to monitor are “Working Set (Memory)” which broadly corresponds to the Unix [Resident Set Size \(RSS\)](#). The Sysinternals `procexp` (see below) is a more sophisticated version.

`perfmom.exe`

This is a Microsoft tool for logging performance and plotting it in real time. It is quite capable but a little fiddly to set up. The third party Python library `psutil` is a useful alternative and the `pymemtrace` `Process` also provides memory plotting of arbitrary processes: `pymemtrace Process` using `gnuplot`.

Sysinternals Suite

The outstanding [Windows Sysinternals tools](#) are a wonderful collection of tools and are essential for debugging any Windows application.

Linux

TODO: Finish this.

`/proc/<PID>`

The `/proc/<PID>` filesystem is full of good stuff.

Valgrind

[Valgrind](#) is essential on any Linux development platform. There is a tutorial [here](#) for building and using Python with Valgrind.

eBPF

The next big thing after DTrace is [eBPF](#). Truly awesome.

Mac OS X

Tools such as `vmmap`, `heap`, `leaks`, `malloc_history`, `vm_stat` can all help. See the man pages for further information.

Some useful information for memory tools from [Apple](#)

DTrace

Mac OS X is DTrace aware, this needs a special build of Python, here is an [introduction](#) that takes you through building and using a DTrace aware version of Python on Mac OS X.

Some examples of using DTrace with [pymemtrace](#).

21.2.2 Python Tools

Modules from the Standard Library

Characteristic	Description
Availability	Always.
Memory Granularity	Various.
Execution Granularity	Various.
Memory Cost	Usually none.
Execution Cost	Usually none.
Developer Cost	Straightforward.

`sys`

The `sys` has a number of useful functions, mostly CPython specific.

Table 2: sys Tools

Tool	Description	Notes
<code>getallocatedblocks()</code>	Returns the number of allocated blocks , regardless of size.	This has no information about the size of any block. CPython only. Implemented in <code>Objects/obmalloc.c</code> as <code>_Py_GetAllocatedBlocks</code> . As implemented in Python 3.9 this returns the total reference count of every <i>pool</i> in every <i>arena</i> .
<code>getrefcount(object)</code>	Returns the reference count of an object.	This is increased by one for the duration of the call.
<code>getsizeof(object)</code>	Returns the size of an object in bytes.	Builtin objects will return correct results. Others are implementation specific. User defined objects can implement <code>__sizeof__</code> which will be called if available.
<code>_debugmallocstats(object)</code>	Prints the state of the Python Memory Allocator <code>pymalloc</code> to <code>stderr</code> . <code>pymemtrace</code> 's pymemtrace Debug Malloc Stats is a very useful wrapper around <code>sys._debugmallocstats()</code> which can report changes to Python's small object allocator.	See pymemtrace Debug Malloc Stats for a <code>pymemtrace</code> wrapper that makes this much more useful.

gc

The `gc` controls the Python garbage collector. Turning the garbage collector off with `gc.disable()` is worth trying to see what effect, if any, it has.

tracemalloc

`tracemalloc` is a useful module that can trace memory blocks allocate by Python. It is invasive and using it consumes a significant amount of memory itself. See [pymemtrace trace_malloc](#) for a `pymemtrace` wrapper that makes this much more useful by report changes to Python's memory allocator..

Third Party Modules

psutil

`psutil` is an excellent, third party, package that can report high level information on a process. [psutil on PyPi](#)

Here are some notes on the [cost of computing](#) the Resident Set Size (RSS).

objgraph

`objgraph` is a wrapper around the Python garbage collector that can take a snapshot of the Python objects in scope. This is quite invasive and expensive but can be very useful in specific cases. It does some excellent visualisations using `graphviz`, `xdot` etc.

Guppy 3

`guppy_3` (source: `guppy_3_source`) is a project that gives a highly detailed picture of the current state of the Python heap.

memory_profiler

Sadly `memory_profiler` (source: `memory_profiler_src`) this excellent project is no longer actively maintained (volunteers welcome, contact f@bianp.net). It gives an annotated version of your source code line by line with the memory usage. It is pure Python and relies just on `psutil`.

21.2.3 Debugging Tools

Debugging Python (and C/C++ extensions) with GDB:

- GDB support for Python: <https://devguide.python.org/gdb/>
- Python debugging with GDB: <https://wiki.python.org/moin/DebuggingWithGdb> and <https://pythondev.readthedocs.io/gdb.html>
- Python debugging tools: https://pythondev.readthedocs.io/debug_tools.html

21.2.4 Building a Debug Version of Python

This is an essential technique however it is limited, due to speed, to a development environment rather than in production.

Building a debug version of Python in a variety of forms: https://pythonextensionpatterns.readthedocs.io/en/latest/debugging/debug_python.html#debug-version-of-python-label

Building a DTrace aware version of Python: <https://github.com/paulross/dtrace-py> Here are some examples of using DTrace with `pymemtrace` with some [technical notes](#) on using DTrace.

21.3 Techniques

This describes some of the techniques I have found useful. Bear in mind:

- Tracking down memory leaks can take a long, long time.
- Every memory leak is its own special little snowflake! So what works will be situation specific.

21.3.1 High Level

It is worth spending a fair bit of time at high level before diving into the code since:

- Working at high level is relatively cheap.
- It is usually non-invasive.
- It will quickly find out the *scale* of the problem.
- It will quickly find out the *repeatability* of the problem.
- You should be able to create the test that shows that the leak is firstly not fixed, then fixed.

At the end of this you should be able to state:

- The *frequency* of the memory leak.
- The *severity* of the memory leak.

Relevant quote: “Time spent on reconnaissance is seldom wasted.”

Using Platform Tools

The high level investigation will usually concentrate on using platform tools such as builtin memory management tools or Python tools such as pymemtrace's *pymemtrace Process* or *psutil* will prove useful.

Specific Tricks

TODO: Finish this.

21.4 pymemtrace

My *pymemtrace* project contains a number of tools that help detect memory usage and leaks. The documentation contains advice on handling memory leaks.

- On PyPi: <https://pypi.org/project/pymemtrace/>
- Project: <https://github.com/paulross/pymemtrace>
- Documentation: <https://pymemtrace.readthedocs.io/en/latest/index.html>

Here is the introduction to that project:

pymemtrace provides tools for tracking and understanding Python memory usage at different levels, at different granularities and with different runtime costs.

Full documentation: <https://pymemtrace.readthedocs.io>

21.4.1 pymemtrace Tools

The tools provided by *pymemtrace*:

- *process* is a very lightweight way of logging the total memory usage at regular time intervals. It can plot memory over time with plotting programs such as *gnuplot*. See [some process examples](#)
- *cPyMemTrace* is a memory tracer written in C that can report total memory usage for every function call/return for both C and Python sections. See some [cPyMemTrace examples](#) and a [technical note on cPyMemTrace](#).
- *DTrace*: Here are a number of D scripts that can trace the low level *malloc()* and *free()* system calls and report how much memory was allocated and by whom. See some [DTrace examples](#) and a [technical note on DTrace](#).
- *trace_malloc* is a convenience wrapper around the Python standard library *tracemalloc* module. This can report Python memory usage by module and line compensating for the cost of *tracemalloc*. This can take memory snapshots before and after code blocks and show the change on memory caused by that code. See some [trace_malloc examples](#)
- *debug_malloc_stats* is a wrapper around the *sys._debugmallocstats* function that can take snapshots of memory before and after code execution and report the significant differences of the Python small object allocator. See some [debug_malloc_stats examples](#)

21.4.2 Tool Characteristics

Each tool can be characterised by:

- *Memory Granularity*: In how much detail is a memory change is observed. An example of *coarse* memory granularity is measuring the [Resident Set Size](#) which is normally in chunks of 4096 bytes. An example of *fine* memory granularity is recording every *malloc()* and *free()*.
- *Execution Granularity*: In how much code detail is the memory change observed. An example of *coarse* execution granularity is measuring the memory usage every second. An example of *fine* execution granularity is recording the memory usage every Python line.

- *Memory Cost*: How much extra memory the tool needs.
- *Execution Cost*: How much the execution time is increased.

Clearly there are trade-offs between these depending on the problem you are trying to solve.

Table 3: **Tool Characteristics**

Tool	Memory Granularity	Execution Granularity	Memory Cost	Execution Cost
process	RSS (total Python and C memory).	Regular time intervals.	Near zero.	Near zero.
cPyMemTrace	RSS (total Python and C memory).	Per Python line, Python function and C function call.	Near zero.	x10 to x20.
DTrace	Every <code>malloc()</code> and <code>free()</code> .	Per function call and return.	Minimal.	x100.
trace_malloc	Every Python object.	Per Python line, per function call.	Significant but compensated.	x900 for small objects, x6 for large objects.
debug_malloc_stats	Python memory pool.	Snapshots the CPython memory pool either side of a block of code.	Minimal.	x2000+ for small objects, x12 for large objects.

Licence

Python memory tracing.

- Free software: MIT license
- Documentation: <https://pymemtrace.readthedocs.io>.
- Project: <https://github.com/paulross/pymemtrace>.

Credits

Phil Smith (AHL) with whom a casual lunch time chat lead to the creation of an earlier, but quite different implementation, of cPyMemTrace in pure Python.

This package was created with [Cookiecutter](#) and the [audreyr/cookiecutter-pypackage](#) project template.

21.4.3 pymemtrace Process

`pymemtrace`'s `process` is an ultralight weight tool for monitoring the memory usage of a process at regular intervals.

`process.log_process` provides a context manager that launches a separate thread that logs the memory usage in JSON of the current process at regular intervals (the CLI version can monitor any user specified process). The log format is designed so that the data can be easily extracted using, say, regular expressions.

Here is an example that creates randomly sized large strings:

```
"""
Example of using process that logs process data to the current log.
"""
import logging
import random
```

(continues on next page)

(continued from previous page)

```

import sys
import time

from pymemtrace import process

logger = logging.getLogger(__file__)

def main() -> int:
    logging.basicConfig(
        level=logging.INFO,
        format= (
            '%(asctime)s - %(filename)s#%(lineno)d - %(process)5d'
            ' - %(threadName)-10s - %(levelname)-8s - %(message)s'
        ),
    )
    logger.info('Demonstration of logging a process')
    # Log process data to the log file every 0.5 seconds.
    with process.log_process(interval=0.5, log_level=logger.getEffectiveLevel()):
        for i in range(8):
            size = random.randint(128, 128 + 256) * 1024 ** 2
            # Add a message to report in the next process write.
            process.add_message_to_queue(f'String of {size:,d} bytes')
            s = ' ' * size
            time.sleep(0.75 + random.random())
            del s
            time.sleep(0.25 + random.random() / 2)
        return 0

if __name__ == '__main__':
    sys.exit(main())

```

Might give this, although verbose, it is quite searchable:

```

$ python3.12 ex_process.py
2025-02-12 14:16:58,675 - ex_process.py#19 - 10193 - (MainThread) - INFO      -
↳ Demonstration of logging a process
2025-02-12 14:16:58,676 - process.py#289 - 10193 - (ProcMon    ) - INFO      -
↳ ProcessLoggingThread-JSON-START {"timestamp": "2025-02-12 14:16:58.676195", "memory_
↳ info": {"rss": 18067456, "vms": 34990526464, "pfaults": 6963, "pageins": 1369}, "cpu_
↳ times": {"user": 0.340946528, "system": 0.991057664, "children_user": 0.0, "children_
↳ system": 0.0}, "elapsed_time": 5.407800197601318, "pid": 10193}
2025-02-12 14:16:59,180 - process.py#293 - 10193 - (ProcMon    ) - INFO      -
↳ ProcessLoggingThread-JSON {"timestamp": "2025-02-12 14:16:59.180476", "memory_info": {
↳ "rss": 199512064, "vms": 35171934208, "pfaults": 51261, "pageins": 1374}, "cpu_times":
↳ {"user": 0.379827552, "system": 1.031979648, "children_user": 0.0, "children_system":
↳ 0.0}, "elapsed_time": 5.912204027175903, "pid": 10193, "label": "String of 181,403,648
↳ bytes"}
2025-02-12 14:16:59,682 - process.py#289 - 10193 - (ProcMon    ) - INFO      -
↳ ProcessLoggingThread-JSON {"timestamp": "2025-02-12 14:16:59.681947", "memory_info": {
↳ "rss": 18104320, "vms": 34990526464, "pfaults": 51262, "pageins": 1374}, "cpu_times": {
↳ "user": 0.380316928, "system": 1.047401792, "children_user": 0.0, "children_system": 0.

```

(continues on next page)

(continued from previous page)

```

→0}, "elapsed_time": 6.413706064224243, "pid": 10193}
...
2025-02-12 14:17:12,312 - process.py#289 - 10193 - (ProcMon ) - INFO -
→ProcessLoggingThread-JSON {"timestamp": "2025-02-12 14:17:12.312343", "memory_info": {
→"rss": 247758848, "vms": 35220168704, "pfaults": 508755, "pageins": 1374}, "cpu_times
→": {"user": 0.820292992, "system": 1.639239552, "children_user": 0.0, "children_system
→": 0.0}, "elapsed_time": 19.044106006622314, "pid": 10193}
2025-02-12 14:17:12,763 - process.py#289 - 10193 - (MainThread) - INFO -
→ProcessLoggingThread-JSON-STOP {"timestamp": "2025-02-12 14:17:12.762896", "memory_info
→": {"rss": 18116608, "vms": 34990526464, "pfaults": 508756, "pageins": 1374}, "cpu_
→times": {"user": 0.820827264, "system": 1.663195264, "children_user": 0.0, "children_
→system": 0.0}, "elapsed_time": 19.49466300010681, "pid": 10193}

Process finished with exit code 0

```

Here is the memory data from one line in more detail.

```

{
  "timestamp": "2025-02-12 14:17:05.719867",
  "memory_info": {
    "rss": 264527872,
    "vms": 35236945920,
    "pfaults": 342856,
    "pageins": 1374
  },
  "cpu_times": {
    "user": 0.658989888,
    "system": 1.418129152,
    "children_user": 0.0,
    "children_system": 0.0
  },
  "elapsed_time": 12.45150899887085,
  "pid": 10193,
  "label": "String of 246,415,360 bytes"
}

```

Of interest is the line:

```
process.add_message_to_queue(f'String of {size:,d} bytes')
```

Which injects a message into the log output and that has the JSON key of “label”.

`pymemtrace.process` provides a number of ways of tabulating and plotting this data that gives a clearer picture, at a high level, of what is happening to the process memory. Some can be seen <here <<https://pymemtrace.readthedocs.io/en/latest/examples/process.html>>`_

21.4.4 pymemtrace cPyMemTrace

`cPyMemTrace` is a Python profiler written in ‘C’ that records the [Resident Set Size](#) for every Python and C call and return.

`cPyMemTrace` writes this data to a log file to the current working directory with a name of the following form, it combines:

- `YYYYMMDD` The date.

- `_HHMMSS` The time.
- `_PID` The process ID.
- `_P` or `_T` depending on whether it is a profile function or a trace function.
- `_n` where `n` is the stack depth of the current profile or trace function as multiple nested profile or trace functions are allowed.
- The Python version such as `PY3.13.0b3`.
- `.log`.

For example `"20241107_195847_62264_P_2_PY3.13.0b3.log"`

Logging Changes in RSS

Here is a simple example:

```
from pymemtrace import cPyMemTrace

def create_string(l: int) -> str:
    return ' ' * l

with cPyMemTrace.Profile():
    l = []
    for i in range(8):
        l.append(create_string(1024**2))
    while len(l):
        l.pop()
```

This produces a log file in the current working directory. For brevity the log file does not show every profile or trace event but only those when the RSS changes by some threshold. By default this threshold is the system page size (typically 4096 bytes)¹.

Here is an example output:

	Event	dEvent	Clock	What	File	#line	Function	RSS	dRSS
NEXT:	0	+0	0.066718	CALL	test.py	# 9	create_string	9101312	9101312
NEXT:	1	+1	0.067265	RETURN	test.py	# 10	create_string	10153984	1052672
PREV:	4	+3	0.067285	CALL	test.py	# 9	create_string	10153984	0
NEXT:	5	+4	0.067777	RETURN	test.py	# 10	create_string	11206656	1052672
PREV:	8	+3	0.067787	CALL	test.py	# 9	create_string	11206656	0
NEXT:	9	+4	0.068356	RETURN	test.py	# 10	create_string	12259328	1052672
PREV:	12	+3	0.068367	CALL	test.py	# 9	create_string	12259328	0
NEXT:	13	+4	0.068944	RETURN	test.py	# 10	create_string	13312000	1052672
PREV:	16	+3	0.068954	CALL	test.py	# 9	create_string	13312000	0
NEXT:	17	+4	0.069518	RETURN	test.py	# 10	create_string	14364672	1052672
PREV:	20	+3	0.069534	CALL	test.py	# 9	create_string	14364672	0
NEXT:	21	+4	0.070101	RETURN	test.py	# 10	create_string	15417344	1052672
PREV:	24	+3	0.070120	CALL	test.py	# 9	create_string	15417344	0
NEXT:	25	+4	0.070663	RETURN	test.py	# 10	create_string	16470016	1052672
PREV:	28	+3	0.070677	CALL	test.py	# 9	create_string	16470016	0
NEXT:	29	+4	0.071211	RETURN	test.py	# 10	create_string	17522688	1052672

So in this example events 2 and 3 are omitted as there is no change in the RSS. Events 4 and 5 are included as there is a change in the RSS between them.

¹ This is obtained from `int getpagesize(void);` in `#include <unistd.h>`.

Logging Every Event

If all events are needed then change the constructor argument to 0:

```
with cPyMemTrace.Profile(0):
    # As before
```

And the log file looks like this:

	Event	dEvent	Clock	What	File	#line	Function	RSS	dRSS
NEXT:	0	+0	0.079408	CALL	test.py	# 9	create_string	9105408	9105408
NEXT:	1	+1	0.079987	RETURN	test.py	# 10	create_string	10158080	1052672
NEXT:	2	+1	0.079994	C_CALL	test.py	# 64	append	10158080	0
NEXT:	3	+1	0.079998	C_RETURN	test.py	# 64	append	10158080	0
NEXT:	4	+1	0.080003	CALL	test.py	# 9	create_string	10158080	0
NEXT:	5	+1	0.080682	RETURN	test.py	# 10	create_string	11210752	1052672
NEXT:	6	+1	0.080693	C_CALL	test.py	# 64	append	11210752	0
NEXT:	7	+1	0.080698	C_RETURN	test.py	# 64	append	11210752	0
NEXT:	8	+1	0.080704	CALL	test.py	# 9	create_string	11210752	0
NEXT:	9	+1	0.081414	RETURN	test.py	# 10	create_string	12263424	1052672
NEXT:	10	+1	0.081424	C_CALL	test.py	# 64	append	12263424	0
NEXT:	11	+1	0.081429	C_RETURN	test.py	# 64	append	12263424	0
NEXT:	12	+1	0.081434	CALL	test.py	# 9	create_string	12263424	0
NEXT:	13	+1	0.081993	RETURN	test.py	# 10	create_string	13316096	1052672
NEXT:	14	+1	0.081998	C_CALL	test.py	# 64	append	13316096	0
...									
NEXT:	59	+1	0.084531	C_RETURN	test.py	# 66	pop	17526784	0
NEXT:	60	+1	0.084535	C_CALL	test.py	# 65	len	17526784	0
NEXT:	61	+1	0.084539	C_RETURN	test.py	# 65	len	17526784	0
NEXT:	62	+1	0.084541	C_CALL	test.py	# 66	pop	17526784	0
NEXT:	63	+1	0.084561	C_RETURN	test.py	# 66	pop	17526784	0
NEXT:	64	+1	0.084566	C_CALL	test.py	# 65	len	17526784	0
NEXT:	65	+1	0.084568	C_RETURN	test.py	# 65	len	17526784	0

There is some discussion about the performance of `cPyMemTrace` here in the [technical note on cPyMemTrace](#). and some more [cPyMemTrace code examples here](#)

Tracers can be nested such as this and each level gets logged to its own file, for example:

```
from pymemtrace import cPyMemTrace

with cPyMemTrace.Profile():
    # Do stuff at stack level 0
    # This gets logged into, say: "20241107_195847_62264_P_0_PY3.13.0b3.log"
    with cPyMemTrace.Profile():
        # Do stuff at stack level 0
        # This gets logged into, say: "20241107_195847_62264_P_1_PY3.13.0b3.log"
        pass
    # Do stuff at stack level 0
    # This gets logged again into: "20241107_195847_62264_P_0_PY3.13.0b3.log"
```

21.4.5 pymemtrace and DTrace

With an OS that supports DTrace (for example Mac OS X) `pymemtrace` provides some D scripts that support memory profiling. DTrace is an extremely powerful tool that can produce an enormous amount of detailed information on memory allocations and de-allocations.

This is beyond the scope of *this* document however some examples are shown in [DTrace examples](#). There is some discussion about the performance of DTrace here in the [technical note on DTrace](#) and some more [DTrace code examples](#) [here](#).

21.4.6 pymemtrace trace_malloc

Python has the `tracemalloc` module ([documentation](#)) that can provide the following information:

- Trace where an object was allocated.
- Statistics on allocated memory blocks per filename and per line number: total size, number and average size of allocated memory blocks.
- Compute the differences between two snapshots to detect memory leaks.

However the `tracemalloc` also consumes memory so this can conceal what is really going on.

`pymemtrace.trace_malloc` contains some utility wrappers around the `tracemalloc` module and the can compensate for the memory used by `tracemalloc` module.

Using trace_malloc Directly

Adding 1Mb Strings

Here is an example of adding 1Mb strings to a list under the watchful eye of `trace_malloc.TraceMalloc`:

```
from pymemtrace import trace_malloc

list_of_strings = []
print(f'example_trace_malloc_for_documentation()')
with trace_malloc.TraceMalloc('filename') as tm:
    for i in range(8):
        list_of_strings.append(' ' * 1024**2)
print(f' tm.memory_start={tm.memory_start}')
print(f' tm.memory_finish={tm.memory_finish}')
print(f' tm.diff={tm.diff}')
for stat in tm.statistics:
    print(stat)
```

Typical output is:

```
example_trace_malloc_for_documentation()
  tm.memory_start=13072
tm.memory_finish=13800
  tm.diff=8388692
pymemtrace/examples/ex_trace_malloc.py:0: size=8194 KiB (+8193 KiB), count=16 (+10), ↵
↪ average=512 KiB
/Library/Frameworks/Python.framework/Versions/3.8/lib/python3.8/tracemalloc.py:0: ↵
↪ size=6464 B (+504 B), count=39 (+10), average=166 B
Documents/workspace/pymemtrace/pymemtrace/trace_malloc.py:0: size=3076 B (-468 B), ↵
↪ count=10 (-1), average=308 B
/Library/Frameworks/Python.framework/Versions/3.8/lib/python3.8/logging/__init__.py:0: ↵
```

(continues on next page)

(continued from previous page)

```

↪size=16.3 KiB (-128 B), count=49 (-2), average=340 B
/Library/Frameworks/Python.framework/Versions/3.8/lib/python3.8/abc.py:0: size=3169 B
↪(+0 B), count=30 (+0), average=106 B
/Library/Frameworks/Python.framework/Versions/3.8/lib/python3.8/posixpath.py:0: size=480
↪B (+0 B), count=1 (+0), average=480 B
/Library/Frameworks/Python.framework/Versions/3.8/lib/python3.8/threading.py:0: size=168
↪B (+0 B), count=2 (+0), average=84 B
/Library/Frameworks/Python.framework/Versions/3.8/lib/python3.8/_weakrefset.py:0:
↪size=72 B (+0 B), count=1 (+0), average=72 B

```

To eliminate the lines that is caused by tracemalloc itself change the last two lines to:

```

for stat in tm.net_statistics:
    print(stat)

```

Which removes the line:

```

/Library/Frameworks/Python.framework/Versions/3.8/lib/python3.8/tracemalloc.py:0:
↪size=6464 B (+504 B), count=39 (+10), average=166 B

```

Using trace_malloc as a Decorator

trace_malloc provides a function decorator that can log the tracemalloc memory usage caused by execution a function. For example:

```

from pymemtrace import trace_malloc

@trace_malloc.trace_malloc_log(logging.INFO)
def example_decorator_for_documentation(list_of_strings):
    for i in range(8):
        list_of_strings.append(create_string(1024**2))

list_of_strings = []
example_decorator_for_documentation(list_of_strings)

```

Would log something like the following:

```

2025-02-13 11:37:39,194 - trace_malloc.py#87 - 10121 - (MainThread) - INFO -
↪TraceMalloc memory delta: 8,389,548 for "example_decorator_for_documentation()"

```

Here are more [examples](#) and some more [trace_malloc code examples here](#)

21.4.7 pymemtrace Debug Malloc Stats

CPython has the function the `sys._debugmallocstats()` that can dump the status of the Python small object memory allocator. For example:

```

Python 3.13.0 (v3.13.0:60403a5409f, Oct 7 2024, 00:37:40) [Clang 15.0.0 (clang-1500.3.9.
↪4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import sys
>>> sys._debugmallocstats()
Small block threshold = 512, in 32 size classes.

```

(continues on next page)

(continued from previous page)

class	size	num pools	blocks in use	avail blocks
0	16	1	90	931
1	32	2	830	190
2	48	9	2916	144
3	64	48	11539	701
4	80	30	6076	44
5	96	6	999	21
6	112	4	454	126
7	128	8	943	73
8	144	2	188	38
9	160	19	1915	23
10	176	2	126	58
11	192	2	100	70
12	208	5	342	48
13	224	4	277	11
14	240	5	271	69
15	256	4	189	63
16	272	3	144	36
17	288	3	125	43
18	304	2	88	18
19	320	2	78	24
20	336	2	54	42
21	352	2	50	42
22	368	2	54	34
23	384	2	49	35
24	400	5	171	29
25	416	2	40	38
26	432	1	26	11
27	448	1	28	8
28	464	1	34	1
29	480	1	28	6
30	496	1	26	6
31	512	2	41	21
# arenas allocated total			=	3
# arenas reclaimed			=	0
# arenas highwater mark			=	3
# arenas allocated current			=	3
3 arenas * 1048576 bytes/arena			=	3,145,728
# bytes in allocated blocks			=	2,654,688
# bytes in available blocks			=	322,672
6 unused pools * 16384 bytes			=	98,304
# bytes lost to pool headers			=	8,784
# bytes lost to quantization			=	12,128
# bytes lost to arena alignment			=	49,152
Total			=	3,145,728
arena map counts				
# arena map mid nodes			=	1

(continues on next page)

(continued from previous page)

```

# arena map bot nodes           =           1

# bytes lost to arena map root   =       262,144
# bytes lost to arena map mid    =       262,144
# bytes lost to arena map bot    =       131,072
Total                            =       655,360

      55 free PyDictObjects * 48 bytes each =       2,640
      6 free PyFloatObjects * 24 bytes each =        144
     80 free PyListObjects * 40 bytes each =       3,200
      3 free 1-sized PyTupleObjects * 32 bytes each =         96
    814 free 2-sized PyTupleObjects * 40 bytes each =      32,560
     58 free 3-sized PyTupleObjects * 48 bytes each =       2,784
     20 free 4-sized PyTupleObjects * 56 bytes each =       1,120
      6 free 5-sized PyTupleObjects * 64 bytes each =         384
      3 free 6-sized PyTupleObjects * 72 bytes each =         216
      1 free 7-sized PyTupleObjects * 80 bytes each =          80
      3 free 8-sized PyTupleObjects * 88 bytes each =         264
      2 free 9-sized PyTupleObjects * 96 bytes each =         192
      1 free 10-sized PyTupleObjects * 104 bytes each =         104
      1 free 11-sized PyTupleObjects * 112 bytes each =         112
      0 free 12-sized PyTupleObjects * 120 bytes each =          0
      3 free 13-sized PyTupleObjects * 128 bytes each =         384
      0 free 14-sized PyTupleObjects * 136 bytes each =          0
      4 free 15-sized PyTupleObjects * 144 bytes each =         576
      1 free 16-sized PyTupleObjects * 152 bytes each =         152
      1 free 17-sized PyTupleObjects * 160 bytes each =         160
      1 free 18-sized PyTupleObjects * 168 bytes each =         168
      0 free 19-sized PyTupleObjects * 176 bytes each =          0
      2 free 20-sized PyTupleObjects * 184 bytes each =         368

>>>

```

The drawback of this is that you really want to see the before and after snapshot when a particular operation is performed and this means comparing quite verbose output.

`pymemtrace` has a module `debug_malloc_stats` that can provide is a wrapper around the `sys._debugmallocstats()` function which take snapshots of memory before and after code execution and report the significant differences of the Python small object allocator. It uses a text parser to show only the differences between before and after. For example:

```

from pymemtrace import debug_malloc_stats

print(f'example_debug_malloc_stats_for_documentation()')
list_of_strings = []
with debug_malloc_stats.DiffSysDebugMallocStats() as malloc_diff:
    for i in range(1, 9):
        list_of_strings.append(' ' * (i * 8))
print(f'DiffSysDebugMallocStats.diff():')
print(f'{malloc_diff.diff()}')

```

The output is:

```
example_debug_malloc_stats_for_documentation()
```

(continues on next page)

(continued from previous page)

DiffSysDebugMallocStats.diff():

class	size	num pools	blocks in use	avail blocks
1	32	+1	+52	+74
2	48	+0	+17	-17
3	64	+0	+33	-33
4	80	+1	+51	-1
5	96	+2	+34	+50
6	112	+0	+2	-2
7	128	+0	+1	-1
10	176	+0	+1	-1
12	208	+0	+1	-1
17	288	+0	+1	-1
18	304	+0	+2	-2
25	416	+0	+3	-3
26	432	+0	+3	-3
27	448	+0	+3	-3
29	480	+0	+3	-3
30	496	+0	+1	-1
31	512	+0	+1	-1
# bytes in allocated blocks				= +19,904
# bytes in available blocks				= -3,808
-4 unused pools * 4096 bytes				= -16,384
# bytes lost to pool headers				= +192
# bytes lost to quantization				= +96
-1 free 1-sized PyTupleObjects * 32 bytes each =				-32
+1 free 5-sized PyTupleObjects * 64 bytes each =				+64
+2 free PyDictObjects * 48 bytes each =				+96
-2 free PyListObjects * 40 bytes each =				-80
+1 free PyMethodObjects * 48 bytes each =				+48

There are more examples in https://pymemtrace.readthedocs.io/en/latest/examples/debug_malloc_stats.html and some more `debug_malloc_stats` code examples [here](#)

THREAD SAFETY

This chapter describes various issues when C extensions interact with Python threads¹.

22.1 When You Need a Lock

If your Extension is likely to be exposed to a multi-threaded environment then you need to think about thread safety. I had this problem in a separate project which was a C++ `SkipList` which could contain an ordered list of arbitrary Python objects.

The problem in a multi-threaded environment that was sharing the same structure was that the following sequence of events could happen:

- Thread A tries to insert a Python object into the `SkipList`. The C++ code searches for a place to insert it preserving the existing order. To do so it must call back into Python code for the user defined comparison function (using `functools.total_ordering` for example).
- At this point the Python interpreter is free to make a context switch allowing thread B to, say, remove an element from the same `SkipList`. This removal may well invalidate C++ pointers held by thread A.
- When the interpreter switches back to thread A it accesses an invalid pointer and a segfault happens.

The solution is to use a lock to prevent a Python context switch until A has completed its insertion, but how?

I found the existing Python documentation misleading and I couldn't get it to work reliably, if at all. It was only when I stumbled upon the [source code](#) for the `bz module` that I realised there was a whole other, low level way of doing this, largely undocumented.

Here is a version that concentrates on those essentials. As an example, here is a subclass of a list that has a `max()` method that returns the maximum value in the list. To do the comparison it must call `PyObject_RichCompareBool` to decide which of two objects is the maximum.

So during that call to `max()` the Python interpreter is free too switch to another thread that might alter the list we are inspecting. What we need to do is to block that thread with a lock so that can't happen. Then once the result of `max()` is known we can release that lock. This class deliberately has `sleep()` calls to allow a thread switch to take place.

The code (C and C++) is in `src/cpy/Threads` and the tests are in `tests/unit/test_c_threads.py`.

Lets walk through it.

¹ I don't cover 'pure' C threads (those that do not `#include <Python.h>`) here as they are not relevant. If your C extension code creates/calls pure C threads this does not affect the CPython state for the *current* thread. Of course *that* C thread could embed another Python interpreter but that would have an entirely different state than the current interpreter.

22.2 Coding up the Lock

First we need to include `pthread.h` as well as the usual includes:

```
#include <Python.h>
#include "structmember.h"

#ifdef WITH_THREAD
#include "pthread.h"
#endif
```

Note

Your Python may have been compiled without thread support in which case we don't have to concern ourselves with thread locking. We can discover this from the presence of the macro `WITH_THREAD` so all our thread support code is conditional on the definition of this macro.

22.2.1 Adding a `PyThread_type_lock` to our object

Then we add a `PyThread_type_lock` (an opaque pointer) to the Python structure we are intending to protect. Here is the object declaration:

```
typedef struct {
    PyListObject list;
#ifdef WITH_THREAD
    PyThread_type_lock lock;
#endif
} SubListObject;
```

22.2.2 Initialising and Deallocating the Lock

If you have a `__new__` method then set the lock pointer to `NULL`. The lock needs to be initialised only in the `__init__` method. In the `__init__` method we allocate the lock by calling `PyThread_allocate_lock()`²:

```
1 static int
2 SubList_init(SubListObject *self, PyObject *args, PyObject *kwds) {
3     if (PyList_Type.tp_init((PyObject *) self, args, kwds) < 0) {
4         return -1;
5     }
6     #ifdef WITH_THREAD
7     self->lock = PyThread_allocate_lock();
8     if (self->lock == NULL) {
9         PyErr_SetString(PyExc_MemoryError, "Unable to allocate thread lock.");
10        return -2;
11    }
12    #endif
13    return 0;
14 }
```

When deallocating the object we should free the lock pointer with `PyThread_free_lock`³:

² The order has to be: set the lock pointer `NULL` in `__new__`, allocate it in `__init__`, free it in `__dealloc__`. If you don't do this then the lock does not get initialised and segfaults, typically in `_pthread_mutex_check_init`.

³ A potential weakness of this code is that we might be deallocating the lock *whilst the lock is acquired* which could lead to deadlock.

```

1 static void
2 SubList_dealloc(SubListObject *self) {
3     /* Deallocate other fields here. */
4     #ifdef WITH_THREAD
5         if (self->lock) {
6             PyThread_free_lock(self->lock);
7             self->lock = NULL;
8         }
9     #endif
10    Py_TYPE(self)->tp_free((PyObject *)self);
11 }

```

22.3 Using the Lock

So now our object has a lock but we need to acquire it and release it.

22.3.1 From C Code

It is useful to declare a couple of macros. These are from the `bz` module:

```

#define ACQUIRE_LOCK(obj) do { \
    if (!PyThread_acquire_lock((obj)->lock, 0)) { \
        Py_BEGIN_ALLOW_THREADS \
        PyThread_acquire_lock((obj)->lock, 1); \
        Py_END_ALLOW_THREADS \
    } } while (0)

#define RELEASE_LOCK(obj) PyThread_release_lock((obj)->lock)

```

The code that acquires the lock is slightly clearer if the `Py_BEGIN_ALLOW_THREADS` and `Py_END_ALLOW_THREADS` macros are fully expanded:

```

if (!PyThread_acquire_lock(_pSL->lock, NOWAIT_LOCK)) {
    {
        PyThreadState *_save;
        _save = PyEval_SaveThread();
        PyThread_acquire_lock(_pSL->lock, WAIT_LOCK);
        PyEval_RestoreThread(_save);
    }
}

```

Before any critical section we need to use `ACQUIRE_LOCK(self)`; (which blocks) then `RELEASE_LOCK(self)`; when done. Failure to call `RELEASE_LOCK(self)`; in any code path *will* lead to deadlocking.

This is very much implementation defined in `pythreads` and may vary from platform to platform. There is no obvious API in `pythreads` that allows us to determine if a lock is held so we can release it before deallocation.

I notice that in the Python threading module (`Modules/_threadmodule.c`) there is an additional `char` field that acts as a flag to say when the lock is held so that the `lock_dealloc()` function in that module can release the lock before freeing the lock.

Example

Here is an example of our `sublist.append()`.

In the body of the function it makes a `super()` call and then introduces a `sleep()` which allows the Python interpreter to switch threads (it should not because of the lock).

```

1 static PyObject *
2 SubList_append(SubListObject *self, PyObject *args) {
3     ACQUIRE_LOCK(self);
4     PyObject *result = call_super_name(
5         (PyObject *) self, "append", args, NULL
6     );
7     // 0.25s delay to demonstrate holding on to the thread.
8     sleep_milliseconds(250L);
9     RELEASE_LOCK(self);
10    return result;
11 }

```

22.3.2 From C++ Code

We can make this a little smoother in C++ by creating a class that will lock and unlock.

Creating a class to Acquire and Release the Lock

We can acquire and release the lock in a RAII fashion in C++ where the constructor blocks until the lock is acquired and the destructor releases the lock. This is a template class for generality.

The code is in `src/cpy/Threads/cThreadLock.h`

```

1 #include <Python.h>
2 #include "structmember.h"
3
4 #ifdef WITH_THREAD
5 #include "pythread.h"
6 #endif
7
8 #ifdef WITH_THREAD
9     /* A RAII wrapper around the PyThread_type_lock. */
10    template<typename T>
11    class AcquireLock {
12    public:
13        AcquireLock(T *pobject) : m_pobject(pobject) {
14            assert(m_pobject);
15            assert(m_pobject->lock);
16            Py_INCREF(m_pobject);
17            if (!PyThread_acquire_lock(m_pobject->lock, NOWAIT_LOCK)) {
18                Py_BEGIN_ALLOW_THREADS
19                PyThread_acquire_lock(m_pobject->lock, WAIT_LOCK);
20                Py_END_ALLOW_THREADS
21            }
22        }
23        ~AcquireLock() {
24            assert(m_pobject);
25            assert(m_pobject->lock);

```

(continues on next page)

(continued from previous page)

```

26         PyThread_release_lock(m_pObject->lock);
27         Py_DECREF(m_pObject);
28     }
29     private:
30         T *m_pObject;
31     };
32
33 #else
34     /* Make the class a NOP which should get optimised out. */
35     template<typename T>
36     class AcquireLock {
37     public:
38         AcquireLock(T *) {}
39     };
40 #endif

```

Using the AcquireLock class

Before any critical section we create an AcquireLock object which blocks until we have the lock. Once the lock is obtained we can make any calls, including calls into the Python interpreter without preemption. The lock is automatically freed when we exit the code block:

```

1  /** append with a thread lock. */
2  static PyObject *
3  SubList_append(SubListObject *self, PyObject *args) {
4      AcquireLock<SubListObject> local_lock((SubListObject *)self);
5      PyObject *result = call_super_name(
6          (PyObject *) self, "append", args, NULL
7      );
8      // 0.25s delay to demonstrate holding on to the thread.
9      sleep_milliseconds(250L);
10     return result;
11 }

```

22.4 Example Code and Tests

The code (C and C++) is in `src/cpy/Threads` and the tests are in `tests/unit/test_c_threads.py`.

22.4.1 Example Code

The example code is here:

- C: `src/cpy/Threads/csublist.c`
- C++: `src/cpy/Threads/cThreadLock.h` and `src/cpy/Threads/cppsublist.cpp`.

`setup.py` creates two extensions; `cPyExtPatt.Threads.csublist` (in C) and `cPyExtPatt.Threads.cppsublist` (in C++):

```

1  Extension(name=f"{PACKAGE_NAME}.Threads.csublist",
2             include_dirs=[
3                 '/usr/local/include',
4                 'src/cpy/Util',

```

(continues on next page)

(continued from previous page)

```

5         "src/cpy/Threads",
6     ],
7     sources=[
8         "src/cpy/Threads/csublist.c",
9         'src/cpy/Util/py_call_super.c',
10    ],
11    extra_compile_args=extra_compile_args_c,
12    language='c',
13    ),
14    Extension(name=f"{PACKAGE_NAME}.Threads.cppsublist",
15              include_dirs=[
16                  '/usr/local/include',
17                  'src/cpy/Util',
18                  "src/cpy/Threads",
19              ],
20              sources=[
21                  "src/cpy/Threads/cppsublist.cpp",
22                  'src/cpy/Util/py_call_super.c',
23              ],
24              language='c++11',
25              ),

```

The individual C and C++ modules can be accessed with:

```

from cPyExtPatt.Threads import cppsublist
from cPyExtPatt.Threads import csublist

```

22.4.2 Example Tests

The tests are in tests/unit/test_c_threads.py. Here are some examples:

Tests in C

First create two function to call max() and append(). These functions print out their progress and which thread they are running in:

```

1  def csublist_max(obj, count):
2      print(
3          f'sublist_max(): Thread name {threading.current_thread().name}',
4          flush=True
5      )
6      for _i in range(count):
7          print(
8              f'sublist_max(): Thread name {threading.current_thread().name}'
9              f' Result: {obj.max()}',
10             flush=True
11         )
12         time.sleep(0.25)
13     print(
14         f'sublist_max(): Thread name {threading.current_thread().name} DONE',
15         flush=True
16     )

```

(continues on next page)

(continued from previous page)

```

17
18
19 def csublist_append(obj, count):
20     print(
21         f'sublist_append(): Thread name {threading.current_thread().name}',
22         flush=True
23     )
24     for _i in range(count):
25         print(
26             f'sublist_append(): Thread name {threading.current_thread().name}',
27             flush=True
28         )
29         obj.append(len(obj))
30         time.sleep(0.25)
31     print(
32         f'sublist_append(): Thread name {threading.current_thread().name} DONE',
33         flush=True
34     )

```

Now a test that creates a single shared sub-list and four threads for each of the `max()` and `append()` functions:

```

1 def test_threaded_c():
2     print()
3     print('test_threaded_c() START', flush=True)
4     obj = csublist.cSubList(range(128))
5     threads = []
6     for i in range(4):
7         threads.append(
8             threading.Thread(
9                 name=f'sublist_max[{i:2d}]',
10                target=csublist_max,
11                args=(obj, 2),
12            )
13        )
14        threads.append(
15            threading.Thread(
16                name=f'sublist_append[{i:2d}]',
17                target=csublist_append,
18                args=(obj, 2),
19            )
20        )
21    for thread in threads:
22        thread.start()
23    print('Waiting for worker threads', flush=True)
24    main_thread = threading.current_thread()
25    for t in threading.enumerate():
26        if t is not main_thread:
27            t.join()
28    print('Worker threads DONE', flush=True)

```

Running this test gives this output, typically:

```
1 test_threaded_c() START
2 sublist_max(): Thread name sublist_max[ 0]
3 sublist_append(): Thread name sublist_append[ 0]
4 sublist_append(): Thread name sublist_append[ 0]
5 sublist_max(): Thread name sublist_max[ 1]
6 sublist_append(): Thread name sublist_append[ 1]
7 sublist_max(): Thread name sublist_max[ 2]
8 sublist_max(): Thread name sublist_max[ 1] Result: 127
9 sublist_append(): Thread name sublist_append[ 2]
10 sublist_max(): Thread name sublist_max[ 3]
11 sublist_append(): Thread name sublist_append[ 3]
12 Waiting for worker threads
13 sublist_append(): Thread name sublist_append[ 1]
14 sublist_max(): Thread name sublist_max[ 0] Result: 128
15 sublist_max(): Thread name sublist_max[ 3] Result: 128
16 sublist_append(): Thread name sublist_append[ 2]
17 sublist_append(): Thread name sublist_append[ 3]
18 sublist_append(): Thread name sublist_append[ 0]
19 sublist_max(): Thread name sublist_max[ 2] Result: 128
20 sublist_append(): Thread name sublist_append[ 2]
21 sublist_append(): Thread name sublist_append[ 1]
22 sublist_max(): Thread name sublist_max[ 1] Result: 131
23 sublist_max(): Thread name sublist_max[ 0] Result: 132
24 sublist_append(): Thread name sublist_append[ 0] DONE
25 sublist_max(): Thread name sublist_max[ 1] DONE
26 sublist_max(): Thread name sublist_max[ 3] Result: 134
27 sublist_append(): Thread name sublist_append[ 3]
28 sublist_append(): Thread name sublist_append[ 1] DONE
29 sublist_max(): Thread name sublist_max[ 2] Result: 134
30 sublist_max(): Thread name sublist_max[ 0] DONE
31 sublist_append(): Thread name sublist_append[ 2] DONE
32 sublist_max(): Thread name sublist_max[ 3] DONE
33 sublist_append(): Thread name sublist_append[ 3] DONE
34 sublist_max(): Thread name sublist_max[ 2] DONE
35 Worker threads DONE
```

Tests in C++

A very similar example in C++ is in `tests/unit/test_c_threads.py`.

SOURCE CODE LAYOUT

I find it useful to physically separate out the source code into different categories:

Table 1: Recommended Code Directories

Category	Language	#include <Python.h>?	Testable?	Where?	Description
Pure Python	Python	No	Yes	py/	Regular Python code tested by pytest or similar.
CPython interface	Mostly C	Yes	No	cpy/	C code that defines Python modules and classes. Functions that are exposed directly to Python.
CPython utilities	C, C++	Yes	Yes	cpy/	Utility C/C++ code that works with Python objects but these functions that are <i>not</i> exposed directly to Python. This code can be tested in a C/C++ environment with a specialised test framework. See C++ RAI Wrappers Around PyObject* for some examples.
C/C++ core	C, C++	No	Yes	cpp/	C/C++ code that knows nothing about Python. This code can be tested in a C/C++ environment with a standard C/C++ test framework.

23.1 Testing CPython Utility Code

When making Python C API calls from a C/C++ environment it is important to initialise the Python interpreter. For example, this small program segfaults:

```

1 #include <Python.h>
2
3 int main(int /* argc */, const char *[] /* argv[] */) {
4     /* Forgot this:
5     Py_Initialize();
6     */
7     PyErr_Format(PyExc_TypeError, "Stuff",);
8     return 0;
9 }
```

The reason is that `PyErr_Format` calls `PyThreadState *thread_state = PyThreadState_Get();` then `thread_state` will be `NULL` unless the Python interpreter is initialised.

So you need to call `Py_Initialize()` to set up statically allocated interpreter data. Alternatively put `if (!Py_IsInitialized()) Py_Initialize();` in every test. See: <https://docs.python.org/3/c-api/init.html>

Here are a couple of useful C++ functions that assert all is well that can be used at the beginning of any function:

```
1  /* Returns non zero if Python is initialised and there is no Python error set.
2  * The second version also checks that the given pointer is non-NULL
3  * Use this thus, it will do nothing if NDEBUG is defined:
4  *
5  * assert(cpython_asserts());
6  * assert(cpython_asserts(p));
7  */
8  int cpython_asserts() {
9      return Py_IsInitialized() && PyErr_Occurred() == NULL;
10 }
11
12 int cpython_asserts(PyObject *pobj) {
13     return cpython_asserts() && pobj != NULL;
14 }
```

USING C++ WITH CPYTHON CODE

Using C++ can take a lot of the pain out of interfacing CPython code, here are some examples.

24.1 C++ RAI Wrappers Around PyObject*

It is sometimes useful to wrap up a PyObject* in a class that will manage the reference count. Here is a base class that shows the general idea, it takes a PyObject * and provides:

- Construction with a PyObject * and access this with operator PyObject*() const.
- PyObject **operator&() to reset the underlying pointer, for example when using it with PyArg_ParseTupleAndKeywords.
- Decrementing the reference count on destruction (potentially freeing the object).

```
1  /** General wrapper around a PyObject*.
2   * This decrements the reference count on destruction.
3   */
4  class DecRefDtor {
5  public:
6      DecRefDtor(PyObject *ref) : m_ref { ref } {}
7      Py_ssize_t ref_count() const { return m_ref ? Py_REFCNT(m_ref) : 0; }
8      // Allow setting of the (optional) argument with PyArg_ParseTupleAndKeywords
9      PyObject **operator&() {
10         Py_XDECREF(m_ref);
11         m_ref = NULL;
12         return &m_ref;
13     }
14     // Access the argument
15     operator PyObject*() const { return m_ref; }
16     // Test if constructed successfully from the new reference.
17     explicit operator bool() { return m_ref != NULL; }
18     ~DecRefDtor() { Py_XDECREF(m_ref); }
19 protected:
20     PyObject *m_ref;
21 };
```

24.1.1 C++ RAI Wrapper for a Borrowed PyObject*

There are two useful sub-classes, one for borrowed references, one for new references which are intended to be temporary. Using borrowed references:

```

1  /** Wrapper around a PyObject* that is a borrowed reference.
2   * This increments the reference count on construction and
3   * decrements the reference count on destruction.
4   */
5  class BorrowedRef : public DecRefDtor {
6  public:
7      BorrowedRef(PyObject *borrowed_ref) : DecRefDtor(borrowed_ref) {
8          Py_XINCREF(m_ref);
9      }
10 };

```

This can be used with borrowed references as follows:

```

void function(PyObject *obj) {
    BorrowedRef(obj); // Increment reference here.
    // ...
} // Decrement reference here.

```

24.1.2 C++ RAI Wrapper for a New PyObject*

Here is a sub-class that wraps a new reference to a PyObject* and ensures it is free'd when the wrapper goes out of scope:

```

/** Wrapper around a PyObject* that is a new reference.
 * This owns the reference so does not increment it on construction but
 * does decrement it on destruction.
 */
class NewRef : public DecRefDtor {
public:
    NewRef(PyObject *new_ref) : DecRefDtor(new_ref) {}
};

```

This new reference wrapper can be used as follows:

```

void function() {
    NewRef(PyLongFromLong(9)); // New reference here.
    // Use static_cast<PyObject*>(NewRef) ...
} // Decrement the new reference here.

```

24.2 Handling Default Arguments

Handling default, possibly mutable, arguments in a pythonic way is described here: *Being Pythonic with Default Mutable Arguments*. It is quite complicated to get it right but C++ can ease the pain with a generic class to simplify handling default arguments in CPython functions.

The actual code is in `src/cpy/ParseArgs/cParseArgsHelper.cpp` but here it is, simplified to its essentials:

```

1  class DefaultArg {
2  public:
3      DefaultArg(PyObject *new_ref) : m_arg(NULL), m_default(new_ref) {}
4      /// Allow setting of the (optional) argument with
5      /// PyArg_ParseTupleAndKeywords

```

(continues on next page)

(continued from previous page)

```

6   PyObject **operator&() {
7       m_arg = NULL;
8       return &m_arg;
9   }
10  /// Access the argument or the default if default.
11  operator PyObject *() const {
12      return m_arg ? m_arg : m_default;
13  }
14  PyObject *obj() const {
15      return m_arg ? m_arg : m_default;
16  }
17  /// Test if constructed successfully from the new reference.
18  explicit operator bool() { return m_default != NULL; }
19 protected:
20     PyObject *m_arg;
21     PyObject *m_default;
22 };

```

24.2.1 Immutable Default Arguments

Suppose we have the Python function equivalent to the Python function:

```

def parse_defaults_with_helper_class(
    encoding_m: str = "utf-8",
    the_id_m: int = 1024,
    log_interval_m: float = 8.0):
    return encoding_m, the_id_m, log_interval_m

```

Here it is in C:

```

1  static PyObject *
2  parse_defaults_with_helper_class(PyObject *Py_UNUSED(module), PyObject *args, PyObject *
   ↪ *kwds) {
3      PyObject *ret = NULL;
4      /* Initialise default arguments. */
5      static DefaultArg encoding_c(PyUnicode_FromString("utf-8"));
6      static DefaultArg the_id_c(PyLong_FromLong(DEFAULT_ID));
7      static DefaultArg log_interval_c(PyFloat_FromDouble(DEFAULT_FLOAT));
8
9      /* Check that the defaults are non-NULL i.e. succesful. */
10     if (!encoding_c || !the_id_c || !log_interval_c) {
11         return NULL;
12     }
13
14     static const char *kwlist[] = {"encoding", "the_id", "log_interval", NULL};
15     /* &encoding etc. accesses &m_arg in DefaultArg because of PyObject **operator&() */
16     if (!PyArg_ParseTupleAndKeywords(args, kwds, "|000",
17                                     const_cast<char *>(kwlist),
18                                     &encoding_c, &the_id_c, &log_interval_c)) {
19         return NULL;
20     }
21 }

```

(continues on next page)

(continued from previous page)

```

22 PY_DEFAULT_CHECK(encoding_c, PyUnicode_Check, "str");
23 PY_DEFAULT_CHECK(the_id_c, PyLong_Check, "int");
24 PY_DEFAULT_CHECK(log_interval_c, PyFloat_Check, "float");
25
26 /*
27  * Use encoding, the_id, must_log from here on as PyObject* since we have
28  * operator PyObject*() const ...
29  *
30  * So if we have a function:
31  * set_encoding(PyObject *obj) { ... }
32  */
33 // set_encoding(encoding);
34 /* ... */
35
36 /* Py_BuildValue("0") increments the reference count. */
37 ret = Py_BuildValue("000", encoding_c.obj(), the_id_c.obj(), log_interval_c.obj());
38 return ret;
39 }

```

The full code is in `src/cpy/cParseArgsHelper.cpp` and the tests in `tests/unit/test_c_parse_args_helper.py`.

Here is an example test:

```

1 @pytest.mark.parametrize(
2     'args, expected',
3     (
4         (
5             (),
6             ('utf-8', 1024, 8.0),
7         ),
8         (
9             ('Encoding', 4219, 16.0),
10            ('Encoding', 4219, 16.0),
11        ),
12    ),
13 )
14 def test_parse_defaults_with_helper_class(args, expected):
15     assert cParseArgsHelper.parse_defaults_with_helper_class(*args) == expected

```

24.2.2 Mutable Default Arguments

The same class can be used for mutable arguments. The following emulates this Python function:

```

def parse_mutable_defaults_with_helper_class(obj, default_list=[]):
    default_list.append(obj)
    return default_list

```

Here it is in C:

```

1 /** Parse the args where we are simulating mutable default of an empty list.
2  * This uses the helper class.
3  *

```

(continues on next page)

(continued from previous page)

```

4  * This is equivalent to:
5  *
6  * def parse_mutable_defaults_with_helper_class(obj, default_list=[]):
7  *     default_list.append(obj)
8  *     return default_list
9  *
10 * This adds the object to the list and returns None.
11 *
12 * This imitates the Python way of handling defaults.
13 */
14 static PyObject *parse_mutable_defaults_with_helper_class(PyObject *Py_UNUSED(module),
15                                                         PyObject *args) {
16     PyObject *ret = NULL;
17     /* Pointers to the non-default argument, initialised by PyArg_ParseTuple below. */
18     PyObject *arg_0 = NULL;
19     static DefaultArg list_argument_c(PyList_New(0));
20
21     if (!PyArg_ParseTuple(args, "O|O", &arg_0, &list_argument_c)) {
22         goto except;
23     }
24     PY_DEFAULT_CHECK(list_argument_c, PyList_Check, "list");
25
26     /* Your code here... */
27
28     /* Append the first argument to the second.
29      * PyList_Append() increments the refcount of arg_0. */
30     if (PyList_Append(list_argument_c, arg_0)) {
31         PyErr_SetString(PyExc_RuntimeError, "Can not append to list!");
32         goto except;
33     }
34
35     /* Success. */
36     assert(!PyErr_Occurred());
37     /* This increments the default or the given argument. */
38     Py_INCREF(list_argument_c);
39     ret = list_argument_c;
40     goto finally;
41 except:
42     assert(PyErr_Occurred());
43     Py_XDECREF(ret);
44     ret = NULL;
45 finally:
46     return ret;
47 }

```

The code is in `src/cpy/ParseArgs/cParseArgsHelper.cpp`.

Here are some tests from `tests/unit/test_c_parse_args_helper.py`. Firstly establish the known Python behaviour:

```

1 def test_parse_mutable_defaults_with_helper_class_python():
2     """A local Python equivalent of cParseArgsHelper.parse_mutable_defaults_with_helper_
   ↪ class()."""

```

(continues on next page)

(continued from previous page)

```

3
4 def parse_mutable_defaults_with_helper_class(obj, default_list=[]):
5     default_list.append(obj)
6     return default_list
7
8 result = parse_mutable_defaults_with_helper_class(1)
9 assert sys.getrefcount(result) == 3
10 assert result == [1, ]
11 result = parse_mutable_defaults_with_helper_class(2)
12 assert sys.getrefcount(result) == 3
13 assert result == [1, 2]
14 result = parse_mutable_defaults_with_helper_class(3)
15 assert sys.getrefcount(result) == 3
16 assert result == [1, 2, 3]
17
18 local_list = []
19 assert sys.getrefcount(local_list) == 2
20 assert parse_mutable_defaults_with_helper_class(10, local_list) == [10]
21 assert sys.getrefcount(local_list) == 2
22 assert parse_mutable_defaults_with_helper_class(11, local_list) == [10, 11]
23 assert sys.getrefcount(local_list) == 2
24
25 result = parse_mutable_defaults_with_helper_class(4)
26 assert result == [1, 2, 3, 4]
27 assert sys.getrefcount(result) == 3

```

And now the equivalent in C:

```

1 from cPyExtPatt import cParseArgsHelper
2
3 def test_parse_mutable_defaults_with_helper_class_c():
4     result = cParseArgsHelper.parse_mutable_defaults_with_helper_class(1)
5     assert sys.getrefcount(result) == 3
6     assert result == [1, ]
7     result = cParseArgsHelper.parse_mutable_defaults_with_helper_class(2)
8     assert sys.getrefcount(result) == 3
9     assert result == [1, 2]
10    result = cParseArgsHelper.parse_mutable_defaults_with_helper_class(3)
11    assert sys.getrefcount(result) == 3
12    assert result == [1, 2, 3]
13
14    local_list = []
15    assert sys.getrefcount(local_list) == 2
16    assert cParseArgsHelper.parse_mutable_defaults_with_helper_class(10, local_list) ==
17    ↪ [10]
18    assert sys.getrefcount(local_list) == 2
19    assert cParseArgsHelper.parse_mutable_defaults_with_helper_class(11, local_list) ==
20    ↪ [10, 11]
21    assert sys.getrefcount(local_list) == 2
22
23    result = cParseArgsHelper.parse_mutable_defaults_with_helper_class(4)
24    assert result == [1, 2, 3, 4]

```

(continues on next page)

(continued from previous page)

```
23 assert sys.getrefcount(result) == 3
```

24.3 Homogeneous Python Containers and C++

Here are some useful generic functions that can convert homogeneous Python containers to and from their C++ STL equivalents in this project: [Python/C++ homogeneous containers on GitHub](#) The project uses a mixture of templates and code generation to provide 300+ functions to convert to and from C++ and Python containers.

Here is the introduction to that project:

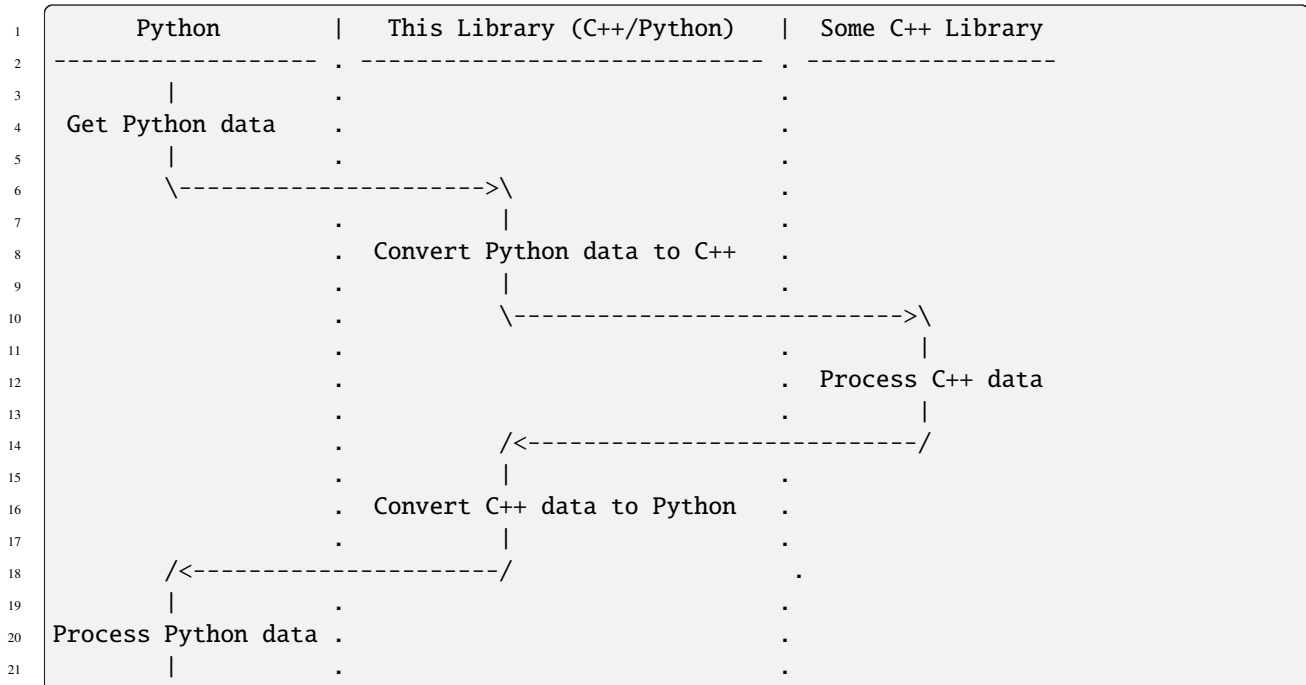
Python is well known for it's ability to handle *heterogeneous* data in containers such as lists like:

```
>>> l = [1, 2.0, "some string", ]
```

But what if you need to interact with C++ containers such as `std::vector<T>` that require *homogeneous* data types?

This project is about converting Python containers such as `list`, `tuple`, `dict`, `set`, `frozenset` containing homogeneous types such as `bool`, `int`, `float`, `complex`, `bytes`, `str` or user defined types to and from their C++ equivalent.

Here is a general example of the use of this library where Python data needs to be passed to and from a C++ library and those results need to be presented in Python. Like this, visually:



Here is a, problematic, example of how to do this:

24.3.1 A Problematic Example

Suppose that you have a Python list of floats and need to pass it to a C++ library that expects a `std::vector<double>`. If the result of that call modifies the C++ vector, or creates a new one, you need to return a Python list of floats from the result.

Your C++ code might look like this (error checking omitted):

```

1 PyObject *example(PyObject *op) {
2     std::vector<double> vec;
3     // Populate the vector, function to be defined...
4     write_to_vector(op, vec);
5     // Do something in C++ with the vector
6     // ...
7     // Convert the vector back to a Python list.
8     // Function to be defined...
9     return read_from_vector(vec);
10 }
```

What should the implementation of `write_to_vector()` and `read_from_vector()` look like?

The answer seems fairly simple; firstly `write_to_vector` converting a Python list to a C++ `std::vector<double>` with Python's C-API:

```

void write_to_vector(PyObject *op, std::vector<double> &vec) {
    vec.clear();
    for (Py_ssize_t i = 0; i < PyList_Size(op); ++i) {
        vec.push_back(PyFloat_AsDouble(PyList_GET_ITEM(op, i)));
    }
}
```

And the inverse, `read_from_vector` creating a new Python list from a C++ `std::vector<double>`:

```

PyObject *read_from_vector(const std::vector<double> &vec) {
    PyObject *ret = PyList_New(vec.size());
    for (size_t i = 0; i < vec.size(); ++i) {
        PyList_SET_ITEM(ret, i, PyFloat_FromDouble(vec[i]));
    }
    return ret;
}
```

There is no error handling shown here, and all errors would be runtime errors.

However if you need to support other object types, say lists of `int`, `str`, `bytes` then each one needs a pair of hand written functions; Python to C++ and C++ to Python. It gets worse when you want to support other containers such as `tuple`, `list`, `set`, `frozenset`, `dict`. You end up with hundreds of functions, all individually named, to handle all the combinations. Then you have to write individual conversion functions, and their tests, for all the combinations of object types *and* containers.

This is tedious and error prone and hard to extend in the general case.

Why This Project

This project simplifies the problem of converting data from Python to C++ and vice versa *in general*.

The project makes extensive use of C++ templates, partial template specialisation and code generation to dramatically reduce the amount of hand maintained code. It also converts many runtime errors to compile time errors.

The types and containers this library supports are:

Table 1: Supported Object types.

C++ Type	Python Type	Notes
bool	True, False	
long	int	
double	float	
std::complex<double>	complex	
std::vector<char>	bytes	bytearray is not supported as we need hashable types for set and dict containers.
std::string	str	Specifically a PyUnicode_1BYTE_KIND ¹ . Python documentation
std::u16string	str	Specifically a PyUnicode_2BYTE_KIND. Python documentation
std::u32string	str	Specifically a PyUnicode_4BYTE_KIND. Python documentation

Used in these containers:

Table 2: Supported Containers.

C++ Container	Python Equivalent
std::vector	Either a tuple or list
std::list	Either a tuple or list
std::unordered_set	Either a set or frozenset
std::unordered_map	dict
std::map	dict

The number of possible conversion functions is worse than the cartesian product of the types and containers as in the case of a dict the types can appear as either a key or a value.

Supporting all these conversions would normally require 352 conversion functions to be written, tested and documented².

This project simplifies this by using a mix of C++ templates and code generators to reduce this number to just **six** hand written templates for all 352 cases.

Using This Library

Python to C++

Using the library is as simple as this, suppose you have data in Python that needs to be passed to a C++ library:

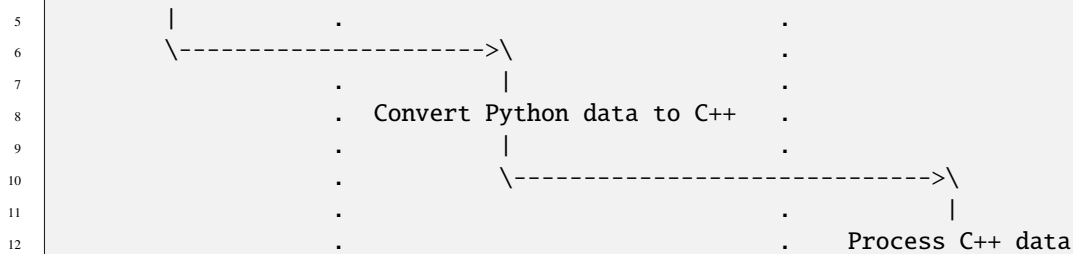
1	Python		This Library (C++/Python)		Some C++ Library
2	-----	.	-----	.	-----
3		.		.	
4	Python data source	.		.	

(continues on next page)

¹ We are currently targeting C++14 so we use `std::string` which is defined as `std::basic_string<char>`. C++20 allows a stricter, and more desirable, definition `std::basic_string<char8_t>` that we could use here. See [C++ reference for std::string](#)

² There are six unary container pairings (`tuple <-> std::list`, `tuple <-> std::vector`, `list <-> std::list`, `list <-> std::vector`, `set <-> std::unordered_set`, `frozenset <-> std::unordered_set`) with eight types (`bool`, `int`, `float`, `complex`, `bytes`, `str[1]`, `str[2]`, `str[4]`). Each container/type combination requires two functions to give two way conversion from Python to C++ and back. Thus $6 \text{ (container pairings)} * 8 \text{ (types)} * 2 \text{ (way conversion)} = 96$ required functions. For dict there are two container pairings (`dict <-> std::map`, `dict <-> std::unordered_map`) with the eight types either of which can be the key or the value so $64 \text{ (} 8 * 2 \text{)} \text{ possible variations}$. Thus $2 \text{ (container pairings)} * 64 \text{ (type pairs)} * 2 \text{ (way conversion)} = 256$ required functions. Thus is a total of $96 + 256 = 352$ functions.

(continued from previous page)



The C++ code using this library looks like this:

C++ Code

```

1  #include "python_convert.h"
2
3  // Create a Python list of floats: [21.0, 42.0, 3.0]
4  PyObject *op = Py_BuildValue("[ddd]", 21.0, 42.0, 3.0);
5
6  // Create the C++ vector that we want to convert this data to...
7  std::vector<double> cpp_vector;
8
9  // The template specialisation will automatically invoke the appropriate
10 // function call.
11 // It will be a compile time error if the container/type function
12 // is not supported.
13 // At run time this will return zero on success, non-zero on failure,
14 // for example if op is not a Python tuple or members of op can not be
15 // converted to C++ doubles.
16 int err = Python_Cpp_Containers::py_list_to_cpp_std_list_like(op, cpp_vector);
17 // Handle error checking if err is non-zero...

```

Note

If you were to change the C++ container to a `std::list<double>` the function call `py_list_to_cpp_std_list_like()` would be the same. Of course `py_list_to_cpp_std_list_like()` would then dispatch to code handling a `std::list<double>`.

Another example, suppose the Python data source is a `typing.Dict[int, str]` and this needs to be converted to a C++ `std::map<long, std::string>` then a function using the conversion code using this library is as simple as this:

```

1  #include "python_convert.h"
2
3  void convert_py_data_to_cpp(PyObject *arg) {
4      std::map<long, std::string> map;
5      if (Python_Cpp_Containers::py_dict_to_cpp_std_map_like(arg, map)) {
6          // Handle error...
7      } else {
8          // Use the map...
9      }
10 }

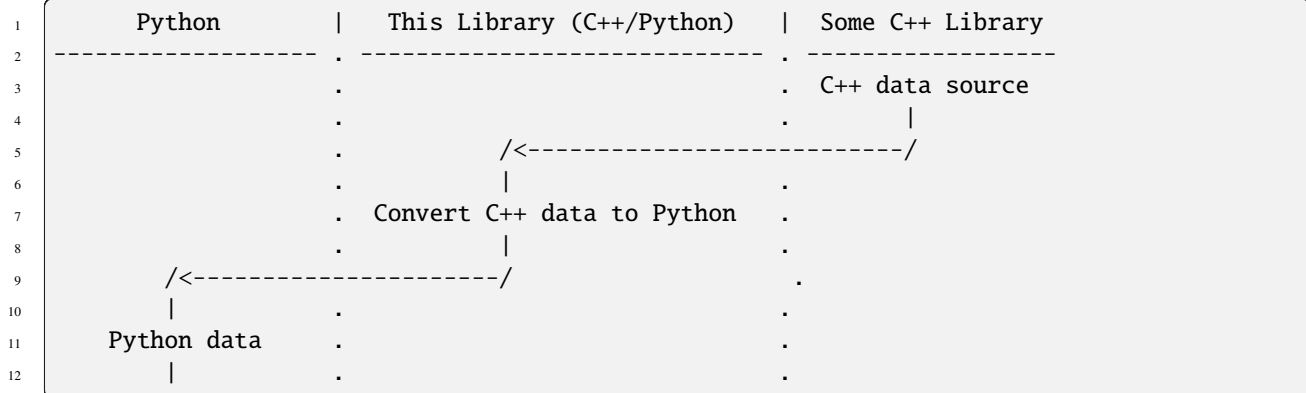
```


Note

If you were to change the C++ container to a `std::unordered_map<long, std::string>` the function call `py_dict_to_cpp_std_map_like()` would be the same. Of course `py_dict_to_cpp_std_map_like()` would then dispatch to code handling a `std::unordered_map<long, std::string>`.

C++ to Python

Suppose that you have data from a C++ library and this data needs to be represented in Python:



The C++ code using this library looks like this:

```

1  #include "python_convert.h"
2
3  std::vector<double> cpp_vector;
4  // Populate the C++ vector...
5  cpp_vector.push_back(21.0);
6  cpp_vector.push_back(42.0);
7  cpp_vector.push_back(3.0);
8
9  // Now convert to Python.
10 // This will be a compile time error if the C++ type is not supported.
11 PyObject *op = Python_Cpp_Containers::cpp_std_list_like_to_py_list(cpp_vector);
12 // op is a Python list of floats: [21.0, 42.0, 3.0]
13 // op will be null on failure and a Python exception will have been set.

```

Note

If you were to change the C++ container to a `std::list<double>` the function call `cpp_std_list_like_to_py_list()` would be the same. Of course `cpp_std_list_like_to_py_list()` would then dispatch to code handling a `std::list<double>`.

Another example, suppose the C++ data source is a `std::map<long, std::string>` and we need this a Python dict typing. `Dict[int, str]` then the conversion code in this library is as simple as this:

```

#include "python_convert.h"

PyObject *convert_cpp_data_to_py() {

```

(continues on next page)

(continued from previous page)

```

std::map<long, std::string> map;
// Populate map from the C++ data source
// ...
// Now convert to a Python dict:
return Python_Cpp_Containers::cpp_std_map_like_to_py_dict(map);
}

```

The Hand Written Functions

At the heart of this library here are just six non-trivial hand written functions along with a much larger of generated functions that successively specialise these handwritten functions. They are defined as templates in `src/cpy/python_object_convert.h`.

- Two C++ templates for Python tuple / list to and from `std::list` or `std::vector` for all types.
- Two C++ templates for Python set / frozenset to and from `std::unordered_set` for all types.
- Two C++ templates for Python dict to and from `std::map` or `std::unordered_map` for all type pairs.

These six handwritten templates are short, simple and comprehensible. Then, for simplicity, a Python script is used to create the final, instantiated, 352 functions.

As an example, here how the function is developed that converts a Python list of float to and from a C++ `std::vector<double>` or `std::list<double>`.

First C++ to Python.

Converting a C++ `std::vector<T>` or `std::list<T>` to a Python tuple or list

The generic function signature looks like this:

```

1  template<
2      template<typename ...> class ListLike,
3      typename T,
4      PyObject *(*ConvertCppToPy)(const T &),
5      PyObject *(*PyUnaryContainer_New)(size_t),
6      int(*PyUnaryContainer_Set)(PyObject *, size_t, PyObject *)
7  >
8  PyObject *
9  very_generic_cpp_std_list_like_to_py_unary(const ListLike<T> &list_like) {
10     // Handwritten code, see "C++ to Python Implementation" below.
11     // ...
12 }

```

Table 3: `very_generic_cpp_std_list_like_to_py_unary()` template parameters.

Template Parameter	Notes
<code>ListLike</code>	The C++ container type, either a <code>std::vector</code> or <code>std::list</code> .
<code>T</code>	The C++ type of the objects in the target C++ container.
<code>ConvertCppToPy</code>	A pointer to a function that converts any C++ <code>T</code> to a <code>PyObject *</code> , for example from <code>double</code> -> <code>float</code> . The function signature is <code>PyObject *ConvertCppToPy(const T&)</code> . This returns <code>NULL</code> on failure.
<code>PyUnaryContainer_New</code>	A pointer to a function that creates a new Python container, for example a list, of a particular length. The function signature is <code>PyObject *PyUnaryContainer_New(Py_ssize_t)</code> . This returns <code>NULL</code> on failure.
<code>PyUnaryContainer_Set</code>	A pointer to a function that sets a <code>PyObject *</code> in the Python container at a given index. The function signature is <code>int PyUnaryContainer_Set(PyObject *container, size_t pos, PyObject *value)</code> . This returns 0 on success.

And the function has the following parameters.

Table 4: `very_generic_cpp_std_list_like_to_py_unary()` parameters.

Type	Name	Notes
<code>ListLike<T> &</code>	<code>list_like</code>	The C++ list like container to read from to.

The return value is non-NULL on success or NULL if there is a runtime error. These errors could be:

- `PyObject *` container can not be created.
- A member of the Python container can not be created from the C++ type `T`.
- The `PyObject *` can not be inserted into the Python container.

C++ to Python Implementation

The implementation is fairly straightforward in `src/cpy/python_object_convert.h` (lightly edited):

```

1  template<
2      template<typename ...> class ListLike,
3      typename T,
4      PyObject *(*ConvertCppToPy)(const T &),
5      PyObject *(*PyUnaryContainer_New)(size_t),
6      int(*PyUnaryContainer_Set)(PyObject *, size_t, PyObject *)
7  >
8  PyObject *
9  very_generic_cpp_std_list_like_to_py_unary(const ListLike<T> &list_like) {
10     assert(!PyErr_Occurred());
11     PyObject *ret = PyUnaryContainer_New(list_like.size());
12     if (ret) {
13         size_t i = 0;
14         for (const auto &val: list_like) {
15             PyObject *op = (*ConvertCppToPy)(val);
16             if (!op) {

```

(continues on next page)

(continued from previous page)

```

17         // Failure, do not need to decref the contents as that will
18         // be done when decrefing the container.
19         // e.g. tupledealloc():
20         // https://github.com/python/cpython/blob/main/Objects/tupleobject.c
21         PyErr_Format(PyExc_ValueError, "C++ value of can not be converted.");
22         goto except;
23     }
24     // PyUnaryContainer_Set wraps a function returning non-zero on error.
25     if (PyUnaryContainer_Set(ret, i++, op)) { // Stolen reference.
26         PyErr_Format(PyExc_RuntimeError, "Can not set unary value.");
27         goto except;
28     }
29 }
30 } else {
31     PyErr_Format(
32         PyExc_ValueError,
33         "Can not create Python container of size %ld",
34         list_like.size()
35     );
36     goto except;
37 }
38 assert(!PyErr_Occurred());
39 assert(ret);
40 goto finally;
41 except:
42     Py_XDECREF(ret);
43     assert(PyErr_Occurred());
44     ret = NULL;
45 finally:
46     return ret;
47 }

```

Partial Specialisation to Convert a C++ `std::vector<T>` or `std::list<T>` to a Python list`

As an example this is specialised for a C++ `std::vector` and a Python list with a handwritten oneliner:

```

1 template<
2     typename T,
3     PyObject *(*ConvertCppToPy)(const T &)
4 >
5 PyObject *
6 generic_cpp_std_list_like_to_py_list(const std::vector<T> &container) {
7     return very_generic_cpp_std_list_like_to_py_unary<
8         std::vector, T, ConvertCppToPy, &py_list_new, &py_list_set
9     >(container);
10 }

```

Note

The use of the function pointers to `py_list_new`, and `py_list_set` that are defined in this project namespace. These are thin wrappers around existing functions or macros in "Python.h". There is no error checking in these

functions.

For example:

```
PyObject *py_list_new(size_t len) {
    return PyList_New(len);
}
int py_list_set(PyObject *list_p, size_t pos, PyObject *op) {
    // No error checking, always "succeeds".
    PyList_SET_ITEM(list_p, pos, op);
    return 0;
}
```

There is a similar partial specialisation for a Python tuple:

```
1 template<
2     typename T,
3     PyObject *(*ConvertCppToPy)(const T &)
4 >
5 PyObject *
6 generic_cpp_std_list_like_to_py_list(const std::vector<T> &container) {
7     return very_generic_cpp_std_list_like_to_py_unary<
8         std::vector, T, ConvertCppToPy, &py_tuple_new, &py_tuple_set
9     >(container);
10 }
```

And the tuple functions are trivial and look like the list ones in the note above. There is no error checking in these functions:

```
PyObject *py_tuple_new(size_t len) {
    return PyTuple_New(len);
}
int py_tuple_set(PyObject *tuple_p, size_t pos, PyObject *op) {
    // No error checking, always "succeeds".
    PyTuple_SET_ITEM(tuple_p, pos, op);
    return 0;
}
```

Converting a Python tuple or list to a C++ `std::vector<T>` or `std::list<T>`

The reverse is converting Python to C++. This generic function that converts unary Python indexed containers (tuple and list) to a C++ `std::vector<T>` or `std::list<T>` for any type has this signature:

```
1 template<
2     template<typename ...> class ListLike,
3     typename T,
4     int (*PyObject_Check)(PyObject *),
5     T (*PyObject_Convert)(PyObject *),
6     int (*PyUnaryContainer_Check)(PyObject *),
7     Py_ssize_t (*PyUnaryContainer_Size)(PyObject *),
8     PyObject *(*PyUnaryContainer_Get)(PyObject *, size_t)>
9 int very_generic_py_unary_to_cpp_std_list_like(
10     PyObject *op, ListLike<T> &list_like
```

(continues on next page)

(continued from previous page)

```

11 ) {
12     // Handwritten code, see "Python to C++ Implementation" below.
13     // ...
14 }

```

This template has these parameters:

Table 5: `very_generic_py_unary_to_cpp_std_list_like()` template parameters.

Template Parameter	Notes
<code>ListLike</code>	The C++ container type, either a <code>std::vector</code> or <code>std::list</code> .
<code>T</code>	The C++ type of the objects in the target C++ container.
<code>PyObject_Check</code>	A pointer to a function that checks that any <code>PyObject *</code> in the Python container is the correct type, for example that it is a bytes object. The function signature is <code>int PyObject_Check(PyObject *)</code> . This returns non-zero if the Python object is as expected.
<code>PyObject_Convert</code>	A pointer to a function that converts any <code>PyObject *</code> in the Python container to the C++ type, for example from bytes -> <code>std::vector<char></code> . The function signature is <code>T PyObject_Convert(PyObject *)</code> .
<code>PyUnaryContainer_Check</code>	A pointer to a function that checks that the <code>PyObject *</code> argument is the correct container type, for example a tuple. The function signature is <code>int PyUnaryContainer_Check(PyObject *)</code> . This returns non-zero if the Python container is not as expected.
<code>PyUnaryContainer_Size</code>	A pointer to a function that returns the size of the Python container. The function signature is <code>Py_ssize_t PyUnaryContainer_Size(PyObject *op)</code> . This returns the size of the Python container.
<code>PyUnaryContainer_Get</code>	A pointer to a function that gets a <code>PyObject *</code> from the Python container at a given index. The function signature is <code>PyObject *PyUnaryContainer_Get(PyObject *, size_t)</code> .

And the function has the following parameters.

Table 6: `generic_py_unary_to_cpp_std_list_like()` parameters.

Type	Name	Notes
<code>PyObject *</code>	<code>op</code>	The Python container to read from.
<code>ListLike<T> &</code>	<code>list_like</code>	The C++ list like container to write to.

The return value is zero on success or non-zero if there is a runtime error. These errors could be:

- `PyObject *op` is not a container of the required type.
- A member of the Python container can not be converted to the C++ type `T` (`PyObject_Check` fails).

Python to C++ Implementation

The implementation is fairly straightforward in `src/cpy/python_object_convert.h` (lightly edited):

```

1 template<
2     template<typename ...> class ListLike,
3     typename T,

```

(continues on next page)

(continued from previous page)

```

4      int (*PyObject_Check)(PyObject *),
5      T (*PyObject_Convert)(PyObject *),
6      int (*PyUnaryContainer_Check)(PyObject *),
7      Py_ssize_t (*PyUnaryContainer_Size)(PyObject *),
8      PyObject *(*PyUnaryContainer_Get)(PyObject *, size_t)
9  >
10  int very_generic_py_unary_to_cpp_std_list_like(PyObject *op, ListLike<T> &list_like) {
11      assert(!PyErr_Occurred());
12      int ret = 0;
13      list_like.clear();
14      Py_INCREF(op); // Increment borrowed reference
15      if (!PyUnaryContainer_Check(op)) {
16          PyErr_Format(
17              PyExc_ValueError,
18              "Can not convert Python container of type %s",
19              op->ob_type->tp_name
20          );
21          ret = -1;
22          goto except;
23      }
24      for (Py_ssize_t i = 0; i < PyUnaryContainer_Size(op); ++i) {
25          PyObject *value = PyUnaryContainer_Get(op, i);
26          if (!value) {
27              ret = -2;
28              goto except;
29          }
30          if (!(*PyObject_Check)(value)) {
31              list_like.clear();
32              PyErr_Format(
33                  PyExc_ValueError,
34                  "Python value of type %s can not be converted",
35                  value->ob_type->tp_name
36              );
37              ret = -3;
38              goto except;
39          }
40          list_like.push_back((*PyObject_Convert)(value));
41          // Check !PyErr_Occurred() which could never happen as we check first.
42      }
43      assert(!PyErr_Occurred());
44      goto finally;
45  except:
46      assert(PyErr_Occurred());
47      list_like.clear();
48  finally:
49      Py_DECREF(op); // Decrement borrowed reference
50      return ret;
51  }

```

Partial Specialisation to Convert a Python list to a C++ `std::vector<T>` or `std::list<T>`

This template can be partially specialised for converting Python *lists* of any type to C++ `std::vector<T>` or `std::list<T>`. This is hand written code but it is trivial by wrapping a single function call.

In the particular case of a `std::vector` we can use `.reserve()` as an optimisations to avoid excessive re-allocations.

```

1  template<
2      typename T,
3      int (*PyObject_Check)(PyObject *),
4      T (*PyObject_Convert)(PyObject *)
5  >
6  int generic_py_list_to_cpp_std_list_like(
7      PyObject *op, std::vector<T> &container
8  ) {
9      // Reserve the vector, but only if it is a list.
10     // If it is any other Python object then ignore it as py_list_len()
11     // may give undefined behaviour.
12     // Leave it to very_generic_py_unary_to_cpp_std_list_like() to error
13     if (py_list_check(op)) {
14         container.reserve(py_list_len(op));
15     }
16     return very_generic_py_unary_to_cpp_std_list_like<
17         std::vector, T, PyObject_Check, PyObject_Convert,
18         &py_list_check, &py_list_len, &py_list_get
19     >(op, container);
20 }
```

Note

The use of the function pointers to `py_list_check`, `py_list_len` and `py_list_get` that are defined in this project namespace. These are thin wrappers around existing functions or macros in "Python.h". There is no error checking in these functions.

For example:

```

int py_list_check(PyObject *op) {
    return PyList_Check(op);
}
Py_ssize_t py_list_len(PyObject *op) {
    return PyList_Size(op);
}
PyObject *py_list_get(PyObject *list_p, size_t pos) {
    return PyList_GET_ITEM(list_p, pos);
}
```

There is a similar partial specialisation for the Python tuple:

```

1  template<typename T, int (*PyObject_Check)(PyObject *), T (*PyObject_Convert)(PyObject_
2  ↪ *)>
3  int generic_py_tuple_to_cpp_std_list_like(PyObject *op, std::vector<T> &container) {
4      // Reserve the vector, but only if it is a tuple.
5      // If it is any other Python object then ignore it as py_tuple_len()
```

(continues on next page)

(continued from previous page)

```

5 // may give undefined behaviour.
6 // Leave it to very_generic_py_unary_to_cpp_std_list_like() to error
7 if (py_tuple_check(op)) {
8     container.reserve(py_tuple_len(op));
9 }
10 return very_generic_py_unary_to_cpp_std_list_like<
11     std::vector, T, PyObject_Check, PyObject_Convert,
12     &py_tuple_check, &py_tuple_len, &py_tuple_get
13 >(op, container);
14 }

```

The functions `py_tuple_len` and `py_tuple_get` are thin wrappers round existing functions or macros in "Python.h" as above.

Generated Functions

The particular function specialisations are created by a Python script that takes the cartesian product of object types and container types and creates functions for each container/object.

C++ to Python

For example, to convert a C++ `std::vector<double>` to a Python list of float the following are created:

A base declaration in *auto_py_convert_internal.h*:

```

template<typename T>
PyObject *
cpp_std_list_like_to_py_list(const std::vector<T> &container);

```

And a concrete declaration for each C++ target type T in *auto_py_convert_internal.h*:

```

template <>
PyObject *
cpp_std_list_like_to_py_list<double>(const std::vector<double> &container);

```

And the concrete definition is in *auto_py_convert_internal.cpp*, this simply calls the generic function:

```

template <>
PyObject *
cpp_std_list_like_to_py_list<double>(const std::vector<double> &container) {
    return generic_cpp_std_list_like_to_py_list<
        double, &cpp_double_to_py_float
    >(container);
}

```

Here is the function hierarchy for converting lists to C++ `std::vector<T>` or `std::list<T>`: This is the function hierarchy for the code that converts C++ `std::vector<T>` or `std::list<T>` to Python list and tuple for all supported object types.

```

1         very_generic_cpp_std_list_like_to_py_unary <-- Hand written
2             |
3         /-----\
4         |               |               Hand written partial
5 generic_cpp_std_list_like_to_py_list  tuples... <-- specialisation for

```

(continues on next page)

(continued from previous page)

```

6          |          |          |          |          |          |
7          |          |          |          |          |          |
8          |          |          |          |          |          |
9          |          |          |          |          |          |
10         cpp_std_list_like_to_py_list<T>          ...          <-- Generated
11          |          |          |          |          |          |
12         /-----\          /-----\
13         |          |          |          |          |          |
14 cpp_std_list_like_to_py_list<double>          ...          ...          ...
15                                     <-- Generated declaration
                                     and implementation
                                     (one liners)

```

Python to C++

For example, to convert a Python list of float to a C++ `std::vector<double>` the following are generated:

A base declaration in *auto_py_convert_internal.h*:

```

template<typename T>
int
py_list_to_cpp_std_list_like(PyObject *op, std::list<T> &container);

```

And a concrete declaration for each C++ target type T in *auto_py_convert_internal.h*:

```

template <>
int
py_list_to_cpp_std_list_like<double>(PyObject *op, std::list<double> &container);

```

And the concrete definition is in *auto_py_convert_internal.cpp*:

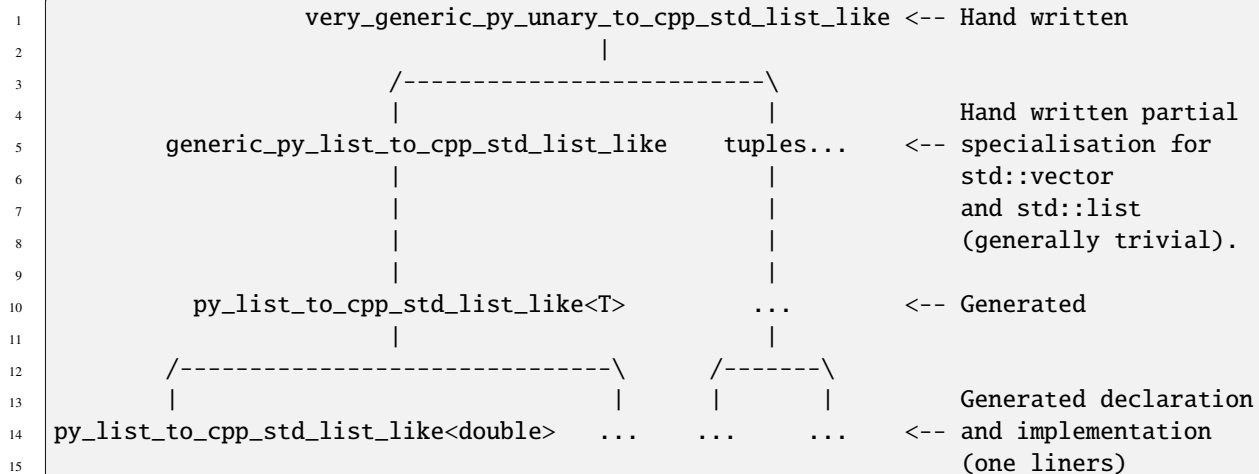
```

template <>
int
py_list_to_cpp_std_list_like<double>(PyObject *op, std::vector<double> &container) {
    return generic_py_list_to_cpp_std_list_like<
        double, &py_float_check, &py_float_to_cpp_double
    >(op, container);
}

```

[Continued on the next page]

This is the function hierarchy for the code that converts Python list and tuple to C++ `std::vector<T>` or `std::list<T>` for all supported object types.



More information can be found from this project [Python/C++ homogeneous containers on GitHub](#).

24.4 A CPython Extension containing C++ Objects

Here is an example of using C++ classes within a CPython extension type (Python object). It shows the various stages of construction and destruction.

Only the important code is shown here. The complete code is in `src/cpy/cpp/placement_new.cpp` and the tests are in `tests/unit/test_c_cpp.py`

24.4.1 Allocation of C++ Objects and Placement new

In `src/cpy/cpp/placement_new.cpp` there is a C++ class `Verbose` which:

- Reports on `stdout` construction and destruction events.
- Allocates an in-memory buffer of 256MB so that the memory usage, and any leaks, will show up in the process RSS.

We are going to create a Python extension that has a Python class that contains the C++ `Verbose` objects in two ways:

- Directly.
- With a dynamically allocated pointer.

This will illustrate the different techniques needed for both. Here is the CPython structure:

```

typedef struct {
    PyObject_HEAD
    Verbose Attr;
    Verbose *pAttr;
} CppCtorDtorInPyObject;

```

Here is the function to allocate a new CPython object. The important point here is that the line below:

```
self = (CppCtorDtorInPyObject *) type->tp_alloc(type, 0);
```

Allocates sufficient, uninitialised, space for the `CppCtorDtorInPyObject` object. This will mean that both the `Verbose Attr;` and `Verbose *pAttr;` are uninitialised. To initialise them two different techniques must be used:

- For `Verbose Attr`; this must be initialised with *placement new* `new(&self->Attr) Verbose;`.
- For `Verbose *pAttr`; this must be initialised with a dynamic *new*: `self->pAttr = new Verbose("pAttr");`.

Here is the complete code:

```

1 static PyObject *
2 CppCtorDtorInPyObject_new(PyTypeObject *type, PyObject *Py_UNUSED(args), PyObject *Py_
  ↳UNUSED(kwds)) {
3     printf("-- %s()\n", __FUNCTION__);
4     CppCtorDtorInPyObject *self;
5     self = (CppCtorDtorInPyObject *) type->tp_alloc(type, 0);
6     if (self != NULL) {
7         // Placement new used for direct allocation.
8         new(&self->Attr) Verbose;
9         self->Attr.print("Initial self->Attr");
10        // Dynamically allocated new.
11        self->pAttr = new Verbose("pAttr");
12        if (self->pAttr == NULL) {
13            Py_DECREF(self);
14            return NULL;
15        } else {
16            self->pAttr->print("Initial self->pAttr");
17        }
18    }
19    return (PyObject *) self;
20 }
```

The complimentary de-allocation function uses different deletion techniques for the two objects.

```

1 static void
2 CppCtorDtorInPyObject_dealloc(CppCtorDtorInPyObject *self) {
3     printf("-- %s()\n", __FUNCTION__);
4     self->Attr.print("self->Attr before delete");
5     // For self->Attr call the destructor directly.
6     self->Attr.~Verbose();
7     self->pAttr->print("self->pAttr before delete");
8     // For self->pAttr use delete.
9     delete self->pAttr;
10    Py_TYPE(self)->tp_free((PyObject *) self);
11 }
```

The C++ `Verbose` class writes to `stdout` the stages of construction and deletion, typically the output is:

```

1 RSS start: 35,586,048
2 -- CppCtorDtorInPyObject_new()
3 Constructor at 0x102afa840 with argument "Default" buffer len: 268435456
4 Default constructor at 0x102afa840 with argument "Default"
5 Initial self->Attr: Verbose object at 0x102afa840 m_str: "Default"
6 Constructor at 0x600003158000 with argument "pAttr" buffer len: 268435456
7 Initial self->pAttr: Verbose object at 0x600003158000 m_str: "pAttr"
8 -- CppCtorDtorInPyObject_buffer_size()
9 Buffer size: 536,871,116
10 RSS new: 572,506,112 +536,920,064
```

(continues on next page)

(continued from previous page)

```

11 -- CppCtorDtorInPyObject_dealloc()
12 self->Attr before delete: Verbose object at 0x102afa840 m_str: "Default"
13 Destructor at 0x102afa840 m_str: "Default"
14 self->pAttr before delete: Verbose object at 0x6000003158000 m_str: "pAttr"
15 Destructor at 0x6000003158000 m_str: "pAttr"
16 RSS del: 35,602,432 +16,384
17 RSS end: 35,602,432 +16,384

```

24.5 Python Unicode Strings and C++

This section looks at how you can bridge between Python and C++ unicode in Python extensions.

Whilst Python is Unicode aware C++ is not, well C++11 added `std::basic_string` specialisations for 2 and 4 byte 'Unicode' characters but these are just containers, they have no real awareness of what they contain.

24.5.1 Basic Handling of Unicode

The task here is to:

1. Take any Python Unicode string as an argument.
2. Convert it into an appropriate C++ container.
3. Dump that C++ container out to `std::cout`.
4. Create and new Python Unicode string from that C++ container and return it.

This is just show that we can round-trip between the internal representations of the two languages.

Here is the despatch function that takes a single Unicode argument (note the "U" specification) and calls the appropriate handling function:

```

1  /* Handler functions, defined later. */
2  PyObject *unicode_1_to_string_and_back(PyObject *py_str);
3  PyObject *unicode_2_to_string_and_back(PyObject *py_str);
4  PyObject *unicode_4_to_string_and_back(PyObject *py_str);
5
6  static PyObject *
7  unicode_to_string_and_back(PyObject *Py_UNUSED(module), PyObject *args) {
8      PyObject *py_str = NULL;
9      PyObject *ret_val = NULL;
10     if (!PyArg_ParseTuple(args, "U", &py_str)) {
11         return NULL;
12     }
13     unicode_dump_as_1byte_string(py_str);
14     std::cout << "Native:" << std::endl;
15     switch (PyUnicode_KIND(py_str)) {
16     case PyUnicode_1BYTE_KIND:
17         ret_val = unicode_1_to_string_and_back(py_str);
18         break;
19     case PyUnicode_2BYTE_KIND:
20         ret_val = unicode_2_to_string_and_back(py_str);
21         break;
22     case PyUnicode_4BYTE_KIND:
23         ret_val = unicode_4_to_string_and_back(py_str);

```

(continues on next page)

(continued from previous page)

```

24         break;
25     default:
26         PyErr_Format(PyExc_ValueError,
27                     "In %s argument is not recognised as a Unicode 1, 2, 4 byte_
↪string",
28                     __FUNCTION__);
29         ret_val = NULL;
30         break;
31     }
32     return ret_val;
33 }

```

The three handler functions are here, they use `std::string`, `std::u16string` and `std::u32string` as appropriate:

```

1  static PyObject *
2  unicode_1_to_string_and_back(PyObject *py_str) {
3      assert(PyUnicode_KIND(py_str) == PyUnicode_1BYTE_KIND);
4      std::string result = std::string((char *) PyUnicode_1BYTE_DATA(py_str));
5      dump_string(result);
6      return PyUnicode_FromKindAndData(PyUnicode_1BYTE_KIND,
7                                       result.c_str(),
8                                       result.size());
9  }
10
11 static PyObject *
12 unicode_2_to_string_and_back(PyObject *py_str) {
13     assert(PyUnicode_KIND(py_str) == PyUnicode_2BYTE_KIND);
14     // NOTE: std::u16string is a std::basic_string<char16_t>
15     std::u16string result = std::u16string((char16_t *) PyUnicode_2BYTE_DATA(py_str));
16     dump_string(result);
17     return PyUnicode_FromKindAndData(PyUnicode_2BYTE_KIND,
18                                     result.c_str(),
19                                     result.size());
20 }
21
22 static PyObject *
23 unicode_4_to_string_and_back(PyObject *py_str) {
24     assert(PyUnicode_KIND(py_str) == PyUnicode_4BYTE_KIND);
25     // NOTE: std::u32string is a std::basic_string<char32_t>
26     std::u32string result = std::u32string((char32_t *) PyUnicode_4BYTE_DATA(py_str));
27     dump_string(result);
28     return PyUnicode_FromKindAndData(PyUnicode_4BYTE_KIND,
29                                     result.c_str(),
30                                     result.size());
31 }

```

Each of these calls `dump_string` which is a template function that spits out the individual character values:

```

1  template <typename T>
2  void dump_string(const std::basic_string<T> &str) {
3      std::cout << "String size: " << str.size();
4      std::cout << " word size: " << sizeof(T) << std::endl;

```

(continues on next page)

(continued from previous page)

```

5     for (size_t i = 0; i < str.size(); ++i) {
6         std::cout << std::setfill('0');
7         std::cout << "0x" << std::hex;
8         std::cout << std::setw(2 * sizeof(T)) << static_cast<int>(str[i]);
9         std::cout << " " << std::dec << std::setw(8) << static_cast<int>(str[i]);
10        std::cout << std::setfill(' ');
11        std::cout << " \"" << str[i] << "\"" << std::endl;
12    }
13 }

```

For completeness here is the module code that creates a cUnicode module with a single show() function:

```

1 static PyMethodDef cUnicode_Methods[] = {
2     {"show", (PyCFunction)unicode_to_string_and_back, METH_VARARGS,
3      "Convert a Python unicode string to std::string and back."},
4     {NULL, NULL, 0, NULL} /* Sentinel */
5 };
6
7 static PyModuleDef cUnicodemodule = {
8     PyModuleDef_HEAD_INIT,
9     "cUnicode",
10    "cUnicode works with unicode strings.",
11    -1,
12    cUnicode_Methods,
13    NULL, NULL, NULL, NULL
14 };
15
16 PyMODINIT_FUNC
17 PyInit_cUnicode(void)
18 {
19     PyObject* m;
20
21     m = PyModule_Create(&cUnicodemodule);
22     if (m == NULL)
23         return NULL;
24     return m;
25 }

```

The full code is in src/cpy/cpp/cUnicode.cpp and the tests are in tests/unit/test_c_cpp.py. Here is an example of using this module:

```

1 >>> from cPyExtPatt.cpp import cUnicode
2 >>> cUnicode.show('Hello')
3 String size: 5 word size: 1
4 0x00000048      72 "H"
5 0x00000065     101 "e"
6 0x0000006c     108 "l"
7 0x0000006c     108 "l"
8 0x0000006f     111 "o"
9 'Hello'
10 >>> s = "a\xac\u1234\u20ac\u00008000"
11 >>> r = cUnicode.show(s)

```

(continues on next page)

(continued from previous page)

```

12 String size: 5 word size: 2
13 0x00000061      97 "97"
14 0x000000ac     172 "172"
15 0x00001234    4660 "4660"
16 0x000020ac    8364 "8364"
17 0x00008000   32768 "32768"
18 >>> r == s
19 True
20 >>> s = "\xac\u1234\u20ac\u000018000"
21 >>> r = cUnicode.show(s)
22 String size: 5 word size: 4
23 0x00000061      97 "97"
24 0x000000ac     172 "172"
25 0x00001234    4660 "4660"
26 0x000020ac    8364 "8364"
27 0x00018000   98304 "98304"
28 >>> r == s
29 True

```

24.5.2 Working with bytes, bytearray and UTF-8 Unicode Arguments

It is fairly common to want to convert an argument that is bytes, bytearray or UTF-8 to a `std::string`. This function will do just that:

```

1  /** Converting Python bytes and Unicode to and from std::string
2   * Convert a PyObject to a std::string and return 0 if successful.
3   * If py_str is Unicode then treat it as UTF-8.
4   * This works with Python 2.7 and Python 3.4 onwards.
5   */
6  static int
7  py_object_to_std_string(const PyObject *py_object, std::string &result, bool utf8_only =
8  ↪true) {
9      result.clear();
10     if (PyBytes_Check(py_object)) {
11         result = std::string(PyBytes_AS_STRING(py_object));
12         return 0;
13     }
14     if (PyByteArray_Check(py_object)) {
15         result = std::string(PyByteArray_AS_STRING(py_object));
16         return 0;
17     }
18     // Must be unicode then.
19     if (!PyUnicode_Check(py_object)) {
20         PyErr_Format(PyExc_ValueError,
21                     "In %s \"py_str\" failed PyUnicode_Check()",
22                     __FUNCTION__);
23         return -1;
24     }
25     if (PyUnicode_READY(py_object)) {
26         PyErr_Format(PyExc_ValueError,
27                     "In %s \"py_str\" failed PyUnicode_READY()",
28                     __FUNCTION__);

```

(continues on next page)

(continued from previous page)

```

28     return -2;
29 }
30 if (utf8_only && PyUnicode_KIND(py_object) != PyUnicode_1BYTE_KIND) {
31     PyErr_Format(PyExc_ValueError,
32                 "In %s \"py_str\" not utf-8",
33                 __FUNCTION__);
34     return -3;
35 }
36 result = std::string((char *) PyUnicode_1BYTE_DATA(py_object));
37 return 0;
38 }

```

And these three do the reverse:

```

1 static PyObject *
2 std_string_to_py_bytes(const std::string &str) {
3     return PyBytes_FromStringAndSize(str.c_str(), str.size());
4 }
5
6 static PyObject *
7 std_string_to_py_bytearray(const std::string &str) {
8     return PyByteArray_FromStringAndSize(str.c_str(), str.size());
9 }
10
11 static PyObject *
12 std_string_to_py_utf8(const std::string &str) {
13     // Equivalent to:
14     // PyUnicode_FromKindAndData(PyUnicode_1BYTE_KIND, str.c_str(), str.size());
15     return PyUnicode_FromStringAndSize(str.c_str(), str.size());
16 }

```

24.6 C++ and the Numpy C API

[Numpy](#) is a powerful array based data structure with fast vector and array operations. It has a fully featured [C API](#). This section describes some aspects of using Numpy with C++.

24.6.1 Initialising Numpy

The Numpy C API must be setup so that a number of static data structures are initialised correctly. The way to do this is to call `import_array()` which makes a number of Python import statements so the Python interpreter must be initialised first. This is described in detail in the [Numpy documentation](#) so this document just presents a cookbook approach.

24.6.2 Verifying Numpy is Initialised

`import_array()` always returns `NUMPY_IMPORT_ARRAY_RETVAL` regardless of success instead we have to check the Python error status:

```

#include <Python.h>
#include "numpy/arrayobject.h" // Include any other Numpy headers, UFuncs for example.

// Initialise Numpy

```

(continues on next page)

(continued from previous page)

```
import_array();
if (PyErr_Occurred()) {
    std::cerr << "Failed to import numpy Python module(s)." << std::endl;
    return NULL; // Or some suitable return value to indicate failure.
}
```

In other running code where Numpy is expected to be initialised then `PyArray_API` should be non-NULL and this can be asserted:

```
assert(PyArray_API);
```

24.6.3 Numpy Initialisation Techniques

Initialising Numpy in a CPython Module

Taking the simple example of a module from the [Python documentation](#) we can add Numpy access just by including the correct Numpy header file and calling `import_numpy()` in the module initialisation code:

```
#include <Python.h>

#include "numpy/arrayobject.h" // Include any other Numpy headers, UFuncs for example.

static PyMethodDef SpamMethods[] = {
    ...
    {NULL, NULL, 0, NULL} /* Sentinel */
};

static struct PyModuleDef spammodule = {
    PyModuleDef_HEAD_INIT,
    "spam", /* name of module */
    spam_doc, /* module documentation, may be NULL */
    -1, /* size of per-interpreter state of the module,
        or -1 if the module keeps state in global variables. */
    SpamMethods
};

PyMODINIT_FUNC
PyInit_spam(void) {
    ...
    assert(! PyErr_Occurred());
    import_numpy(); // Initialise Numpy
    if (PyErr_Occurred()) {
        return NULL;
    }
    ...
    return PyModule_Create(&spammodule);
}
```

That is fine for a singular translation unit but you have multiple translation units then each has to initialise the Numpy API which is a bit extravagant. The following sections describe how to manage this with multiple translation units.

Initialising Numpy in Pure C++ Code

This is mainly for development and testing of C++ code that uses Numpy. Your code layout might look something like this where `main.cpp` has a `main()` entry point and `class.h` has your class declarations and `class.cpp` has their implementations, like this:

```

.
├── src
│   └── cpp
│       ├── class.cpp
│       ├── class.h
│       └── main.cpp

```

The way of managing Numpy initialisation and access is as follows. In `class.h` choose a unique name such as `awesome_project` then include:

```

#define PY_ARRAY_UNIQUE_SYMBOL awesome_project_ARRAY_API
#include "numpy/arrayobject.h"

```

In the implementation file `class.cpp` we do not want to import Numpy as that is going to be handled by `main()` in `main.cpp` so we put this at the top:

```

#define NO_IMPORT_ARRAY
#include "class.h"

```

Finally in `main.cpp` we initialise Numpy:

```

#include "Python.h"
#include "class.h"

int main(int argc, const char * argv[]) {
    // ...
    // Initialise the Python interpreter
    wchar_t *program = Py_DecodeLocale(argv[0], NULL);
    if (program == NULL) {
        fprintf(stderr, "Fatal error: cannot decode argv[0]\n");
        exit(1);
    }
    Py_SetProgramName(program); /* optional but recommended */
    Py_Initialize();
    // Initialise Numpy
    import_array();
    if (PyErr_Occurred()) {
        std::cerr << "Failed to import numpy Python module(s)." << std::endl;
        return -1;
    }
    assert(PyArray_API);
    // ...
}

```

If you have multiple `.h`, `.cpp` files then it might be worth having a single `.h` file, say `numpy_init.h` with just this in:

```

#define PY_ARRAY_UNIQUE_SYMBOL awesome_project_ARRAY_API
#include "numpy/arrayobject.h"

```

Then each implementation `.cpp` file has:

```
#define NO_IMPORT_ARRAY
#include "numpy_init.h"
#include "class.h" // Class declarations
```

And `main.cpp` has:

```
#include "numpy_init.h"
#include "class_1.h"
#include "class_2.h"
#include "class_3.h"

int main(int argc, const char * argv[]) {
    // ...
    import_array();
    if (PyErr_Occurred()) {
        std::cerr << "Failed to import numpy Python module(s)." << std::endl;
        return -1;
    }
    assert(PyArray_API);
    // ...
}
```

Initialising Numpy in a CPython Module using C++ Code

Supposing you have laid out your source code in the following fashion:

```
.
├── src
│   ├── cpp
│   │   ├── class.cpp
│   │   └── class.h
│   └── cpython
│       └── module.c
```

This is a hybrid of the above and typical for CPython C++ extensions where `module.c` contains the CPython code that allows Python to access the pure C++ code.

The code in `class.h` and `class.cpp` is unchanged and the code in `module.c` is essentially the same as that of a CPython module as described above where `import_array()` is called from within the `PyInit_<module>` function.

How These Macros Work Together

The two macros `PY_ARRAY_UNIQUE_SYMBOL` and `NO_IMPORT_ARRAY` work together as follows:

	PY_ARRAY_UNIQUE_SYMBOL NOT defined	PY_ARRAY_UNIQUE_SYMBOL defined as <NAME>
NO_IMPORT_ARRAY not defined	C API is declared as: static void **PyArray_API Which makes it only available to that translation unit.	C API is declared as: void **<NAME> so can be seen by other translation units.
NO_IMPORT_ARRAY defined	C API is declared as: extern void **PyArray_API so is available from another translation unit.	C API is declared as: extern void **<NAME> so is available from another translation unit.

Adding a Search Path to a Virtual Environment

If you are linking to the system Python this may not have numpy installed, here is a way to cope with that. Create a virtual environment from the system python and install numpy:

```
python -m venv <PATH_TO_VIRTUAL_ENVIRONMENT>
source <PATH_TO_VIRTUAL_ENVIRONMENT>/bin/activate
pip install numpy
```

Then in your C++ entry point add this function that manipulates `sys.path`:

```
/** Takes a path and adds it to sys.paths by calling PyRun_SimpleString.
 * This does rather laborious C string concatenation so that it will work in
 * a primitive C environment.
 *
 * Returns 0 on success, non-zero on failure.
 */
int add_path_to_sys_module(const char *path) {
    int ret = 0;
    const char *prefix = "import sys\nsys.path.append(\"";
    const char *suffix = "\")\n";
    char *command = (char*)malloc(strlen(prefix)
                                   + strlen(path)
                                   + strlen(suffix)
                                   + 1);

    if (!command) {
        return -1;
    }
    strcpy(command, prefix);
    strcat(command, path);
    strcat(command, suffix);
    ret = PyRun_SimpleString(command);
#ifdef DEBUG
    printf("Calling PyRun_SimpleString() with:\n");
    printf("%s", command);
    printf("PyRun_SimpleString() returned: %d\n", ret);
    fflush(stdout);
#endif
    free(command);
    return ret;
}
```

main() now calls this with the path to the virtual environment site-packages:

```
int main(int argc, const char * argv[]) {
    wchar_t *program = Py_DecodeLocale(argv[0], NULL);
    if (program == NULL) {
        fprintf(stderr, "Fatal error: cannot decode argv[0]\n");
        exit(1);
    }
    // Initialise the interpreter.
    Py_SetProgramName(program); /* optional but recommended */
    Py_Initialize();
    const char *multiarray_path = "<PATH_TO_VIRTUAL_ENVIRONMENT_SITE_PACKAGES>";
    add_path_to_sys_module(multiarray_path);
    import_array();
    if (PyErr_Occurred()) {
        std::cerr << "Failed to import numpy Python module(s)." << std::endl;
        return -1;
    }
    assert(PyArray_API);
    // Your code here...
}
```

24.7 C++ and the Python Buffer Protocol

Python's buffer protocol is a general purpose wrapper around data structures that contain homogeneous types with a regular structure. Examples are numpy ndarrays, PIL images and Python types such as bytes, bytearray and array.array types.

The buffer protocol is described in [PEP 3118](#).

The great advantage of this is that it uses a shared memory model so that the data can be passed between Python or C++ without copying.

It is fairly straightforward to create a C++ wrapper class around the buffer protocol.

Todo

Complete the Buffer Protocol chapter with examples from RaPiVot and the C++ wrapper code.

24.7.1 References:

- Python documentation on objects that support the [Buffer protocol](#).
- Python standard library for the [array module](#).

MISCELLANEOUS

This chapter covers various miscellaneous issues that the author has found with creating Python Extensions over the years.

25.1 No PyInit_... Function Found

This is probably Mac OS X and Clang specific but when you import your extension and you get an error like:

`ImportError: dynamic module does not define module export function (PyInit_Foo)`

Have a look at the binary.

```
$ nm -m Foo.cpython-36m-darwin.so | grep Init
000000000000010d0 (__TEXT,__text) non-external (was a private external) _PyInit_Foo
```

Sometimes (why?) clang does not make the symbol external. I have found that adding `__attribute__((visibility("default")))` to the module initialisation function can fix this:

```
__attribute__((visibility("default")))
PyMODINIT_FUNC
PyInit_Foo(void) {
    /* ... */
}
```

And the binary now looks like this:

```
$ nm -m Foo.cpython-36m-darwin.so | grep Init
000000000000010d0 (__TEXT,__text) external _PyInit_Foo
```

25.2 Migrating from Python to a C Extension

Suppose you have followed my advice in *Summary Advice* in that you write your code in Python first then, when profiling shows the slow spots, rewrite in C. You might not want to do this all at once so here is a technique that allows you to migrate to C in a flexible way, say over a number of releases, without your users having to change *their* code.

Suppose you have a bunch of functions and classes in a Python module `spam.py`. Then take all that Python code and put it in a file, say, `py_spam.py`. Now create an empty C Extension calling it, say, `c_spam`.

Change `spam.py` to be merely:

```
from py_spam import *
from c_spam import *
```

Your users, including your test code, just uses `import spam` and they get all of `py_spam` for now and nothing from `c_spam` as it is empty.

You can now, judiciously, add functionality and classes to `c_spam` and your users will automatically get those as `spam` overwrites the appropriate imports from `py_spam` with the ones from `c_spam`.

INSTALLATION

This project is primarily a [documentation project](#) however it does contain a lot of code examples and tests.

26.1 Project Links

- Source is [on GitHub](#).
- Documentation [Read the Docs](#).
- Project is [on PyPi](#).

This code can be installed as follows.

26.2 Setup

First make a virtual environment in your `<PYTHONVENV>`, say `~/pyvenvs`:

```
$ python3 -m venv <PYTHONVENV>/cPyExtPatt
$ source <PYTHONVENV>/cPyExtPatt/bin/activate
(cPyExtPatt) $
```

26.3 Stable release

To install `cPyExtPatt`, run this command in your terminal:

```
$ pip install cPyExtPatt
```

This is the preferred method to install this project, as it will always install the most recent stable release.

If you don't have `pip` installed, this [Python installation guide](#) can guide you through the process.

26.4 From sources

The sources for `cpip` can be downloaded from the [Github repo](#).

You can clone the public repository:

```
(cPyExtPatt) $ git clone https://github.com/paulross/PythonExtensionPatterns.git
```

26.5 Install Requirements

Then install the requirements in your virtual environment (these are pretty minimal):

```
(cPyExtPatt) $ pip install -r requirement.txt
```

Once you have a copy of the source, you can install it with:

```
(cPyExtPatt) $ python setup.py install
```

Or:

```
(cPyExtPatt) $ python setup.py develop
```

As you prefer.

26.6 What is in the Package

Within the cPyExtPatt package are several modules.

Any module can be imported with:

```
from cPyExtPatt import <module>
```

The modules are:

Module	Description
cExceptions	Examples of creating and using exceptions.
cModuleGlobals	Accessing module globals.
cObject	Exploring PyObject.
cParseArgs	Parsing Python function arguments in all their ways.
cParseArgsHelper	More parsing Python function arguments.
cPyRefs	Exploring reference counts.
cPickle	Pickling objects.
cFile	Working with Python and C files.
Capsules.spam	A simple capsule.
Capsules.spam_capsule	A target capsule.
Capsules.spam_client	A client capsule.
Capsules.datetimetz	Using an existing capsule from Python's stdlib.
cpp.placement_new	Example of using C++ placement new.
cpp.cUnicode	Example of working with unicode to and from C++ and Python.
SimpleExample.cFibA	A simple example of a Python C extension.
SimpleExample.cFibB	A simple example of a Python C extension.
Iterators.cIterator	Iterators in C.
SubClass.sublist	Subclassing, in this case a list.
Threads.csublist	Illustrates thread contention in C.
Threads.cppsublist	Illustrates thread contention in C++.
Logging.cLogging	Examples of logging.
cRefCount	Reference count explorations.
cCtxMgr	Example of a context manager.
cStructSequence	Example of a named tuple in C.
cWatchers	Example of a dictionary watcher in C.

In addition there are these modules available in Python 3.12+:

Module	Description
cWatchers	Examples of watchers.

26.7 Running the Tests

Then you should be able to run the tests:

```
(cPyExtPatt) $ pytest tests/
===== test session starts =====
platform darwin -- Python 3.13.1, pytest-8.3.4, pluggy-1.5.0
rootdir: /Users/engun/GitHub/paulross/PythonExtensionPatterns
collected 275 items

tests/unit/test_c_capsules.py ..S.....S. [ 8%]
tests/unit/test_c_cpp.py ..... [ 13%]
tests/unit/test_c_ctxmgr.py ..... [ 14%]
tests/unit/test_c_custom_pickle.py ..... [ 16%]
tests/unit/test_c_exceptions.py .S..S..... [ 21%]
tests/unit/test_c_file.py .....SS.....S [ 26%]
tests/unit/test_c_iterators.py .S.S..... [ 33%]
tests/unit/test_c_logging.py .. [ 34%]
tests/unit/test_c_module_globals.py ..... [ 36%]
tests/unit/test_c_object.py .S..S..SS. [ 40%]
tests/unit/test_c_parse_args.py ...S..S.....SS.....SSSSSS. [ 57%]
.....SSSS..... [ 64%]
tests/unit/test_c_parse_args_helper.py ..... [ 65%]
tests/unit/test_c_py_refs.py ..... [ 67%]
tests/unit/test_c_ref_count.py s..... [ 84%]
..... [ 88%]
tests/unit/test_c_simple_example.py ..... [ 91%]
tests/unit/test_c_struct_sequence.py . [ 92%]
tests/unit/test_c_subclass.py .S... [ 93%]
tests/unit/test_c_threads.py ..S.S..... [100%]

===== 242 passed, 33 skipped in 33.99s =====
```

The skipped tests are specific to a Python version that is not the current version in your virtual environment.

26.8 Building the Documentation

If you want to build the documentation you need to:

```
(cPyExtPatt) $ cd doc/sphinx
(cPyExtPatt) $ make html latexpdf
```

This takes about 40 seconds from clean.

The landing page is *build/html/index.html* in *doc/sphinx*. The PDF is *build/latex/PythonExtensionPatterns.pdf* in *doc/sphinx*.

26.9 Building Everything

At the project root there is a script `build_all.sh` which, for every supported version of Python:

- Builds and tests the C/C++ code.
- Creates a Python virtual environment (optionally deleting any existing one).
- Run `pip install -r requirements.txt` on the virtual environment.
- Run `python setup.py develop` in that virtual environment.
- Run `pytest tests/`.
- Run `python setup.py bdist_wheel`.
- Run `python setup.py sdist`.
- Optionally, create the documentation.
- Report the results.

The script will halt on the first error returning the error code.

Takes about 70 seconds per Python version.

FURTHER READING

Some other sources of information for you.

27.1 C Extensions - General

- [python.org Extension tutorial](#)
- [python.org C/C++ reference](#)
- Joe Jevnik’s “How to Write and Debug C Extension Modules” [Documentation and Code](#)

27.2 Books

- The [CPython Internals book](#) (RealPython)
- The [Python Cookbook](#) by David Beazley and Brian Jones.

27.3 Projects

- Python memory tracing: <https://github.com/paulross/pymemtrace>
- Python/C++ homogeneous containers: <https://github.com/paulross/PyCppContainers>

28.1 Existing TODOs

 Todo

PyDict_SetDefaultRef() failure modes.

original entry

 Todo

PyDict_GetItemRef() failure modes.

original entry

 Todo

Finish Dictionary PyDict_Pop() Failure

original entry

 Todo

Finish Dictionary “Other APIs”

original entry

 Todo

Complete PyDict_DelItem() with code examples.

original entry

 Todo

Complete `PyDict_Items()` with code examples.

[original entry](#)

 **Todo**

Complete `PyDict_Keys()` with code examples.

[original entry](#)

 **Todo**

Complete `PyDict_Values()` with code examples.

[original entry](#)

 **Todo**

Complete `PyDict_Next()` with code examples.

[original entry](#)

 **Todo**

Complete the summary of dictionary APIs and their documentation lacunae.

[original entry](#)

 **Todo**

Complete the Buffer Protocol chapter with examples from RaPiVot and the C++ wrapper code.

[original entry](#)

 **Todo**

Review the Debugging chapter as some of it might be obsolete.

[original entry](#)

 **Todo**

“Creating New Types”: Add a section on making an object act like a number using `tp_as_number`. See also [Number Object Structures](#)

original entry

Todo

“Creating New Types”: Add a section on making an object act like a mapping object using `tp_as_mapping`. See also [Mapping Object Structures](#)

NOTE: This is rarely used in the Python stdlib, really only for dictionaries.

original entry

28.2 Work Estimate for v0.3.0 Release

28.2.1 New Chapter “Debugging Python with CLion”

Estimate: High [4]

28.2.2 TOTAL

4 points.

HISTORY

29.1 0.3.1rc0 (2025-04-01)

- Add a single index entry. This is documentation for 0.3.1rc0, 341 pages.
- Add link to compiler flags resources.
- Add explicit `#include` that was failing on some platforms.
- Include note on `ml_flags` failure on import.
- Suggest change for `Py_SETREF()`.
- Add warning on `Py_SETREF` and `Py_XSETREF`. Fixes to Python 3 links.
- Fix issue #33: Sections in the “Parsing Python Arguments” chapter are of the wrong depth.

29.2 0.3.0 (2025-03-20)

29.2.1 Added Chapters

- “Containers and Reference Counts” which corrects the Python documentation where that is wrong, misleading or missing.
- “Struct Sequences (namedtuple in C)” which corrects the Python documentation where that is wrong, misleading or missing.
- “Context Managers” with practical C code examples.
- “Watchers” with practical examples for dictionary watchers (Python 3.12+).
- “Installation” for the project.
- “Source Code Layout” for the project.

29.2.2 Changed Chapters

- Update the “Homogeneous Python Containers and C++” chapter.
- Expand the “Memory Leaks” chapter.
- Extended the “Logging” chapter to show how to access the CPython Frame from C.
- Add “Emulating Sequence Types” to the “Creating New Types” chapter.
- Expand the Index.

29.2.3 Other

- Python versions supported: 3.9, 3.10, 3.11, 3.12, 3.13.
- Development Status :: 5 - Production/Stable
- The documentation content, example and test code has roughly doubled since version 0.2.2.
- PDF Documentation is 339 pages.

29.2.4 TODO

- Add “Debugging Python with CLion”.

29.3 0.2.2 (2024-10-21)

- Expand note on `PyDict_SetItem()`, `PySet_Add()` with code in `src/cpy/RefCount/cRefCount.c` and tests.

29.4 0.2.1 (2024-07-29)

- Python versions supported: 3.9, 3.10, 3.11, 3.12, 3.13 (possibly backwards compatible with Python 3.6, 3.7, 3.8)
- Almost all example code is built and tested against these Python versions.
- Added a chapter on managing file paths and files between Python and C.
- Added a chapter on subclassing from your classes or builtin classes.
- Added a chapter on pickling from C.
- Added a chapter on Capsules.
- Added a chapter on Iterators and Generators.
- Added a chapter on memory leaks and how to detect them.
- Added a chapter on thread safety.
- Update “Homogeneous Python Containers and C++” to refer to <https://github.com/paulross/PyCppContainers>
- All the documentation has been extensively reviewed and corrected where necessary.
- Development Status :: 5 - Production/Stable

29.4.1 Contributors

Many thanks!

Pull Requests

- <https://github.com/marioemmanuel>
- <https://github.com/miurahr>
- <https://github.com/gdevanla>
- <https://github.com/joelwhitehouse>
- <https://github.com/dhermes>
- <https://github.com/gst>
- <https://github.com/adamchainz>

- <https://github.com/nnathan>

Issues

- <https://github.com/ngoldbaum>
- <https://github.com/niki-sp>
- <https://github.com/ldo>
- <https://github.com/1a1a11a>
- <https://github.com/congma>

29.5 0.1.0 (2014-09-09)

- First release.
- Originally “Examples of reliable coding of Python ‘C’ extensions by Paul Ross.”.
- Development Status :: 3 - Alpha

SEARCH

- `genindex`
- `modindex`
- `search`

Symbols

`__getitem__()`, 179
`__len__()`, 179
`__reversed__()`, 179

A

Access After Free, 12, 230
 Debugging, 221

C

C Functions

Coding Pattern, 72
 Full Coding Pattern, 75

C++, 276

Borrowed PyObject* Wrappers, 277
 Borrowed References, 277
 Buffer Protocol, 308
 bytearray, 302
 bytes, 302
 Homogeneous Containers, 283
 Initialising Numpy, 303
 New PyObject* Wrappers, 278
 New References, 278
 Numpy, 303
 Numpy Initialisation Techniques, 304
 Project PyCppContainers, 283
 PyObject* Wrappers, 277
 Reference Counts, 277
 References, 277
 Unicode, 299

C/C++

Libraries, 2

Capsules, 153

Exporting, 156
 Simple Example, 155
 Using Existing API, 160
 Using Exported API, 159

CMake

Compiler Flags, 213

Code Generators, 2

Compiler Flags, 210
 CLI, 211

CMake, 213

Programmatically, 211

Resources, 215

setup.py, 212

sysconfig, 212

Containers

Documentation Lacunae, 24

Context Managers, 181

`__enter__`, 183

`__exit__`, 183

C Functions, 183

Minimal in C, 185

With target, 184

Without target, 184

cPyMemTrace, 259

CPython Memory Management, 247

Cyclic References, 248

Garbage Collection, 248

Interned Objects, 248

Memory Allocator, 249

pymalloc, 249

pymalloc deallocation, 251

Reference Counts, 247

sys._debugmallocstats, 249

ctypes, 2

D

Debug Malloc Stats, 263

Debugging, 215

A Debug Python Version, 217

Access After Free, 221, 230

Building Python, 219

Debug Version of Python, 219

gcov, 231

IDEs, 234

LLTRACE, 220

Memory Monitor, 217

Py_DEBUG, 220

Py_REF_DEBUG, 220

Py_TRACE_REFS, 220

pyatexit, 241

PyMalloc Statistics, 222

- PYMALLOC_DEBUG, 220
- Python Build Macros, 220
- Python's Memory Allocator, 220
- sysconfig, 224
- Tactics, 230
- Tools, 217
- Valgrind, 217
- WITH_PYMALLOC, 220
- Xcode, 240
- Default Arguments
 - C, 84
 - C++, 278
- Default Arguments, Immutable
 - C, 92
 - C++, 278, 279
- Default Arguments, Mutable
 - C, 87, 93
 - C++, 278, 280
- Dictionaries
 - Documentation Lacunae, 38
- Dictionary, 38
 - Getters, 42, 43
 - Gotchas, 50
 - Other APIs, 45
 - Py_BuildValue(), 47
 - PyDict_DelItem(), 46
 - PyDict_GetItem(), 42
 - PyDict_GetItemRef(), 43
 - PyDict_GetItemWithError(), 45
 - PyDict_Items(), 46
 - PyDict_Keys(), 46
 - PyDict_Next(), 47
 - PyDict_Pop(), 43
 - PyDict_Pop(); Failure, 45
 - PyDict_Pop(); Key Does not Exist, 44
 - PyDict_Pop(); Key Exists, 43
 - PyDict_SetDefault(), 50
 - PyDict_SetDefaultRef(), 40
 - PyDict_SetDefaultRef(); Failure, 42, 43
 - PyDict_SetDefaultRef(); Key Does not Exist, 41
 - PyDict_SetDefaultRef(); Key Exists, 40
 - PyDict_SetItem(), 38, 50
 - PyDict_Values(), 47
 - Setters, 43
 - Summary, 50
 - Watchers, 201
- Documentation Lacunae
 - Containers, 24
 - Dictionaries, 38
 - General, 3
 - Lists, 35
 - Py_SETREF(), 14
 - Py_XSETREF(), 14
 - PyDict_SetDefault(), 39
 - PyDict_SetDefaultRef(), 40
 - PyList_Insert(), 36
 - PyList_SET_ITEM(), 35
 - PyList_SetItem(), 35
 - PyTuple Replacing NULL, 32
 - PyTuple Setting NULL, 32
 - PyTuple_SET_ITEM() Failures, 32
 - PyTuple_SET_ITEM() Replacement, 30
 - PyTuple_SetItem() Failures, 30
 - PyTuple_SetItem() Replacement, 28
 - sq_ass_item, 110
 - Struct Sequence Unnamed Fields, 62
 - Struct Sequence vs namedtuple, 53
- DTrace, 261
- E
- Exceptions
 - Common Exception Patterns, 68
 - General, 66
 - Specialised Exceptions, 69
- F
- faulthandler, 20
- File Path Codecs
 - PyUnicode_DecodeFSDefault(), 133
 - PyUnicode_DecodeFSDefaultAndSize(), 133
 - PyUnicode_EncodeFSDefault(), 133
 - PyUnicode_FSConverter(), 133
 - PyUnicode_FSDecoder(), 133
- File Paths, 133
 - Parsing Arguments, 133
- File Paths and Files, 132
- Files, 134
 - Python Files; C++ Wrapper, 137
 - Python Files; Reading, 135
 - Python Files; Writing, 136
- Frames, 130
 - C, 132
 - Python, 130
- Further Reading, 314
 - Books, 315
 - General, 315
 - Projects, 315
- G
- Generators, 180
- Getters
 - Dictionary, 42, 43
 - List, 37
 - Tuple, 34
- GIL, 2
- Gotchas
 - Dictionary, 50

List, 50
 Tuple, 50

H

History, 319

I

Installation, 310

Iterators, 169

Adding an Iterator, 172
 Iterating Python In C, 176
 Module Initialisation, 175
 Reverse, 179

K

Keyword Only Arguments, 85

L

List, 35

API Summary, 37
 Getters, 37
 Gotchas, 50
 Py_BuildValue(), 37, 50
 PyList_Append(), 35, 50
 PyList_GET_ITEM(), 37, 50
 PyList_GetItem(), 37
 PyList_GetItemRef(), 37
 PyList_Insert(), 36, 50
 PyList_SET_ITEM(), 35, 50
 PyList_SetItem(), 35, 50
 Summary, 50

Lists

Documentation Lacunae, 35

LLTRACE

Debugging, 220

Logging, 123

M

Memory Leaks, 13, 246

Is it a Leak?, 247
 Platform Tools, 251
 pymemtrace, 256
 Python Debugging Tools, 255
 Python Tools, 253
 Sources, 247
 Techniques, 255
 Third Party Python Tools, 254
 Tools, 251

Memory Management, *see* CPython Memory Management

Memory Monitor, *see* pymemtrace, 217

Memory Usage, 2

METH_KEYWORDS, 78, 82

METH_NOARGS, 77, 80

METH_O, 77, 80

METH_VARARGS, 78, 82

Miscellaneous, 308

ml_flags, 77

Module Globules

Getting, 118
 Getting From C, 122
 Getting From Python, 121
 Initialising, 119
 Setting, 118
 Setting From C, 123

N

New Types

Created Python Properties, 98
 Creating, 95
 Delete Attributes Dynamically, 98
 Dynamic Python Properties, 98
 Examples, 98
 Existing C Properties, 97
 Existing Python Properties, 97
 Get Attributes Dynamically, 98
 PySequenceMethods, 116
 Sequence Types, 103
 Set Attributes Dynamically, 98
 sq_ass_contains, 113
 sq_ass_item, 110
 sq_concat, 105
 sq_inplace_concat, 115
 sq_inplace_repeat, 115
 sq_item, 108
 sq_length, 105
 sq_repeat, 107

Numpy

C++, 303
 C++ Initialisation Techniques, 304

P

Parsing Arguments, 76

METH_KEYWORDS, 78
 METH_NOARGS, 77, 80
 METH_O, 77, 80
 METH_VARARGS, 78, 82
 ml_flags, 77
 Multiple Arguments, 78
 No Arguments, 77
 One Argument, 77
 Positional and Keyword Arguments, 78

Parsing Arguments Example

Default Bytes Arguments, 84
 Default Immutable Arguments, 279
 Default Mutable Arguments, 87, 278, 280
 Default String Arguments, 84

- Functional Conversion to C, 86
 - Helper Class, 278
 - Helper Macros, 91
 - Keyword Arguments, 82
 - Keyword Arguments and C++11, 84
 - Keyword Only Arguments, 85
 - No Arguments, 80
 - One Argument, 80
 - Positional Only Arguments, 85
 - Variable Number of Arguments, 82
 - Personal Note, 1
 - Pickling, 190
 - __getstate__, 191
 - __setstate__, 192
 - __setstate__ Error Checking, 192
 - External State, 200
 - pickletools, 197
 - References, 200
 - Version Control, 191
 - Positional Only Arguments, 85
 - Py_buffer, 84
 - Py_DEBUG
 - Debugging, 220
 - Py_REF_DEBUG
 - Debugging, 220
 - Py_SETREF(), 14
 - Documentation Lacunae, 14
 - Py_TRACE_REFS
 - Debugging, 220
 - Py_XSETREF(), 14
 - Documentation Lacunae, 14
 - PyDict_SetDefault()
 - Documentation Lacunae, 39
 - PyDict_SetDefaultRef(), 40
 - Documentation Lacunae, 40
 - PyDict_SetItem(), 38
 - PyList_Append(), 35
 - PyList_GET_ITEM(), 37
 - PyList_GetItem(), 37
 - PyList_GetItemRef(), 37
 - PyList_Insert(), 36
 - Documentation Lacunae, 36
 - PyList_SET_ITEM(), 35
 - Documentation Lacunae, 35
 - PyList_SetItem(), 35
 - Documentation Lacunae, 35
 - PYMALLOC_DEBUG
 - Debugging, 220
 - pymemtrace, *see* Memory Monitor, 256
 - cPyMemTrace, 259
 - Debug Malloc Stats, 263
 - DTrace, 261
 - process, 257
 - Tool Characteristics, 256
 - Tools, 256
 - tracemalloc, 262
 - PySequenceMethods, 116, 179
 - PySet_Add(), 47
 - PySet_Discard(), 48
 - PySet_Pop(), 49
 - PyTuple Replacing NULL
 - Documentation Lacunae, 32
 - PyTuple Setting NULL
 - Documentation Lacunae, 32
 - PyTuple_GET_ITEM(), 34
 - PyTuple_GetItem(), 34
 - PyTuple_Pack(), 33
 - PyTuple_SET_ITEM(), 30
 - Failures, 32
 - Replacement, 31
 - Replacing NULL, 32
 - Setting NULL, 32
 - PyTuple_SET_ITEM() Failures
 - Documentation Lacunae, 32
 - PyTuple_SET_ITEM() Replacement
 - Documentation Lacunae, 30
 - PyTuple_SetItem(), 27
 - Failures, 30
 - Replacement, 28
 - Replacing NULL, 32
 - Setting NULL, 32
 - PyTuple_SetItem() Failures
 - Documentation Lacunae, 30
 - PyUnicode_DecodeFSDefault(), 133
 - PyUnicode_DecodeFSDefaultAndSize(), 133
 - PyUnicode_EncodeFSDefault(), 133
 - PyUnicode_FSConverter(), 133
 - PyUnicode_FSDecoder(), 133
- ## R
- Reference Counts, 9
 - Abandoned, 25
 - Borrowed, 9, 19
 - Discarded, 25
 - Finding Leaks, 226
 - New, 9, 16
 - Recognising Leaks, 227
 - Stolen, 9, 17
 - Valgrind, 229
 - References
 - Finding Leaks, 226
 - Recognising Leaks, 227
 - Strong, 21
 - Valgrind, 229
 - Weak, 21
 - Reverse Iterators, 179

S

- Sequence Types, 103
 - Function Table, 104, 116
 - sq_ass_contains, 113
 - sq_ass_item, 110
 - sq_concat, 105
 - sq_inplace_concat, 115
 - sq_inplace_repeat, 115
 - sq_item, 108
 - sq_length, 105
 - sq_repeat, 107
- Set, 47
 - PySet_Add(), 47
 - PySet_Discard(), 48
 - PySet_Pop(), 49
 - Setters, 47
- Setters
 - Dictionary, 43
 - Set, 47
- setup.py, 6
- Source Code Layout, 274
- sq_ass_contains, 113
- sq_ass_item, 110
- sq_concat, 105
- sq_inplace_concat, 115
- sq_inplace_repeat, 115
- sq_item, 108
- sq_length, 105
- sq_repeat, 107
- Strong References, 21
- Struct Sequence, 51
 - Basic Example, 53
 - C structs, 60
 - Controlling Member Access, 62
 - Creating an Instance, 54
 - Differences from namedtuple, 53
 - Documentation String, 54
 - Exposing the Type, 57
 - Field Specifications, 54
 - Hiding the Type, 58
 - n_in_sequence, 62
 - PyStructSequence_UnnamedField, 62
 - Type Specification, 54
 - Unnamed Fields, 62
- Struct Sequence Unnamed Fields
 - Documentation Lacunae, 62
- Struct Sequence vs namedtuple
 - Documentation Lacunae, 53
- Subclassing, 139
 - Basic, 141
 - Calling super() From builtins, 148
 - Calling super() from C, 143
 - datetime Example, 152, 160
 - Directly Calling super() from C, 145

- Using super(), 139
 - With Overloading, 150
- sys._debugmallocstats(), 263
- sysconfig
 - CLI, 212
 - Debugging, 224
 - importing, 211

T

- Thread Safety, 266
 - Creating a Lock, 267
 - Examples, 271
 - Using a Lock, 269
- To Do, 315
- tracemalloc, 262
- Tuple, 27
 - API Summary, 35
 - Getters, 34
 - Gotchas, 50
 - Py_BuildValue(), 34, 50
 - PyTuple_GET_ITEM(), 34, 50
 - PyTuple_GetItem(), 34
 - PyTuple_Pack(), 33, 50
 - PyTuple_SET_ITEM(), 50
 - PyTuple_SET_ITEM();, 30
 - PyTuple_SetItem(), 27, 50
 - Summary, 50

U

- Unicode
 - C++, 299

V

- Valgrind, 225
 - Building, 225
 - Building Python For, 225
 - Using, 226

W

- Watchers, 200
 - Dictionary, 201
 - Dictionary; Context Manager, 207
 - Dictionary; Implementation, 202
 - Dictionary; Module, Setup and Test, 208
 - Dictionary; No Context Manager, 210
- Weak References, 21
- WITH_PYMALLOC
 - Debugging, 220